

PrimeSim™ HSPICE® User Guide: Advanced Analog Simulation and Analysis

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	13
Related Products and Trademarks	13
Conventions	14
Customer Support	14
Statement on Inclusivity and Diversity	15

Part 1: PrimeSim HSPICE Advanced Analog Features

1. Introduction to PrimeSim HSPICE Advanced Analog Analyses	17
PrimeSim HSPICE Advanced Analog Analyses	17
PrimeSim HSPICE Advanced Analog Features	18
Recommended Analysis Method for Typical RF Design Blocks	20
PrimeSim HSPICE Advanced Analog Output Files	21
Using PrimeWave Design Environment	22
Creating a Configuration File	23
Inserting Comments in a .hspicrf File	25
Using Wildcards	25
Limiting Output Data Size	26
Generating Measurement Output Files	26
Advanced Analog Demonstration Input Files	27
2. Use of Example Syntax	29

Part 2: Periodic Steady-State Analysis

3. Harmonic Balance Analysis	31
Tutorial Examples Harmonic Balance Analysis	31
Example 1: Using HB Analysis for a Power Amplifier	31

Example 2: Using HB Analysis for a Low Noise Amplifier	36
Harmonic Balance Analysis	42
Features Supported	43
Harmonic Balance Equations	44
Steady-State HB Sources	45
Examples	45
Harmonic Balance Output Measurements	46
HB .PRINT and .PROBE Output Syntax	47
HB Output Data Files and Examples	49
HB Error and Warning Messages	50
HB Analysis Error Messages	50
HB Analysis Warning Messages	51
Calculating Power Measurements After HB Analyses	52
Power Dissipated in a Resistor	52
Power Delivered to a Port Element	52
Examples	54
Calculations for Time-Domain Output	55
Using .MEASURE with .HB Analyses	56
Examples	57
Troubleshooting Convergence Issues with Harmonic Balance Analysis	58
Phase Differences Between HB and SIN Sources	59
Example	59
Optimization	60
Optimizing with HB Measurements	61
References	61
4. Shooting Newton Analysis	63
Shooting Newton Steady-State Time Domain Analysis (.SN)	63
Shooting Newton Analysis – Tutorial Example	64
Shooting Newton Analysis — Setup	64
Clock Buffer Analysis Example	65
SN .PRINT and .PROBE Output Syntax	70
Shooting Newton with Fourier Transform (.SNFT)	72
Other Shooting Newton Analyses	73
SN Error and Warning Messages	74
Troubleshooting Convergence Issues with Shooting Newton Analysis	74

Part 3: Oscillator and PLL Analysis

5. Oscillator Analysis	76
Oscillator Analysis (.HBOSC and .SNOSC)	77
Tutorial Examples Using HBOSC Analysis	78
Example 1 — Colpitts Oscillator	78
Example 2 — Using HBOSC for a CMOS GPS VCO	83
.HBOSC Output Syntax	90
Using the .MEASURE Command with .HBOSC	90
HB Simulation of Ring Oscillators	91
Troubleshooting Convergence Problems	91
General Convergence Issues	92
Outer Loop Convergence	94
Inner Loop Convergence	95
Shooting Newton Oscillator Analysis (.SNOSC)	96
Ring Oscillator Example	97
References	100
6. Phase Noise Analysis	102
Introduction to Phase Noise Analysis (.PHASENOISE)	102
Identifying Phase Noise Spurious Signals	105
Phase Noise Algorithms	105
PHASENOISE Output Syntax	106
Whole Circuit and Specified Element Phase Noise	107
Frequency-Dependent and Frequency-independent Sources	107
Frequency and Bias Dependencies	108
Examples	108
See Also	109
Measuring Phase Noise with .MEASURE PHASENOISE	109
Optimizing with PHASENOISE Measurement	110
Amplitude Modulation/Phase Modulation Separation	111
AM/PM .PRINT and .PROBE Statement Syntax	111
Examples	112
Output Files	112
Important Note for AM/PM Users	113
Interpreting Phase Noise Analysis Results	114
Accumulated Jitter Measurement for Closed Loop PLL Analysis	114

Jitter Measurements from Phase Noise	115
Jitter Definitions	115
Jitter Output Syntax	117
.MEASURE Statements for Jitter	118
Peak-to-Peak Jitter	119
Errors/Warnings	120
Clock Source with Random Jitter	121

7. Behavioral PLL Noise Analysis (.ACPHASENOISE)	122
ACPHASENOISE Analysis .PRINT and .PROBE Syntax	122
.MEASURE Support for ACPHASENOISE	123
Behavioral Noise Sources	123
Using Noise Analysis Results as Input Noise Sources	125
Syntax Rules for Behavioral Noise Source Files	126
Noise File Examples	127
Power Supply Current and Voltage Noise Sources	128
Syntax	128
Example	129

Part 4: Periodically-driven AC, Transfer Function, and Noise Analyses

8. Large Signal AC, Transfer Function, and Noise Analyses	131
Multitone Harmonic Balance AC Analysis (.HBAC)	131
Prerequisites and Limitations	132
Tutorial Example - Using Multi-Tone HB and HBAC Analyses for a Mixer	132
Two-tone HB Approach	134
HBAC Approach	134
Comparing Results	135
Input Syntax	138
HBAC Analysis Options	140
Output Syntax	141
Time-Sampled (TS) HBAC Output Format	142
Time-Averaged (TA) HBAC Output Format	142
Using the .MEASURE Command with .HBAC	143
Errors and Warnings	143
Error Messages	143
Warning Messages	144

Contents

Shooting Newton AC Analysis (.SNAC)	144
Prerequisites and Limitations	145
SNAC Example	145
Input Syntax	146
Output Syntax	146
SNAC Output Data Files	147
Using the .MEASURE Command with .SNAC	148
Errors and Warnings	148
Error Messages	148
Warning Messages	149
Steady-State Voltage and Current Sources	149
I and V Element Syntax	149
Examples	151
Phase Noise and Buffer Chains	152
Multitone Harmonic Balance Noise (.HBNOISE)	153
Supported Features	154
Prerequisites and Limitations	154
HBNOISE Example	154
Input Syntax	155
Output Syntax	157
.PRINT and .PROBE Statements	157
Output Data Files	158
Measuring HBNOISE Analyses with .MEASURE	158
Optimizing with HBNOISE Measurement	160
Errors and Warnings	161
Shooting Newton Noise Analysis (.SNNOISE)	161
Supported Features	162
Prerequisites and Limitations	162
SNNOISE Analysis Example	162
Input Syntax	163
Specifying Variant Indices	164
Output Syntax	164
.PRINT and .PROBE Statements	164
Output Data Files	165
Measuring SNNOISE Analyses with .MEASURE	166
Multitone Harmonic Balance Transfer Function Analysis (.HBXF)	168
Supported Features	169
Prerequisites and Limitations	169

Example	169
HBXF Test Listing	169
Input Syntax	170
Output Syntax	170
.PRINT and .PROBE Statements	170
Output Data Files	171
Using the .MEASURE Command with .HBXF	171
Shooting Newton Transfer Function Analysis (.SNXF)	171
Prerequisites and Limitations	172
Example	172
SNXF Test Listing	172
Input Syntax	173
Output Syntax	174
Output Data Files	174
Using the .MEASURE Command with .SNXF	175
HB- and SN-Based Stability Analysis	175
Algorithm Description	175
.HBSTB and .SNSTB	178
References	178
9. Large Signal S-parameter Analyses	180
Frequency Translation S-Parameter (HBLIN) Extraction	181
Limitations	182
HB Analysis	183
Port Element	183
Port Element Syntax	183
HBLIN Analysis	184
Input Syntax	184
Examples	185
Output Syntax	185
.PRINT and .PROBE Statements	186
Output Data Files	186
Large-Signal S-parameter (HBLSP) Analysis	187
Input Syntax	188
Examples	189
Output Syntax	190
.PRINT and .PROBE Statements	190

Output Data Files	190
HBLSP Analysis Limitations	191

10. Envelope Analysis	192
Envelope Simulation	192
Envelope Analysis Commands	193
Nonautonomous Form	193
Oscillator Analysis Form	194
Fast Fourier Transform Form	195
Output Syntax	196
.PRINT or .PROBE	196
.MEASURE	196
Envelope Output Data File Format	196

Part 5: Large Signal Noise Analyses

11. Simulation of Random Noise	199
Introduction to Noise Sources	199
Characteristics of Random Signals	200
Probability Distribution Function versus Power Spectral Density	200
Multiple Noise Sources in a Circuit	204
Summary	205
Noise Types	205
Thermal Noise	205
Flicker Noise	207
Shot Noise	210
Summary	210
Component Noise Models and PrimeSim HSPICE Noise Simulation	210
Element Noise Models	210
PrimeSim HSPICE Noise Simulation	211
RX Transfer Function	212
Example	212
Summary	216

12. Transient Noise Analysis	218
---	------------

Contents

Overview of PrimeSim HSPICE Transient Noise Analysis	219
Modeling Frequency-Dependent Noise Sources	220
Monte Carlo Noise Analysis	221
Setting up a Monte Carlo Transient Noise Analysis	222
Transient Noise and Monte Carlo Noise Analysis Techniques Available in PrimeSim HSPICE	223
Traditional Monte Carlo with Transient Noise Disabled	223
Transient Noise Analysis without Monte Carlo (Single Sample)	223
Transient Noise Analysis with Monte Carlo	224
Both Transient Noise and Traditional Monte Carlo Enabled	224
Monte Carlo Output Data	225
Monte Carlo Examples	225
Controlling Noise	225
Setting Noise Bandwidth	226
TIE and Spectrum Jitter Measurements from Transient Noise Results	227
Measure TIE Jitter and Jitter Spectrum Example	227
Autocorrelation Function Example	230
Phase Noise and Jitter from Transient Noise	231
Delay Line Method	232
Using Delay Line Method	232
Phase Detector Method	233
Using the Phase Detector Method	233
Phase Noise Output	233
Jitter Output	234
Recommendations	234
Power Spectral Density from Transient Noise	235
Using PSD from Transient Noise	236
Stochastic Differential Equation Analysis	236
SDE Output Data	238
SDE Examples	239
Error Handling, Error Recovery, Status Reporting	239
References	239
13. Periodic Time-Dependent Noise Analysis (.PTDNOISE)	241
Introduction to Periodic Time-Dependent Noise Analysis	241
PTDNOISE Input Syntax	242

PTDNOISE Output Syntax and File Format	244
Output File Format	245
.MEASURE Syntax and File Format	246
Errors and Warnings	247
Usage Example	247
Troubleshooting .PTDNOISE Simulation	248
References	248

Part 6: Behavioral Modeling Applications

14. Behavioral Modeling of Nonlinear Circuits	250
Behavioral Design Process	250
Using Behavioral Elements	251
Controlled Sources	253
Libraries	253
Voltage and Current Controlled Elements	253
Modeling with Digital Behavioral Components	254
Behavioral AND and NAND Gates	254
Behavioral D-Latch	254
Behavioral Double-Edge Triggered Flip-Flop	256
Calibrating Digital Behavioral Components	258
Building Behavioral Lookup Tables	258
Subcircuit Definition	259
Behavioral N-Channel MOSFET	259
Creating a Behavioral Inverter Lookup Table	260
Optimizing Behavioral CMOS Inverters	262
Optimizing Behavioral Ring Oscillators	263
Analog Behavioral Elements	264
Behavioral Integrator	265
Behavioral Differentiator	266
Behavioral Amplitude Modulator	268
Behavioral Data Sampler	268
Op-Amps, Comparators, and Oscillators	269
741 Op-Amp from Controlled Sources	269
Inverting Comparator with Hysteresis	271

Voltage-Controlled Oscillator (VCO)	273
LC Oscillator	275
Phase-Locked Loops (PLL)	277
Phase Detector, with Multi-Input NAND Gates	277
Phase Locked Loop Modeling	279
References	282
15. Laplace Transform Modeling of Filters and Networks	283
Transient Modeling	283
Using G- and E-elements	284
Laplace Transform Function Call	285
G- and E-element Notes	286
Laplace Band-Reject Filter	287
Laplace Low-Pass Filter	288
Circular Convolution Example	289
Notes	290
30-Degree Phase Shift Circuit File	290
Laplace and Pole-Zero Modeling	291
Laplace Transform (LAPLACE) Function	291
General Form of the Transfer Function	292
Finding the Transfer Function	292
Determining the Laplace Coefficients	294
Laplace Transform POLE (Pole/Zero) Function	298
POLE Function Call	299
General Form of the Transfer Function	299
Reduced Form of the Transfer Function	300
RC Line Modeling	302
AWE Transfer Function Modeling	304
Y-parameter Line Modeling	307
Comparison of Circuit and Pole/Zero Models	310
Simulation Time Summary	310
Modeling Switched Capacitor Filters	312
Switched Capacitor Network	312
Switched Capacitor Filter Example	313
Input File for Switched Capacitor Filter	314
References	316

About the PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis

The *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis* focuses on advanced analog analysis and modeling and concentrates on advanced analog circuit characterization, noise modeling and analysis, and behavioral analysis including Verilog-A.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the PrimeSim HSPICE Release Notes on the SolvNetPlus site.

Related Products and Trademarks

For additional information about the PrimeSim HSPICE simulation, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys PrimeWave[™] Design Environment

Synopsys SolvNetPlus support site

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Part 1: PrimeSim HSPICE Advanced Analog Features

Describes how to use PrimeSim HSPICE advanced analog analyses, take advantage of its specialized features, and use the WaveView tool; introduces the PrimeSim HSPICE solutions for noise analysis.

PrimeSim HSPICE advanced analog analyses is a set of analysis and design capabilities that support the design of advanced analog and high-speed circuits. PrimeSim HSPICE advanced analog analyses includes several modeling, simulation, and measurement additions that augment the ultimate-accuracy analog circuit simulation capabilities.

PrimeSim HSPICE advanced analog analyses accepts a netlist file from the standard input and delivers the ASCII text simulation results to HTML or to the standard output. Standard error output reports the error and warning messages.

These following sections cover these topics:

- [Introduction to PrimeSim HSPICE Advanced Analog Analyses](#)
- [Use of Example Syntax](#)

1

Introduction to PrimeSim HSPICE Advanced Analog Analyses

The following sections discuss the advanced analog features:

- [PrimeSim HSPICE Advanced Analog Analyses](#)
- [PrimeSim HSPICE Advanced Analog Features](#)
- [Recommended Analysis Method for Typical RF Design Blocks](#)
- [PrimeSim HSPICE Advanced Analog Output Files](#)
- [Using PrimeWave Design Environment](#)
- [Creating a Configuration File](#)
- [Using Wildcards](#)
- [Limiting Output Data Size](#)
- [Generating Measurement Output Files](#)
- [Advanced Analog Demonstration Input Files](#)

PrimeSim HSPICE Advanced Analog Analyses

PrimeSim HSPICE advanced analog analyses provide several analyses that support the simulation and analysis of radio-frequency integrated circuits (RFICs). The advanced analog analyses include:

- [Harmonic Balance Analysis](#)
- [Shooting Newton Analysis](#) and [Shooting Newton with Fourier Transform \(.SNFT\)](#)
- [Oscillator Analysis \(.HBOSC and .SNOSC\)](#)
- [Shooting Newton Oscillator Analysis \(.SNOSC\)](#)
- [Large Signal AC, Transfer Function, and Noise Analyses](#) and [Multitone Harmonic Balance AC Analysis \(.HBAC\)](#)

- [Shooting Newton AC Analysis \(.SNAC\)](#)
- [Large Signal AC, Transfer Function, and Noise Analyses](#)
- [Shooting Newton Noise Analysis \(.SNNOISE\)](#)
- [Multitone Harmonic Balance Transfer Function Analysis \(.HBXF\)](#)
- [Shooting Newton Transfer Function Analysis \(.SNXF\)](#)
- [Frequency Translation S-Parameter \(HBLIN\) Extraction](#)
- [Envelope Analysis](#)
- [Transient Noise Analysis](#)

Caution:

You can enable the analytical derivative computation of expression-based element evaluations. You can enable this in all advanced analog analyses, for extensive accuracy, by using the `.OPTION EQN_ANALYTICAL_DERIV` described in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

PrimeSim HSPICE Advanced Analog Features

This section briefly introduces the features of both the simulation engine and the waveform display tool.

- Steady-state frequency-domain analyses for linear and nonlinear circuits.
- The `.HBLSP` command invokes periodically-driven nonlinear circuit analyses for power-dependent S parameters.
- Harmonic Balance (`.HB`) analysis by using Direct and Krylov solvers. The `.HB` command invokes the single- and multi-tone Harmonic Balance algorithm for periodic steady-state analysis.
- `TRANFORHB` element parameter to recognize V/I sources that include SIN and PULSE transient descriptions as well as PWL and VMRF sources.
- Harmonic balance-based periodic AC analysis. The `.HBAC` command invokes periodic AC analysis for analyzing small-signal perturbations on circuits that operate in a large-signal periodic steady-state.
- Harmonic Balance-based Periodic Noise analysis (`.HBNOISE`) for noise analysis of periodically modulated circuits; includes stationary, cyclo-stationary, and frequency-dependent noise effects.

- Autonomous Harmonic Balance analysis. The `.HBOSC` command invokes the multi-tone, oscillator-capable Harmonic Balance algorithm for periodic steady-state analysis.
- Perturbation analysis for Oscillator Phase Noise. The `.HBAC` command invokes phase periodic AC noise for oscillators circuits that operate in a large-signal steady-state.
- Oscillator phase noise analysis, including both a nonlinear perturbation method and a PAC method; includes stationary, cyclo-stationary, frequency-dependent, and correlated noise effects.
- Frequency translation S-parameter and noise figure extraction with the `.HBLIN` command.
- Envelope analysis. The `.ENV` command invokes standard envelope simulation. The `.ENVOSC` command invokes envelope startup simulation. The `.ENVFFT` command invokes envelope Fast Fourier Transform simulation.
- `.OPTION HBTRANINIT, HBTRANPTS, and HBTRANSTEP`, to use transient analysis to initialize all state variables for HB analysis.
- Calculation of the transfer function from an arbitrary source and harmonic in the circuit to a designated output with the `.HBXF` command.
- `.OPTION SIM_ACCURACY` provides simplified accuracy control for all simulations while `.OPTION SIM_ORDER` and `SIM_TRAP` improve transient analysis simulation controls.
- DSPF Flow for fast analysis by using parasitic data from layout.
- Shooting Newton steady-state time domain analysis; the Shooting Newton algorithm provides functionality to support the following commands: `.SN`, `.SNAC`, `.SNFT`, `.SNNOISE`, `.SNOSC`, and `.SNXF`.
- Periodic Time-Dependent Noise Analysis (`.PTDNOISE`) calculates the noise spectrum and the total noise at a point in time. This analysis determines jitter in a digital threshold circuit from the total noise and the digital signal slew rate.
- PrimeSim HSPICE advanced analog analyses supports the ISUB syntax with the exception of wildcard support with the “?” sign. For example, PrimeSim HSPICE advanced analog analyses does not support `isub(x1.a?)`.
- PrimeSim HSPICE advanced analog analyses supports case sensitivity.

PrimeSim HSPICE advanced analog analyses also includes the following measurement capabilities:

- 1dB compression point.
- Intercept points (for example, IP2, IP3).

- Mixer conversion gain and noise figure.
- VCO output spectrum.
- Oscillator phase noise.
- Options simplify specifying levels of accuracy. As a result, PrimeSim HSPICE provides effective simulation solutions for advanced analog, high-speed, and PCB signal integrity circuit challenges.
- Verilog-A is supported for all advanced analog analyses.
- Standard restrictions for Verilog-A in periodic steady-state analysis are the same as other advanced analog simulators that use Verilog-A. For example:
 - Verilog-A modules that are time-dependent are illegal for HB or SN unless the time dependence is periodic with a period that matches the HB or SN setup.
 - Verilog-A modules with “internal states” may not work correctly in HB or SN because the engine cannot track the internal state. So, HB or SN may accept convergence to a periodic steady-state even though the internal state may not be in periodic steady state.
 - Some event-driven constructs in Verilog-A may not be compatible with HB.
- For netlist input guidelines, see Input Netlist and Data Entry in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.
- For information on the use of parameters and functions, see Parameters and Functions in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.
- For information on the use of Monte Carlo sweeps, see Variation Analysis in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Recommended Analysis Method for Typical RF Design Blocks

Design	SN Analysis	HB Analysis
Oscillators	Yes	Yes
Crystal oscillators	-	Yes
Frequency dividers	Yes	-
Switch cap circuits	Yes	-
Mixers	-	Yes
Charge pumps	Yes	-

Design	SN Analysis	HB Analysis
LNA	Yes	Yes

As a general rule of thumb, consider using:

- Shooting Newton analysis for highly nonlinear circuits or circuits having waveforms with sharp transitions.
- Harmonic Balance analysis for the other types of circuits.

PrimeSim HSPICE Advanced Analog Output Files

The following table shows the output file extensions that PrimeSim HSPICE advanced analog analyses produce. The base file name of each output file is the same as the base name of the input netlist file. The # at the end of each file extension represents the .ALTER run from which the file came.

In general, text output from .PRINT commands is for users, while binary output from .PROBE or .OPTION POST is the input for the WaveView tool.

Command	Description	Text Output	Output for WaveView
.ENV	Envelope analysis	.printev#	.ev#
.ENVFFT	Envelope FFT	(none)	.fe#
.ENVOSC	Oscillator Startup	.printev#	.ev#
.HB	Harmonic Balance	.printhb#	.hb#
.HBAC	Harmonic Balance AC	.printhb#	.hb#
.HBLIN	Harmonic Balance Linear Analysis	.PRINT output: .printhl# S-param output: .SnP	.PROBE output: .hl# S-param output: .SnP
.HBLSP	.HBLSP large-signal	.PRINT output: .printls# S-param output: .p2d	.PROBE output: .ls# S-param output: .p2d
	.HBLSP small-signal	.PRINT output: .printss# S/noise output: .S2P#	.PROBE output: .ss# S/noise output: .S2P#
.HBNOISE	HBAC noise	.printsnpn#	.pn#
.HBOSC	Harmonic Balance OSC	.printhb#	.hb#
.HBXF	Transfer Functions	.printxf#	.xf#
.PHASENOISE	Phase Noise (SNOSC)	.printsnpn#	.pn#

Command	Description	Text Output	Output for WaveView
.PTDNOISE	Phase Noise (HBOSC)	.printtpn#	.pn#
	Jitter	.printjt#	.jt#
	Periodic Time Dependent Noise (HB)	.printptn#	.ptn#
	Periodic Time Dependent Noise (SN)	.printsnptn#	.snptn#
.SN	Shooting Newton Analysis	.printsn#	.sn#
.SNAC	Shooting Newton AC	.printsnac#	.snac#
.SNNOISE	Shooting Newton Noise	.printsnpn#	.snpn#
.SNOSC	Shooting Newton OSC	.printsn#	.sn#
.SNXF	Shooting Newton Transfer Function	.printsnxf#	.snxf#
.TRANNOISE	Transient Noise	.printtrpn#	.trpn#
	Jitter	.printtrjt#	.trjt#
	Autocorrelation Function	none	.trzn#

Using PrimeWave Design Environment

Synopsys WaveView supports viewing and processing of PrimeSim HSPICE output files. This section presents a basic overview of how to use the tool.

1. To start the WaveView tool, type `wv` on the UNIX/Linux command line.
2. Click the **Open Waveform** icon (or press CTRL+O) to open the Open: Waveform Files dialog box, to select waveform files.
3. Use the **File Filters** to limit the file names to waveform files. The preceding table lists the PrimeSim HSPICE advanced analog file types. When you open a file, its contents appear in the file browser. The file browser lists all open plot files. Click on the '+' near a waveform file name, to display the hierarchy of the waveform file. Click on the top level or any hierarchy level to display the contents of the waveform file to appear in the signal browser. To plot one of the signals listed in the signal browser, you can either double-click the signal label or select, drag, and drop the signal label into the wave view.
4. To create a panel, use the Panel section of the **Waveform** tab and select the panel type: **X-Y**, **Smith Chart**, or **Polar Plot**.

5. Use the WaveView toolbar to change how signals look, to delete signals, and to group or ungroup signals.
6. Right-click on a frequency-domain signal name and use the **To Time Domain** command to convert the histogram waveform (for example, from a `.hb0` file) to a time domain waveform.
7. Configure the axis scale and grid by right-clicking a horizontal or vertical axis and selecting the desired scale or grid from the context-sensitive menu.
8. Zoom in and out using the zoom icons on the toolbar, or use the mouse cursor to select the area to zoom directly on the waveform.
9. You can use dynamic meters to see the precise value of the signal at different points. From the **WaveView** tab, click the **Measure Tools** icon. Select and configure the desired dynamic meter from the Measurement Tool window. You can then move the meter to the desired location on the selected signal.

The following advanced analog measurements are available:

- 1db compression point (dBCP).
- Intercept points (IPn).
- 3rd order intercept point and spurious free dynamic range (IP3/SFDR).

Creating a Configuration File

You can create a configuration file, called `.hspicerrf`, to customize your PrimeSim HSPICE advanced analog simulation. PrimeSim HSPICE first searches for `.hspicerrf` in your current working directory and then in your home directory as defined by `$HOME`. Following are the configuration options available in PrimeSim HSPICE:

Keyword	Description	Example
flush_waveform	Flushes a waveform. If you do not specify a percentage, then the default value is 20%.	<code>flush_waveform percent%</code>
ground_floating_node	Uses .IC statements to set floating nodes in a circuit to ground. You can select three options for grounding floating nodes: • If set to 1, grounds only floating nodes (gates, bulk, control nodes, and non-rail bulk) that the .IC set includes. • If set to 2, adds unconnected terminals to this set. • If set to 3, uses .IC statements to ground all floating nodes, including dangling terminals.	<code>ground_floating_ node 1</code>
hier_delimiter	Changes the delimiter for subcircuit hierarchies from “.” to the specified symbol.	<code>hier_delimiter /</code>

Keyword	Description	Example
html	Stores all PrimeSim HSPICE output in HTML format.	htmlhspicerf test This example creates a file named test.html in the current directory.
integer_node	Removes leading zeros from node names. For example, PrimeSim HSPICE considers 0002 and 2 to be the same node. Without this keyword, 0002 and 2 are two separate nodes.	integer_node
max_waveform_size	Automatically limits the waveform file size. - If the number is less than 5000, PrimeSim HSPICE resets it to 2G. - If you do not set the number, PrimeSim HSPICE uses the default, 2G. - If you do not set the line, the file size has no limit.	max_waveform_size 2000000000
negative_td	Allows negative time delay input in pwl (piecewise linear with repeat), pl (piecewise linear), exp (exponential, rising time delay only), sin (damped sinusoidal), pulse (trapezoidal pulse), and am (amplitude modulation) formats.	If you do not set negative_td, the negative time delay defaults to zero.
port_element_voltage_matchload	Allows the alternate Port element definition. A Port element consists of a voltage source in series with a resistor. For the explanation that follows, let the user-specified DC, AC, or transient value of the Port element be V, and let the voltage across the overall port element be Vp. By default, PrimeSim HSPICE advanced analog analyses set the internal voltage source value to V. The value of Vp is lower than V, depending on the internal impedance and the network's input impedance. With the alternate definition, the internal voltage source value is adjusted to 2*V, so that Vp=V when the Port element's impedance matches the network input impedance. The actual value of Vp still depends on the port and network impedances.	port_element_voltage_matchload
rcxt_divider	Defines the hierarchy delimiter in the active nodes file in RCXT format.	rcxt_divider /
skip_nrd_nrs	Directs PrimeSim HSPICE advanced analog analyses to consider transistors with matching geometries (except for NRD and NRS) as identical for pre-characterization purposes.	skip_nrd_nrs
unit_atto	Activates detection of the "atto." unit. Otherwise, PrimeSim HSPICE advanced analog analyses assumes that "a" represents "amperes."	unit_atto
v_supply	Changes the default voltage supply range for characterization.	v_supply 3
wildcard_left_range	Begins range expression.	wildcard_left_range [
wildcard_match_all	Matches any group of characters.	wildcard_match_all *

Keyword	Description	Example
wildcard_match_one	Matches any single character.	wildcard_match_ one ?
wildcard_right_range	Ends range expression.	wildcard_right_ range]

Note:

For more information about wildcards, see [Using Wildcards](#).

This section covers the following topics:

- [Inserting Comments in a .hspicerrf File](#)

Inserting Comments in a .hspicerrf File

To insert comments into your `.hspicerrf` file, include a number sign character (#) as the first character in a line. For example, this configuration file shows how to use comments in a `.hspicerrf` file:

```
# sample configuration file
# the next line of code changes the delimiter
# for subcircuit hierarchies from "," to "^"
hier_delimiter ^
# the next line of code matches any groups of "*" characters
wildcard_match_all *
# the next line of code matches one "?" character
wildcard_match_one ?
# the next line of code begins the range expression with
# the "[" character
wildcard_left_range [
# the next line of code ends the range expression with
# the "]" character
wildcard_right_range ]
```

Using Wildcards

You can use wildcards to match node names. PrimeSim HSPICE advanced analog uses wildcards somewhat differently than standard PrimeSim HSPICE.

Before you use wildcards, you must define the wildcard configuration in a `.hspicerrf` file. For example, you can define the following wildcards in the `.hspicerrf` file:

```
file .hspicerrf
wildcard_match_one      ?
wildcard_match_all      *
wildcard_left_range     [
wildcard_right_range    ]
```

The `.PRINT`, `.PROBE`, `.LPRINT`, and `.CHECK` statements support wildcards in PrimeSim HSPICE.

For more information about using wildcards in an PrimeSim HSPICE configuration file, see Using Wildcards in PRINT and PROBE Statements in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Limiting Output Data Size

For multi-million transistor simulations, an unrestricted waveform file can grow several gigabytes in size. The file becomes unreadable in some waveform viewers, and requires excessive space on the hard drive.

This section describes options that limit the number of nodes output to the waveform file, to reduce the file size. PrimeSim HSPICE supports the following options to control the output:

Control Option	Description
<code>.OPTION SIM_POSTTOP</code>	Use this option to limit the data written to your waveform file to data from only the top <code>n</code> level nodes. This option outputs instances up to <code>n</code> levels deep. Note: To enable the waveform display interface, you also need the <code>POST</code> option.
<code>.OPTION SIM_POSTSKIP</code>	Use this option to have the <code>SIM_POSTTOP</code> option skip any instances and their children that the <code>subckt_definition</code> defines.
<code>.OPTION SIM_POSTAT</code>	Use this option to limit the waveform output to only the nodes in the specified subcircuit instance. This option can operate in conjunction with the <code>SIM_POSTTOP</code> option and when present, has precedence over the <code>SIM_POSTSKIP</code> option.
<code>.OPTION SIM_POSTDOWN</code>	Use this option to include an instance and all children of that instance in the output. This option can operate in conjunction with the <code>SIM_POSTTOP</code> option and when present, has precedence over the <code>SIM_POSTSKIP</code> option.
<code>.OPTION SIM_POSTSCOPE</code>	Use this option to specify the signal types to probe from within a scope.

Generating Measurement Output Files

You can make measurements with the `.MEASURE` statement when using PrimeSim HSPICE advanced analog analyses.

The results of the `.MEASURE` statements appear in a file with one of the following filename extensions:

- `.mb#` for measurements in HB analysis
- `.mp#` for measurements in HBNOISE and SNNOISE analysis
- `.mpn#` for measurements from PHASENOISE analysis when using HB to obtain the steady state solution
- `.msnpn#` for measurements from PHASENOISE analysis when using SN to obtain the steady state solution
- `.msnptn#` for measurements in PTDNOISE analysis

For more information about `.MEASURE` statements, see PrimeSim HSPICE Netlist Commands in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Advanced Analog Demonstration Input Files

The following is a listing of the shipped demonstration files for illustrating the PrimeSim HSPICE advanced analog analyses functionality. All of these example files are available at:

`$installdir/demo/hspice/rf_examples`

File Name	Location: \$installdir/demo/hspice/rf_examples/	Detailed Demo Description
acpr.sp	Envelope simulation example	See Envelope Simulation in this guide.
bjt.inc	Transistor model library used by osc.sp	
clockbuf4.sp	Transient noise analysis example	See Example: Measure TIE Jitter and Jitter Spectrum in this guide.
clockbuf4sn.sp	Shooting Newton-based analysis of clock buffer circuit example	See Shooting Newton Analysis — Tutorial Example in this guide.
cmos49_model.inc	Transistor model library used by example circuits	
cmos90nmWflicker.lib	Transistor model library used by phasefreqdet.sp	
gpsvco.sp	Oscillator and Phase Noise analysis example	See Using HBOSC for a CMOS GPS VCO in this guide.

File Name	Location: \$installdir/demo/hspice/rf_examples/	Detailed Demo Description
gsmlna.sp	LNA Linear analysis example	See .LIN Analysis for a NMOS Low Noise Amplifier in the <i>PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis</i> .
gsmlnaIP3_A.sp	3rd order intercept point example	See Using HB Analysis for a Low Noise Amplifier in this guide.
mix_hb.sp	Mixer HB analysis example	See Two-Tone HB Analysis in this guide.
mix_hbac.sp	Mixer HBAC analysis example	See HBAC Analysis of a Simple Mixer in this guide.
mix_snac.sp	Mixer Shooting Newton AC example	See SNAC Analysis Example in this guide.
mix_tran.sp	Mixer transient analysis example	See Mixer Example: Transient Analysis in this guide.
osc.sp	Oscillator tuning curve and phase noise analysis example	See HBOSC Analysis: Colpitts Oscillator in this guide.
pa.sp	Power amplifier HB analysis example	See Using HB Analysis for a Power Amplifier in this guide.
pfdcGain.sp	Shooting Newton analysis example	See Shooting Newton Analysis in this guide.
phasefreqdet.sp	Shooting Newton and noise analysis example	See Shooting Newton Noise Analysis in this guide.
rcInvPTDnoise.sp	Periodic Time-Dependent Noise analysis (.PTDNOISE) example	See Periodic Time-Dependent Noise Analysis in this guide.
ringoscSN.sp	Shooting Newton and Phase Noise analysis example	See Ring Oscillator Example in this guide.
tsmc018.m	Transistor model library used by ringoscSN.sp	

To run a demonstration file, see Running the Demonstration Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

2

Use of Example Syntax

To copy and paste proven syntax, use the demonstration files shipped with your installation of PrimeSim HSPICE (see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists*). Attempting to copy and paste from the book or help documentation may present unexpected results, as text used in formatting may include hidden characters and white space for visual clarity.

Part 2: Periodic Steady-State Analysis

Describe the usage of harmonic balance analysis for frequency-driven and steady-state analysis and PrimeSim HSPICE advanced analog steady-state time domain analysis based on an advanced Shooting-Newton algorithm.

The following chapters/topics are included in this Part:

- [Chapter 3, Harmonic Balance Analysis](#)
- [Chapter 4, Shooting Newton Analysis](#)

3

Harmonic Balance Analysis

Describes how to use harmonic balance analysis for frequency-driven and steady-state analysis.

This chapter provides an introduction to the Harmonic Balance (`.HB`) analysis command. You can use steady-state analysis on a circuit if it contains only DC and periodic sources. These analyses assume that all “start-up” transients have completely died out with only the steady-state response remaining. These analyses treat sources that are not periodic or DC as zero-valued.

The following sections discuss these topics:

- [Tutorial Examples Harmonic Balance Analysis](#)
- [Harmonic Balance Analysis](#)
- [Troubleshooting Convergence Issues with Harmonic Balance Analysis](#)
- [Phase Differences Between HB and SIN Sources](#)
- [Optimization](#)
- [Optimizing with HB Measurements](#)
- [References](#)

Tutorial Examples Harmonic Balance Analysis

The following sections present these tutorial examples:

- [Example 1: Using HB Analysis for a Power Amplifier](#)
- [Example 2: Using HB Analysis for a Low Noise Amplifier](#)

Example 1: Using HB Analysis for a Power Amplifier

The `.HB` command computes periodic steady-state solutions of circuits. This analysis uses the Harmonic Balance (HB) technique for computing such solutions in the frequency

domain. The circuit can be driven by a voltage, power, or current source, or it may be an autonomous oscillator. The HB algorithm represents the circuit's voltage and current waveforms as a Fourier series; that is, a series of sinusoidal waveforms.

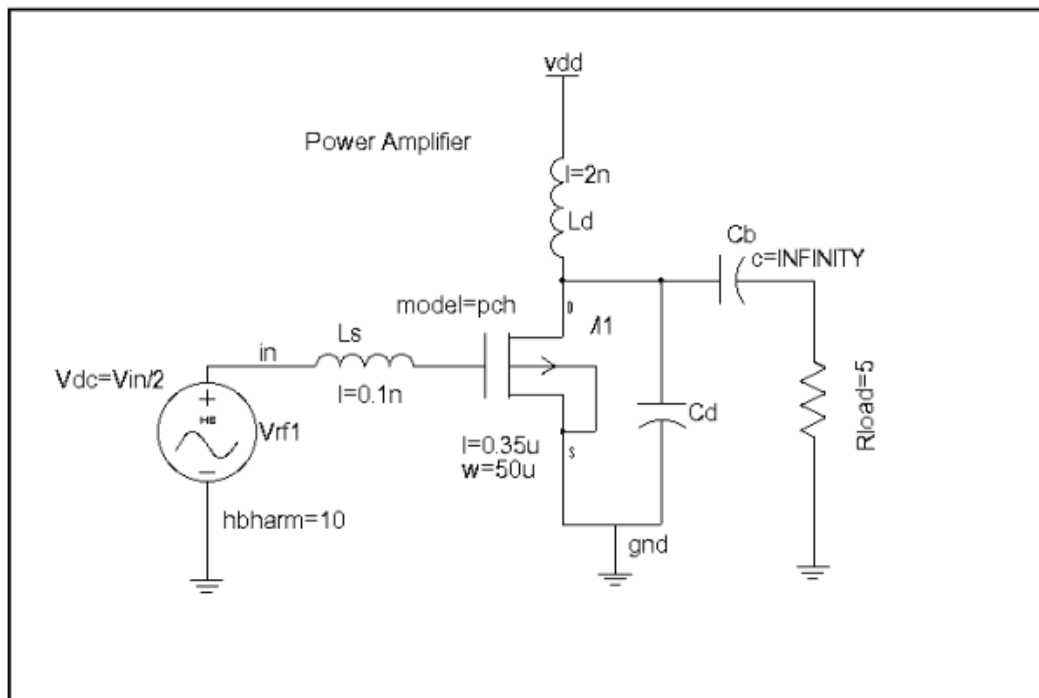
To set up a periodic steady-state analysis, the PrimeSim HSPICE input netlist must contain:

- An `.HB` command to activate the analysis. The `.HB` command specifies the base frequency (or frequencies, also called tones) for the analysis, and the number of harmonics to use for each tone. The `.HB` command can specify base tones so that the circuit solution is represented as a multi-dimensional Fourier series. The number of terms in the series are determined by the number of harmonics; more harmonics result in higher accuracy, but also longer simulation times and higher memory usage.
- One or more signal sources for driving the circuit in HB analysis, if the circuit is driven. In the case of autonomous oscillator analysis, no signal source is required. Signal sources are specified using the HB keyword in the voltage or current source syntax. Power sources are specified by setting the power switch on the voltage/current sources to 1; in this case, the source value is treated as a power value in Watts, instead of a voltage or current.

Optionally, the netlist can also contain a set of control options for optimizing the HB analysis performance.

The following example shows how to set up an HB analysis on an NMOS Class C Power Amplifier. The example compares transient analysis results to the HB results.

Figure 1 Power Amplifier



The following netlist performs both a transient and an HB analysis of the amplifier driven by a sinusoidal input waveform. The `accurate` option is set to ensure sufficient number of time points for comparison with HB. This example is included with the PrimeSim HSPICE advanced analog analyses distribution as `pa.sp` and is available in directory `$installdir/demo/hspice/rf_examples/`.

```
.options POST accurate
.param f0=950e6 PI=3.1415926 Ld=2e-9 Rload=5 Vin=3.0
.param Lin=0.1n Vdd=2 Cd='1.0/(4*PI*PI*f0*f0*Ld)'
M1 drain gt 0 0 CMOSN L=0.35u W=50u AS=100p AD=100p
PS=104u PD=104u M=80
Ls in gt Lin $ gate tuning
Ld drain vdd Ld $ drain tuning
Cd drain 0 Cd
Cb drain out INFINITY $ DC block
Rload out 0 Rload
Vdd vdd 0 DC Vdd
Vrf1 in 0 DC 'Vin/2.0'
+ SIN ('Vin/2' 'Vin/2' 'f0' 0 0 90)
+ HB 'Vin/2' 0.0 1 1
.hb tones=f0 nharms=10
.tran 10p 10n
.probe hb p(Rload)
.probe tran p(Rload)
```

```
.include cmos49_model.inc  
.end
```

An HB analysis uses the following:

- An `.HB` command:

```
.hb tones=f0 nharms=10
```

For a single-tone analysis with base frequency 950 MHz and 10 harmonics.

- The HB source in `Vrf1`:

```
HB 'Vin/2' 0.0 1 1.
```

This creates a sinusoidal waveform matching the transient analysis one. The amplitude is $V_{in}/2 = 1.5$ V, and it applies to the first harmonic of the first tone, 950 MHz.

- A `.PROBE` command for plotting the output power:

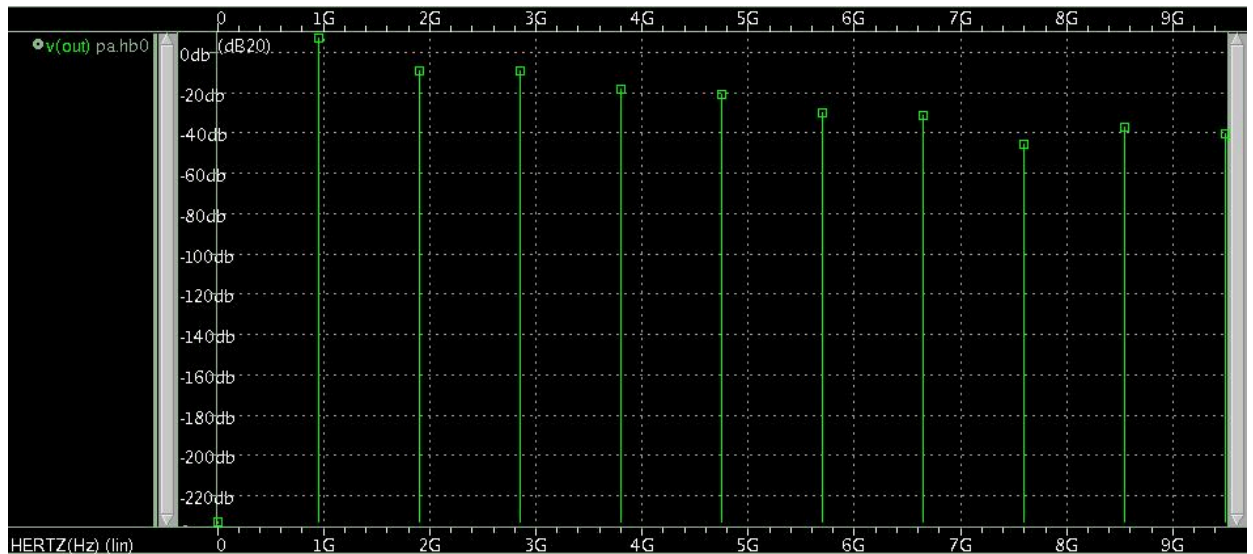
```
.probe hb p(Rload)
```

To run this netlist, type the following command:

```
hspice pa.sp
```

This produces two output files named `pa.tr0` and `pa.hb0`, containing the transient and HB output, respectively. To view and compare the output:

1. Type `wv` at the prompt to invoke WaveView.
2. Click the **Open Waveform** icon and select the `pa.tr0` and `pa.hb0` files from the Open: Waveform Files dialog box.
3. Select the `v(out)` signal from `pa.hb0` in the signal browser. Double-click on the signal name, or drag and drop the signal into the waveview. The histogram shows lines at 950MHz and multiples thereof, up to 9.5GHz.

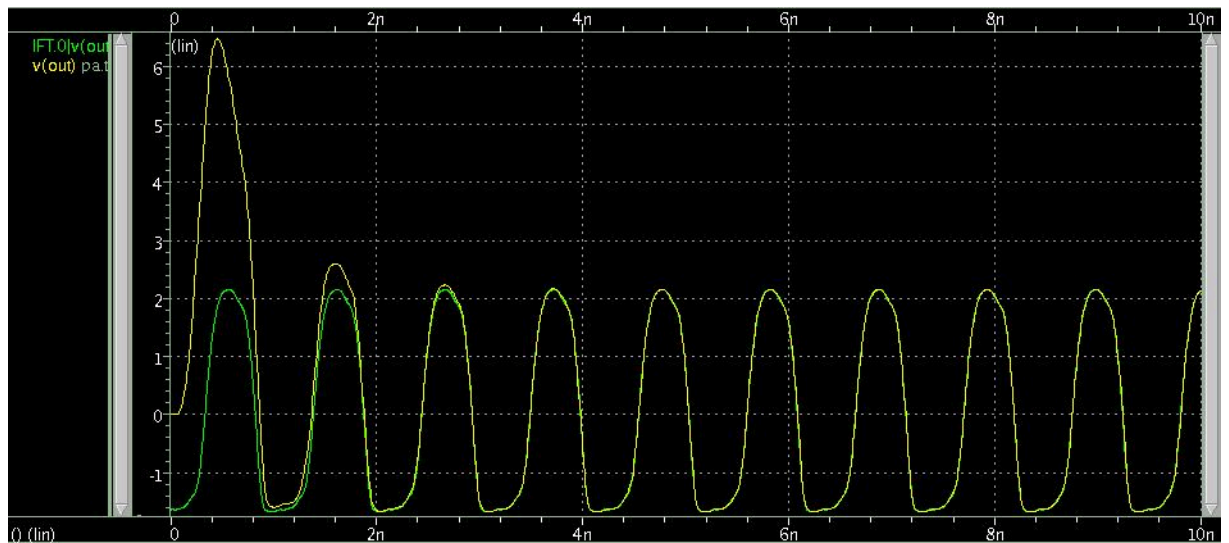


4. In the waveview, right click in the name area of the panel containing the signal v(out), left-click on the waveform label for v(out) from the `pa.hb0` file. From the **Panel** menu, choose **Signal 'v(out)' > To Time-Domain**.
5. In the Convert to Time Domain window, change the X-End (sec) value to 10n.
6. Click **OK** to accept the settings.
7. The new waveview shows a new time domain waveform named IFT.0|v(out).



8. Select the v(out) signal from `pa.tr0` in the signal browser. Drag and drop the signal into the waveview containing IFT.0|v(out). This should overlay the IFT.0|v(out) and

v(out) signals in the same waveview. Zoom into the transitions to see the slight differences between the waveforms.

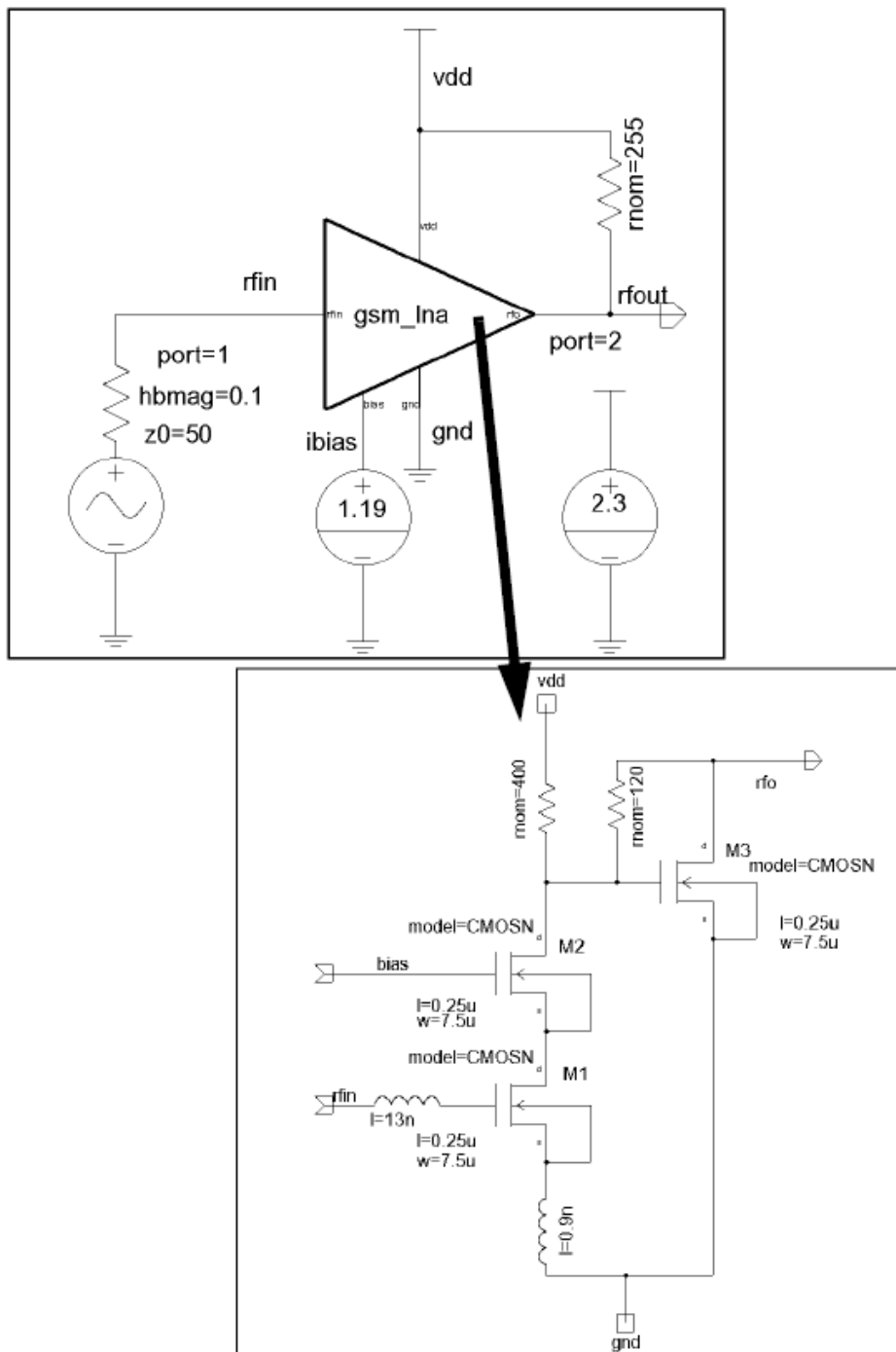


Example 2: Using HB Analysis for a Low Noise Amplifier

This example performs a simulation on a Low Noise Amplifier circuit using two closely-spaced steady-state tones to study the compression and the third-order distortion properties of the amplifier. The example file `gsm1naIP3_A.sp` is located at:
`$installdir/demo/hspice/rf_examples/`.

See [Figure 2](#) for the schematic view.

Figure 2 Schematic showing instantiation for Low Noise Amplifier



Example 1 Low Noise Amplifier Netlist: Part 1

```

** NMOS 0.25um Cascode LNA for GSM applications
** setup for s-parameter and noise parameter measurements
.temp 27
.options post
.param Vdd=2.3
**
** Cascode LNA tuned for operation near 1 GHz
**
M1 _n4 _n3 _n5 _n5 CMOSN l=0.25u w=7.5u
+ as=15p ad=15p ps=19u pd=19u m=80
M2 _n6 _n1 _n4 _n4 CMOSN l=0.25u w=7.5u
+ as=15p ad=15p ps=19u pd=19u m=80
M3 rfo _n6 gnd gnd CMOSN l=0.25u w=7.5u
+ as=15p ad=15p ps=19u pd=19u m=40
r1 _vdd _n6 400
l1 _n5 gnd l=0.9nH
l2 rfin _n3 l=13nH
lchk rfin rfinb INFINITY
cblk rfin rfind INFINITY
vzb _n1 gnd dc=1.19 $ bias for common base device
vinb rfinb gnd dc=0.595
vzdd _vdd gnd dc=Vdd
rfb rfo _n6 120 $ feedback
**
**
** Two-tone input source (DC blocked at this time)
**
Vin rfind gnd dc=0 power=1 z0=50 $ 50 Ohm src
+ HB Pin 0 1 1 $ tone 1
+ HB Pin 0 1 2 $ tone 2
Rload rfo _vdd R=255
**
** HB test bench to measure IP3 and IP2
.HB tones=900MEG, 910MEG nharms=11 11 intmodmax=7
+ SWEEP Pin dec 10 1e-8 1e-3
.print HB P(Rload) P(Rload) [1,0] P(Rload) [2,0] P(Rload) [2,-1]
.probe HB P(Rload) P(Rload) [1,0] P(Rload) [2,0] P(Rload) [2,-1]
**
**

```

Example 2 Low Noise Amplifier Netlist: Part 2

```

*
.meas hb fund1_mag max v(rfo) [1,0]
.meas hb fund2_mag max v(rfo) [0,1]
.meas hb harm1_mag max v(rfo) [2,-1]
.meas hb harm2_mag max v(rfo) [-1,2]
.meas hb fund1_param ='20*log10(fund1_mag) '
.meas hb fund2_param ='20*log10(fund2_mag) '
.meas hb harm1_param ='20*log10(harm1_mag) '
.meas hb harm2_param ='20*log10(harm2_mag) '

```

```
.meas hb IIP31 param ='fund1+(fund1-harm1)/2'  
.meas hb IIP32 param ='fund1+(fund2-harm2)/2'  
*
```

First, a voltage source element is used as a two-tone power source by setting the power flag. A source impedance of 50 ohms is specified. The `HB` keyword is used to identify the amplitude (interpreted as Watts with the power flag set to 1), phase, harmonic index, and tone index for each tone.

```
Vin rfind gnd dc=0 power=1 z0=50 $ 50 Ohm src  
+ HB Pin 0 1 1 $ tone 1  
+ HB Pin 0 1 2 $ tone 2
```

Second, the `.HB` command designates the frequencies of the two tones and establishes the power sweep. The `intmodmax` parameter has been set to 7 to include intermodulation harmonic content up to 7th order effects.

```
.HB tones=900MEG,910MEG nharms=11 intmodmax=7  
+ SWEEP Pin dec 10 1e-8 1e-3
```

Last, the PrimeSim HSPICE advanced analog analyses ability to specify a specific harmonic term is used in the `.PRINT` and `.PROBE` statements to pull out the signals of particular interest. Notice the three different formats:

1. The following reference dumps a complete spectrum in RMS Watts for the power across resistor `Rload`.

```
.PRINT HB P(Rload)
```

2. The following reference selectively dumps the power in resistor `Rload` at the first harmonic of the 1st tone.

```
.PRINT HB P(Rload) [1,0]
```

3. The following reference selectively dumps the power in resistor `Rload` at the 3rd intermodulation product frequency (890 MHz).

```
.PRINT HB P(Rload) [2,-1]
```

To run this simulation, type the following at the command line:

```
hspice -i gsmlnaIP3_A.sp -o
```

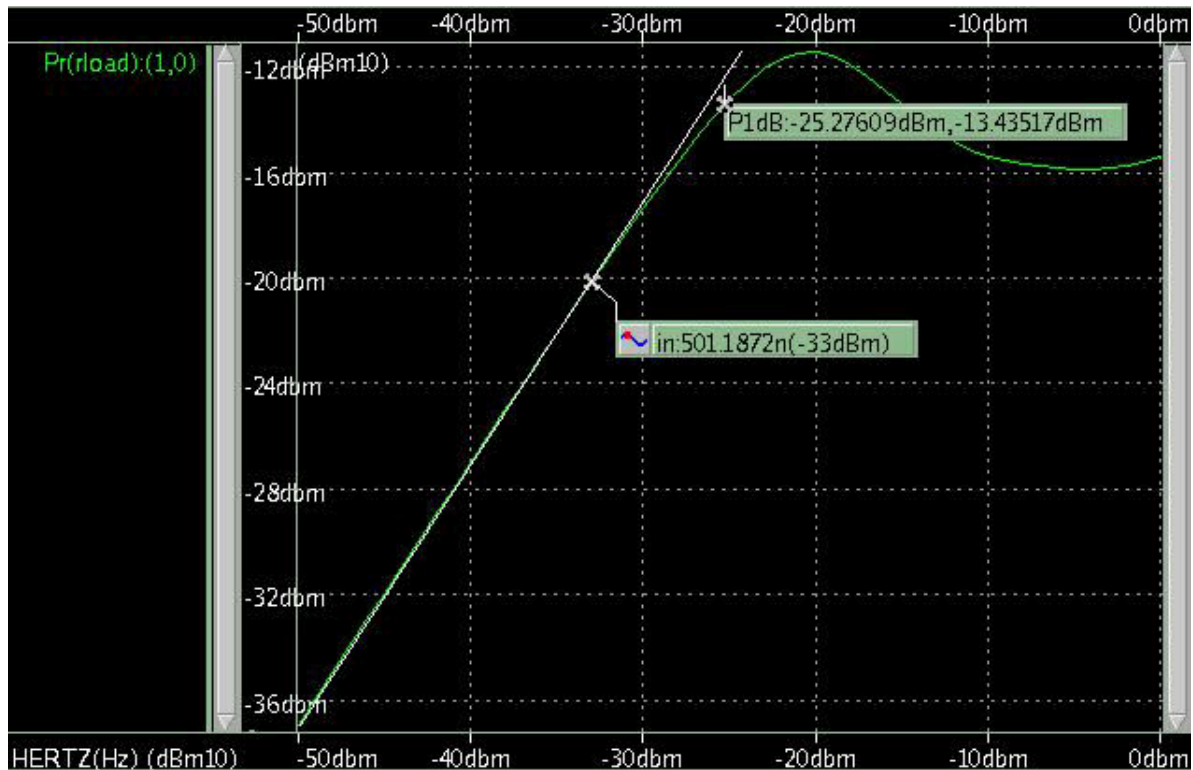
Viewing Results Using WaveView

For this analysis, the `.PRINT` statement will generate a `gsmlnaIP3_A.printhb0` file. Assume you want to find the output power through the load resistor at the first tone, when the input power is 0.1mW.

To view the file:

1. Type **vv** at the prompt to invoke WaveView.
2. Click the **Open Waveform** icon and select the `gsmlnaIP3_A.hb0` file.
3. Select the signal `Pr(rload) [1,0]` in the signal browser. Drag and drop the signal into the waveview. The X-axis is the input power and the Y-axis is the output power. The output power on the Y-axis is displayed in dBm but the input power on the X-axis is displayed in watts. To make the 1dB compression point measurement, it is necessary to change the X-axis scale to dBm.
4. To change the X-axis scale to dBm, right-click the X-axis and select **Attributes** from the menu. At the bottom right of the X-axis Attributes menu, select **dBm10** from the type pull-down menu and click **Apply**. Close the menu by clicking **Close**.
5. To measure the 1dB compression point of the amplifier, click **Measurement Tools** from the Utilities section of the **Waveform** tab. In the Measurement Tools window, the available measurements are listed by category on the left side of the window. Scroll down to the RF measurements and select **dBCP**. Click **OK** to place a dynamic meter in the waveview.
6. Move the dynamic meter near the signal, the meter will show the 1dB compression point, the linear gain of the amplifier, and the input power where the 1dB compression point is measured. For best results, the asymptotic line drawn by the dynamic meter should overlay the linear portion of the amplifier power curve.

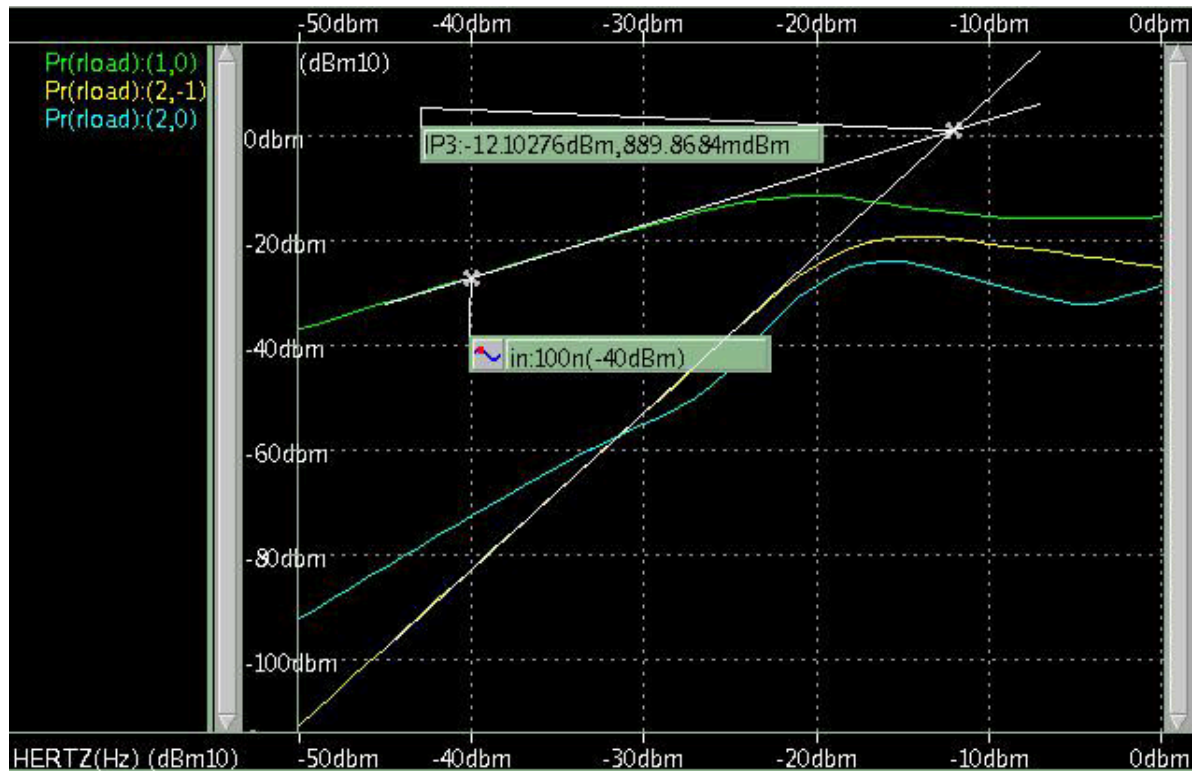
Figure 3 1dB Compression Point



7. Click the **Waveview** icon in the Add section of the **Waveform** tab (or press CTRL+w) to open a new waveview.
8. Select the signals `Pr(rload) [1, 0]`, `Pr(rload) [2, -1]`, and `Pr(rload) [2, 0]` in the signal browser. Drag and drop the signals in the waveview. Change the X-axis scale to dBm as described in step 4.
9. The 3rd order intercept point is also measured by using the measurement tool. In the measurement section, select **IP3/SFDR**. Click **OK**; this will place a dynamic meter in the waveview.
10. Move the dynamic meter near the `Pr(rload) [1, 0]` signal, the meter will show the 3rd order intercept point and the input power where the 3rd order intercept point is

measured. For best results, the asymptotic lines drawn by the dynamic meter should overlay the linear portion of the $\text{Pr}(\text{rload})[1,0]$ and $\text{Pr}(\text{rload})[2,-1]$ signals.

Figure 4 3^{rd} Order Intercept Point



Harmonic Balance Analysis

Harmonic balance analysis (HB) is a frequency-domain, steady-state analysis technique. Use this analysis technique on a circuit excitable by DC and periodic sources of one or more fundamental tones. The solution that HB finds is a set of phasors for each harmonic signal in the circuit. You can think of this solution as a set of truncated Fourier series. PrimeSim HSPICE advanced analog analyses allows you to specify the solution spectrum to use in an analysis. HB analysis then finds the set of phasors at these frequencies that describes the circuit response. The result is a set of complex-valued Fourier series coefficients that represent the waveforms at each node in the circuit.

Evaluation of linear circuit elements takes place in the frequency domain, while evaluation of nonlinear elements occurs in the time domain. The nonlinear response then transforms to the frequency domain where it adds to (or “balances” with) the linear response. The resulting composite response satisfies KCL and KVL (Kirchoff’s current and voltage laws) with the circuit solution.

Typical applications include performing intermodulation analysis, oscillator analysis, and gain compression analysis, on amplifiers and mixers. HB analysis also serves as a starting point for periodic AC and noise analyses.

For more information on control options, see `.HB` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [Features Supported](#)
- [Harmonic Balance Equations](#)
- [Steady-State HB Sources](#)
- [Harmonic Balance Output Measurements](#)
- [HB `.PRINT` and `.PROBE` Output Syntax](#)
- [HB Output Data Files and Examples](#)
- [HB Error and Warning Messages](#)
- [Calculating Power Measurements After HB Analyses](#)
- [Calculations for Time-Domain Output](#)
- [Using `.MEASURE` with `.HB` Analyses](#)

Features Supported

HB supports the following features:

- All existing PrimeSim HSPICE advanced analog analyses models.
- An unlimited number of independent input tones.
- Sources with multiple HB specifications.
- `SIN`, `PULSE`, `VMRF`, and `PWL` sources with `TRANFORHB=1`.

Prerequisites and Limitations

The following prerequisites and limitations apply to HB:

- Requires one `.HB` statement.
- Treats sources without a DC, HB, or `TRANFORHB` description as a zero-value for HB, unless the sources have a transient description, in which case, the `time=0` value becomes a DC value.

Harmonic Balance Equations

We can write Kirchoff's current law in the time domain as:

Equation 1

$$f(v, t) = i(v(t)) + \frac{d}{dt} q(v(t)) + \int_{-\infty}^t y(t-\tau) v(\tau) d\tau + i_s(t) = 0$$

- $i(v(t))$ represents the resistive currents from nonlinear devices
- q represents the charges from nonlinear devices
- y represents the admittance of the linear devices in the circuit
- i_s represents the vector of independent current sources
- v is a variable that represents the circuit unknowns, both node voltages and branch currents, and $f(v, t)$ is an error term that goes to zero to satisfy Kirchoff's current law.

Transforming this equation to the frequency domain results in:

Equation 2

$$F(V) = I(V) + \Omega Q(V) + Y(\omega)V + I_s = 0$$

Note:

Time-differentiation transforms to multiplication by $j\omega$ terms (which make up the Ω matrix) in the frequency domain. The convolution integral transforms to a simple multiplication. The Y matrix is the circuit's modified nodal admittance matrix.

All terms above are vectors that represent the circuit response at each analysis frequency.

The following equation shows the vector of (complex-valued) unknowns in the frequency domain for a circuit with K analysis frequencies and N unknowns.

Equation 3

$$V = [V_{(1,0)} \ V_{(1,1)} \ \dots \ V_{(1,K-1)} \ V_{(2,0)} \ \dots \ V_{(N,K-1)}]$$

PrimeSim HSPICE advanced analog analyses finds the unknown vector (V), to satisfy the system of nonlinear equations, in the equation above. The Newton-Raphson technique uses either a direct solver to factor the Jacobian matrix, or an indirect solver. The PrimeSim HSPICE advanced analog analyses indirect solver is in the Generalized Minimum Residual (GMRES) Solver, a Krylov technique, and uses a matrix-implicit algorithm.

Steady-State HB Sources

The fundamental frequencies used with harmonic balance analysis are specified with the `.HB TONES` command. These frequencies can then be referenced by their integer indices when specifying steady-state signal sources. For example, the `.HB` specification given by the following line:

```
.HB TONES=1900MEG,1910MEG INTMODMAX=5
```

This specifies two fundamental frequencies: $f[tone = 1] = 1.9GHz$ and $f[tone = 2] = 1.91GHz$. Their mixing product at 10 MHz can then be referenced using indices as $|f[2]-f[1]|$, while their 3rd order intermodulation product at 1.89 GHz can be referenced as $|2f[1]-f[2]|$.

Steady-state voltage and current sources are identified with the `HB` keyword according to `[HB [mag [phase [harm [tone [mod harm [modtone]]]]]]]`

The source is mathematically equivalent to a cosine signal source that follows the equation

$$A \cos(\omega t + \phi)$$

where:

$$A = mag$$

$$\omega = 2\pi |harm \cdot f[tone] + modharm \cdot f[modtone]|$$

$$\phi = \frac{\pi}{180} \cdot phase$$

Values for `tone` and `modtone` (an optional modulating tone) must be non-negative integers that specify index values for the frequencies specified with the `.HB TONES` command. Values for `harm` (harmonic) and `modharm` (modulating tone harmonic) must be integers (negative values are valid) that specify harmonic indices.

Examples

Example 3 The following example is a 1.0 Volt (peak) steady-state cosine voltage source, which is at the fundamental HB frequency with zero phase and with a zero volt DC value:

```
Vsrc in gnd DC 0 HB 1.0 0 1 1
```

Example 4 The following example is a steady-state cosine power source with 1.0mW available power, which is implemented with a Norton equivalent circuit and a 50 ohm input impedance:

```
Isrc in gnd HB 1.0e-3 0 1 1 power=1 z0=50
```

Example 5 Five series voltage sources sum to produce a stimulus of five equally spaced frequencies at and above 2.44 GHz using *modharm* and *modtone* parameters. These are commensurate tones (an integer relation exists); therefore, you only need to specify two tones when invoking the HB analysis.

```
.param Vin=1.0
.param f0=2440MEG
.param deltaf=312.5K
.param fcenter='f0 + 2.0*deltaf'
Vrfa    in    ina    HB    'Vin'    0    1    1                $ 2.440625    GHz
Vrfb    ina    inb    HB    'Vin'    0    1    1    -1    2    $ 2.4403125    GHz
Vrfc    inb    inc    HB    'Vin'    0    1    1    -2    2    $ 2.440    GHz
Vrfd    inc    ind    HB    'Vin'    0    1    1    +1    2    $ 2.4409375    GHz
Vrfe    ind    gnd    HB    'Vin'    0    1    1    +2    2    $ 2.44125    GHz
.HB tones=fcenter,deltaf intmodmax=5
```

Harmonic Balance Output Measurements

This section explains the harmonic balance output measurements you receive after PrimeSim HSPICE runs an HB simulation.

The HB cosine sources interpret in real/imaginary and polar formats according to [Equation 4](#):

Equation 4

$$\begin{aligned}
 v(t) &= A \cos(\alpha t + \phi) = \operatorname{Re}\{Ae^{j(\alpha t + \phi)}\} = \operatorname{Re}\{Ae^{j\phi}e^{j\omega t}\} \\
 &= \operatorname{Re}\{Ae^{j\phi}[\cos(\alpha t) + j\sin(\alpha t)]\} \\
 &= \operatorname{Re}\{[V_R + jV_I][\cos(\alpha t) + j\sin(\alpha t)]\} \\
 &= V_R \cos(\alpha t) - V_I \sin(\alpha t) \\
 &= A \cos(\phi) \cos(\alpha t) - A \sin(\phi) \sin(\alpha t)
 \end{aligned}$$

This equation relates real/imaginary and polar formats with the standard convention:

Equation 5

$$\begin{aligned}
 V_R + jV_I &= Ae^{j\phi} \\
 V_R &= A \cos(\phi) \\
 V_I &= A \sin(\phi) \\
 A &= \sqrt{V_R^2 + V_I^2} \\
 \tan \phi &= \frac{V_I}{V_R}
 \end{aligned}$$

The result of HB analysis is a complex voltage (current) spectrum at each circuit node (or specified branch). Let $a[i]$ be the real part and $b[i]$ be the imaginary part of the complex

voltage at the `ith` frequency index. The Fourier series expansion gives the conversion to a steady-state time-domain waveform as in [Equation 6](#):

Equation 6

$$\begin{aligned} v(t) = & d[0] + d[1]*\cos(2\pi f[1]*t) - b[1]*\sin(2\pi f[1]*t) \\ & + d[2]*\cos(2\pi f[2]*t) - b[2]*\sin(2\pi f[2]*t) \\ & + d[3]*\cos(2\pi f[3]*t) - b[3]*\sin(2\pi f[3]*t) \\ & + \dots \\ & + d[N]*\cos(2\pi f[N]*t) - b[N]*\sin(2\pi f[N]*t) \end{aligned}$$

Where:

- `v(t)` is the resulting time domain waveform.
- `N+1` is the total number of harmonics (including DC) in the frequency domain spectrum of the `*.hb0` file (the zero-th data point represents DC).
- `a[i]` is the real component at the `ith` frequency
- `b[i]` is the imaginary component at the `ith` frequency
- `f[i]` is the `ith` frequency value (with `i=0` representing the zero frequency DC term). These frequencies do not need a harmonic relationship.

This frequency domain (Fourier coefficient) representation converts to a steady-state time domain waveform when you use the `.PRINT` or `.PROBE HBTRAN` output option or you invoke the **To Time Domain** function on complex spectra within WaveView.

HB .PRINT and .PROBE Output Syntax

This section describes the syntax for the HB `.PRINT` and `.PROBE` statements.

```
.PRINT HB TYPE(NODES|ELEMENTS) [INDICES]
.PROBE HB TYPE(NODES|ELEMENTS) [INDICES]
```

Parameter	Description
TYPE	Specifies a harmonic type node or element. TYPE can be one of the following: • Voltage type: - V = voltage magnitude and phase in degrees - VR = real component - VI = imaginary component - VM = magnitude - VP - Phase in degrees - VPD - Phase in degrees - VPR - Phase in radians - VDB - dB units - VDBM - dB relative to 1 mV • Current type: - I = current magnitude and phase in degrees - IR = real component - II = imaginary component - IM = magnitude - IP - Phase in degrees - IPD - Phase in degrees - IPR - Phase in radians - IDB - dB units - IDBM - dB relative to 1 mV • Power type: - P = Power in Watts or Pdbm = Power in dBm • Frequency type: 'HERTZ[i]' (for single tone analysis), \ 'HERTZ[i][j]' (for two-tone analysis), 'HERTZ[i][j][k]' (for 3-tone analysis), etc. You must specify the harmonic index integer for the HERTZ keyword. The frequency of the specified harmonics results.
INDICES	Index to tones in the form [n1, n2,..., nN], where nj is the index of the HB tone and the HB statement contains N tones. Wildcards are illegal if you use the INDICES keyword.
TYPE can be one of the following: • Voltage type – a single node name (n1), or a pair of node names, (n1,n2) • Current type – an element name (elemname) • Power type – a resistor (resistorname) or port (portname) element name.	

Use the following output syntax to transform HB data into the time domain:

```
.PRINT hbtran ov1 [ov2...]
.PROBE hbtran ov1 [ov2...]
```

Where `ov1 ...` are the output variables to print or probe.

Outputting Phase Noise Source as ASCII Data Files Using *.printpn0

HB phase noise and phase noise analyses can output simulation results as ASCII data in *.printpn0 files for HBOSC and HBNOISE. Extend the E- and G- voltage-controlled source syntax to make the phase noise data in ASCII phase noise files double as input for specifying behavioral noise sources.

Usage Model

The syntax for the voltage controlled voltage (E) or current (G) source is as follows:

```
Exxx node1 node2 noise file='filename' [mname='measname']  
Gxxx node1 node2 noise file='filename' [mname='measname']
```

Where `file='filename'` is the name of the ASCII phase noise data file. The file '`design.printpn0`', name is typically designates an .HBOSC phase noise analysis or .HBNOISE analysis output file.

Use `mname='measname'` to select the appropriate noise measurement name from the *.printpn0 file.

`measname` can be one of the following:

- `NLP_L(f)` - selects the `nlp_L(f)` phase noise data in units of dBc/Hz
- `PAC_L(f)` - selects the `pac_l(f)` phase noise data in units of dBc/Hz
- `ONoise` - selects the `onoise` data based on .HBNOISE or .SNNOISE analysis

HB Output Data Files and Examples

The results of an HB analysis are complex spectral components at each frequency point. The `a[i]` is the real part, and `b[i]` is the imaginary part of the complex voltage at frequency index `i`. The conversion to a steady-state time-domain is then given by the Fourier series expansion.

An HB analysis produces these output data files:

- Output from the `.PRINT HB` statement is written to a `.printhb#` file.
 - The header contains the large signal fundamental frequencies.
 - The columns of data are labeled as `HERTZ`, followed by frequency indices, and then the output variable names.
 - The sum of the frequency indices, multiplied by the corresponding fundamental frequencies, add up to the frequency in the first column.

- Output from the `.PROBE HB` statement is written to a `.hb#` file. It is in the same format as the PrimeSim HSPICE transient analysis `.tr#` file. Besides the output waveform, it contains the information of harmonic indices and basic tone frequencies.
- Output from the `.PRINT HBTRAN` statement is written to a printer file. The format is identical to a `.print#` file.
- Output from the `.PROBE HBTRAN` statement is written to a `.hr#` file. The format is identical to a `.tr#` file.
- Reported performance log statistics are written to a `.lis` file.

.HB Output Example

```
.PRINT HB P(rload)      $ RMS power (spectrum)
                        $ dissipated at the rload resistor
.PROBE HB V(n1,v2)      $ Differential voltage (spectrum)
                        $ between the n1,n2 nodes
.PRINT HB VP(out)[1]    $ Phase of voltage at the out
                        $ node, at the fundamental
                        $ frequency
.PROBE HB P(Pout)[2,-1] $ RMS power delivered to the Pout
                        $ port, at third-order intermod
.PRINT HBTRAN V(n1)     $ Voltage at n1 in time domain
.PROBE HBTRAN V(n1, n2) $ Differential voltages between n1
                        $ and n2 node in time domain.
```

HB Error and Warning Messages

The following sections discuss these topics:

- [HB Analysis Error Messages](#)
- [HB Analysis Warning Messages](#)

HB Analysis Error Messages

Following are the error messages issued by PrimeSim HSPICE simulator when using the `.HB` analysis:

File	Description
HB_ERR.1	Harmonic numbers must be positive non-zero.
HB_ERR.2	No <code>.hb</code> frequencies given.
HB_ERR.3	Negative frequency given.
HB_ERR.4	Number of harmonics should be greater than zero.
HB_ERR.5	Different number of tones, <code>nharms</code> .

File	Description
HB_ERR.6	Bad probe node format for oscillator analysis.
HB_ERR.7	Bad format for FSPTS.
HB_ERR.8	Bad .hb keyword.
HB_ERR.9	Tones must be specified for .hb analysis.
HB_ERR.10	Nharms or intrmodmax must be specified for .hb analysis.
HB_ERR.11	Source harmonic out of range.
HB_ERR.12	Source named in the tones list is not defined.
HB_ERR.13	Source named in the tones list does not have TRANFORHB specified.
HB_ERR.14	Source named in the tones list has no transient description.
HB_ERR.15	Source named in the tones list must be HB, SIN, PULSE, PWL, or VMRF.
HB_ERR.16	Tone specification for the source is inconsistent with its frequency.
HB_ERR.17	HB oscillator analysis has reached the NULL solution.
HB_ERR.18	Bad subharms format.
HB_ERR.19	Modtone may not be set to the same value as tone.

HB Analysis Warning Messages

Following are the warning messages issued by PrimeSim HSPICE when using the .HB analysis:

File	Description
HB_WARN.1	.hb multiply defined. Last one will be used.
HB_WARN.2	Tone specified for V/I source not specified in .HB command.
HB_WARN.3	HB convergence not achieved.
HB_WARN.4	Source specifies both HB and transient description. HB description will be used.
HB_WARN.5	Source specifies exponential decay. HB will ignore it.
HB_WARN.6	Source specifies a non-positive frequency.
HB_WARN.7	Source does not fit the HB spectrum.
HB_WARN.8	Source cannot be used with the TRANFORHB option.

File	Description
HB_WARN.9	Frequency not found from transient analysis
HB_WARN.10	.hb/.hbosc will be ignored due to .env/.envosc.

Calculating Power Measurements After HB Analyses

Two types of power measurements are available: dissipated power in resistors and delivered power to port elements.

The following sections discuss these topics:

- [Power Dissipated in a Resistor](#)
- [Power Delivered to a Port Element](#)
- [Examples](#)

Power Dissipated in a Resistor

All power calculations make use of the fundamental phasor power relationship given in the following equation, where voltage V and current I are complex phasors given in peak values (not rms nor peak-to-peak):

Equation 7

$$P_{rms} = \frac{1}{2} \text{Re}\{VI^*\}$$

In the case of a simple resistor, its current and voltage relate to each other according to $V_n = I_n R$. [Equation 8](#) gives the power dissipated in a resistor of (real) value R at frequency index n :

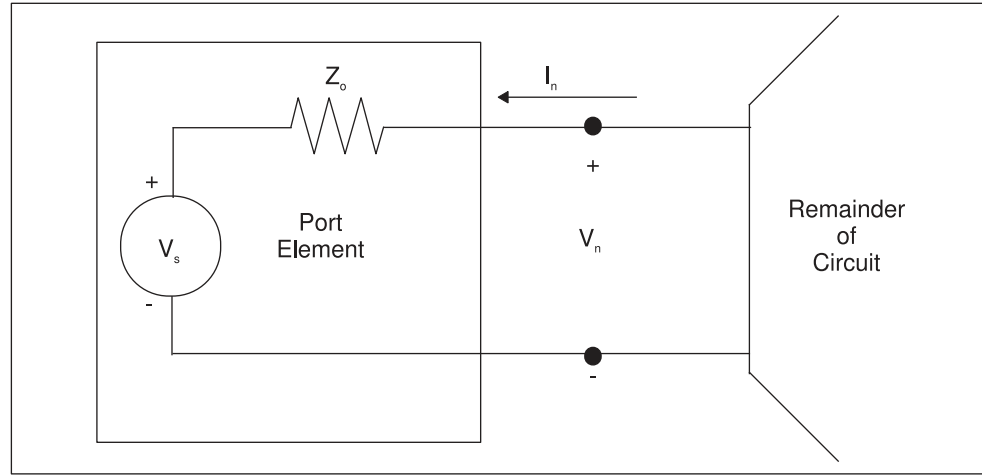
Equation 8

$$P_{rms}(\text{resistor})[n] = \frac{|V_n|^2}{2R}$$

Power Delivered to a Port Element

The port element can be either a source or sink for power. You can use a special calculation that computes the power flowing into a port element even if the port element itself is the source of that power. In [Figure 5](#) a Port element connects to a circuit (the Port element may or may not include a voltage source).

Figure 5 Port Element



Let V_n be the (peak) voltage across the terminals of the port element (at frequency index n). Let I_n be the (peak) current into the (1st) terminal of the port element (at frequency index n). Let Z_o be the impedance value of the $z0$ port element. Then, you can compute the power wave flowing into the terminals of the port element (at frequency index n) according to:

Equation 9

$$P_{in}[n] = \frac{1}{2} \left| \frac{V_n + Z_o I_n}{2\sqrt{Z_o}} \right|^2$$

This power expression remains valid whether or not the port element includes an internal voltage source at the same frequency. If the port element includes a voltage source at the same frequency, you can use this power calculation to compute the magnitude of the related large-signal scattering parameters.

If you expand the preceding formula, you can determine the power delivered to a port element with (real) impedance Z_o :

Equation 10

$$P_{rms}(port)[n] = \frac{1}{2} \left\{ \frac{|V_n|^2 + Z_o^2 |I_n|^2}{4Z_o} + \frac{1}{2} \text{Re}\{V_n I_n^*\} \right\}$$

This power value represents the power incident upon and delivered to the port element's load impedance (Z_o) due to other power sources in the circuit, and due to reflections of its own generated power.

If you use the port element as a load resistor (no internal source), the preceding equation reduces to that for the simple resistor.

If you use the port element as a power source (with non-zero available power, that is, a non-zero V_s) and it terminates in a matched load (Z_o), the port power-measurement returns 0 W, because no power is reflected.

You can request power measurements in the form of complete spectra or in the form of scalar quantities that represent power at a particular element. To request a complete power spectrum, use the following syntax.

```
.PRINT HB P(Elem)
.PROBE HB P(Elem)
```

To request a power value at a particular frequency tone, use the following syntax:

```
.PRINT HB P(Elem) [<n1<,n2<n3,...>>>]
.PROBE HB P(Elem) [<n1<,n2<,n3,...>>>]
```

The `Elem` is the name of either a Resistor (R) or Port (P) element, and $n1$, $n2$, and $n3$ are integer indices used for selecting a particular frequency in the Harmonic Balance output spectrum.

Examples

Example 6 Prints a table of the RMS power (spectrum) dissipated by resistor R1.

```
.PRINT HB P(R1)
```

Example 7 Outputs the RMS power dissipated by resistor R1 at the fundamental HB analysis frequency following a one-tone analysis.

```
.PROBE HB P(R1) [1]
```

Example 8 Prints the power dissipated by resistor R1 at DC following a one-tone analysis.

```
.PRINT HB P(R1) [0]
```

Example 9 Outputs the RMS power dissipated by resistor R1 at the (low-side) 3rd order intermodulation product after an HB two-tone analysis.

```
.PROBE HB P(R1) [2,-1]
```

Example 10 Prints the RMS power dissipated by resistor R1 at the (high-side) 3rd order intermodulation product after an HB two-tone analysis.

```
.PRINT HB P(R1) [-1,2]
```

Example 11 Outputs the RMS power (spectrum) delivered to port element Pload.

```
.PROBE HB P(Pload)
```

Example 12 Prints the RMS power delivered to port element Pload at the fundamental HB analysis frequency following a one-tone analysis.

```
.PRINT HB P(Pload) [1] $
```

Example 13 Outputs the RMS power delivered to port element Pload at the (low-side) 3rd order intermodulation product after an HB two-tone analysis.

```
.PROBE HB P(Pload) [2,-1]
```

Calculations for Time-Domain Output

In addition to a frequency-domain output, HB analysis also supports a time-domain output. The simulation generates an equivalent time-domain waveform according to the Fourier series expansion by way of

Equation 11

$$V(n1)@time\ t = \sum_m (REAL\ V(n1)[m] \cdot \cos(\Omega[m] \cdot t) - IMAG\ V(n1)[m] \cdot \sin(\Omega[m] \cdot t))$$

Where m starts from 0 to the number of frequency points in the HB simulation.

The output syntax is

```
.PRINT [HBTRAN | HBTR] V(n1)
.PROBE [HBTRAN | HBTR] V(n1)
```

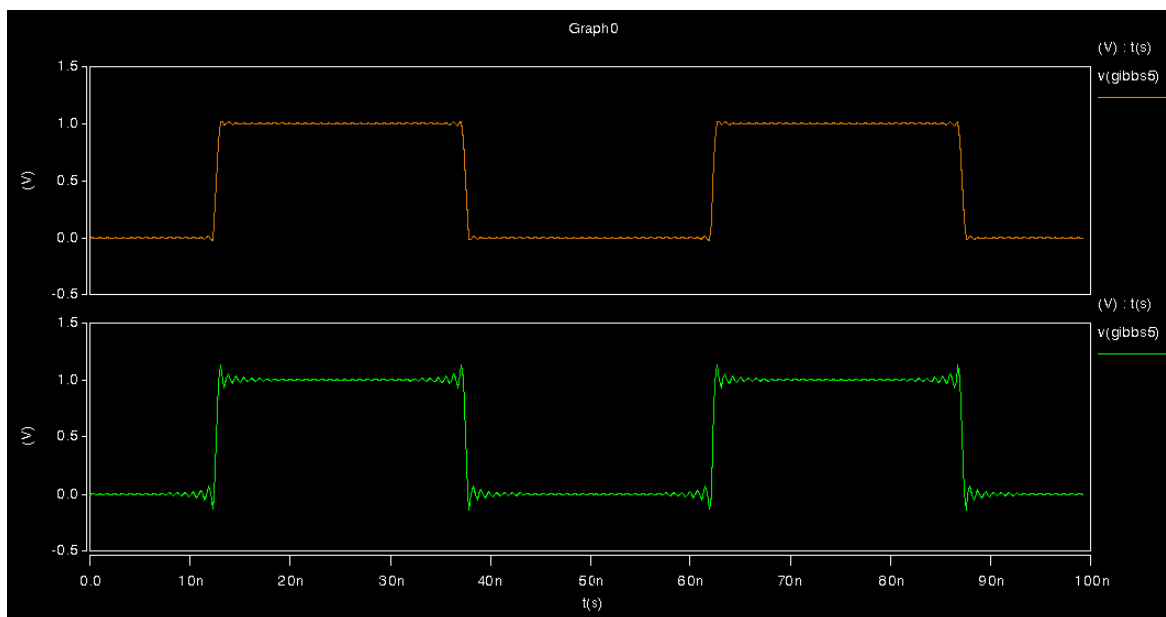
The output time ranges from 0 to twice the period of the smallest frequency in the HB spectra.

Minimizing Gibbs Phenomenon

You can use the HB_GIBBS option for HBTRAN output to minimize Gibbs' phenomenon that may occur in transforming a square-wave signal from the frequency domain to the time domain. The syntax is `.OPTION HB_GIBBS= $n$` (defaults to zero, which is equivalent to not using it at all). The result is that the $A(\sin c(x))^N$ function filters HBTRAN waveforms before they transform to the time domain via FFT. This option applies only to single-tone output. For example:

```
.option hb_gibbs = 2
...
.print hbtran v(2)
```

Figure 6 Upper square-wave signal shows $HB_GIBBS = 2$, while the lower shows the option = 0



Using .MEASURE with .HB Analyses

- For transient analysis (TRAN), the independent variable for calculating .MEASURE is time.
- For AC analysis, the independent variable for calculating .MEASURE is frequency.
- However, as with DC analysis, the use of a .MEASURE command is peculiar for HB analysis, because it has no obvious independent variable.

In PrimeSim HSPICE advanced analog analyses, the independent variable for HB .MEASURE analysis is the first swept variable specified in the .HB simulation control statement. This variable can be anything: frequency, power, voltage, current, a component value, and so on.

If you do not specify an HB sweep, then .MEASURE assumes a single-valued independent variable sweep.

You can apply the measurements to current, voltage, and power waveforms. The independent variable for measurements is the swept variable (such as power), not the frequency axis that corresponds to a single HB steady state point.

PrimeSim HSPICE advanced analog analyses also support the .MEASURE [HBTRAN | HBTR] . . . syntax. Similar to the .PROBE and .PRINT HBTR statements in the section [Calculations for Time-Domain Output](#), you apply a .MEASURE HBTR statement on the

signals obtained in the same way. Moreover, like a `.MEASURE` statement in transient analysis, the independent variable in a `.MEASURE HBTR` statement is time.

PrimeSim HSPICE advanced analog analyses optimization can read the data from `.MEASURE HB` and `.MEASURE HBTR` statements. Due to the difference in the independent variable between the `.MEASURE HB` and `.MEASURE HBTR` statements, you cannot mix these two types of measurements in an PrimeSim HSPICE advanced analog analyses optimization. But you can combine a `.MEASURE HBTR` statement with a `.MEASURE PHASENOISE` statement (see [Measuring Phase Noise with .MEASURE PHASENOISE](#)) and a `.MEASURE HBNOISE` statement (see [Measuring HBNOISE Analyses with .MEASURE](#)) in an PrimeSim HSPICE advanced analog analyses optimization flow.

Examples

Example 14 For the following `.HB` simulation control statement, the independent variable is the swept tone frequency, and the `.MEASURE` command values return results based on this frequency sweep:

```
* HARMONIC BALANCE tone-frequency sweep for amplifier
.param freq1=1.91e9 power=1e-3
.HB tones=freq1 nharms=10 sweep freq1 LIN 10 1.91e9 2.0e9
.MEASURE HB Patf0 FIND P(Rload)[1] AT=1.95e9 $ Power at
+ f0=1.95Ghz
.MEASURE HB Frq1W WHEN P(Rload)[1]=1. $ freq1 @ 1 Watt
.MEASURE HB BW1W TRIG AT=1.92e9 TARG P(Rload)[1] VAL=1.
+ CROSS=2 $ 1 Watt bandwidth
.MEASURE HB MaxPwr MAX P(Rload)[1] FROM=1.91e9 TO=2.0e9
+ $ Finds max output power
.MEASURE HB MinPwr MIN P(Rload)[1] FROM=1.91e9 TO=2.0e9
+ $ Finds min output power
```

Example 15 In the following example, the independent variable is the power variable, and the `.MEASURE` values return results based on the power sweep. Units are in Watts.

```
* HARMONIC BALANCE power sweep for amplifier
.param freq1=1.91e9 power=1e-3
.HB tones=freq1 nharms=10 sweep power DEC 10 1e-6 1e-3
.MEASURE HB Pat1uW FIND P(Rload)[1] AT=1e-6 $ Pout at 1uW
.MEASURE HB Pin1W WHEN P(Rload)[1]=1. $ Pin @ 1 Watt Pout
.MEASURE HB Prange1W TRIG AT=1.92e9 TARG P(Rload)[1] VAL=1.
+ CROSS=2 $ 1W oper. range

.MEASURE HB ssGain DERIV P(Rload)[1] AT=1e-5
+ $ relative power gain at 10uW input
.MEASURE HB Gain3rd DERIV P(Rload)[3] AT=1e-5
+ $ 3rd harmonic gain at 10uW input
.MEASURE HB PAE1W FIND '(P(Rload)[1]-power)/P(Vdc)[0]'
+ WHEN P(Rload)[1]=1 $ PAE at 1 Watt output
```

Example 16 In this example, the independent variable is again the power variable, and the `.MEASURE` values return results based on the power sweep. This is a two-tone sweep, where both input frequency sources are at the same power level in Watts.

```
* HARMONIC BALANCE two-tone sweep for amplifier
* An IP3 calculation is made at 10uW in the sweep
.param freq1=1.91e9 freq2=1.91e9 power=1e-3
.HB tones=freq1,freq2 nharms=6,6 sweep power DEC 10 1e-6 1e-3
.MEASURE HB Pfl dBm FIND '10.*LOG(P(Rload)[1,0]/1.e-3)'
+ AT=1e-5 $ P(f1) at 10uW input
.MEASURE HB P2f1_f2 dBm FIND '10.*LOG(P(Rload)[2,-1]/1.e-3)'
+ AT=1e-5 $ P(2f1-f2) at 10uW input
.MEASURE HB OIP3 dBm PARAM = '0.5*(3.*Pfl dBm-P2f1_f2 dBm)'
.MEASURE HB IIP3 dBm PARAM = 'OIP3 dBm-Pfl dBm+20.0'
.MEASURE HB AM2PM DERIV VP(outp,outn)[1] AT=1e-5
+ $ AM to PM Conversion in Deg/Watt
```

Troubleshooting Convergence Issues with Harmonic Balance Analysis

To troubleshoot convergence issues with harmonic balance analysis, perform the following steps:

- Ensure that the transient sources are activated by setting `.option tranforhb`.
- The number of harmonics (`nharms`) specified in the `.HB` command will affect the accuracy. The number of harmonics should be set to approximately 40 per the fastest period in the circuit. For example, for a circuit containing a divide by 6, at least 240 harmonics should be used.
- To verify that the correct number of harmonics is used, observe the time domain waveforms. To output the time domain waveforms, include `.PROBE hbtr` statements in the netlist. For example, `.probe hbtr v(*)`. The time domain waveform file will have the extension `hr0`.
- Increase the initial transient analysis time by increasing the setting of the option `HBTRANINIT` to a value equal to 100 times the waveform period.
- Increase the value of the number of harmonics (`nharms`) in the `.HB` command.

Phase Differences Between HB and SIN Sources

The HB steady-state cosine source has a phase variation compared to the `TRAN` time-domain `SIN` source. The `SIN` source (with no offset, delay or damping) follows the equation:

Equation 12

$$A \sin(\omega t + \phi)$$

while the HB sources follow:

Equation 13

$$A \cos(\omega t + \phi)$$

In order for the two sources to yield identical results it is necessary to align them by setting their phase values accordingly using:

Equation 14

$$A \cos(\omega t + \phi) = A \sin(\omega t + \phi + 90^\circ)$$

Equation 15

$$A \sin(\omega t + \phi) = A \cos(\omega t + \phi - 90^\circ)$$

Example

Example 17 To specify sources with matching phase for HB and TRAN analysis, use a convention similar to:

```
** Example #1 with equivalent HB and SIN sources
** SIN source is given +90 phase shift
.param freq1=2400MEG Vin=1.0
Vsrc in gnd DC 0 HB 'Vin' 0 1 1 SIN(0 'Vin' 'freq1' 0 0 90)
.HB tones=freq1 intmodmax=7
** Example #2 with equivalent HB and SIN sources
** HB source is given -90 phase shift to align with SIN
.param freq1=2400MEG Vin=1.0
Vsrc in gnd DC 0 HB 'Vin' -90 1 1 SIN(0 'Vin' 'freq1' 0)
.HB tones=freq1 intmodmax=7
** Example #3 with equivalent .HB and .TRAN sources
** SIN source is activated for HB using "TRANFORHB"
.param freq1=2400MEG Vin=1.0
Vsrc in gnd DC 0 SIN(0 'Vin' 'freq1' 0) TRANFORHB=1
.HB tones=freq1 intmodmax=7
```

Optimization

To perform optimization, create an input netlist file that specifies:

- Optimization parameters with upper and lower boundary values along with an initial guess.
- An HB or HBOSC optimization statement.
- An optimization model statement.
- Optimization measurement statements for optimization parameters.

If you provide the input netlist file, optimization specifications, limits, and initial guess, then the optimizer reiterates the simulation until it finds an optimized solution.

Usage Notes and Examples

- Optimization works for HB, HBOSC, and HBAC analyses.
- You can add the `GOAL` options in every meaningful `.MEASURE` statement, like `FIND-WHEN`, `FIND-AT`, and so on.
- A data sweep does not need definition in the `.HB` statement for HB optimization to use the measured result from `.MEASURE HBNOISE`, or `PHASENOISE` statements. Therefore, this type of optimization does not support parameter sweep.
- Optimize multiple parameters with multiple goals by selecting `.MODEL OPT LEVEL=0` (modified Lavenberg-Marquardt method).
- Optimize single parameters in single measurement situations by selecting `.MODEL OPT LEVEL=1` (bisection method).
- Examples

- Setting optimization parameters

```
.param W=opt1(231u, 100u, 800u)
.param Rs=opt1(10, 8, 20)
```

- Optimization analysis statement

```
.HB tones=2.25g 2.5g nharms=6, 3
+ sweep Pin_dbm -30 0 2
+ sweep optimize = opt1
+ results = gain $measure result to tune the parameters
+ model= optmod1
```

- Selecting an optimization model

```
.model optmod1 opt level=1 $Bisection method
+ itropt=40 relin=1e-4 relout=1e-6 $ accuracy settings
```

- Measurement statements to tune the optimization parameters

```
.measure HB vif find vdb(if+)[-1,1] at 10e-6
.measure HB vrf find vdb(rf+)[0,1] at 10e-6
.measure HB gain=param('vif-vrf') goal=-2
```

- Measurement statement to find the fundamental frequency from HB analysis:

```
.measure HB frequency_max FIND 'HERTZ[1]' at=0
```

Optimizing with HB Measurements

The required statements are:

- Analysis statement

```
.HB TONES=f1[f2 ... fn] [NHARMS=h1 [,h2 ... hn]]
+ SWEEP parameter_sweep OPTIMIZE=OPTxxx RESULT=measname
+ MODEL=mname
```

- Measure statement

```
.MEASURE HB measname FIND out_var1 AT=val GOAL=val
```

References

- [1] S. Maas, *Nonlinear Microwave Circuits*, Chapter 3, IEEE Press, 1997.
- [2] R. Gilmore and M.B. Steer, "Nonlinear Circuit Analysis Using the Method of Harmonic Balance - A Review of the Art, Part I, Introductory Concepts." *International Journal of Microwave and Millimeter-wave Computer-Aided Engineering*, Volume 1, No. 1, pages 22-37, 1991.
- [3] R. Gilmore and M.B. Steer, "Nonlinear Circuit Analysis Using the Method of Harmonic Balance - A Review of the Art. Part II. Advanced Concepts." *International Journal of Microwave and Millimeter-wave Computer-Aided Engineering*, Volume 1, No. 2, pages 159-180, 1991.
- [4] V. Rizzoli, F. Mastri, F. Sgallari, G. Spaletta, "Harmonic-Balance Simulation of Strongly Nonlinear Very Large-Size Microwave Circuits by Inexact Newton Methods," *MTT-S Digest*, pages 1357-1360, 1996.
- [5] S. Skaggs, *Efficient Harmonic Balance Modeling of Large Microwave Circuits*, Ph.D. thesis, North Carolina State University, 1999.
- [6] R.S. Carson, *High-Frequency Amplifiers*, 2nd Edition, John Wiley & Sons, 1982
- [7] S.Y. Liao, *Microwave Circuit Analysis and Amplifier Design*, Prentice-Hall, 1987.

- [8] J. Roychowdhury, D. Long, P. Feldmann, "Cyclostationary Noise Analysis of Large RF Circuits with Multitone Excitations", *IEEE JSCC*, volume 33, number 3, March 1998.
- [9] Y. Saad, *Iterative Methods for Sparse Linear Systems*, PWS Publishing Company, 1995.
- [10] J. Roychowdhury, D. Long, and P. Feldmann, "Cyclostationary Noise Analysis of Large RF Circuits with Multitone Excitations," *IEEE Journal of Solid-State Circuits*, volume 33, pages 324–336, March 1998.
- [11] K. Kurakawa, "Power waves and the Scattering Matrix," *IEEE Trans. Microwave Theory Tech.*, vol. MTT-13, pp. 194-202, March 1965.

4

Shooting Newton Analysis

Describes PrimeSim HSPICE advanced analog steady-state time domain analysis based on an advanced Shooting-Newton algorithm.

The following sections discuss these topics:

- [Shooting Newton Steady-State Time Domain Analysis \(.SN\)](#)
- [Shooting Newton with Fourier Transform \(.SNFT\)](#)
- [Troubleshooting Convergence Issues with Shooting Newton Analysis](#)

Shooting Newton Steady-State Time Domain Analysis (.SN)

An advanced Shooting Newton (SN) algorithm provides additional performance and functionality to PrimeSim HSPICE advanced analog analyses for time-domain, steady-state analysis.

Shooting-Newton adds analysis capabilities for PLL components, digital circuits/logic such as ring oscillators, frequency dividers, phase/frequency detectors (PFDs), switched capacitor filters, and for other digital logic circuits and RF components that require steady-state analysis, but operate with waveforms that are more square wave than sinusoidal.

For more information on control options, see .SN command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The Shooting-Newton algorithm effectively analyzes applications including:

- Ring oscillators (see [Chapter 5, Oscillator Analysis](#))
- Frequency dividers (prescalers)
- Mixer conversion gain
- Phase-frequency detectors (PFDs)
- Mixer noise figure

Functionality includes:

- Both driven and oscillator (autonomous) analyses
- Time domain or frequency analysis based on advanced Shooting Newton algorithm
- [Shooting Newton with Fourier Transform \(.SNFT\)](#)
- [Shooting Newton AC Analysis \(.SNAC\)](#)
- [Shooting Newton Oscillator Analysis \(.SNOSC\)](#)
- [Oscillator Analysis](#)

This section covers the following topics:

- [Shooting Newton Analysis – Tutorial Example](#)
- [SN .PRINT and .PROBE Output Syntax](#)

Shooting Newton Analysis – Tutorial Example

While the Harmonic Balance (HB) algorithm represents the circuit's voltage and current waveforms as a Fourier series (a series of sinusoidal waveforms), the Shooting Newton (SN) algorithm provides analysis capability for digital logic circuits and advanced analog components that require steady-state analysis, but operate with waveforms that tend to be square instead of sinusoidal.

This section covers the following topics:

- [Shooting Newton Analysis — Setup](#)
- [Clock Buffer Analysis Example](#)

Shooting Newton Analysis — Setup

To set up a time-domain, steady-state analysis, the PrimeSim HSPICE input netlist must contain:

- A `.SN` command to activate the analysis. The `.SN` command specifies:
 - The expected period of the steady-state waveforms, which must match the period of any input waveforms. The period is specified in time domain units (seconds). Alternatively, you may specify a frequency in Hz.
- A time resolution, which is analogous to the transient analysis (`.TRAN`) command's `TSTEP` parameter and will affect the time-step size selection. It also affects the number of frequency terms used in small-signal analyses, such as periodic AC or noise analysis. The time resolution is typically specified in seconds but, alternatively, may be specified in the frequency domain as a number of harmonics.

- A transient initialization time that is used by PrimeSim HSPICE advanced analog analyses to run a basic transient simulation of this length before attempting Newton-Raphson iterations to converge on a steady-state solution. This parameter is optional. If it is not specified, the specified period is used as the initialization time. The initial transient analysis is used for circuit stabilization before the steady state solution is found. Larger initialization values typically result in convergence that is more robust.
- For oscillator circuits, a `.SNOSC` command is used to activate the analysis. The `.SNOSC` command specifies:
 - The approximate frequency of oscillation specified either as a frequency (in Hertz) or as the time domain period.
 - The number of high frequency harmonics. Alternatively, a time resolution in seconds can be specified.
- A transient initialization time that is used by PrimeSim HSPICE advanced analog analyses to run a basic transient simulation of this length before attempting Newton-Raphson iterations to converge on a steady state solution. This parameter is optional. If it is not specified, the period of the specified frequency of oscillation is used as the initialization time. For oscillators we recommend specifying a transient initialization time since the default initialization time is usually too short to effectively stabilize the circuit.
- A node at which to probe for oscillation conditions.
- If the tuning curve of a VCO is to be analyzed, the optional parameter `MAXTRINITCYCLES` can be specified.
- One or more signal sources for driving the circuit in SN analysis, if the circuit is driven. In the case of autonomous oscillator analysis, no signal source is required. The sources are required to be time domain sources and must match the period specified in the `.SN` command.

Clock Buffer Analysis Example

This example demonstrates the Shooting Newton-based analysis of a clock buffer circuit. Extracted portions of the input file are presented in the following pages. The complete `clockbuf4sn.sp` input file for this example is located at `$installdir/demo/hspice/rf_examples/`.

Example 18 Clock Buffer Netlist

```
**
** Shooting Newton analysis example
**
.temp 30.0
.option scale=50n
.option post probe
.subckt inverter      VDD      in      out
M1      out      in      0      0      nmos      L=1      W=10
```

Chapter 4: Shooting Newton Analysis

Shooting Newton Steady-State Time Domain Analysis (.SN)

```

M2      out      in      VDD      VDD      pmos      L=1      W=20
.ends
*-----
* reference clock
*
.param vdd=0.80
.param freq=2000MEG
.param period='1.0/freq'
Vsrc     ref     gnd     DC     0
+ sin (0 'vdd' 'freq')

*-----
* Circuit:
*
Vdd      VDD      0      DC      'vdd'
Xrout    VDD      ref     outa    inverter
* Chain of 4 inverters
Xb11 VDD outa outb11 inverter
Xb12 VDD outb11 outb12 inverter
Xb13 VDD outb12 outb13 inverter
Xb14 VDD outb13 outb14 inverter
RLb1     outb14    0    100k
* Chain of 6 inverters
Xb21 VDD outa outb21 inverter
Xb22 VDD outb21 outb22 inverter
Xb23 VDD outb22 outb23 inverter
Xb24 VDD outb23 outb24 inverter
Xb25 VDD outb24 outb25 inverter
Xb26 VDD outb25 outb26 inverter
RLb2     outb26    0    100k
*-----
* SN analysis setup
*
*
* Using frequency domain syntax
.sn tone='freq' nharms=63 trinit='10*period' numperout=2
*
* Using time domain syntax
.sn tres='0.02*period' period='period' trinit='10*period'
+ numperout=2
*
.snnoise v(outb26) Vsrc dec 10 100 100meg [0,1]
*
*
.probe sn v(ref) v(outa) v(outb14) v(outb26)
.probe snfd v(ref) v(outb26)
.probe snnoise onoise
*

```

The `.SN` command can be specified using either the frequency domain syntax that specifies the tone and number of harmonics, or the time domain syntax that specifies

the period and time resolution. In both cases, a transient initialization time is set and the number of output periods of the signal are specified using the `NUMBEROUT` keyword.

```
* Using frequency domain syntax
.sn tone='freq' nharms=63 trinit='10*period' numberout=2

* Using time domain syntax
.sn tres='0.05*period' period='period' trinit='10*period' numberout=2
```

```
The steady state noise analysis
.snnoise v(outb26) Vsrc dec 10 100 100meg [0,1]
```

A periodic, time-varying AC noise analysis based on the Shooting Newton algorithm is performed using the `.SNNOISE` command. The `.SNNOISE` analysis requires an output node (`v(outb26)`) where the noise is to be measured, an input noise source (`Vsrc`) which serves as the reference for the noise computation, and a frequency sweep for the noise analysis. Optionally, an index term can be defined. The index term specifies the output frequency band at which the noise is evaluated. For this case, you will evaluate the low frequency noise of the clock buffer.

During the simulation, the simulation status is displayed on the computer screen. In addition to the screen display, a more detailed status, cpu time, and memory usage information are also written to the `clockbuf4sn.lis` file.

To run this simulation, type the following at the command line:

```
hspice -i clockbuf4sn.sp -o
```

Viewing Results Using WaveView

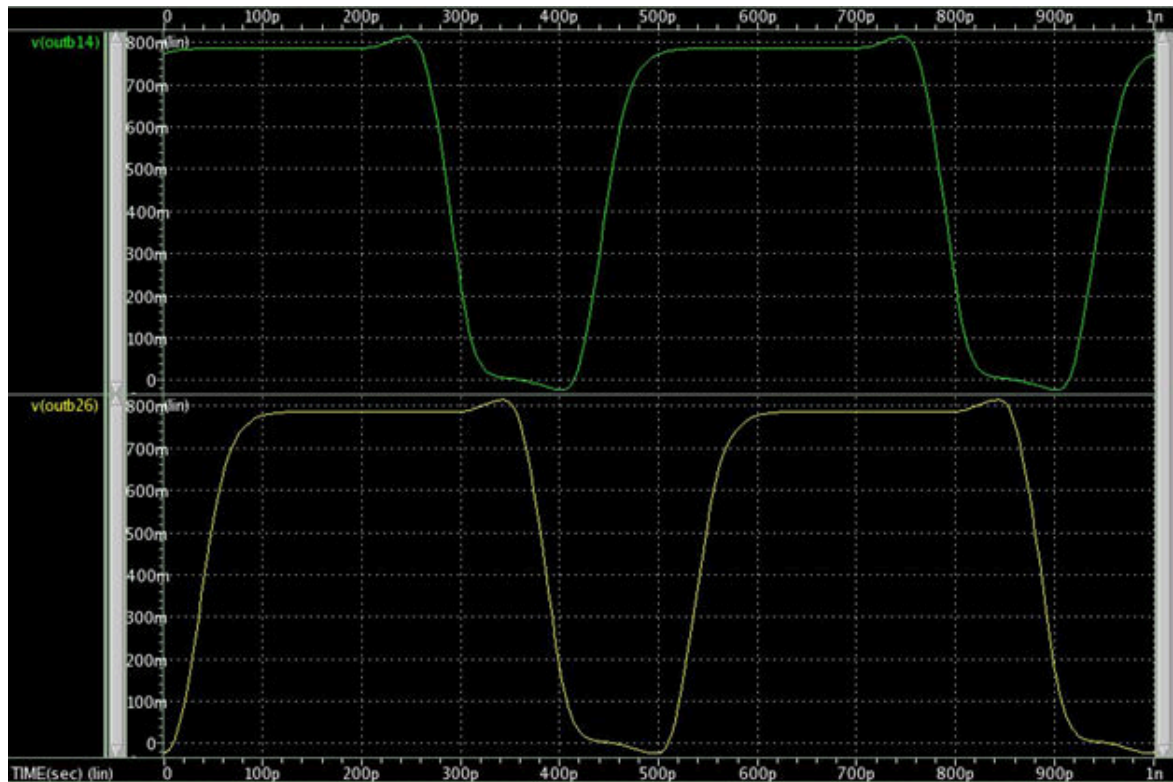
You can view the steady-state time domain, frequency domain, and noise results.

To view the time domain results:

1. Type `wv` at the prompt to invoke WaveView.
2. Click the **Open Waveform** icon and select the `clockbuf4sn.sn0` file.

3. Select the signals `v(outb26)` and `v(outb14)` in the signal browser. Drag and drop the signals in the waveform.

Figure 7 SN Analysis Output: Time Domain

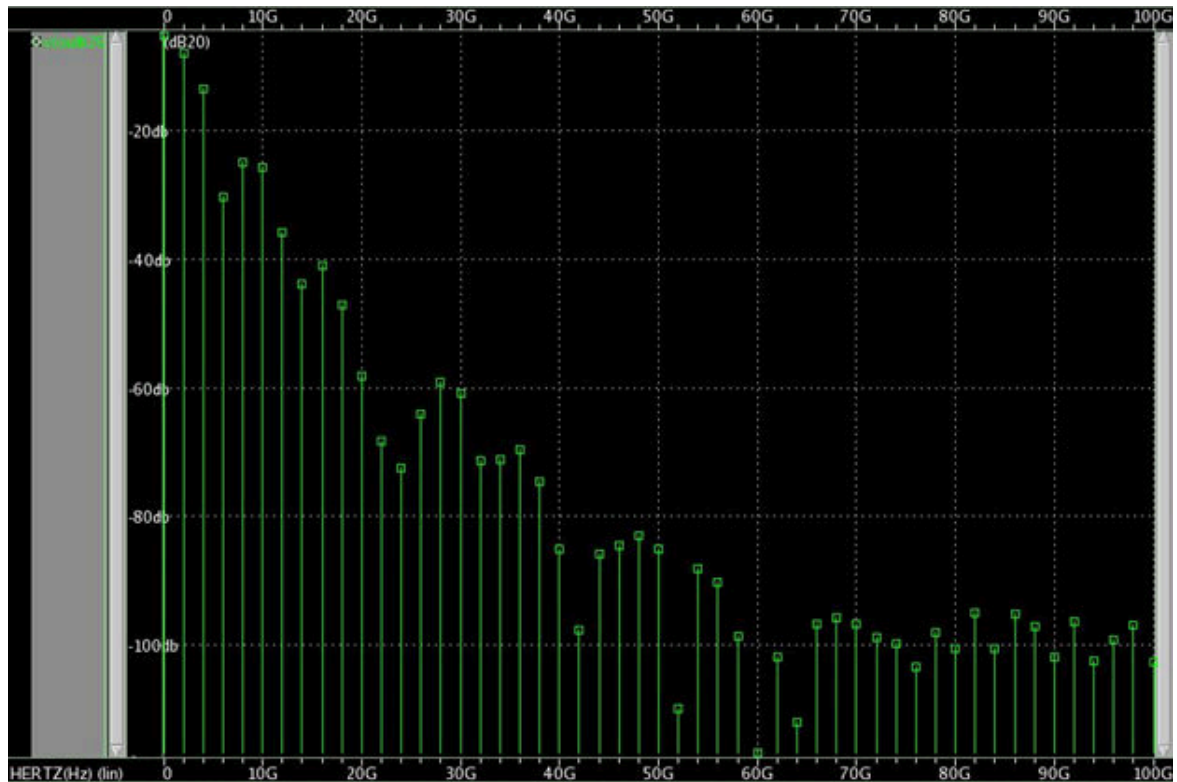


To view the frequency domain results:

1. Click the **Open Waveform** icon and select the `clockbuf4sn.snfd0` file.
2. Click the **Waveview** icon in the Add section of the **Waveform** tab (or press CTRL+w) to open a new waveview.

3. Select the signal `v(outb26)` in the signal browser. Drag and drop the signal into the waveview.

Figure 8 SN Analysis Output: Frequency Domain

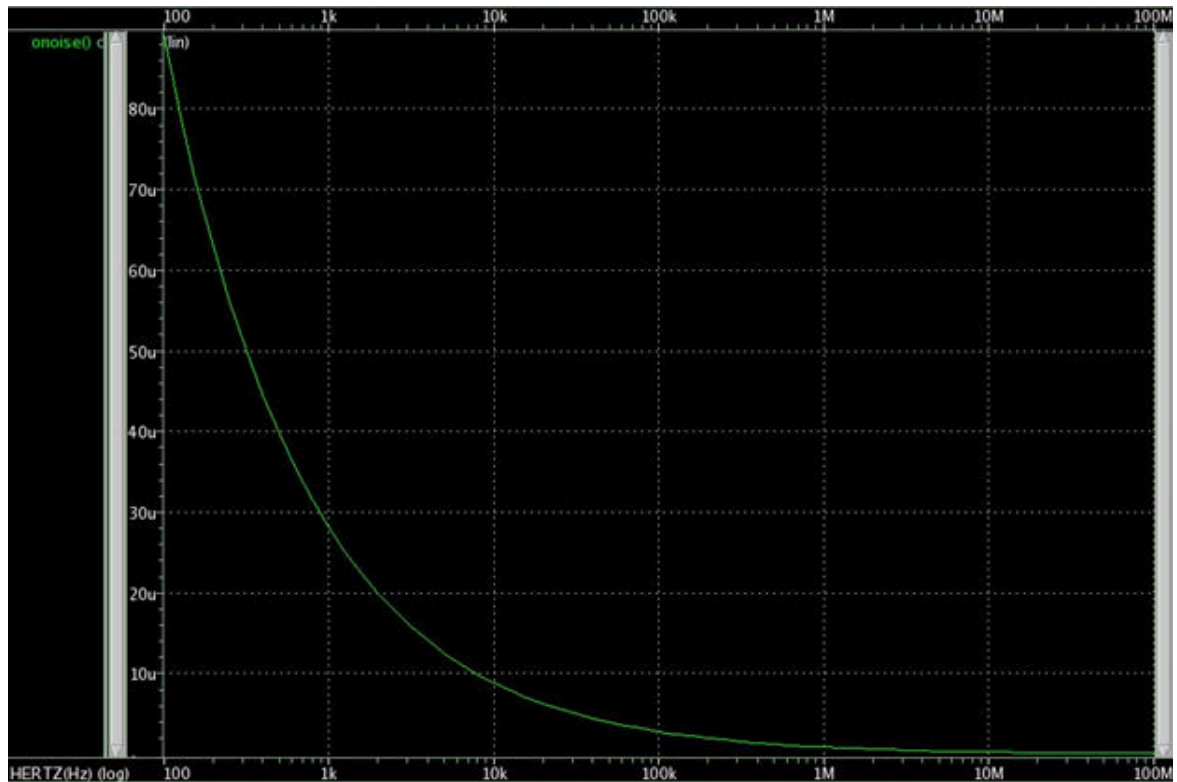


To view the noise analysis results:

1. Click the **Open Waveform** icon and select the `clockbuf4sn.sn` file.
2. Click the **Waveview** icon in the Add section of the **Waveform** tab (or press CTRL+w) to open a new waveform.
3. Select the signal **noise()** in the signal browser. Drag and drop the signal into the waveview.
4. The X-axis scale will be linear and it is preferred to use the log scale.

To change the X-axis scale to the log scale, right-click the X-axis. A menu will appear. Select **log** in this menu.

Figure 9 SNNOISE Analysis Results



SN .PRINT and .PROBE Output Syntax

The output from .SN analysis is generated in both time and frequency domains.

The time domain output variables are the same as for standard transient analysis:

- individual nodal voltages: $V(n1 [, n2])$
- branch currents: $I(V_{xx})$
- element power dissipation: $In(element)$

It is also possible to output the results from Shooting Newton analysis in terms of complex, frequency-domain output variables. This output format is activated by using the “SNFD” keyword in the output syntax.

For output in the frequency domain, the syntax is identical to the Harmonic Balance output syntax:

```
.PRINT SNFD TYPE (NODES|ELEMENTS) [INDICES]
.PROBE SNFD TYPE (NODES|ELEMENTS) [INDICES]
```

Parameter	Description
SNFD TYPE	Specifies a harmonic type node or element. TYPE can be one of the following: • Voltage type: V = voltage magnitude and phase in degrees VR = real component VI = imaginary component VM = magnitude VP - Phase in degrees VPD - Phase in degrees VPR - Phase in radians VDB - dB units VDBM - dB relative to 1 mV • Current type: I = current magnitude and phase in degrees IR = real component II = imaginary component IM = magnitude IP - Phase in degrees IPD - Phase in degrees IPR - Phase in radians IDB - dB units IDBM - dB relative to 1 mV • Power type – P • Frequency type: hertz[index], hertz[index1, index2, ...]. You must specify the harmonic index for the hertz variable. The frequency of the specified harmonics is dumped.
INDICES	Is the harmonic index of the SNFD tone. Index is limited to the single tone associated with the SN analysis.
SNFD TYPE can be one of the following: • Voltage type – a single node name (n1), or a pair of node names, (n1,n2) • Current type – an element name (elemname) • Power type – a resistor (resistorname) or port (portname) element name.	

Output Files

The time domain data are output to `printsn0` and `.sn0` files. Frequency domain data are output to `.printsnf0` and `.snf0` files.

Output Format

The format for time domain output is the same as standard transient analysis. For frequency domain output, the format is similar to HB. The main difference is that the Shooting Newton output in the frequency domain is single tone only.

The results of an SN analysis are complex spectral components at each frequency point. The $a[i]$ is the real part and $b[i]$ is the imaginary part of the complex voltage at frequency index i . The conversion to a steady-state time-domain is then given by the Fourier series expansion.

An SN analysis produces these output data files:

- Output from the `.PRINT SN` statement is written to a `.printsn#` file.
 - The header contains the large signal fundamental frequencies.
 - The columns of data are labeled as `HERTZ`, followed by frequency indices, and then the output variable names.
 - The sum of the frequency indices, multiplied by the corresponding fundamental frequencies, add up to the frequency in the first column.
- Output from the `.PROBE SN` statement is written to a `.sn#` file in the same format as the PrimeSim HSPICE transient analysis `.tr#` file. It contains the information on harmonic indices and basic tone frequencies plus the output waveform.
- Reported performance log statistics are written to a `.lis` file:
 - Name of SN data file.
 - Simulation time:
 - DC operating point (op) time
 - SN time
 - Total simulation time
 - Memory used
 - Size of matrix (nodes * harmonics)
 - Final SN residual error

Shooting Newton with Fourier Transform (.SNFT)

The `.SNFT` command is to the `.SN` analysis what `.FFT` is to the `TRAN` analysis, a means to provide spectrum analysis. Spectrum analysis represents a time-domain signal, within the frequency domain. `.SNFT` uses the Fourier transform: a Discrete Fourier Transform (DFT)

uses sequences of time values to determine the frequency content of analog signals, in circuit simulation. The `.SNFT` statement uses the internal time point values.

By default, the `.SNFT` statement uses a second-order interpolation to obtain waveform samples, based on the number of points that you specify.

Note:

True distributed components (such as ideal delays or transmission lines) are not supported; components with hidden states are not supported.

SN analysis assumes that all stimuli are periodic with period T. If the circuit is driven with more than one periodic stimulus, then the frequencies must be all co-periodic and T must match the common period or some integer multiple of it. The `.SN` analysis only supports `.tran` (time-domain) periodic signal sources. (Refer to the `.tran` analysis for a detailed documentation on transient signal sources).

You can use windowing functions to reduce the effects of waveform truncation on the spectral content. You can also use the `.SNFT` command to specify:

- output format
- frequency
- number of harmonics
- total harmonic distortion (THD)

For more information, see the `.SNFT` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

This section covers the following topics:

- [Other Shooting Newton Analyses](#)
- [SN Error and Warning Messages](#)

Other Shooting Newton Analyses

The following Shooting Newton Analyses are also supported by PrimeSim HSPICE advanced analog analyses.

- `.SNFT` is equivalent to the `.FFT` command in transient (`.TRAN`) analysis. `.SNFT` uses Fourier transform to represent a time domain signal in the frequency domain. For more information, see [Shooting Newton with Fourier Transform \(.SNFT\)](#).
- `.SNAC` is used to perform a linear analysis of a driven (or non-autonomous) circuit, where the linear coefficients are modulated by a periodic, steady-state signal. The

functionality is similar to the `.HBAC` command. For more information, see [Shooting Newton AC Analysis \(.SNAC\)](#).

- `.SNXF` is used to calculate transfer functions from an arbitrary number of small signal sources to a designated output in a circuit under periodic steady-state conditions. For more information, see [Shooting Newton Transfer Function Analysis \(.SNXF\)](#).

SN Error and Warning Messages

Error messages are displayed with convergence recommendations in cases of non-convergence within the maximum number of Shooting-Newton iterations.

Error messages are displayed for software errors such as segmentation violations, and abort conditions such as:

- unrecognized format such as unrecognized V/I source.
- faulty input values such as wrong sign and out of range values.
- unspecified values such as unspecified tone.
- inconsistent values such as non-commensurable tones.
- duplicate values such as same entry provided more than once, the last one is always taken.

Troubleshooting Convergence Issues with Shooting Newton Analysis

To troubleshoot convergence issues with shooting newton analysis, perform the following steps:

- Increase the accuracy using either the `RUNLVL` (default is 3) or the `SIM_ACCURACY` (default is 10) option.
- Increase the initial transient simulation time by increasing the value of `TRINIT` in the `.SN` or the `.SNOSC` command.
- Limit the maximum time-step size used in the simulation by setting the option `DELMAX` to a value equal to the waveform period divided by 100.
- Ensure that the correct frequency is selected. In the case of a driven circuit containing dividers, use the final divider (slowest) output frequency.

Part 3: Oscillator and PLL Analysis

Describes the usage of oscillator analysis, phase noise analysis and behavioral PLL noise analysis.

The following chapters/topics are included in this Part:

- [Chapter 5, Oscillator Analysis](#)
- [Chapter 6, Phase Noise Analysis](#)
- [Chapter 7, Behavioral PLL Noise Analysis \(.ACPHASENOISE\)](#)

5

Oscillator Analysis

Describes how to use the PrimeSim HSPICE advanced analog functions to perform oscillator and phase noise analysis on oscillator circuits.

Two main groups categorize oscillators:

- Ring oscillators: These oscillators tend to have a low Q and operate based on the delay of digital cells such as inverters. Ring oscillators have a strong nonlinear behavior and the output signals are often square-wave-like. You can analyze ring oscillators in either the frequency domain using harmonic balance analysis or in the time domain using Shooting Newton analysis.
- Harmonic oscillators: Common harmonic oscillators are LC and crystal oscillators. These oscillators tend to have a high Q, making it difficult to find the oscillation frequency. Their behavior tends to be only mildly nonlinear and their output signals tend to be close to purely sinusoidal. harmonic balance analysis is most effective for analyzing harmonic oscillators.

PrimeSim HSPICE advanced analog analyses includes special analysis algorithms for finding the steady-state solution for oscillator circuits. No driving sources set the frequencies of operation in oscillators. The fundamental oscillation frequency is one of the unknowns that the simulator solves for. PrimeSim HSPICE advanced analog analyses provide two approaches: harmonic balance analysis or a Shooting Newton algorithm-based analysis.

This section covers the following topics:

- [Oscillator Analysis \(.HBOSC and .SNOSC\)](#)
- [Shooting Newton Oscillator Analysis \(.SNOSC\)](#)
- [Ring Oscillator Example](#)
- [References](#)

Oscillator Analysis (.HBOSC and .SNOSC)

Because the frequencies of the driving sources do not determine the frequency of oscillation, the simulator solves for a slightly different set of nonlinear equations as in the following equation:

Equation 16

$$F(V, \omega_0) = I(V, \omega_0) + \Omega Q(V, \omega_0) + Y(\omega_0)V + I_s$$

PrimeSim HSPICE harmonic balance oscillator analysis (.HBOSC) adds the fundamental frequency of oscillation to the list of unknown circuit quantities. To accommodate the extra unknown, PrimeSim HSPICE sets the phase (or equivalently, the imaginary part of one unknown variable — generally a node voltage) to zero. The phases of all circuit quantities are relative to the phase at this reference node (referred to as the “PROBENODE”).

Additionally, the HBOSC analysis tries to avoid the “degenerate solution,” where all non-DC quantities are zero. Although this is a valid solution of the above equation (it is the correct solution, if the circuit does not oscillate), the HBOSC analysis might find this solution incorrectly, if the algorithm starts from a bad initial solution.

The HBOSC analysis follows a technique similar to that described by Ngoya et al., which uses an internally-applied voltage probe to find the oscillation voltage and frequency. The source resistance of this probe is a short circuit at the oscillation frequency, and an open circuit otherwise. PrimeSim HSPICE advanced analog analyses use a two-tier Newton approach to find a non-zero probe voltage which results in a zero probe current.

HB analysis of the oscillator circuit uses the DC solution as a starting point. This analysis requires, in addition to the DC solution, initial values for both the oscillation frequency and the probe voltage. HBOSC analysis calculates the small-signal admittance that the voltage probe sees over a range of frequencies, in an attempt to find potential oscillation frequencies. Oscillation is likely to occur where the real part of the probe current is negative and the imaginary part is zero. You can use the `FSPTS` parameter to specify the frequency search. You must also supply an initial guess for the large signal probe voltage. A value of one-half the supply voltage is often a good starting point.

For more information on control options, see the .HBOSC command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [Tutorial Examples Using HBOSC Analysis](#)
- [.HBOSC Output Syntax](#)
- [Using the .MEASURE Command with .HBOSC](#)

- [HB Simulation of Ring Oscillators](#)
- [Troubleshooting Convergence Problems](#)

Tutorial Examples Using HBOSC Analysis

The following tutorial examples illustrate:

- [Example 1 — Colpitts Oscillator](#)
- [Example 2 — Using HBOSC for a CMOS GPS VCO](#)

Example 1 — Colpitts Oscillator

This section demonstrates the PrimeSim HSPICE advanced analog oscillator analysis by using a single transistor oscillator circuit. Oscillator analysis is an extension of harmonic balance in which you solve for the base frequency. In oscillator analysis, the user supplies a guess of the base frequency. It requires no voltage or current source stimulus.

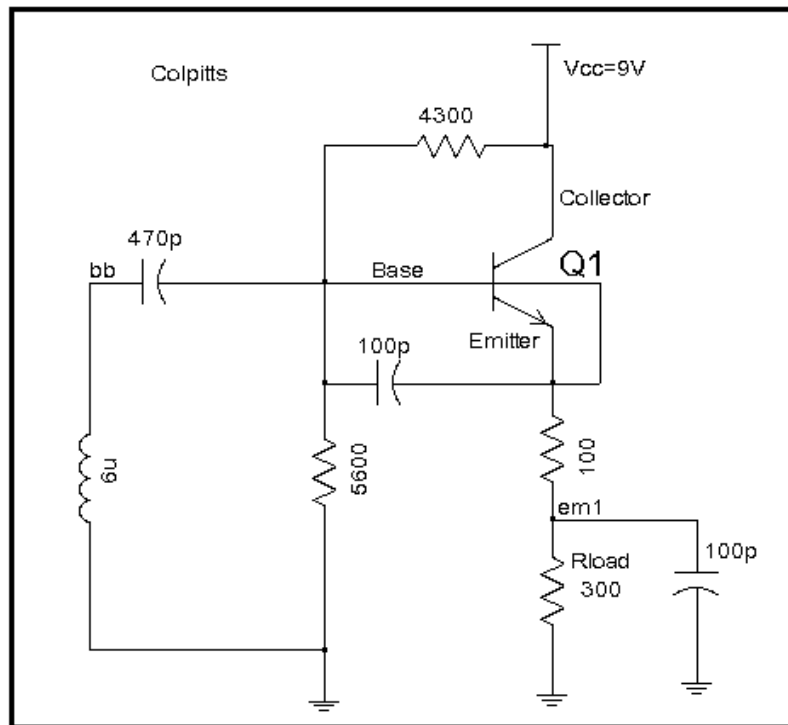
To activate oscillator analysis, include a `.HBOSC` command with:

- The `TONE` parameter set to a guess of the oscillation frequency.
- The `PROBENODE` parameter set to identify an oscillating node or a pair of nodes. Always specify a pair of nodes; if only one node oscillates, specify ground as the second node. To speed up the simulation, you also supply a guess of the magnitude of the oscillating voltage across these nodes.
- The `FSPTS` parameter set to a frequency range and the number of search points. When you set `FSPTS`, PrimeSim HSPICE advanced analog analyses precede the HBOSC analysis, with a frequency search in the specified range to obtain an optimal initial guess for the oscillation frequency. This can accelerate the HB oscillator convergence.

In conjunction with oscillator analysis, PrimeSim HSPICE advanced analog analyses can perform phase noise analysis. Phase noise analysis measures the effect of transistor noise on the oscillator frequency. You activate phase noise analysis by using the `.PHASENOISE` command; this command sets a set of frequency points for phase noise analysis. The `.PRINT` and `.PROBE` commands output the phase noise values.

The following netlist, `osc.sp`, simulates an oscillator and performs phase noise analysis. This example file is available in the PrimeSim HSPICE distribution in the directory `$installdir/demo/hspice/rf_examples/`.

Figure 10 Colpitts Oscillator



Use the `.HBOSC` command with the `PROBENODE` and `FSPTS` parameters set.

```
PROBENODE=emitter,0,4.27
```

Identifies the emitter node as an oscillating node and provides a guess value of 4.27 volts for the oscillation amplitude at the emitter node.

```
FSPTS=40,9e6,1.1e7
```

Causes an initial frequency search using 40 equally-spaced points between 9 and 11 MHz.

In the `.PHASENOISE`, `.PRINT`, and `.PROBE` commands:

```
.PHASENOISE V(emitter) dec 10 10k 1meg
```

Runs phase noise analysis at the specified offset frequencies, measured from the oscillation carrier frequency. The frequency points specified here are on a logarithmic scale, 10 points per decade, 10 kHz to 1 MHz.

- **.PROBE PHASENOISE PHNOISE** and the similar **.PRINT** command instruct PrimeSim HSPICE advanced analog analyses to output phase noise results to the `osc.pn0` and `osc.printpn0` files.

```

**
** Uses emitter resistor limiting to keep output sinusoidal.
** Output can be taken at the emitter (eml node).
**
*-----
* Options for Oscillator Harmonic Balance Analysis...
*
.OPTIONS post sim_accuracy=100 hbsolver=0
*-----
* Bias NPN transistor for 5V Vce, 10mA Ic
* Emitter follower Colpitts design
Vcc collector 0      9V
Q1 collector base emitter emitter RF_WB_NPN
Re1 emitter   eml    100
Rload eml      0      300
Rb1 collector base 4300
Rb2 base       0      5600
*
*-----
* Capacitive feedback network
Ce 0      eml    100pF
Cfb base eml    100pF
Cbb base bb     470pF
Lb bb      0      6uH
*
*-----
* Simulation control for automated oscillator analysis
*
.HBOSC tones=1.0e7 nharms=15
+PROBENODE=emitter,0,4.27
+FSPTS=40,9.e6,1.1e7
*
.PHASENOISE V(emitter) DEC 10 10K 1MEG
+METHOD=0 CARRIERINDEX=1
*
.print hbosc vm(eml) vp(eml) vr(emitter) vi(emitter)
.print hbosc vm(emitter) vp(emitter) P(Rload)
.print phasenoise phnoise
.probe phasenoise phnoise
.probe hbosc v(emitter) v(eml)
.include bjt.inc
.END

```

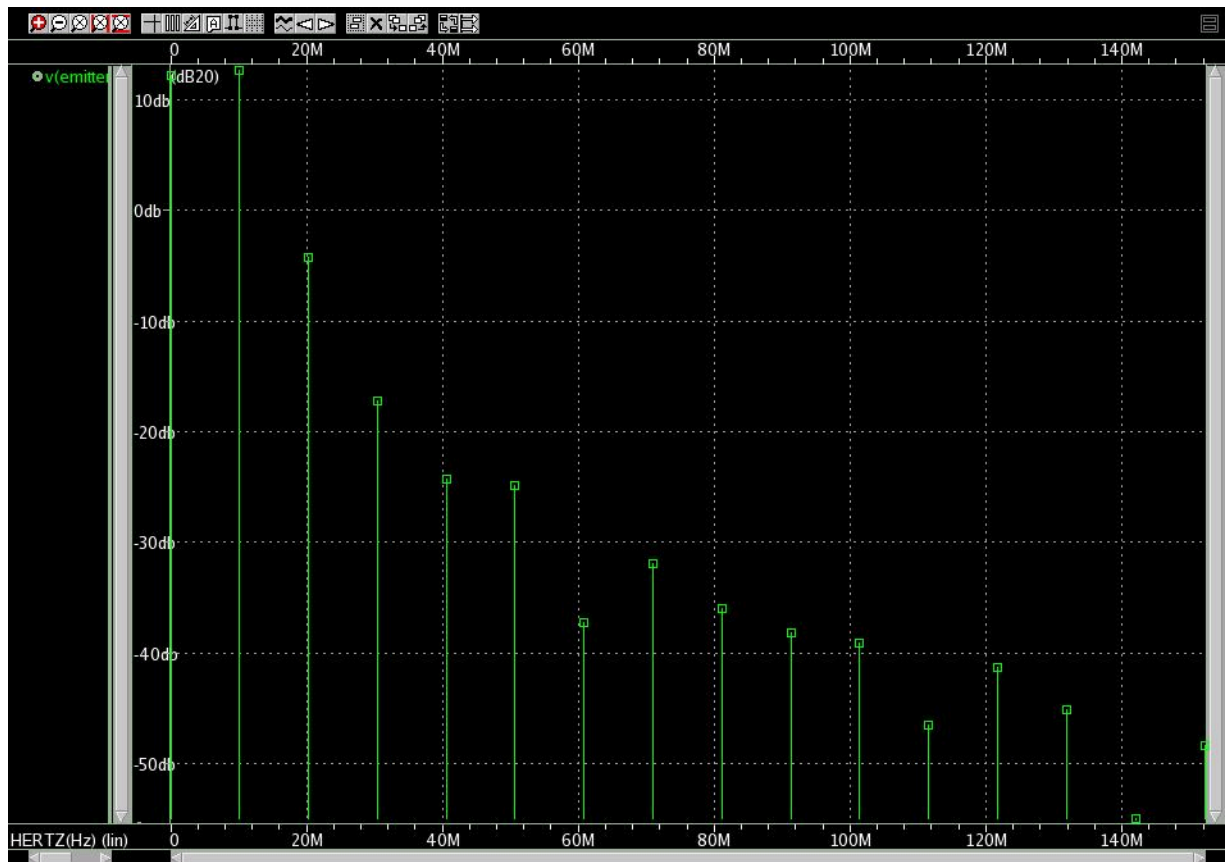

After you run this netlist, examine the `osc.printhb0` file.

- At the top is the oscillator frequency (about 10.14 MHz) and the `.PRINT HBOSC` output.
- The first two lines show that the `eml` node oscillates around 3 V with an amplitude of about 2.85 V.
- The `emitter` node oscillates around 4V with an amplitude of about 4.27 V.

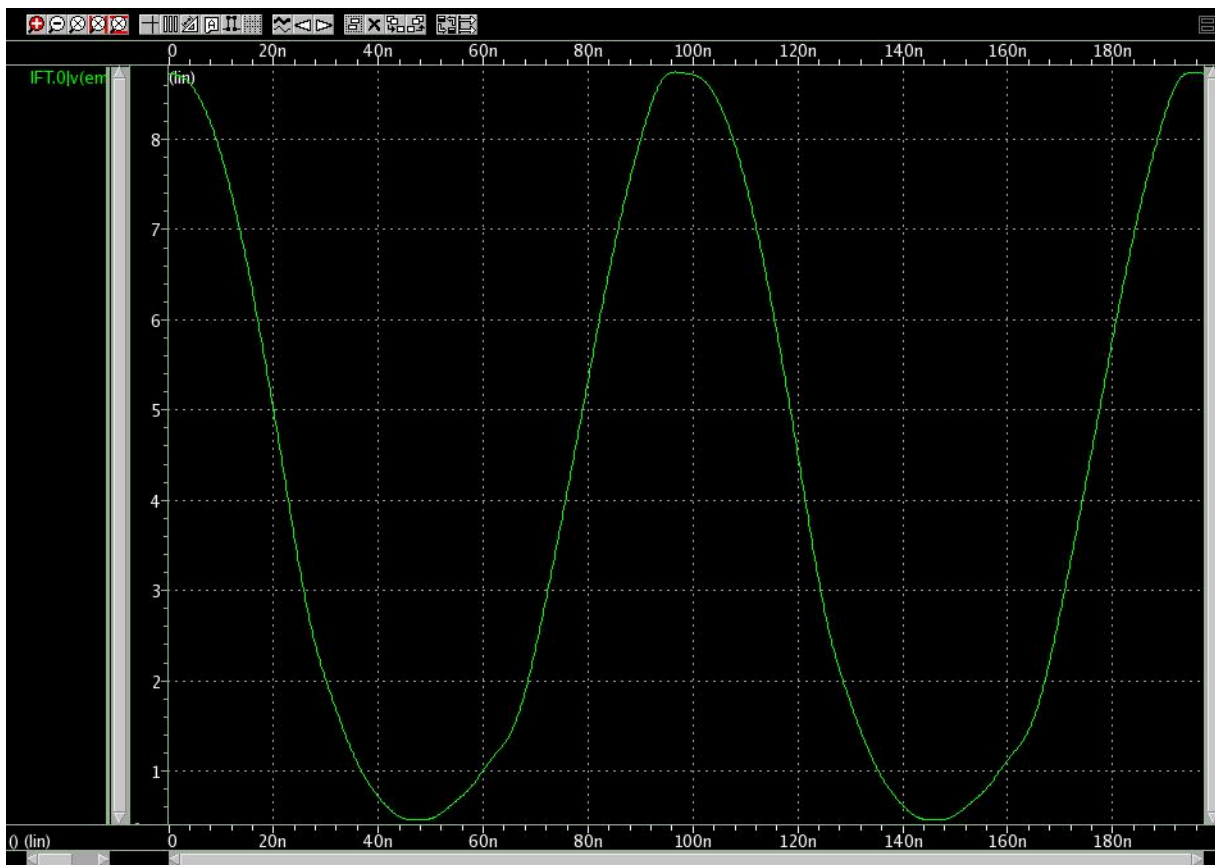
Also examine the `osc.printpn0` file which contains the phase noise results in text form.

You can view the `osc.hb0` and `osc.pn0` files in WaveView.

1. Type **wv** at the prompt to invoke WaveView.
2. Click the **Open Waveform** icon and select the `osc.hb0` file from the Open: Waveform Files dialog box.
3. Select the `v(emitter)` signal in the signal browser. Double-click on the signal name or drag and drop the signal into the waveview.



4. In the waveview, right-click in the name area of the panel containing the signal `v(emitter)`, left-click on the waveform label for `v(emitter)` in the waveview. From the Panel menu, choose **Signal 'v(emitter)' > To Time-Domain**.
5. To accept the defaults for range and interval, click **OK** in the Convert to Time domain window.
6. In a new waveview, you should now see a time domain waveform named IFT.0|v(emitter).



To run a transient simulation for comparison:

1. Use the `.TRAN 1n 10u` command.
2. Add `ic=10n` to the `Lb` inductor.

The resulting waveforms should be the same as those from the HB oscillator analysis.

Time domain waveforms can also be output from HB analysis. In the netlist, include the following `.PROBE` statement:

```
.probe hbtr v(emitter)
```

Example 2 — Using HBOSC for a CMOS GPS VCO

You activate the oscillator analysis by using the `.HBOSC` command:

- The following example uses the demonstration netlist `gpsvco.sp`, which is available in the directory `$installdir/demo/hspice/rf_examples/`. This netlist simulates the oscillator schematic [Figure 11](#) and performs phase noise analysis.

```

**
** NMOS IC Quadrature VCO circuit for GPS local oscillator
**
** Twin differential negative resistance VCOs using NMOS
** transistors for varactors, coupled to produce quadrature
** resonances.
** Design based on 0.35um CMOS process.
**
** References:
** >P. Vancorenland and M.S.J. Steyaert, "A 1.57-GHz fully
** integrated very low-phase-noise quadrature VCO,"
** IEEE Trans. Solid-State Circuits, May 2002, pp.653-656.
** >J. van der Tang, P. van de Ven, D. Kasperkovitz, and A. Roermund,
** "Analysis and design of an optimally coupled 5-GHz quadrature
** LC oscillator," IEEE Trans. Solid-State Circuits, May 2002,
** pp.657-661.
** >F. Behbahani, H. Firouzkouhi, R. Chokkalingam, S. Delshadpour,
** A. Kheirkhani, M. Nariman, M. Conta, and S. Bhatia,
** "A fully integrated low-IF CMOS GPS radio with on-chip analog
** image rejection," IEEE Trans. Solid-State Circuits, Dec. 2002,
** pp. 1721-1727.
** Setup for Harmonic Balance Analysis
** Oscillation Frequency: ~ 1575 MHz (GPS L1 frequency)
** Amplitude: ~5 Volts peak-to-peak (zero to 5V)
** Vdd: 2.5 V
**
** PrimeSim HSPICE Simulation Options:
*.option delmax=1n ACCURATE LIST NODE
**
** PrimeSim HSPICE advanced analog simulation options :
.option sim_accuracy=10
**
*.option savehb='a.hbs' loadhb='a.hbs'
.option POST
.param Vtune=2.0 $ Failures: vtune=1
.param Cval=0.2p
*-----
Vtune vc gnd DC Vtune
Vdd vdd gnd 2.5
*-----
* First oscillator section
** Low-Q resonator with Vdd at center tap of inductors
R1a IP ri 100k $ These R's set the Q
R1b ri IN 100k
L1 IP vdd 16.5nH
L2 vdd IN 16.5nH
Cc1 IP gnd Cval $ I to Q
Cc2 IN gnd Cval $ -I to Q
** Differential fets
M1 IP IN cs gnd NMOS l=0.35u w=15u
M2 IN IP cs gnd NMOS l=0.35u w=15u
** Bias fet - bias at Vdd -- too high?
Mb cs vdd gnd gnd NMOS l=0.35u w=15u

```

Chapter 5: Oscillator Analysis

Oscillator Analysis (.HBOSC and .SNOSC)

```

** fets used as varactors
Mt1 vc IP vc gnd NMOS l=0.35u w=2u M=50
Mt2 vc IN vc gnd NMOS l=0.35u w=2u M=50
*-----
** Second oscillator section
** Low-Q resonator with Vdd at center tap of inductors
R1a_b QP ri_b 100k $ These R's set the Q
R1b_b ri_b QN 100k
L1_b QP vdd 16.5nH
L2_b vdd QN 16.5nH
Cc1_b QP gnd Cval $ -Q to -I
Cc2_b QN gnd Cval $ -Q to I
** Differential fets
M1_b QP QN cs_b gnd NMOS l=0.35u w=15u
M2_b QN QP cs_b gnd NMOS l=0.35u w=15u
** Bias fet - bias at Vdd -- too high? 2nd in parallel
Mb_b cs_b vdd gnd gnd NMOS l=0.35u w=15u
** fets used as varactors
Mt1_b vc QP vc gnd NMOS l=0.35u w=2u M=50
Mt2_b vc QN vc gnd NMOS l=0.35u w=2u M=50
*
*-----
* Differentiators Coupling transistors for quadrature
*
.param Cdiff=0.14p difMsize=50u
vidiff dbias gnd 1.25
viqdiff vdcdif gnd 1.75
Midiff1 dQP dbias gnd gnd NMOS l=0.35u w=difMsize
Midiff2 dQN dbias gnd gnd NMOS l=0.35u w=difMsize
Midiff3 dIN dbias gnd gnd NMOS l=0.35u w=difMsize
Midiff4 dIP dbias gnd gnd NMOS l=0.35u w=difMsize
Cdiff1 dQP QP Cdiff
Cdiff2 dQN QN Cdiff
Cdiff3 dIN IN Cdiff
Cdiff4 dIP IP Cdiff
Mc_QP1 IP vdcdif dQP gnd NMOS l=0.35u w=difMsize
Mc_QN2 IN vdcdif dQN gnd NMOS l=0.35u w=difMsize
Mc_QN3 QP vdcdif dIN gnd NMOS l=0.35u w=difMsize
Mc_QP4 QN vdcdif dIP gnd NMOS l=0.35u w=difMsize
*-----
* Transient Analysis Test Bench
* stimulate oscillation with 2mA pulse
*iosc IP IN PULSE ( 0 2m .01n .01n .01n 10n 1u )
*.probe tran v(IP) v(IN)
*.print tran v(IP) v(IN)
*.TRAN .01n 10n
*-----
* Harmonic Balance Test Bench
*
.sweepblock vtune_sweep
+ 0 5 0.2
+ 2 3 0.1

```

Chapter 5: Oscillator Analysis

Oscillator Analysis (.HBOSC and .SNOSC)

```
.HBOSC tones=1550e6 nharms=12
+ PROBENODE=IP,QN,4
+ sweep Vtune sweepblock=vtune_sweep
**
.phasenoise v(IP,IN)dec 10 100 1e7
.print phasenoise phnoise
.probe phasenoise phnoise
.print hb v(IP,IN) v(IP,IN) [1] v(QP,QN) v(QP,QN) [1]
.probe hb v(IP,IN) v(IP,IN) [1] v(QP,QN) v(QP,QN) [1]
.probe hbtr v(IP,IN) v(IP,IN)
.probe hb hertz[1]
*
.END
```

Viewing Results Using WaveView

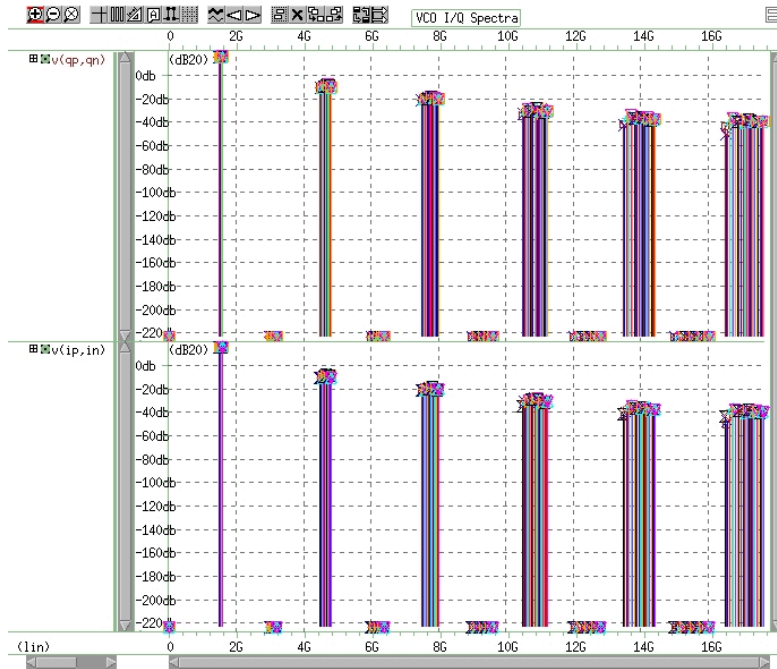
You can view the VCO frequency and time domain waveforms, tuning curves, and phase noise response using WaveView.

To view the VCO frequency domain waveform:

1. Type `wv` at the prompt to invoke WaveView.
2. Click the **Open Waveform** icon and select the `gpsvco.hb0` file.

3. Select the signals $v(qp, qn)$ and $v(ip, in)$ in the signal browser. Drag and drop the signals into the waveview.

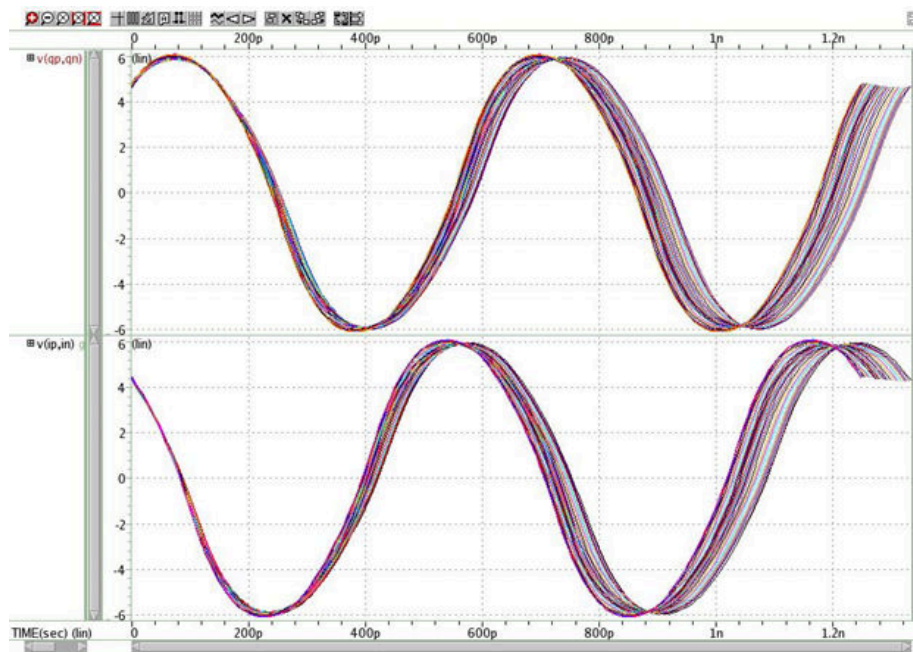
Figure 12 VCO Spectra Output



To view the time domain waveforms:

1. Click the **Open Waveform** icon and select the `gpsvco.hr0` file.
2. Select the signals `v(qp,qn)` and `v(ip,in)` in the signal browser. Drag and drop the signals into the waveview.

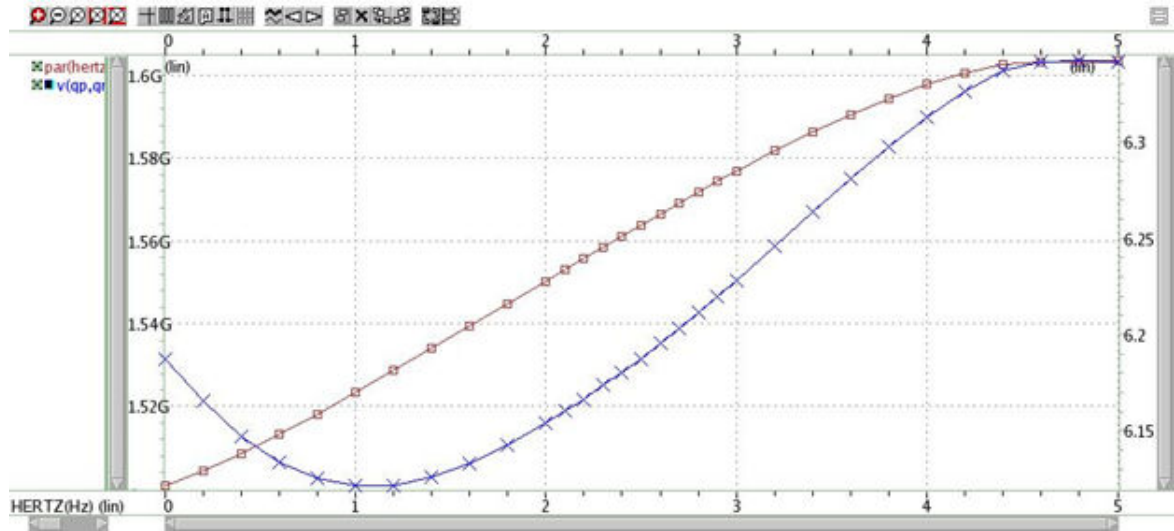
Figure 13 VCO Waveform Output



To view the tuning curves of the VCO:

1. Click the **Waveview** icon in the Add section of the **Waveform** tab (or press CTRL+W) to open a new waveview.
2. Select the signal `par(hertz[1]) : (1)` in the signal browser. Drag and drop the signal into the waveview. This will show the change in frequency as the tuning voltage is changed from 0 to 5 volts.
3. Select the signal `v(qp,qn) : (1)` in the signal browser. Drag and drop the signal into the same waveview as the previous signal. This will show the change in magnitude of the signal as the tuning voltage is changed.
4. Change to the signal mode by clicking on the signal mode icon in the waveview icon bar.
5. Select the `v(qp,qn) : (1)` signal in the waveview. The signal will now be highlighted.

6. Right-click on the highlighted signal. In the menu, scroll down to **Highlighted Signals**. A menu will appear. Scroll down to **Right Axis** and click on it. The right axis will now be the magnitude of the signal $v(qp, qn) : (1)$.
7. To show the data points, click on the toggle data points icon in the icon bar.

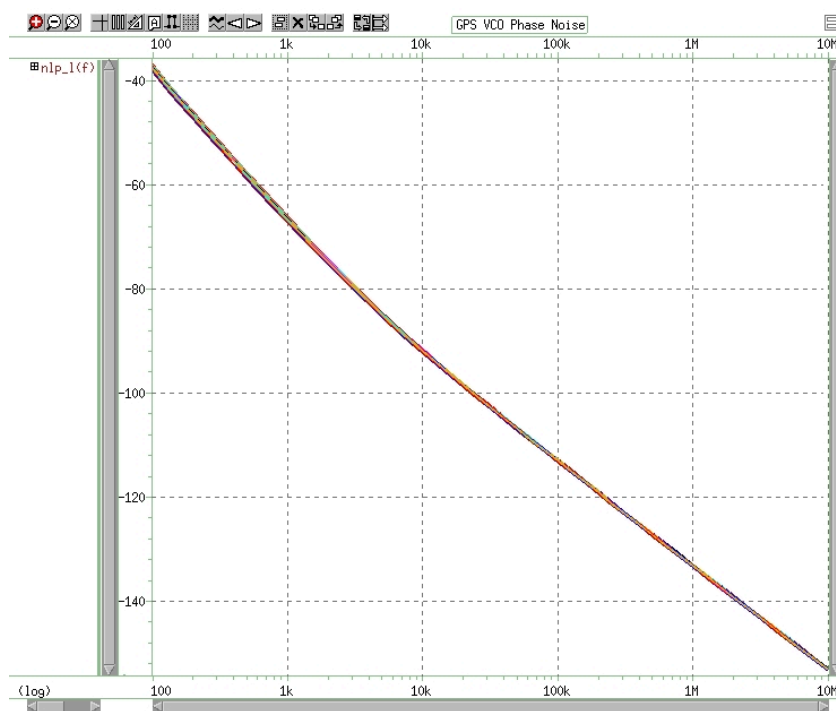


To view the phase noise analysis results:

1. Click the **Open Waveform** icon and select the `gpsvco.pn0` file.
2. Click the **Waveview** icon in the Add section of the **Waveform** tab (or press CTRL+w) to open a new waveview.
3. Select the signal `nlp_1(f)` in the signal browser. Drag and drop the signal into the waveview.

4. The X-axis scale will be linear and it is preferred to use the log scale. To change the X-axis scale to the log scale, right-click the X-axis. A menu will appear. Select **log** in this menu.

Figure 14 VCO Phase Noise Response



.HBOSC Output Syntax

The output syntax for .HBOSC analysis is identical to that for HB analysis (see [Chapter 3, Harmonic Balance Analysis](#)). To output the final frequency of oscillation, use the `HERTZ` keyword. For example, `HERTZ[1]` identifies the fundamental frequency of oscillation.

Note:

For `PROBENODE = n1 n2 vp`, where `vp` is a voltage, the units must be in volts.

See also [Outputting Phase Noise Source as ASCII Data Files Using *.printpn0](#).

Using the .MEASURE Command with .HBOSC

Since .HBOSC requires an .HB analysis, the measure statements for this analysis are the same as for .HB analysis. For example,

```
.MEASURE HB result FIND out_var AT=val
```

HB Simulation of Ring Oscillators

Ring oscillators require a slightly different simulation approach in HB analysis. Since their oscillation is due to the inherent delay in the inverters of the ring, they are best modeled in the time domain and not in the frequency domain.

In addition, ring oscillator waveforms frequently approach square waves, which require a large number of harmonics in the frequency domain. An accurate initial guess is important for an accurate HB simulation.

The PrimeSim HSPICE advanced analog HBOSC analysis typically starts from the DC solution and looks for potential resonances in the linear portion of the circuit to determine the initial guess for the oscillation frequency. However, these resonances generally do not exist in ring oscillators which do not contain linear resonant elements.

HB analysis provides a second method of obtaining a good initial guess for the oscillation frequency, specifically intended for ring oscillators. Instead of starting from the results of a DC analysis, this method starts from the result of a transient analysis. This method, *Transient Initialization*, also provides a good initial guess for all the voltages and currents in the circuit.

The recommended setup for ring oscillators is therefore:

- Set up `.HBOSC` without `FSPTS`.
- Choose one of the nodes in the ring as the `PROBENODE`.
- Recommendation: Since ring oscillators tend to have square-wave-like output signals which have significant high frequency content, use a relatively large value, perhaps 50, for `nharms`. Ring oscillators with more stages tend to need more harmonics.
- Set `HBTRANINIT` to a value that represents ~5-10 oscillator periods. Ensure that you include an `.ic` command or other transient analysis setup to start the oscillator in transient simulation. Longer `HBTRANINIT` times may result in faster HBOSC convergence, at the expense of additional CPU time spent on `HBTRANINIT`.

Troubleshooting Convergence Problems

This section lists the most common causes of convergence problems and how to recognize them and resolve them.

The PrimeSim HSPICE advanced analog harmonic balance oscillator analysis consists of a two-tier iterative analysis of *inner loop* and *outer loop* iterations. In the outer loop iteration, HBOSC iterates to reduce any reported “probe errors” for each outer loop iteration. Each outer loop iteration involves a non-autonomous HB circuit solution; this non-autonomous solve is the *inner loop iteration*.

If HBOSC has inner loop convergence problems, the simulation may hang on the first outer loop iteration or you may see warning messages such as:

```
Warning: HB_WARN.3: Final HB residual value > HB_TOL.  
Rank of HB_Jacobian = 155  
Warning: HB_WARN.3: HB convergence failure in non-autonomous HB.
```

The simulation lists the probe voltage and probe frequency for each outer loop iteration. If an outer loop convergence problem occurs, you may see the following:

- Decreasing probe voltage values.
- Wildly fluctuating values of probe frequency.

```
Osc probe : voltage = 0.218234 frequency = 6.240794122744832e+09
```

- A warning message which indicates that the oscillator simulation has reached a non-oscillating DC solution.

```
Warning: HB_ERR.18: HB oscillator analysis has reached the NULL  
solution.
```

The following sections discuss these topics:

- [General Convergence Issues](#)
- [Outer Loop Convergence](#)
- [Inner Loop Convergence](#)

General Convergence Issues

The following sections discuss these topics:

- [Probe Node Location](#)
- [Incorrect Source Values](#)
- [GMRES Convergence](#)
- [Accuracy of Initial Guess](#)

Probe Node Location

Since convergence is sensitive to the probe node location, you can often track convergence problems to this setting.

A common scenario is that the oscillator's output signal passes through one or more buffers and a designer may think of the buffer output as the oscillator output. A frequent mistake places the probe node at the output of the buffer, but this causes HB convergence problems. In this case, move the probe node to the oscillator part of the circuit. Often, it is necessary to select an internal node of a subcircuit to achieve this.

The most typical symptom of this problem is inner loop convergence failure. For ring oscillators, always choose a node that is part of the ring, that is, connecting two stages of the ring; for harmonic oscillators, choose a node close to the oscillator.

Incorrect Source Values

If the original netlist simulates the oscillator in transient analysis, some voltage or current sources may have transient descriptions (for example, PWL) to start the oscillator. For example, you can ramp a voltage supply to simulate a power-up to start the oscillator:

```
Vvdd vdd 0 PWL (0 0 1n 3)
```

In this case, the user would like HBOSC to use 3 as the voltage source value, but HBOSC uses 0 because harmonic balance uses the explicit DC value of the source. PrimeSim HSPICE advanced analog analyses tries to interpret your sources intelligently but, in some cases, it may not be able to determine what you intended.

For the above example, there are a few ways to ensure that PrimeSim HSPICE advanced analog analyses correctly interprets the source.

- Remove the explicit DC value. If you only provide a transient description, HB uses the time=infinity value of the source.
- Add `TRANFORHB=1`

```
Vvdd vdd 0 PWL (0 0 1n 3) TRANFORHB=1
```

The `TRANFORHB=1` keyword causes HB to use the transient analysis description in HB and HBOSC.

- Add an explicit HB value

```
Vvdd vdd 0 PWL (0 0 1n 3) HB 3 0 0
```

This causes HB to treat the source as a 3V DC source since the HB value specifies the 0th harmonic). If you provide an HB value, PrimeSim HSPICE advanced analog analyses ignore the PWL description and uses “HB 3 0 0” (amplitude=3, phase=0, harmonic=0) instead. The PWL description still applies for HBTRANINIT.

Incorrect source values usually result in the following:

- High residual value after HBTRANINIT. Usually, HBTRANINIT should produce a good starting point for HB or HBOSC. Typical residuals after HBTRANINIT are 1e-4 or 1e-5. If the initial residual, printed immediately after HBTRANINIT completes, is high, there may be a source problem. In the VDD ramping example above, you might see

a residual value of 3. Outer loop may converge to a DC solution because incorrect source values result in a non-oscillatory circuit:

```
Warning: HB_ER.18: HB oscillator analysis has reached the NULL  
solution.
```

- In some cases, inner loop non-convergence may occur.

GMRES Convergence

PrimeSim HSPICE advanced analog analyses use a GMRES iterative solver to solve the linear systems that arise on each inner loop Newton-Raphson step. If GMRES does not solve the linear systems accurately enough, then the inner loop may not converge.

The GMRES solver is controlled by two options:

- **HBKRYLOVTOL**: relative tolerance for GMRES solver. Default is 0.01, or 1%. For some circuits, setting this option helps inner loop convergence:

```
.option HBKRYLOVTOL=1e-3
```

- **HBKRYLOVDIM**: dimension of Krylov subspace to use in GMRES iteration. Also controls maximum number of GMRES iterations. The PrimeSim HSPICE advanced analog analyses .lis file lists the number of GMRES iterations taken for each Newton-Raphson step. If that number is equal to **HBKRYLOVDIM**, you may improve convergence by increasing **HBKRYLOVDIM**. For example:

```
.option HBKRYLOVDIM=80
```

The symptom for GMRES convergence difficulty is always inner loop convergence failure, or slow inner loop convergence. If this problem occurs, the inner loop convergence is often good until the residual reaches a fairly low value like 1e-8 or 1e-7, and then stagnates.

Accuracy of Initial Guess

Both inner loop and outer loop convergence improves significantly if the starting point or initial guess of the iterative method is good.

Outer Loop Convergence

For outer loop convergence, the initial guess consists simply of the oscillation frequency and the first harmonic amplitude at the probe node location. If inner loop convergence is successful but outer loop convergence is not, then you may need to provide a better frequency or amplitude guess.

If you use HBTRANINIT, then you can improve the accuracy of the initial guess by use of one of the following methods:

- Increase the HBTRANINIT time, simply by increasing the value of the HBTRANINIT option.
- Increase the HBTRANINIT accuracy. You can increase the transient analysis accuracy by setting `.option DELMAX` or `.option SIM_ACCURACY`. For example, you may set:

```
.option SIM_ACCURACY=10 HBTOL=1e-8
```

SIM_ACCURACY simultaneously tightens transient and HB accuracy tolerances. If you want HB accuracy to remain unaffected, you may also want to set HBTOL as in the example above.

- Increase accuracy of the time domain to frequency domain conversion of HBTRANINIT results, by increasing HBTRANPTS or equivalently, decreasing HBTRANSTEP. For example:

```
.option HBTRANSTEP=1p
```

If you are using FSPTS, you can increase the number of points. Sometimes, it is best to supply a guess manually by removing FSPTS and adjusting the TONES value.

If you do not use HBTRANINIT, you may be able to improve convergence by manually adjusting the PROBENODE amplitude guess.

To evaluate the effectiveness of your option settings, look at the “probe error” reported after the first outer loop iteration:

```
Iteration 1
Osc probe : voltage = 0.2 frequency = 5.980000000000000e+09
hb residual = 7.628260e-10
Rank of HB Jacobian = 9102
Probe error = 0.000154462
dv = -0.0411324 df = -2.30464430e+08
```

A smaller probe error value indicates a better initial guess.

Inner Loop Convergence

If inner loop convergence is a problem, it may be because the initial voltage waveform values are not close to the solution. The only way to improve the voltage values is by using HBTRANINIT. While this does not work well for harmonic oscillators, it does work well for ring oscillators. You can improve the accuracy of HBTRANINIT as described in the outer loop convergence section above.

If the initial residual is large after HBTRANINIT, you may want to check to make sure that the voltage and current sources are consistent between HB and transient analysis.

The following section discuss these topics:

- [Insufficient Number of Harmonics](#)
- [Presence of Frequency Divider](#)

Insufficient Number of Harmonics

If the number of harmonics specified is too small to represent the signals present in the circuit, you may see convergence problems in either the inner or the outer loop, or the solution may converge to an unreasonable frequency value.

It is difficult to know when the number of harmonics is insufficient, but if you suspect an insufficiency, it is a simple experiment to increase the value of NHARMS. If you achieved convergence and the number of harmonics is large enough, then the magnitude of the spectral data for all signals should significantly decay with increasing frequency. If the spectral data for node voltages has not decayed at the highest harmonics included in the simulation, increase the value of NHARMS.

Presence of Frequency Divider

If a frequency divider is present and not accounted for by the SUBHARMS setting, convergence is not possible because the Harmonic Balance spectrum does not include the necessary low frequency components. As a result, inner loop convergence fails. When debugging HBOSC convergence problems, it is necessary to rule out the possibility of the presence of frequency dividers early in the process.

If a frequency divider is present, you can simulate the circuit if you set SUBHARMS to the largest frequency division present in the circuit. If a frequency divider is present, it is almost always necessary to use the HBTRANINIT option to achieve convergence.

Shooting Newton Oscillator Analysis (.SNOSC)

The analysis described in [Chapter 4, Shooting Newton Analysis](#) also provides a very effective means for finding the steady-state for oscillator circuits.

Ring oscillators are best suited for time domain analysis by using Shooting Newton because they tend to:

- have a low Q
- operate based on digital delays
- have a strongly nonlinear behavior
- output signals that are piece-wise-linear or square-wave-like

HBOSC is superior for sinusoidal waveforms. As with the Harmonic Balance approach, the goal is to solve for the additional unknown oscillation frequency. Shooting Newton

accomplishes this by considering the period of the waveform as an additional unknown, and solving the boundary conditions at the waveform endpoints that coincide with the steady-state operation. As with regular Shooting Newton analysis, you can specify the input in terms of time or frequency values.

For more information on the control options, see the `.SNOSC` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

.SNOSC Output Syntax

The output syntax for `.SNOSC` analysis is identical to that for SN analysis (see [Chapter 4, Shooting Newton Analysis](#)). To output the final frequency of oscillation, use the `HERTZ` keyword. For example, `HERTZ[1]` identifies the fundamental frequency of oscillation.

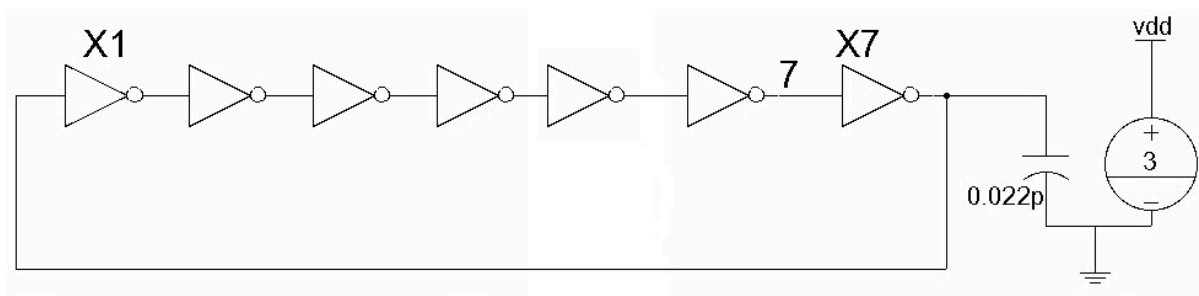
See also [Using Noise Analysis Results as Input Noise Sources](#).

Ring Oscillator Example

The Shooting Newton algorithm provides a fast and effective analysis for ring oscillators. The `ringoscSN.sp` input file for this example is located at

```
$installdir/demo/hspice/rf_examples/.
```

Figure 15 Ring Oscillator



```
.title ringosc

.subckt inv in out vdd
mn1 out in 0 0 nmos l=0.25u w=2u
mp1 out in vdd vdd pmos l=0.25u w=6u
.ends

vdd vdd 0 3
x1 1 2 vdd inv
x2 2 3 vdd inv
x3 3 4 vdd inv
x4 4 5 vdd inv
x5 5 6 vdd inv
x6 6 7 vdd inv
```

```
x7 7 1 vdd inv
c1 1 0 0.022p
.ic v(1)=3
.options post

.snosc tones=335meg nharms=10 oscnode=1 trinit=10n
.phasenoise v(7) dec 10 100 10meg

.probe sn v(7)
.probe snfd v(7)
.print phasenoise phnoise v(7)
.probe phasenoise phnoise v(7)

.end
```

This analysis finds the oscillation frequency of the ring oscillator. Since the circuit is an oscillator, no input source is required. The oscillator is started by setting an initial condition at the input of the ring (node 1). In the `.SNOSC` command, the node that the analysis will probe for oscillation conditions is specified, as well as the approximate frequency of oscillation. The number of harmonics to include in the analysis is specified as well.

The phase noise characteristics of the oscillator are analyzed by using the `.PHASENOISE` command. The `.PHASENOISE` command requires that an output node, a pair of nodes, or a two-port element and a frequency sweep be specified. The frequency sweep is used to calculate the phase noise analysis at the specified offset frequencies, measured from the oscillation carrier frequency. For this example of phase noise analysis, the default Nonlinear Perturbation (NLP) method is used.

The signal `v(7)` will be probed in both the frequency and time domain. The `measure` statement is used to measure the fundamental frequency of the oscillator.

Simulation Status Output

During the simulation, the simulation status is displayed on the screen. In addition to the screen display, a more detailed status, CPU time, and memory usage information are also written to the `ringoscSN.lis` file.

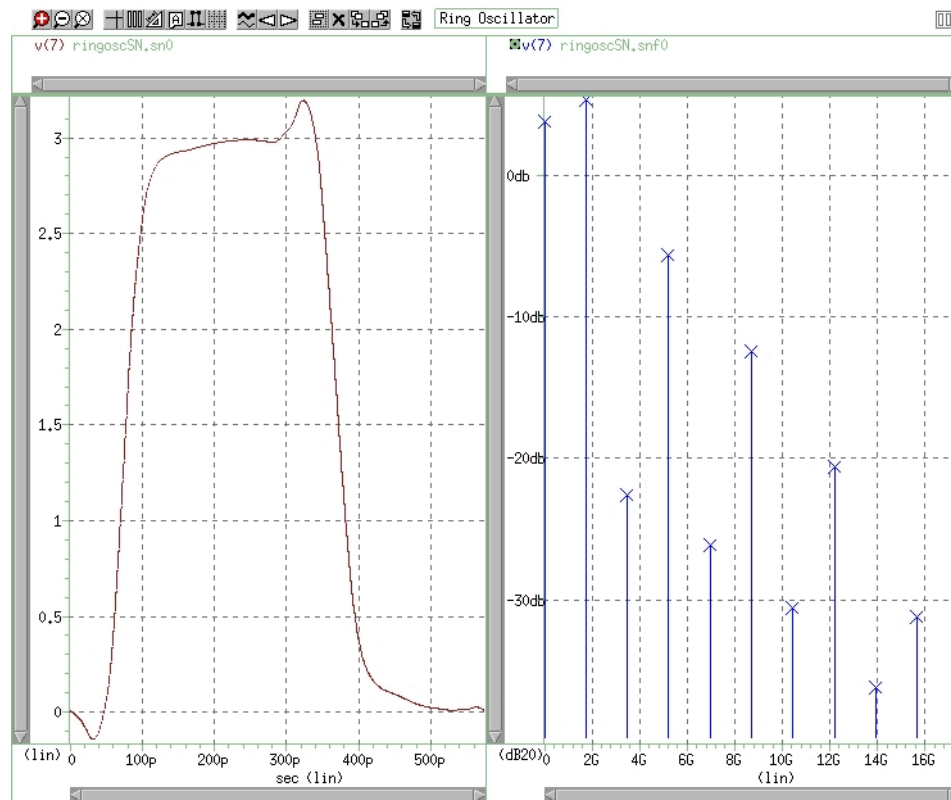
Viewing Results in WaveView

You can view the time-domain `ringoscSN.sn0` file, the frequency domain `ringoscSN.snf0` file, and the phase noise `ringoscSN.snpn0` file in WaveView.

1. Enter **wv** at the prompt to start WaveView.
2. Click the **Open Waveform** icon and select the `ringoscSN.sn0` file.
3. Select the signal `v(7)` from the signal browser. Drag and drop the signal `v(7)` to the right side of the waveform so that the panels are opened in row / column format. The time domain trace is shown at the right side of [Figure 16 on page 99](#).

4. Click the **Open Waveform** icon and select the `ringoscSN.snf0` file.
5. Select the signal `v(7)` from the signal browser. Drag and drop the signal `v(7)` to the right side of the waveform so that the panels are opened in row / column format. The frequency domain spectrum is shown at the left side of [Figure 16](#).

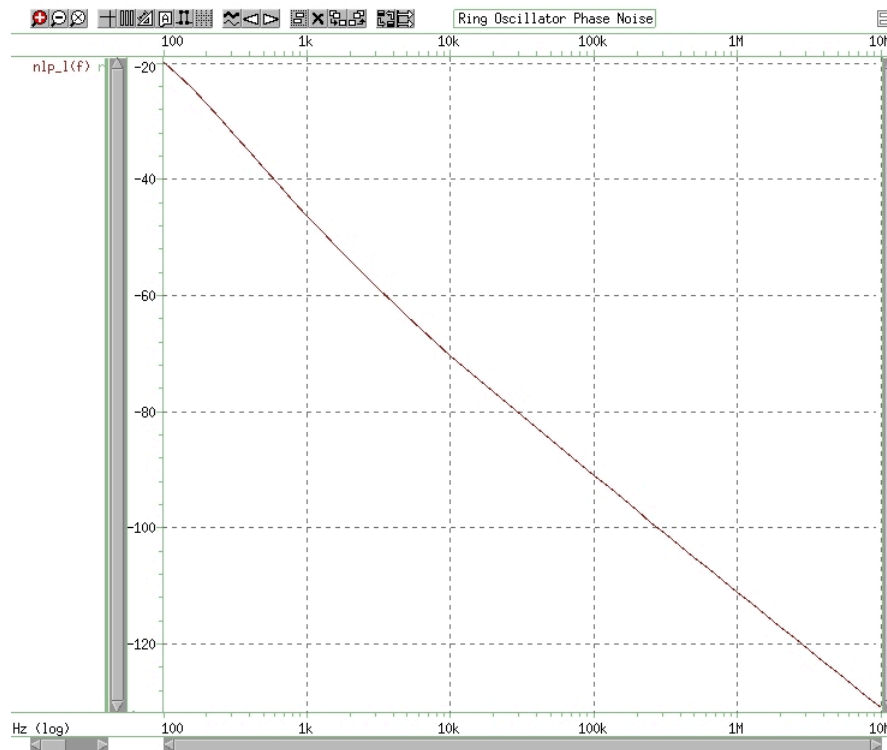
Figure 16 Ring Oscillator Output



6. Click the **Open Waveform** icon and select the `ringoscSN.snpn0` file.
7. Click the **Waveview** icon in the Add section of the **Waveform** tab (or press CTRL+w) to open a new waveview.

8. Select the signal `nlp_1(f)` from the signal browser. Drag and drop the signal `nlp_1(f)` into the waveview. [Figure 17 on page 100](#) shows the resulting phase noise results for the oscillator.

Figure 17 Ring Oscillator Phase Noise



References

- [1] E. Ngoya, A. Suarez, R. Sommet, R. Quere, "Steady State Analysis of Free or Forced Oscillators by Harmonic Balance and Stability Investigation of Periodic and Quasi-Periodic Regimes," *International Journal of Microwave and Millimeter-Wave Computer-Aided Engineering*, Volume 5, Number 3, pages 210-223 (1995)
- [2] C.R. Chang, M.B. Steer, S. Martin, E. Reese, "Computer-Aided Analysis of Free-Running Microwave Oscillators," *IEEE Trans. on Microwave Theory and Techniques*, Volume 39, No. 10, pages 1735-1745, October 1991.
- [3] G.D. Vendelin, *Design of Amplifiers and Oscillators by the S-Parameter Method*, John Wiley & Sons, 1982
- [4] A. Demir, A. Mehrotra, J. Roychowdhury, "Phase Noise in Oscillators: A Unifying Theory and Numerical Methods for Characterization" in Proc. IEEE DAC, pages 26-31, June 1998.

- [5] A. Demir, A. Mehrotra, and J. Roychowdhury, "Phase Noise in Oscillators: A Unifying Theory and Numerical Methods for Characterization," *IEEE Trans. Circuits System I*, Volume 47, pages 655–674, May 2000.
- [6] A. van der Ziel, *Noise in Solid State Devices and Circuits*, John Wiley & Sons, 1986.
- [7] A. Hajimiri, S. Limotyrakis, and T.H. Lee, "Jitter and phase noise in ring oscillators," *IEEE J. Solid-State Circuits*, vol. 34, no. 6, pp. 790-804, June 1999.
- [8] *Jitter Analysis Techniques for High Data Rates*, Application Note 1432, Agilent Technologies, Feb. 2003.
- [9] *Characterization of Clocks and Oscillators*, NIST Technical Note 1337, National Institute of Standards and Technology.
- [10] G.V. Klimovitch, "Near-carrier oscillator spectrum due to flicker and white noise," *Proc. ISCAS 2000 (Geneva)*, May 2000.

6

Phase Noise Analysis

Describes phase noise analysis.

Phase Noise analysis requires first running either the harmonic balance (HBOSC) or Shooting Newton (SNOSC) analysis, and then PHASENOISE analysis. The PHASENOISE analysis itself is identical whether you run SNOSC or HBOSC.

For more information on control options, see the .PHASENOISE command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

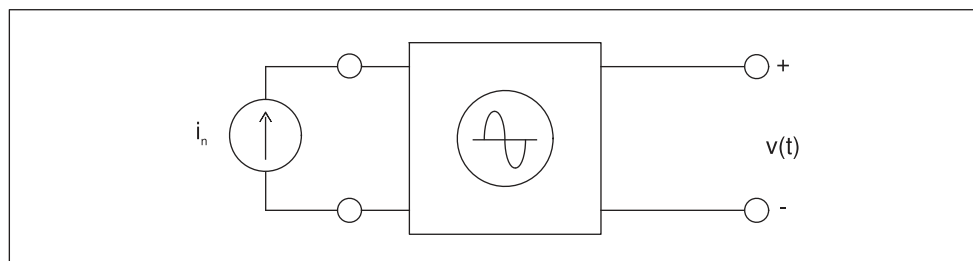
This chapter discusses the following topics:

- [Introduction to Phase Noise Analysis \(.PHASENOISE\)](#)
- [Accumulated Jitter Measurement for Closed Loop PLL Analysis](#)
- [Clock Source with Random Jitter](#)

Introduction to Phase Noise Analysis (.PHASENOISE)

The following figure shows a simple free-running oscillator, which includes a port with injected current.

Figure 18 *Oscillator with Injected Current*



The following equation presents how an ideal oscillator would be insensitive to perturbations with a fixed amplitude and frequency:

Equation 17

$$v(t) = A \cos [\omega_0 t + \phi_0]$$

A noisy oscillator has amplitude and phase fluctuations we can write as:

Equation 18

$$v(t) = A(t) \cos [\omega_0 t + \phi(t)]$$

In the preceding equation:

- $A(t)$ is the time varying amplitude for the noisy oscillator.
- $\phi(t)$ is the time varying phase for the noisy oscillator.
- ω_0 is the frequency of oscillation.

In most applications, the phase noise is of particular interest, because it represents frequency fluctuations about the fundamental, which you cannot remove. These fluctuations are random processes. Typical expressions are in terms of their power spectral density. For most oscillators, the phase noise is a low-frequency modulation that creates sidebands in the oscillator's spectrum, about ω_0 .

For example, the following equation represents a simple sinusoidal variation in the phase:

Equation 19

$$v(t) = A \cos [\omega_0 t + \theta_p \sin \omega_m t]$$

- θ_p is the peak phase deviation, specified as $\theta_p = \Delta\omega / \omega_m$.
- $\Delta\omega$ is the peak angular frequency deviation.

For $\theta_p \ll 1$, the following equation approximates the output:

Equation 20

$$v(t) = A \left\{ \cos(\omega_0 t) - \frac{\theta_p}{2} [\cos(\omega_0 + \omega_p)t - \cos(\omega_0 - \omega_p)t] \right\}$$

That is, when the peak phase deviation is small, the result is frequency components on each side of the fundamental with amplitude $\theta_p / 2$.

The Single-Sideband Phase Noise $L(f_m)$ is the ratio of noise power to carrier power in a 1Hz bandwidth, at offset $\omega_m = 2\pi f_m$, which in this case can be written as:

Equation 21

$$L(f_m) = \left(\frac{V_{sb}}{A} \right)^2 = \frac{\theta_p^2}{4} = \frac{\theta_{rms}^2}{2}$$

This model for oscillator noise shows that sidebands about the fundamental, due to noise, directly relate to the spectrum of the phase fluctuations $\theta(t)$. The more general definition of phase noise relates it to the spectral density of phase fluctuations, that is:

Equation 22

$$S_\phi(\omega_m) = \frac{\theta_p^2}{2} = 2L(f_m)$$

PrimeSim HSPICE advanced analog analyses use several sophisticated analysis techniques for computing the power spectrum of the phase variations to yield the phase noise response. This information describes the spectrum of the oscillator about the fundamental frequency, and its random jitter characteristics.

Any .PHASENOISE analysis results in the calculation of a curve fit for a power-law model according to:

Equation 23

$$L(f) = 10 \cdot \log \left\{ \frac{a3}{f^3} + \frac{a2}{f^2} + \frac{a1}{f} + a0 \right\}$$

The .lis file reports the coefficients $a3$, $a2$, $a1$, and $a0$.

The .lis file includes a table which models the phase noise, including the behavioral model fit and its fit error.

```
|-----|
| L(f) = 10*log10( a3/f^(2+ef) + a2/f^2 + a1/f^(ef) + a0 ) dBc/Hz |
| a3 = 0.000000e+00 |
| a2 = 3.165111e-02 |
| a1 = 0.000000e+00 |
| a0 = 0.000000e+00 |
| ef = 1.000000e+00 |
| Average fit error = 1.6185e+00 dB |
| Maximum fit error = 8.4862e+00 dB @ 1.0000e+07 Hz |
```

The following sections discuss these topics:

- [Identifying Phase Noise Spurious Signals](#)
- [Phase Noise Algorithms](#)

- [PHASENOISE Output Syntax](#)
- [Measuring Phase Noise with .MEASURE PHASENOISE](#)
- [Optimizing with PHASENOISE Measurement](#)
- [Amplitude Modulation/Phase Modulation Separation](#)
- [Interpreting Phase Noise Analysis Results](#)

Identifying Phase Noise Spurious Signals

Realistic phase noise responses include spurs. Spurs are contributions to the phase noise that result from deterministic signals present within the circuit. In most cases, the spurs are very small signals and do not interfere with the steady-state operation of the oscillator, but do add energy to the output spectrum of the oscillator. You may need to include the energy that the spurs add in jitter measurements. The phase noise spurs feature adds an additional analysis option that can predict the spurious contributions to the jitter.

To activate the new phase noise spur analysis, use the `SPURIOUS` keyword in the `.PHASENOISE` command. An additional `.HBAC` analysis predicts the spurious contributions to the phase noise.

The use of a voltage or current source can to add spurious signals to an oscillator circuit. The keyword `SPUR` identifies the spurious signal.

Syntax

```
Exxxx n1 n2 ... [SPUR mag phase freq] ... $ voltage spur  
Gxxxx n1 n2 ... [SPUR mag phase freq] ... $ current spur
```

where:

- *mag* is the amplitude in volts or amps
- *phase* is the phase in degrees
- *freq* is the frequency in Hz

The source is equivalent to a steady-state sinusoidal source at the specified amplitude, phase, and frequency values and only the spurious analysis uses it. All other analyses ignore it. The `SPUR` keyword is combinable into a source that other analyses use.

Recommendation: Add `SPUR` sources as separate sources.

Phase Noise Algorithms

PrimeSim HSPICE provides two approaches for oscillator phase noise analysis: Nonlinear Perturbation (NLP) and Periodic AC (PAC). Setting the `METHOD` parameter to 0 or 1 selects

the first or the second approach respectively. The two methods are provided for flexible control of the simulation speed and accuracy. The NLP method is the fastest, while the PAC method has superior accuracy over a wide range of offset frequencies.

This section covers the following algorithms:

- [Nonlinear Perturbation Algorithm](#)
- [Periodic AC Algorithm](#)

Nonlinear Perturbation Algorithm

The NLP algorithm, `METHOD=0`, has the fastest run time and gives very good accuracy for core oscillator circuits and for circuits that do not include power supply noise filtering.

The NLP approach performs a single frequency noise analysis about the fundamental oscillation frequency, and then extrapolates this solution for offset sweep frequencies based on the noise source behavior. This generally gives very good results at low offset frequencies.

The NLP computation time has only a small dependence on the number of frequency points in the phase noise frequency sweep.

Periodic AC Algorithm

The PAC algorithm, `METHOD=1` (default), is intended for maximum accuracy over a broad range of offset frequencies. Use the PAC approach to capture accurate phase noise for oscillators that may have additional buffering, filtering, dividers, and frequency dependent power supply noise effects.

The PAC approach performs a full analysis at each frequency given in the phase noise frequency sweep. This makes it slower than the NLP algorithm, but very accurate for capturing far-carrier noise filtering effects and measuring the floor of the phase noise response.

The computation time for the PAC algorithm is approximately linearly dependent on the number of frequency points in the phase noise sweep. So, use a conservative number of sweep points for best performance. PAC accuracy can generally be improved by using larger values for `NHARMS` (>16) and `RUNLVL` (5 or 6). At low offset frequencies, the PAC approach will give results identical to the NLP method.

PHASENOISE Output Syntax

You can analyze element phase noise through the `.PRINT` and `.PROBE` statements. PrimeSim HSPICE supports producing the output of the phase noise for the whole circuit and the phase noise due to a specified element. In addition, by using specialized keywords, you can output phase noise by noise source types. To generate the output, you must enable (`=on`) the `listsources` option of the `.PHASENOISE` command.

The following sections discuss these topics:

- [Whole Circuit and Specified Element Phase Noise](#)
- [Frequency-Dependent and Frequency-independent Sources](#)
- [Frequency and Bias Dependencies](#)
- [Examples](#)
- [See Also](#)

Whole Circuit and Specified Element Phase Noise

A single `phnoise` keyword specifies the phase noise for the whole circuit and the `phnoise(element_name)` specifies the phase noise value of a specified element in the circuit.

```
.PRINT PHASENOISE phnoise phnoise(element_name)
.PROBE PHASENOISE phnoise phnoise(element_name)
```

In this syntax, the standalone `phnoise` is the phase noise parameter. For example:

```
.PROBE PHASENOISE phnoise
```

The `.PHASENOISE` statement outputs raw data to the `*.pn#` and `*.printpn#` files. PrimeSim HSPICE advanced analog analyses output the `phnoise` data in decibels, relative to the carrier signal, per hertz, across the output nodes in the `.PHASENOISE` statement (Units: dBc/Hz). The data plot is a function of the offset frequency.

When multiple output nodes are specified, multiple `*.printsnpn#` and `*.snpn#` files are generated, each with one output node name in its file name as in `*.printsnpn#@output` and `*.snpn#@output`.

PrimeSim HSPICE advanced analog analyses output `phnoise` to the `.pn#` file if you set `.OPTION POST`.

Frequency-Dependent and Frequency-independent Sources

- The `phnoise_fdep` keyword variable collects all *frequency-dependent* noise sources' contributions to the phase noise.
- The `phnoise_findep` keyword variable collects all *frequency independent* noise sources' contributions.

```
.print phasenoise phnoise_fdep
.print phasenoise phnoise_findep
```

Frequency and Bias Dependencies

Table 1 Summary of Noise Type Dependencies

Noise type	frequency-dependent	bias-dependent
phnoise_stationary	No	No
The following syntax is frequency-independent and bias-independent: .print phasenoise phnoise_stationary		
phnoise_cyclostationary or phnoise_cyclo	No	Yes
The following syntax is frequency-independent and bias-dependent: .print phasenoise phnoise_cyclo or .print phasenoise phnoise_cyclostationary Where: cyclo or cyclostationary means anything bias-dependent.		
phnoise_flicker	Yes	No
The following syntax is bias-independent and frequency-dependent: .print phasenoise phnoise_flicker		
phnoise_cycloflicker	Yes	Yes
The following syntax is frequency-dependent and bias-dependent: .print phasenoise phnoise_cycloflicker or .print phasenoise phnoise_cyclostationaryflicker		
phnoise_fdep	is the union of phnoise_Flicker and phnoise_cycloflicker noise types	
phnoise_findep	is the union of phnoise_stationary and phnoise_cyclostationary noise types	

Examples

Example 1

This example performs an oscillator analysis by searching for frequencies in the vicinity of 900 MHz, followed by a phase noise analysis at frequency offsets from 100 Hz to 10 MHz.

```
.HBOSC TONE=900MEG NHARMS=9
+ PROBENODE=gate,gnd,0.65
.PHASENOISE V(gate,gnd) DEC 10 100 1.0e7
+ METHOD=0 CARRIERINDEX=1 $use NLP algorithm
+ listsources=on
.PROBE PHASENOISE phnoise
.PRINT PHASENOISE phnoise(X1)
```

Example 2

This example performs a VCO analysis by searching for frequencies in the vicinity of 2.4 GHz. This example uses eleven harmonics and sweeps the VCO tuning voltage from 0 to 5 V. PrimeSim HSPICE advanced analog analyses use the nonlinear perturbation (NLP) algorithm to perform a phase noise analysis based on the fundamental frequency for each tuning voltage value.

```
.HBOSC TONE=2400MEG NHARMS=11
+ PROBENODE=drainP,drainN,1.0
+ FSPTS=20,2100MEG,2700MEG
+ SWEEP Vtune 0.0 5.0 0.2
.PHASENOISE V(drainP,drainN) DEC 10 100 1.0e7
+ METHOD=0 CARRIERINDEX=1 $use NLP algorithm
+ listsources=on
.PROBE PHASENOISE phnoise
.PRINT PHASENOISE phnoise(X2)
```

See Also

- [Using Noise Analysis Results as Input Noise Sources](#)

Measuring Phase Noise with .MEASURE PHASENOISE

The PrimeSim HSPICE advanced analog optimization flow can read the measured data from a .MEASURE PHASENOISE analysis. You can combine this flow in the PrimeSim HSPICE advanced analog analyses optimization routine with a .MEASURE HBTR analysis (see [Using .MEASURE with .HB Analyses](#)) and a .MEASURE HBNOISE analysis (see [Measuring HBNOISE Analyses with .MEASURE](#)). The .MEASURE PHASENOISE syntax supports the following measurements:

- FIND

```
.MEASURE PHASENOISE result FIND phnoise at = IFB_value
```

— yields the result of a variable value at a specific input frequency band point. For example:

```
.MEASURE PHASENOISE np1 find PHNOISE at=100K
```

- WHEN

```
.MEASURE PHASENOISE result WHEN phnoise=value
```

—yields the input frequency point at a specific phnoise value. For example:

```
.MEASURE PHASENOISE fcorn1 when PHNOISE=-120
```

- RMS, average, min, max, and peak-to-peak

```
.MEASURE PHASENOISE result func phnoise + [FROM = IFB1] [TO = IFB2]
```

—yields the average, RMS, minimum, maximum, or peak-to-peak value of the phase noise from frequency IFB1 to frequency IFB2, where the value of `func` can be RMS, AVG, MIN, MAX or PP. If you do not specify FROM and TO, PrimeSim HSPICE calculates the value over the frequency range that you specify in the .PHASENOISE command. For example:

```
.measure PHASENOISE agn1 AVG phnoise from=100k to=10meg
```

- Integral evaluation

```
.MEASURE PHASENOISE result INTEGRAL phnoise + [FROM = IFB1] [TO = IFB2]
```

—integrates the phase noise value from the IFB1 frequency to the IFB2 frequency. For example:

```
.MEASURE PHASENOISE rns1 INTEGRAL phnoise from=50k to 500k
```

- Derivative evaluation

```
.MEASURE PHASENOISE result DERIVATIVE phnoise AT = IFB1
```

—finds the derivative of phase noise at the IFB1 frequency point. For example:

```
.MEASURE PHASENOISE fdn1 DERIVATIVE phnoise at=10meg
```

Note:

.MEASURE PHASENOISE cannot contain an expression that uses a phase noise variable as an argument. You also cannot use .MEASURE PHASENOISE for error measurement and expression evaluation of the .PHASENOISE command.

See also the .MEASURE PHASENOISE command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Optimizing with PHASENOISE Measurement

The required statements are:

- Analysis statement

```
.HB TONES=f1[f2...fn] [NHARMS=h1 [,h2...hn]]  
+ SWEEP OPTIMIZE=OPTxxx RESULT=measname MODEL=mname
```

For example:

```
.HBOSC tone=1g nharms=5 PROBENODE=out,gnd,0.8  
+ SWEEP OPTIMIZE=opt1 RESULT=y1,y2 MODEL=m1  
.MODEL m1 OPT level=0
```

```
.PHASENOISE v(out) DEC 1 1k 1G
.MEAS PHASENOISE y1 FIND PHNOISE at 10k goal=-150dBc
.MEAS PHASENOISE y2 RMSJITTER PHNOISE units=sec goal=1p
```

- Measure statement

```
.MEASURE PHASENOISE measname FIND out_var1 AT=val
+ GOAL=val
```

Amplitude Modulation/Phase Modulation Separation

You can separate the Amplitude Modulation (AM) and Phase Modulation (PM) components of the total noise by calculating components in-phase (AM component) and in quadrature (PM component) with the carrier, by using PAC PHASENOISE analysis. The output and measure syntax separates AM/PM noise.

Turn this feature on by setting the `.OPTION PHNOISEAMPM=1` (see `.OPTION PHNOISEAMPM` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*. See also, [Important Note for AM/PM Users](#) at the end of this section).

- If you use the NLP algorithm (`METHOD=0`), PrimeSim HSPICE advanced analog analyses calculate only the phase noise component.
- If you use the PAC algorithm (`METHOD=1`), PrimeSim HSPICE advanced analog analyses add both the phase and amplitude noise components together to show the total noise at the output.

The following sections discuss these topics:

- [AM/PM .PRINT and .PROBE Statement Syntax](#)
- [Examples](#)
- [Output Files](#)
- [Important Note for AM/PM Users](#)

AM/PM .PRINT and .PROBE Statement Syntax

```
.PROBE PHASENOISE phnoise [la] [ltotal] [onoise]
.PRINT PHASENOISE phnoise [la] [ltotal] [onoise]
```

Keywords for AM/PM separations are:

- Phase Modulation term only: `phnoise`
- Amplitude Modulation term only: `la`
- Total phase noise term: `ltotal`
- Voltage noise term: `onoise`

For example:

```
.probe phasenoise phnoise la ltotal $ AM modulation and  
total phase noise
```

Add a Noise type suffix to each of these noise terms, `phnoise`, `la`, `ltotal`, `onoise`, to select specific noise-type components:

Table 2 *Summary of Noise_term*

Noise type	frequency-dependent	bias-dependent
Noise_term_phnoise_stationary	No	No
Noise_term_phnoise_cyclostationary	No	Yes
Noise_term_phnoise_flicker	Yes	No
Noise_term_phnoise_cycloflicker	Yes	Yes
Noise_term_phnoise_fdep	The union of <code>phnoise_Flicker</code> and <code>phnoise_cycloflicker</code> noise types	
Noise_term_phnoise_findep	The union of <code>phnoise_stationary</code> and <code>phnoise_cyclostationary</code> noise types	

Examples

Example 1

```
.PROBE PHASENOISE la_phnoise_cyclostationary
```

You can also show the AM/PM separation for individual noise elements. (To enable this capability, the `listsources` option of `.PHASENOISE` must = `on`.) This example outputs the phase modulation noise associated only with cyclo-stationary sources (that is, sources that are bias dependent, but not frequency dependent).

Noise Element output is of the form `Noise_term(element_name)`, where `Noise_term` can be `phnoise`, `la`, `ltotal`, or `onoise`, and `element_name` is a valid netlist element name.

Example 2

```
.PROBE PHASENOISE la(x1)
```

Output Files

- File `*.printpn#`: Writes output from the `.PRINT` statement when using HB to obtain the steady state solution.
- File `*.pn#`: Writes output from the `.PROBE` statement when using HB to obtain the steady state solution.

- File *.printsnpn#: Writes output from the .PRINT statement when using SN to obtain the steady state solution.
- File *.snpn#: Writes output from the .PROBE statement when using SN to obtain the steady state solution.

The .PHASENOISE command line parameters Listfreq, ListCount, Listfloor, and Listsources control and list noise source contributions sequentially. The listsources argument must be =on to generate a noise list block for each output parameter specified in the .PRINT/.PROBE statement such as phnoise, la, ltotal, and onoise.

.MEASURE Syntax and Outputs

.MEASURE PHASENOISE extends output variables to the set: am[noise] pm[noise]

.MEASURE output files include:

- File *.mpn#: Writes output from the .MEASURE statement when using HB to obtain the steady state solution.
- File *.msnpn#: Writes output from the .MEASURE statement when using SN to obtain the steady state solution.

Important Note for AM/PM Users

There are discrepancies that may occur between this feature and the traditional PAC phase noise analysis in PrimeSim HSPICE advanced analog analyses. Total phase noise (ltotal) is the sum of two terms, the amplitude modulation (am) and phase modulation (phnoise). Traditionally, PAC phase noise reports the phnoise (phase modulation component) and ltotal (total phase noise) terms as identical, with the assumption that the am term (amplitude modulation component) is zero.

The PAC phase noise AM/PM feature described in this section separately calculates the am and phnoise components. The AM/PM feature affects all phase noise measurements that involve PAC. In most cases the differences between PHNOISEAMPM=1 and =0 are small, unless you expect a significant AM component. You may see a slight decrease in the new phase noise (phase modulation) component when compared to the old calculation.

For example, the random jitter calculations are accurate only when they involve the PM component of phase noise. When basing the calculation on ltotal, or if you include the AM noise, the process may introduce a small error.

NLP phase noise (method=0) only calculates the phnoise component; the AM/PM option does not affect this method.

Workaround: Ensure that .OPTION PHNOISEAMPM=0 (the default). This assures that periodic AC phase noise amplitude-modulation (AM) component is zero to maintain backward compatibility and produce the traditional results for phnoise and jitter.

Interpreting Phase Noise Analysis Results

A typical phase noise plot consists of a line which drops off as a function of frequency, at a slope of -20dbc/decade where white noise dominates, or -30dbc/decade where flicker noise dominates. At very low offset frequencies, the phase noise rolls off according to a Lorentzian shape, such that it never exceeds 0 dbc/Hz even for very low offset frequencies. The 0 dbc/Hz value represents the power of the carrier oscillation, at 0 offset frequency. At very high offset frequencies, the slope can deviate from -20 dbc/decade due to the existence of a noise floor or a circuit feedback effect.

Numerical methods for phase noise analysis have limitations. The main limitation in the PAC phase noise algorithm is that it rolls off too quickly at low offset frequencies. In the low frequency region, you can trust NLP. The main limitation of the NLP algorithm is that it does not cover all high frequency effects, so you can trust PAC in the high frequency region.

If you suspect the phase noise results to be inaccurate, check the following:

1. Is the `.HBOSC` steady state solution fully converged?

The NLP or PAC small-signal noise analysis requires a highly accurate steady state solution.

2. Did the phase noise analysis fully converge?

Phase noise analysis uses a GMRES iterative linear solver. If this iterative solver reaches its iteration limit before full convergence, the results are not reliable. Check the number of Krylov iterations that the phase noise analysis required. If it took the maximum number of iterations (as set by `PHASENOISE_KRYLOV_ITR`, default=1000), then the results did not fully converge and you should not trust them. You can use the options `PHASENOISE_KRYLOV_DIM` and `PHASENOISE_KRYLOV_ITR` to control the GMRES solver. You can increase `PHASENOISE_KRYLOV_DIM` to improve the convergence rate at the expense of memory, or increase `PHASENOISE_KRYLOV_ITR` to allow more iterations.

3. Is the phase noise analysis accurate?

In rare circumstances, the linear solver accuracy needs to be improved. The linear solver accuracy is controlled by the `PHASENOISE_TOL` option. The default value of the option is 1e-9. The suggested setting for `PHASENOISE_TOL` is 1e-10, if the accuracy needs to be improved.

Accumulated Jitter Measurement for Closed Loop PLL Analysis

Enhancements to PrimeSim HSPICE advanced analog analyses include considerable support for a variety of jitter measurements. Many of these are important in a PLL flow,

where you use the HBOSC or SNOSC analyses to compute a phase noise response for an oscillator or VCO, to derive the resulting random jitter from phase noise. In the PLL methodology, you use other PrimeSim HSPICE advanced analog analyses to compute phase noise contributions from the other PLL building blocks. You then perform a closed loop analysis by using phase-domain models for both signal and noise responses that take into account the noise contributions from all such blocks. To complete this flow is the ability to compute “Accumulated Jitter” or “Timing Jitter” for the closed loop PLL. The accumulated jitter response is essentially an integral transformation of the closed-loop PLL response. The following sections show how you can measure accumulated jitter directly from the phase noise-output of an open loop oscillator analysis.

In the PLL flow, you interpret the closed loop phase noise from the results of a linear PrimeSim HSPICE advanced analog `.AC/.NOISE` analysis. The following sections describe a capability that allows direct measurement of accumulated jitter from the results of this closed loop noise analysis, without any special interpretation of the results.

This section covers the following topics:

- [Jitter Measurements from Phase Noise](#)

Jitter Measurements from Phase Noise

This section discusses the following topics:

- [Jitter Definitions](#)
- [Jitter Output Syntax](#)
- [.MEASURE Statements for Jitter](#)
- [Peak-to-Peak Jitter](#)
- [Errors/Warnings](#)

Jitter Definitions

PrimeSim HSPICE advanced analog analyses provide several random jitter (RJ) measurements. This section defines, describes, and compares the various jitter measurements provided. You derive random jitter measurements from the results of an PrimeSim HSPICE advanced analog phase noise analysis. The following presents the relationships between phase noise and the random jitter measurements, and their means for calculation. The types of random jitter measurements include: Timing, Phase, Period, Tracking, Long-Term, and Cycle-to-Cycle Jitter.

Timing Jitter is a measurement of the oscillator uncertainty in the time domain. For clock applications, time domain measurements are preferable, since most specifications of concern involve time domain values.

Timing Jitter is the standard deviation of the timing uncertainty, which is a function of the auto-correlation function in the power spectrum of the phase variations. Timing jitter is the square root of the variance (standard deviation squared) of the timing uncertainty between two clock edges separated by an interval given by $\tau = N \cdot T_o$, where T_o is the ideal clock period. You can write it as a function of the auto-correlation function of the power spectrum of phase variations as:

Equation 24

$$\sigma_{TIE}^2(\tau) = \frac{2}{\omega_o^2} [R_\phi(0) - R_\phi(\tau)]$$

where TIE refers to the Time Interval Error. Call this measurement *Timing Jitter*, *Accumulated Jitter*, or *N-Cycle Jitter*, since it represents the jitter that may accumulate over an interval of many periods.

The Weiner-Khintchine Theorem [1] relates the auto correlation function to the power spectrum of phase variations as in the following equation:

Equation 25

$$R_\phi(\tau) = \frac{1}{2\pi} \int_{-\infty}^{\infty} S_\phi(\omega) e^{j\omega\tau} d\omega = 2 \int_0^{\infty} L(f) \cos(2\pi f\tau) df$$

where $S_\phi(\omega)$ is the double-sided power spectrum of phase variations, and $L(f)$ is the single-sideband phase noise. The auto-correlation for $\tau = 0$ is given by:

Equation 26

$$R_\phi(0) \equiv \phi_{rms}^2 = 2 \int_0^{\infty} L(f) df$$

which defines ϕ_{rms} in PrimeSim HSPICE advanced analog analyses known as RMS Phase Jitter.

Using the identity $2 \sin^2 \alpha = 1 - \cos 2\alpha$ we can then write:

Equation 27

$$\sigma_{TIE}^2(\tau) = \frac{8}{\omega_o^2} \int_0^{\infty} L(f) \sin^2(\pi f\tau) df$$

to enable currently supported PrimeSim HSPICE advanced analog analyses jitter measurements to be written as:

Equation 28 *RMS Phase Jitter*

$$\phi_{rms} = \sigma_{ph} \cdot \omega_o = \sqrt{2 \int_0^{\infty} L(f) df}$$

Equation 29 Timing (Time Interval Error) Jitter

$$\sigma_{TIE}(\tau) = \frac{2}{\omega_0} \sqrt{2 \int_0^{\infty} L(f) \sin^2(\pi f \tau) df}$$

From these definitions, several other key jitter measurements can be derived, including Period Jitter, Tracking Jitter, Long-Term Jitter, and Cycle-to-Cycle Jitter.

Period Jitter is equivalent to the value for Timing Jitter for a one period interval. We therefore have:

Equation 30 Period Jitter

$$\sigma_{PER} = \sigma_{TIE}(T_0) = \frac{2}{\omega_0} \sqrt{2 \int_0^{\infty} L(f) \sin^2(\pi f T_0) df}$$

Tracking Jitter is equivalent to the value (in units of seconds) for RMS Phase Jitter, or:

Equation 31 Tracking Jitter

$$\sigma_{tr} = \sigma_{ph} = \frac{\phi_{rms}}{\omega_0} = \frac{1}{\omega_0} \sqrt{2 \int_0^{\infty} L(f) df}$$

Long-Term Jitter is equivalent to $\sqrt{2}$ times the Tracking Jitter, that is:

Equation 32 Long-Term Jitter

$$\sigma_{\Delta T \rightarrow \infty} = \sigma_{TIE}(\tau \rightarrow \infty) = \sqrt{2} \frac{\phi_{rms}}{\omega_0} = \frac{2}{\omega_0} \sqrt{\int_0^{\infty} L(f) df}$$

Cycle-to-Cycle Jitter is based on the difference between adjacent Period Jitter measurements. It is given by:

Equation 33 Cycle-to-Cycle Jitter

$$\sigma_{CTC} = \sqrt{4\sigma_{PER}^2 - \sigma_{TIE}^2(2T_0)}$$

In general, each of the above calculations must be performed carefully over limits of integration to accurately calculate jitter expressions based on the finite frequency limits provided for the phase noise analysis. Linear interpolation is used, but the phase noise generally follows more of a power law expansion.

Jitter Output Syntax

The timing jitter calculations are derived from the results of phase noise analysis. The phase noise output syntax supports the `JITTER` keyword as an output keyword in addition to the `PHNOISE` keyword.

```
.PRINT PHASENOISE PHNOISE JITTER
.PROBE PHASENOISE PHNOISE JITTER
```

If the `JITTER` keyword is present, the `.PHASENOISE` statement also outputs the raw jitter data to `*.jtt0` and `*.printjtt0` data files. The PHNOISE data is given in units of dBc/Hz, that is, dB relative to the carrier, per Hz, across the output nodes specified by the PHASENOISE statement. The data plot is a function of offset frequency. If the `JITTER` keyword is present, `.PHASENOISE` outputs the TIE Timing Jitter (Accumulated Jitter) data to `*.jtt0` and `*.printjtt0` data files. These data are plotted as a function of time in units of seconds. The jitter calculations make use of some of the parameters given in the `.PHASENOISE` syntax.

The time samples for the timing jitter output make use of the same number of points as the phase noise frequency sweep specification.

The output of the timing jitter information uses a corresponding time sampling derived via:

Equation 34

$$\tau_1 = \frac{1}{T_0}, \tau_2 = \frac{2}{T_0}, \dots, \tau_N = \frac{N}{T_0}$$

.MEASURE Statements for Jitter

The jitter-specific `.MEASURE` statements specify the jitter keywords as follows. (For a discussion of the `BER` parameter, see below.)

```
.MEASURE PHASENOISE Jname PERJITTER phnoise
+ [UNITS=(sec|rad|UI)] [BER=val]
.MEASURE PHASENOISE Jname CTCJITTER phnoise
+ [UNITS=(sec|rad|UI)] [BER=val]
.MEASURE PHASENOISE Jname RMSJITTER phnoise
+ [FROM start_frequency] [TO end_frequency]
+ [UNITS=(sec|rad|UI)] [BER=val]
.MEASURE PHASENOISE Jname PHJITTER phnoise
+ [FROM start_frequency] [TO end_frequency]
+ [UNITS=(sec|rad|UI)] [BER=val]
.MEASURE PHASENOISE Jname TRJITTER phnoise
+ [FROM start_frequency] [TO end_frequency]
+ [UNITS=(sec|rad|UI)] [BER=val]
.MEASURE PHASENOISE Jname LTJITTER phnoise
+ [FROM start_frequency] [TO end_frequency]
+ [UNITS=(sec|rad|UI)] [BER=val]
```

`RMSJITTER`, `PHJITTER`, and `TRJITTER` are synonymous measurements, all based on the calculations described related to the RMS Phase Jitter value in units of seconds given by $\sigma_{ph} = \phi_{rms} / \omega_0$. These measurements allow control of the integration range using the `FROM` and `TO` parameters. The measurements for `PERJITTER` and `CTCJITTER` use the full offset frequency sweep range given for the phase noise analysis to compute values (the `FROM` and `TO` parameters are ignored if entered).

Note:

It is essential to add probing Jitter statement of PHASENOISE like `.PROBE PHASENOISE PHNOISE JITTER` when doing Jitter measurements.

In PrimeSim HSPICE advanced analog analyses, the frequency intervals can be modified for these jitter calculations (if desired, although not recommended), and `UNITS` can be selected between seconds, radians, and Unit Intervals. The following table specifies the calculation used for `units=seconds` for each jitter measurement.

MEASURE name	Calculation used (Units=sec)
RMSJITTER	$\sigma_{ph} = \phi_{rms} / \omega_0$
PHJITTER	$\sigma_{ph} = \phi_{rms} / \omega_0$
TRJITTER	$\sigma_{ph} = \phi_{rms} / \omega_0$
PERJITTER	σ_{PER}
LTJITTER	$\sigma_{\Delta T \rightarrow \infty} = \sqrt{2} \phi_{rms} / \omega_o$
CTCJITTER	σ_{CTC}

Example:

```
.meas phasenoise rj RMSJITTER phnoise from 1K to 100K
+ units = rad
```

Peak-to-Peak Jitter

As noted in [.MEASURE Statements for Jitter](#), an additional Bit Error Rate (`BER`) parameter is supported. This parameter allows you to convert any jitter value from an RMS value into a Peak-to-Peak value. The RMS jitter values correspond to an 1-sigma standard deviation value for the Gaussian distribution of the jitter. Peak-to-peak values represent the full span of the Gaussian distribution. Since this span is theoretically unbounded for truly random distributions, the conversion to peak-to-peak values has to be interpreted as spanning some number of sigma values. You can arrive at this number (in other words, “sigma multiplier”) by specifying a corresponding `BER`.

The term “BER” corresponds to the unitless Bit Error Rate that allows for this conversion. The following table shows some sample conversions from various BER values into a “sigma multiplier” value, which corresponds to the number of sigma standard deviations in converting from RMS to peak-to-peak values:

Bit Error Rate	Sigma Multiplier
10^{-3}	6.180

Bit Error Rate	Sigma Multiplier
10^{-4}	7.438
10^{-5}	8.530
10^{-6}	9.507
10^{-7}	10.399
10^{-8}	11.224
10^{-9}	11.996
10^{-10}	12.723
10^{-11}	13.412
10^{-12}	14.069
10^{-13}	14.698
10^{-14}	15.301
10^{-15}	15.883
10^{-16}	16.444

These conversions are done in accordance with the relationship:

Equation 35

$$\frac{1}{2} \operatorname{erfc} \frac{\alpha}{2 \cdot \sqrt{2}} = \text{BER}$$

where, erfc is the complementary error function and α is the Sigma Multiplier. Support for peak-to-peak conversions is included for a continuous range of BER values from $10^{-16} \leq \alpha \leq 10^{-3}$ (and some values extrapolated outside this range).

Specification of the `BER` parameter results in the output of the peak-to-peak jitter value and not the RMS value. Labels for the measurements show appropriate “rms” and “p-p” labels. A `BER` parameter set to `BER=0` is equivalent to having no parameter and only results in the RMS calculation.

Errors/Warnings

Error handling and recovery is exercised to capture obvious errors in input specifications. The following error checks are performed:

- Calculations to be performed if oscillator or phase noise analysis fails.
- ERROR if $L(f) > 1$ over any part of the frequency sweep (non-dB form).

- ERROR if $L(f) < 0$ over any part of the frequency sweep (non-dB form).
- Error if any time or frequency samples are negative values.
- ERROR if $BER < 0$ for any Jitter measurement.
- WARNING if $BER > 1$ for any Jitter measurement.
- WARNING if $f_0 < 10$ Hz. Message: "Jitter calculations may be ineffective for offset frequencies under 10 Hz."

Clock Source with Random Jitter

For .AC-related .NOISE analysis, see Clock Source with Random Jitter in the *PrimeSim HSPICE® User Guide: Elements*.

7

Behavioral PLL Noise Analysis (.ACPHASENOISE)

Describes behavioral PLL noise analysis.

To see the influence that oscillator or VCO phase noise can have on a system where it is present, it is necessary to perform a phase-domain analysis where the circuit variables are phase and the input noise stimuli are phase noise. This is the purpose of the `.ACPHASENOISE` analysis in PrimeSim HSPICE advanced analog analyses.

This type of analysis is critical, for example, in analyzing the effects of noise in a phase-locked loop (PLL). In a PLL design flow, the HBOSC or SNOSC analyses are used to compute a phase noise response for an oscillator or VCO. PrimeSim HSPICE advanced analog analyses can be used to compute phase noise contributions from the other PLL building blocks. A closed loop PLL analysis can then be performed by using phase-domain models for both signal and noise responses, where the noise contributions from all blocks are input as phase noise stimuli. Such an analysis can be performed to determine the PLL closed-loop phase noise, based on the contributions of each block, determined by an open loop analysis.

For more information on control options, see the `.ACPHASENOISE` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

This chapter discusses the following topics:

- [ACPHASENOISE Analysis .PRINT and .PROBE Syntax](#)
- [Behavioral Noise Sources](#)

ACPHASENOISE Analysis .PRINT and .PROBE Syntax

The unique aspect of the `.ACPHASENOISE` analysis is that it allows the small signal noise calculation results to be interpreted as phase noise values. The available `.print/.probe` measurements reflect this. The `.print/.probe` output syntax are the “JITTER”

and “PHNOISE” keywords consistent with the PrimeSim HSPICE advanced analog .phasenoise analysis, namely:

```
.PRINT ACPHASENOISE PHNOISE JITTER
.PROBE ACPHASENOISE PHNOISE JITTER
```

As with the .PHASENOISE analysis, the .ACPHASENOISE analysis outputs raw data to *.pn0 and *.printpn0 files. The PHNOISE data is given in units of dBc/Hz, that is, dB relative to the carrier, per Hz, across the output nodes specified by the .ACPHASENOISE statement. The data plot is a function of offset frequency. If the “JITTER” keyword is present, .ACPHASENOISE also outputs the accumulated TIE jitter data to *.jto and *.printjto data files. These data are plotted as a function of time in units of seconds. The Timing Jitter data itself has units of seconds. The timing jitter calculations make use of the parameters given in the .ACPHASENOISE syntax, such as “freq” and “interval”.

See also [Using Noise Analysis Results as Input Noise Sources](#).

.MEASURE Support for ACPHASENOISE

Single-valued jitter measurements are available from .MEASURE statements. Examples include period jitter, cycle-to-cycle jitter, and phase jitter measurements, as shown respectively in the following syntax:

```
.MEASURE ACPHASENOISE Jname PERJITTER phnoise
+ [UNITS=(sec|rad|UI)] [BER=val]

.MEASURE ACPHASENOISE Jname CTCJITTER phnoise
+ [UNITS=(sec|rad|UI)] [BER=val]

.MEASURE ACPHASENOISE Jname PHJITTER phnoise
+ [FROM start_frequency [TO end_frequency]
+ [UNITS=(sec|rad|UI)] [BER=val]
```

Behavioral Noise Sources

In PrimeSim HSPICE advanced analog analyses, you can use the G-element to specify noise sources. Frequency domain noise analyses (.NOISE, .HBNOISE, and .PHASENOISE) take these noise sources into account.

You can attach noise sources to behavioral models. For example, you can use a G-element with the VCCAP parameter to model a varactor, which includes a noise model. You can also simulate effects such as substrate noise, including its effect on oscillator phase noise. You can also use this G-element syntax to simulate behavioral descriptions

of substrate noise during any frequency domain noise analysis, which includes phase noise analysis. For example,

```
gname node1 node2 noise='noise_equation'
gname node1 node2 node3 node4 noise='noise_equation'
```

The first line creates a simple two-terminal current noise source, whose value is described in $A^2/(Hz)$. The output noise generated from this noise source is:

*noise_equation**H

Where H is the transfer function from the terminal pair (node1,node2) to the circuit output, where PrimeSim HSPICE advanced analog analyses measures the output noise.

The second line produces a noise source correlation between the (node1,node2) and (node3,node4) terminal pairs. The resulting output noise is calculated as

*noise_equation**sqrt(H1*H2*); where:

- H1 is the transfer function from (node1,node2) to the output.
- H2 is the transfer function from (node3,node4) to the output.
- The * on H1*H2* represents the complex conjugate of H1 and H2.

The *noise_equation* expression can involve node voltages and currents through voltage sources.

For the PAC phase noise simulation to evaluate the frequency-dependent noise, the frequency-dependent noise factor in the phase noise must be expressed between parentheses. For example:

```
gname node1 node2 noise = '(frequency_dependent_noise)*
bias_dependent_noise'
```

This is only true when the total noise can be expressed in this form and when the frequency-dependent noise can be evaluated in the PAC phase noise simulation. You can also input the behavioral noise source as a noise table with the help of the predefined `Table()` function. The `Table()` function takes two formats:

- Noise table can be input directly through the `Table()` function. For example:

```
gname node1 node2 noise = 'Table(arg1,f1,v1,f2,v2,.....)'
```

- The f1,v1,f2,v2,..... parameters describe the noise table. When `arg1 == f1`, the function returns v1. The `arg1` can be an expression of either HERTZ, bias, or both. For example, `arg1 = 'HERTZ * 1.0E+3'`.
- The noise table can be input through a .DATA structure:

```
.DATA d1
+ x y
+ f1 v1
```

```
+ f2 v2
.ENDDATA
```

```
gname node1 node2 noise = 'TABLE(arg1,d1)'
```

The *x*, *y* parameters in the `DATA` structure are two placeholder strings that can be set to whatever you prefer even if they are in conflict with other parameters in the netlist. The `arg1` parameter can be an expression of `HERTZ` and bias. When `arg1 == f2`, the function returns `v2`.

This section covers the following topics:

- [Using Noise Analysis Results as Input Noise Sources](#)
- [Syntax Rules for Behavioral Noise Source Files](#)
- [Power Supply Current and Voltage Noise Sources](#)

Using Noise Analysis Results as Input Noise Sources

By extending the E- and G- voltage-controlled source syntax, the phase noise or noise data in the ASCII phase noise and noise files can be used as input for specifying behavioral noise sources.

Usage Model

The syntax for the voltage-controlled voltage (E) or current (G) source is:

```
Exxx node1 node2 noisefile='filename' [mname='measname']
Gxxx node1 node2 noisefile='filename' [mname='measname']
```

where:

`noisefile='filename'` is the name of the ASCII noise data file. Phase noise analysis and noise analyses can output simulation results as ASCII data using `.PRINT` statements. The file name is designated as `'design.printsnpn0'` for `.SNOSC` phase noise and `.SNNOISE` analyses. For `.PHASENOISE`, `.HBNOISE` and `.ACPHASENOISE` analyses, the file name is designated as `'design.printtpn0'`. For `.NOISE` analyses, the file name is designated as `'design.printac0'`.

`mname='measname'` is used to select the appropriate noise measurement name to be taken from the `*.printtpn0`, `*.printsnpn0`, or `*.printac0` file.

`measname` can be one of the following:

- `NLP_L(f)` – selects the `nlp_L(f)` phase noise data in units of dBc/Hz.
- `PAC_L(f)` – selects the `pac_L(f)` phase noise data in units of dBc/Hz.
- `ONoise` – selects the `onoise` data based on `.SNNOISE` or `.NOISE` analysis.

The following syntaxes are supported in both PrimeSim HSPICE advanced analog analyses and PrimeSim HSPICE:

- `Exxx n1 n2 noise data=dataname`
- `Exxx n1 n2 noise data=datablock`
- `Exxx n1 n2 noisefile='filename'`
- `Exxx n1 n2 noise='expression'`
- `Exxx n1 n2 noise='Table(arg1,f1,v1,f2,v2...)'`
- `Exxx n1 n2 noise='Table(arg1,dotDataBlockName) ', where dotDataBlockName is the .data statement reference.`

Syntax Rules for Behavioral Noise Source Files

You can use ASCII noise data files not generated by `.PRINT` statements when you follow these syntax rules:

- A noise file always needs a header (do not include frequency or noise points in the header line, as they will be ignored).
- The items in the header must be consistent with the number of data columns.
- You can use noise keywords in the header (see the next item in this list).
- The noise value units depend on your use of the `MNAME` keyword.

You can use noise keywords like `ONoise`, and the phase noise keywords, `PAC_L(f)` and `NLP_L(f)`. This is useful for `*.printac#`, `*.printpn#`, or `*.printsnpn#` files and use of the `MNAME` keyword to specify the noise data column to be used.

If `MNAME` is not specified with the behavioral noise source, then PrimeSim HSPICE assumes that the second column in the noise file represents the desired noise values, and the `MNAME` string is read from the noise file header. The first column is always assumed to contain frequency values.

The `MNAME` keyword is used to differentiate units. If you use `MNAME=ONoise` and the file contains `ONoise` in the header, then the noise units are interpreted as $V/\sqrt{\text{Hz}}$. The same is true with `MNAME=ONoise(M)` and `MNAME=ONoise(MAG)`. If `MNAME` is set to `PAC_L(f)`, `NLP_L(f)`, or `PHNoise`, then units are interpreted as dBc/Hz . With `MNAME=ONoise(DB)`, units are taken as dB/Hz ($20 \cdot \text{Log}(\text{noise})$), as are other `MNAME` keywords that end in `DB` or `(DB)`. Most other `MNAME` keywords are generally interpreted in V^2/Hz units, unless the name ends in `(M)` or `(MAG)`, which are taken to be $V/\sqrt{\text{Hz}}$. Depending on the noise units, different interpolation schemes are used for frequencies needed between noise data points. Noise in dB/Hz or dBc/Hz units use logarithmic interpolation (in x and y), while non-dB units use linear interpolation in V^2/Hz .

Noise File Examples

The following examples provide sample noise results noting the various noise value units:

Example 1: Phase noise file generated by using `.PRINT phasenoise phnoise` statement. The noise units are dBc/Hz:

```
HERTZ  NLP_L(f)
10.0000000000k      -130.2612492663
12.5892541179k      -132.2612484153
15.8489319246k      -134.2612470666
19.9526231497k      -136.2612449295
25.1188643151k      -138.2612415429
31.6227766017k      -140.2612361768
39.8107170553k      -142.2612276743
50.1187233627k      -144.2612142034
63.0957344480k      -146.2611928626
79.4328234724k      -148.2611590575
100.0000000000k     -150.2611055150
125.8925411794k     -152.2610207237
158.4893192461k     -154.2608864692
199.5262314969k     -156.2606739394
251.1886431510k     -158.2603375701
316.2277660168k     -160.2598053187
398.1071705535k     -162.2589632779
501.1872336273k     -164.2576312765
630.9573444802k     -166.2555240055
794.3282347243k     -168.2521885144
1.0000000000x       -170.2469020563
```

Example 2: Noise file generated by using `.PRINT snnoise onoise` statement. The noise units are V/sqrt(Hz):

```
HERTZ  ONOISE()
100.0000      1.6699u
526.4105k     46.8034n
1.0527x       45.1538n
1.5790x       44.6096n
2.1053x       44.3405n
2.6317x       44.1804n
3.1580x       44.0744n
3.6843x       43.9990n
4.2106x       43.9425n
4.7369x       43.8986n
5.2632x       43.8634n
5.7895x       43.8345n
6.3158x       43.8102n
6.8421x       43.7894n
7.3684x       43.7714n
7.8948x       43.7556n
8.4211x       43.7415n
8.9474x       43.7288n
```

```
9.4737x      43.7172n
10.0000x     43.7067n
```

Example 3: User-generated noise file. Units are in dBc/Hz:

```
HERTZ  NOISE (DB)
10.0    -120
100.0   -125
1.0k    -130
5.0k    -135
10.0k   -140
100.0k  -150
```

Power Supply Current and Voltage Noise Sources

You can implement the power supply noise source with G- and E- elements. The G-element is for the current noise source and the E-element for the voltage noise source. As noise elements, they are two-terminal elements that represent a noise source connected between two specified nodes.

- [Syntax](#)
- [Example](#)

Syntax

Expression form:

```
Gxxx node1 node2 noise='expression'
Exxx node1 node2 noise='expression'
```

The G noise element represents a noise current source and the E noise element represents a noise voltage source. The *xxx* parameter can be set with a value up to 1024 characters. The *node1* and *node2* are the positive and negative nodes that connect to the noise source. The noise expression can contain the bias, frequency, or other parameters.

Data form:

```
Gxxx node1 node2 noise data=dataname
Exxx node1 node2 noise data=dataname
.data dataname
+ pname1 pname2
+ freq1 noise1
+ freq2 noise2
+ ...
.enddata
```

The data form defines a basic frequency-noise table. The `.DATA` statement contains two parameters: frequency and noise to specify the noise value at each frequency point. The

unit for frequency is hertz, and the unit for noise is A^2/Hz (for G current noise source) or V^2/Hz (for E voltage noise source).

Example

The following netlist shows a 1000 ohm resistor (`g1`) using a G-element. The `g1noise` element, placed in parallel with the `g1` resistor, delivers the thermal noise expected from a resistor. The `r1` resistor is included for comparison. The noise due to `r1` should be the same as the noise due to `g1noise`.

```
* Resistor implemented using g-element
v1 1 0 1
r1 1 2 1k
g1 1 2 cur='v(1,2)*0.001'
g1noise 1 2
+ noise='4*1.3806266e-23*(TEMPER+273.15)*0.001'
rout 2 0 1meg
.ac lin 1 100 100
.noise v(2) v1 1
.end
```

Part 4: Periodically-driven AC, Transfer Function, and Noise Analyses

Describes the usage of large signal AC, transfer function and noise analyses, large signal S-parameter analyses and envelope analysis.

The following chapters/topics are included in this Part:

- [Chapter 8, Large Signal AC, Transfer Function, and Noise Analyses](#)
- [Chapter 9, Large Signal S-parameter Analyses](#)
- [Chapter 10, Envelope Analysis](#)

8

Large Signal AC, Transfer Function, and Noise Analyses

Describes how to use both harmonic balance-based and Shooting Newton-based AC and transfer function analyses, as well as nonlinear, steady-state noise analysis.

This chapter discusses the following topics:

- [Multitone Harmonic Balance AC Analysis \(.HBAC\)](#)
- [Shooting Newton AC Analysis \(.SNAC\)](#)
- [Steady-State Voltage and Current Sources](#)
- [Phase Noise and Buffer Chains](#)
- [Multitone Harmonic Balance Noise \(.HBNOISE\)](#)
- [Shooting Newton Noise Analysis \(.SNNOISE\)](#)
- [Multitone Harmonic Balance Transfer Function Analysis \(.HBXF\)](#)
- [Shooting Newton Transfer Function Analysis \(.SNXF\)](#)
- [HB- and SN-Based Stability Analysis](#)
- [References](#)

Multitone Harmonic Balance AC Analysis (.HBAC)

Use the `.HBAC` (Harmonic Balance AC) statement for analyzing linear behavior in large-signal periodic systems. The `.HBAC` statement computes a time-averaged HBAC using a periodic AC (PAC) algorithm to perform linear analysis of autonomous (oscillator) or non-autonomous (driven) circuits, where the linear coefficients are modulated by a periodic, steady-state signal. HBAC also allows the computation of a time-sampled HBAC. This feature is only supported for non-autonomous, single-tone harmonic balance.

Multitone HBAC analysis extends single-tone HBAC to quasi-periodic systems with more than one periodic, steady-state tone. One application of multitone HBAC is to more

efficiently determine mixer conversion gain under the influence of a strong interfering signal than is possible by running a swept three-tone HB simulation.

The following sections discuss these topics:

- [Prerequisites and Limitations](#)
- [Tutorial Example - Using Multi-Tone HB and HBAC Analyses for a Mixer](#)
- [Input Syntax](#)
- [Output Syntax](#)
- [Using the .MEASURE Command with .HBAC](#)
- [Errors and Warnings](#)

Prerequisites and Limitations

The following prerequisites and limitations apply to HBAC:

- Requires one and only one `.HBAC` statement. If you use multiple `.HBAC` statements, PrimeSim HSPICE advanced analog analyses use only the last `.HBAC` statement.
- Requires one and only one `.HB` statement.
- Supports arbitrary number of tones.
- Requires placing the parameter sweep in the `.HB` statement.
- Requires at least one HB source.
- Requires at least one HBAC source.
- Supports unlimited number of HB and HBAC sources.
- The requested maximum harmonic in a `.PROBE` or `.PRINT` statement must be less than or equal to half the number of harmonics specified in harmonic balance (that is, $\text{max_harm} \leq \text{num_hb_harms} / 2$).

Tutorial Example - Using Multi-Tone HB and HBAC Analyses for a Mixer

The example in this section shows how to use PrimeSim HSPICE advanced analog analyses to analyze a circuit driven by multiple input stimuli with different frequencies. Mixer circuits provide a typical example of this scenario: in this case, there might be two

input signals (LO and RF), which are mixed to produce an IF output signal. In this case, PrimeSim HSPICE advanced analog analyses offers two options:

- Multi-tone HB analysis: specify the `LO` and `RF` base frequencies as two separate tones on the `.HB` command.
- Periodic AC analysis (HBAC): if one of the inputs is a small-signal, you can use a faster linear analysis to analyze its effect. For example, if a mixer's `LO` is a large signal, but `RF` is a small signal, a single-tone HB analysis using the `LO` frequency can be combined with HBAC in place of a 2-tone HB analysis.

To demonstrate both techniques, this example analyzes an ideal mixer built using behavioral elements. It is based on the demonstration netlist `mix_tran.sp`, which is available in the directory: `$installdir/demo/hspice/rf_examples/`.

```
* Ideal mixer example: transient analysis
.OPTIONS POST
vlo lo 0 1.0 sin (1.0 0.5 1.0g 0 0 90)
rrf1 rf1 rf 1.0
g1 0 if cur='1.0*v(lo)*v(rf)' $ mixer element
c1 0 if q='1.0e-9*v(lo)*v(rf)' $ mixer element
rout if ifg 1.0
vctrl ifg 0 0.0
h1 out 0 vctrl 1.0 $ convert I to V
rh1 out 0 1.0
vrf rf1 0 sin (0 0.001 0.8GHz 0 0 114)
.tran 10p 10n
.opt sim_accuracy=100
.end
```

This example uses behavioral-controlled current and charge sources to simulate a mixer. The `LO` signal is driven by a 0.5 Volt sinusoid at 1 GHz and `RF` is driven by a 10mV signal at 800 MHz. The mixer output is the voltage at node out, `v(out)`.

- [Two-tone HB Approach](#)
- [HBAC Approach](#)
- [Comparing Results](#)

Two-tone HB Approach

To analyze this circuit using 2-tone HB, add:

- HB source for LO: add `HB 0.5 0 1 1` to the LO voltage source; this sets the amplitude to 0.5, no phase shift for the first harmonic of the first tone, which is 1 GHz.
- HB source for RF: add `HB 0.001 24 1 2` to the RF voltage source; this sets the amplitude to 0.001, 24 degrees phase shift for the first harmonic of the second tone (0.8 GHz).
- An `.HB` command specifying both tones: `.hb tones=1g 0.8g nharms=6 3`; only a small number of harmonics are required to resolve the signals.

The complete `mix_hb.sp` netlist for 2-tone HB analysis is then:

```
* Ideal mixer example: 2-tone HB analysis
.OPTIONS POST
vlo lo 0 1.0 sin (1.0 0.5 1.0g 0 0 90) HB 0.5 0 1 1
rrf1 rfl rf 1.0
g1 0 if cur='1.0*v(lo)*v(rf)' $ mixer element
c1 0 if q='1.0e-9*v(lo)*v(rf)' $ mixer element
rout if ifg 1.0
vctrl ifg 0 0.0
h1 out 0 vctrl 1.0 $ convert I to V
rh1 out 0 1.0
vrf rfl 0 sin (0 0.001 0.8GHz 0 0 114) HB 0.001 24 1 2
.opt sim_accuracy=100
.hb tones=1g 0.8g nharms=6 3
.end
```

This example is available in directory `$installdir/demo/hspice/rf_examples/`.

HBAC Approach

To analyze this circuit using HBAC, start with the 2-tone HB analysis setup and modify it as follows:

- Replace the RF HB signal with an HBAC signal: change `HB 0.001 24 1 2` to `HBAC 0.001 24`; this deactivates the source for HB and activates it for HBAC with the same magnitude and phase.

- Specify the frequency in the `.HBAC` command.

- Change the `.HB` command to single tone:

```
.HB tones=1g nharms=6
```

HBAC takes care of the second tone.

- Add a `.HBAC` command

```
.HBAC lin 1 0.8g 0.8g
```

This command runs an analysis at a single frequency point, 0.8 GHz. In general, HBAC analysis can sweep the advanced analog frequency over a range of values.

The following is the complete `mix_hbac.sp` netlist for HBAC analysis of this simple mixer. This netlist also contains commands for performing periodic noise analysis. It is available in the `$installdir/demo/hspice/rf_examples/` directory.

```
* Ideal mixer example: HBAC analysis
.OPTIONS POST
vlo lo 0 1.0 sin (1.0 0.5 1.0g 0 0 90)
+ HB 0.5 0 1 1
rrf1 rfl rf 1.0
g1 0 if cur='1.0*v(lo)*v(rf)' $ mixer element
c1 0 if q='1.0e-9*v(lo)*v(rf)' $ mixer element
rout if ifg 1.0
vctrl ifg 0 0.0
hl out 0 vctrl 1.0 $ convert I to V
rhl out 0 1.0
vrf rfl 0 sin (0 0.001 0.8GHz 0 0 114)
+ HBAC 0.001 24
.opt sim_accuracy=100
.hb tones=lg nharms=6
.hbac lin 1 0.8g 0.8g
* Noise analysis
.hbnoise v(out) rrf1 lin 40 0.1g 4g
.print hbnoise onoise nf
.probe hbnoise onoise nf
.end
```

Comparing Results

After running all three netlists above, you will have generated 3 output files:

- `mix_tran.tr0`
- `mix_hb.hb0`
- `mix_hbac.hb0`

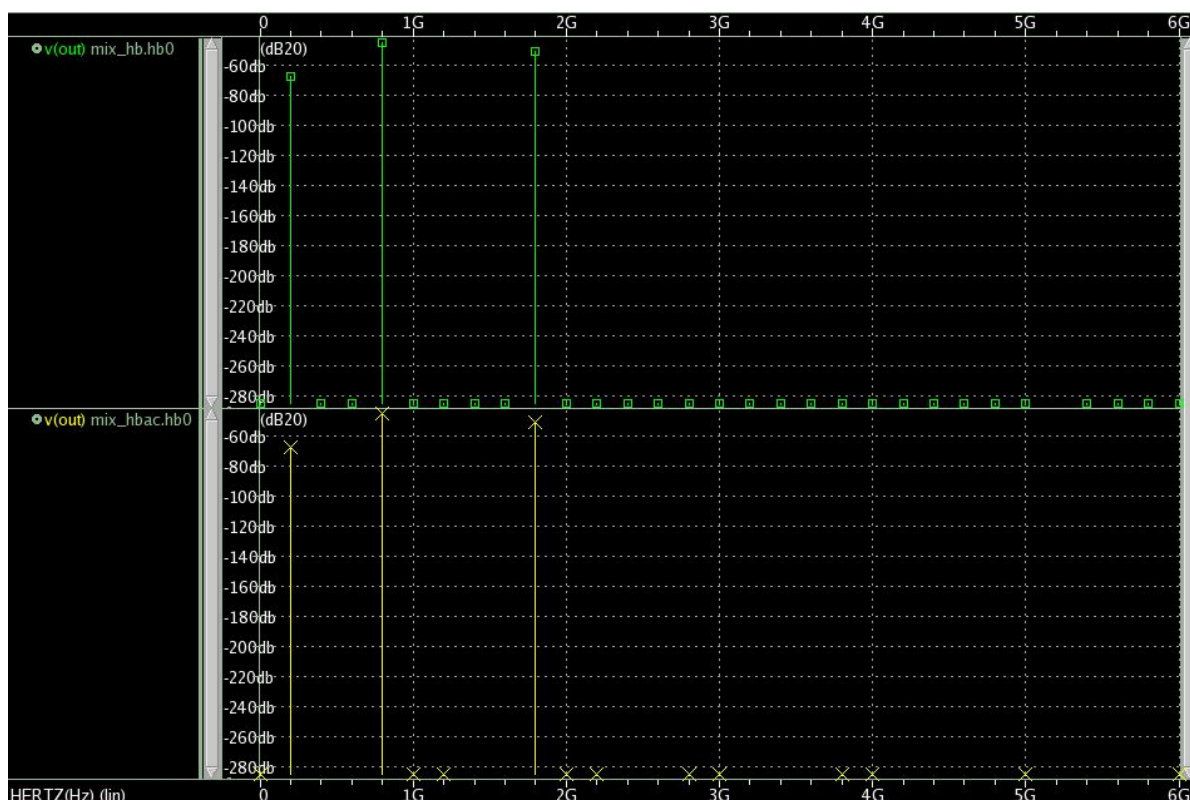
You can compare the results of the three analyses in WaveView.

1. To run the netlists and start WaveView, type:

```
hspice -i mix_tran.sp -o
hspice -i mix_hb.sp -o
hspice -i mix_hbac.sp -o
wv &
```

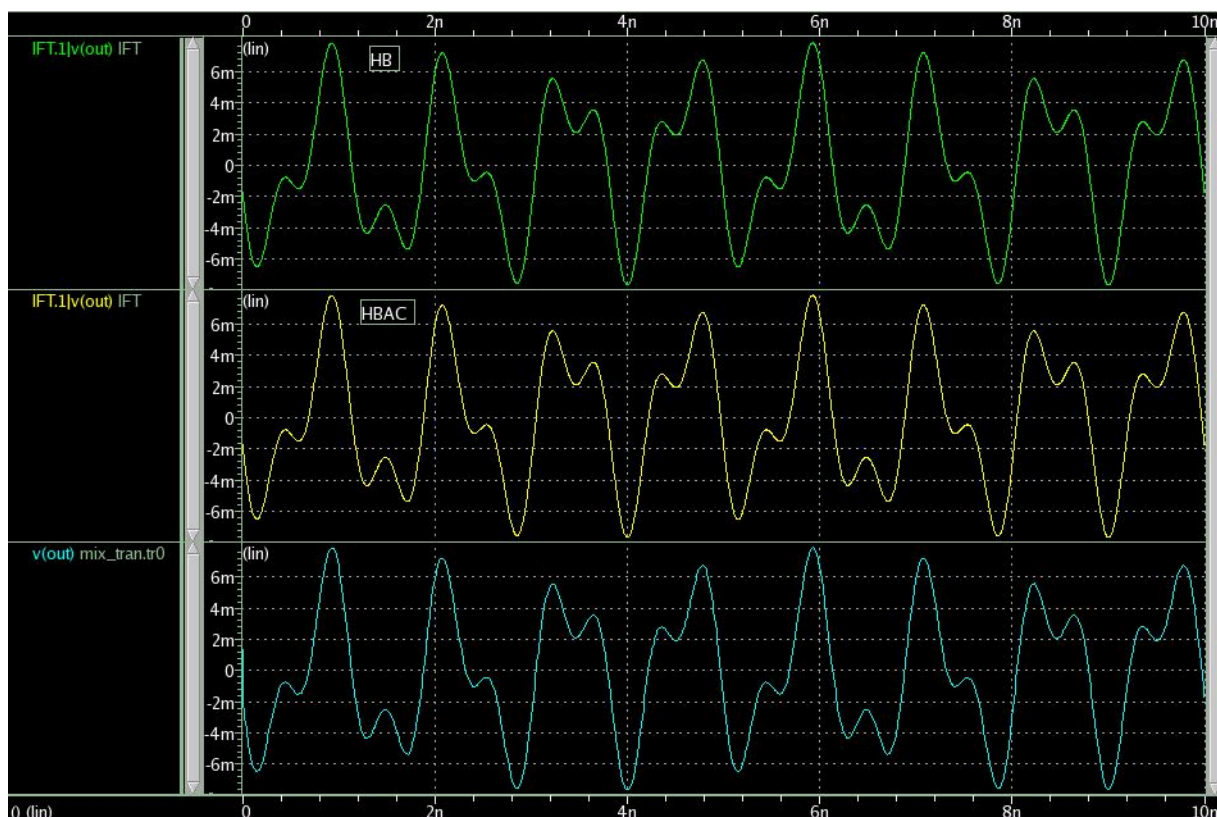
2. Click the **Open Waveform** icon and select the `mix_tran.tr0`, `mix_hb.hb0`, and `mix_hbac.hb0` files from the Open: Waveform Files dialog box.

3. Select the `v(out)` signal from the `mix_hb.hb0` file in the signal browser. Double-click on the signal name or drag and drop the signal into the waveview. You should see a frequency spectrum similar to the one from the `mix_hbac.hb0` file.
4. Select the `v(out)` signal from the `mix_hbac.hb0` file in the signal browser. Double-click on the signal name or drag and drop the signal into the waveview. You should see a frequency spectrum similar to the one from the `mix_hb.hb0` file.



5. Convert the HB and HBAC histograms to time domain. In the waveview, right-click in the name area of the panel containing the signal `v(out)`, left-click on the waveform label for `v(out)` from the `mix_hb.hb0` file. From the Panel menu, choose **Signal 'v(out)' > To Time-Domain**. To accept the defaults for range and interval, click **OK** in the Convert to Time domain window.
6. Repeat Step 5 for the `v(out)` signal from the `mix_hbac.hb0` file.
7. Click the **Waveview** icon in the Add section of the **Waveform** tab (or press CTRL+w) to open a new waveview.
8. Select the `v(out)` signal from the `mix_tran.tr0` file in the signal browser. Double-click on the signal name or drag and drop the signal into the waveview.

- Compare the time domain waveforms from the `mix_hb.hb0` and `mix_hbac.hb0` files with the time domain waveform from the `mix_tran.tr0` file. In the file browser, click **IFT** under derived waveforms. The signals `0|v(out)` and `1|v(out)` should appear in the signal browser. Select the `0|v(out)` and `1|v(out)` signals and drag and drop them into the waveview. All the three time domain signals should be displayed in the same panel. The three signals are almost indistinguishable from each other.



You can also use HBAC to perform noise analysis on advanced analog circuits by using the `.HBNOISE` command, which is included in the `mix_hbac.sp` netlist.

- The `.HBNOISE` command invokes noise analysis, identifying an output node where the noise is measured, an input noise source (in this case, `rrf1`) which serves as a reference for noise figure computation and a frequency sweep for the noise analysis.
- The `.PRINT` and `.PROBE hbnoise` commands instruct PrimeSim HSPICE advanced analog analyses to save the output noise and noise figure at each frequency in the `mix_hbac.printpn0` and `mix_hbac.pn0` output files.

This ideal mixer is noiseless, except for the resistors at the input and output.

The `mix_hbac.lis` file contains detailed data on the individual noise source contributions of the resistors. You can view `mix_hbac.printpn0` to see the output noise and noise

figure at each frequency. In WaveView, you can view `mix_hbac.pn0` to plot the output noise and noise figure data as a function of frequency.

Input Syntax

```
.HBAC
+ frequency_sweep
+ TIME = [val],[meas_name],[time_sweep]
+ [SAMPLEDELAY = value]
+ [MAXSIDE BAND = value]
+ [FREQAXIS] = [in|out]
+ [TIMEAXIS = value]
+ [TOL = value]
+ [KRYLOVDIM = value]
+ [KRYLOVITER = value]
```

Parameter	Description
<i>frequency_sweep</i>	Frequency sweep range for the input signal (also referred to as the input frequency band or fin). You can specify LIN, DEC, OCT, POI, or SWEEPBLOCK. Specify the nsteps, start, and stop frequencies using the following syntax for each type of sweep: • LIN <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • POI <i>nsteps freq_values</i> • SWEEPBLOCK=<i>swblockname</i> • DATA=<i>dataname</i>

Parameter	Description
<i>TIME</i>	Determines the time points, which the time domain HBAC is evaluated. If <i>TIME</i> is specified, time-sampled HBAC is assumed. If <i>TIME</i> is not specified, a time-averaged HBAC is assumed. Time values are specified using 1 or more of 3 types as follows: • <i>TIME</i> = [<i>val</i>] Supports one single <i>val</i> or single param. • <i>TIME</i> = [<i>meas_name</i>]. Where <i>meas_name</i> is a single ".MEASURE" name. • <i>TIME</i> = [<i>time_sweep</i>] Specifies the type, <i>nsteps</i> or <i>npoints</i> and <i>tstart</i> and <i>tstop</i> or time points for each sweep type, where: ◦ <i>type</i> = the time sweep type which can be OCT, DEC, LIN, POI, MPOI(to support multiple <i>meas_names</i>) or SWEEPBLOCK. ◦ LIN <i>npoints</i> <i>tstart</i> <i>tstop</i> ◦ OCT <i>nsteps</i> <i>tstart</i> <i>tstop</i> ◦ DEC <i>nsteps</i> <i>tstart</i> <i>tstop</i> ◦ POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ...<i>pn</i> ◦ MPOI <i>npoints</i> <i>mp1</i>, <i>mp2</i>, ...<i>mpn</i> ◦ SWEEPBLOCK <i>swblockname</i>, where <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT) ◦ <i>npoints</i> = number of points for linear (LIN) or points of interest (POI) ◦ <i>tstart</i> = starting time ◦ <i>tstop</i> = ending time ◦ <i>p1</i>, <i>p2</i>, ... <i>pn</i> = time points Notice that the time=POI <i>npoints</i>, <i>p1</i>,<i>p2</i>,...,<i>pn</i> to get multiple time points. And similar usage of time=MPOI <i>npoints</i>, <i>mp1</i>,<i>mp2</i>,...,<i>mpn</i> for multiple measure name support.. • Note: The <i>MEASURE</i>name must generate at least 1 or more time point values. All generated time points is used as <i>TIME</i> values in the <i>TS HBAC</i>. This recommended to use with <i>TIME</i> = [<i>meas_name</i>]. Where <i>meas_name</i> is a single ".MEASURE" name. When a time value lies outside the range (0,<i>Tp</i>), where <i>Tp</i> is the period associated with the fundamental frequency, the time value will be appropriately folded into the acceptable range. Note: Time-sampled HBAC is only supported for single-tone harmonic balance. <i>TIME</i> values specified with multitone harmonic balance will be ignored and HBAC will proceed with a time-averaged HBAC simulation.
<i>SAMPLEDELAY</i>	Adds <i>VAL</i> to the <i>TIME</i> values. Default is 0. Note: The new time values is folded into range (0,<i>Tp</i>), where <i>Tp</i> is the period associated with the fundamental frequency. <i>SAMPLEDELAY</i> = <i>VAL</i>

Parameter	Description
<i>MAXSIDEBAND</i>	Limits the number of sidebands used in generating the time-dependent response. Default uses all sidebands. <code>MAXSIDEBAND = INT</code>
<i>FREQAXIS</i>	Determines the set of frequencies used in the Print/Probe output files. <code>FREQAXIS = [in out]</code> • <i>In</i> = input frequency values that is the frequencies specified in the <code>frequency_sweep</code> argument on the HBAC command line. • <i>Out</i> = output frequency values that is the frequencies specified in the <code>[INDICES]</code> term on the <code>.PRINT/PROBE</code> statement. Note: The <i>FREQAXIS</i> argument is used only by time averaged HBAC. HBAC ignores this parameter.
<i>TIMEAXIS</i>	Determines primary axis time or frequency used in the Print/Probe output files. <code>TIMEAXIS = [0 1]</code> . 0 indicates frequency as primary axis (default). 1 indicates time as primary axis. Note: The <i>TIMEAXIS</i> argument is used only by <code>TS HBAC</code>. The <code>TA HBAC</code> ignores this parameter.
<i>TOL</i>	Specifies the absolute error tolerance for determining HBAC solver convergence. The value parameter must specify a real number greater than zero. The range is 1.e-14 to infinity. Default is 1e-9. This argument remains unchanged from existing HBAC.
<i>KRYLOVDIM</i>	Specifies the maximum Krylov subspace dimension used by the HBAC Krylov solver. The value parameter must specify an integer greater than zero. The range is 1 to infinity. Default is 300. This argument remains unchanged from existing HBAC.
<i>KRYLOVITER</i>	Specifies the maximum number of Generalized Minimum Residual (GMRES) solver iterations used by the Krylov solver. The value parameter must specify an integer greater than zero. The range is 1 to infinity. Default is 1500.

HBAC Analysis Options

The following options directly relate to an HBAC analysis and override the corresponding parameters on the HBAC statement line if specified in the netlist:

- `.OPTION HBACTOL`, default = 1x10-8, Range = 1x10-14 to Infinity
- `.OPTION HBACKRYLOVDIM`, default = 300, Range = 1 to Infinity
- `.OPTION HBACKRYLOVITER | HBAC_KRYLOV_ITER`, default = 1500, Range = 1 to Infinity

If these parameters are not specified, then the following conditions apply:

- If `HBACTOL > HBTOL`, then `HBACTOL = HBTOL`
- If `HBACKRYLOVDIM < HBKRYLOVDIM`, then `HBACKRYLOVDIM = HBKRYLOVDIM`

Output Syntax

This section describes the syntax for the HBAC `.PRINT` and `.PROBE` statements. These statements are similar to those used for HB analysis.

.PRINT and .PROBE Statements

```
.PRINT HB TYPE(NODES | ELEM) [INDICES]
.PROBE HB TYPE(NODES | ELEM) [INDICES]
```

Parameter	Description
TYPE	Specifies a harmonic type node or element. TYPE can be one of the following: • Voltage type – V = voltage magnitude and phase in degrees VR = real component VI = imaginary component VM = magnitude VP - Phase in degrees VPD - Phase in degrees VPR - Phase in radians VDB - dB units VDBM - dB relative to 1 mV • Current type – I = current magnitude and phase in degrees IR = real component II = imaginary component IM = magnitude IP - Phase in degrees IPD - Phase in degrees IPR - Phase in radians IDB - dB units IDBM - dB relative to 1 mV • Power type – P • Frequency type – hertz[index], hertz[index1, index2, ...] You must specify the harmonic index for the hertz variable. The frequency of the specified harmonics is dumped.
NODES ELEM	NODES or ELEM can be one of the following: • Voltage type – a single node name (n1), or a pair of node names, (n1,n2) • Current type – an element name (elemname) • Power type – a resistor (resistorname) or port (portname) element name • Frequency type – the harmonic index for the hertz variable. The frequency of the specified harmonics is dumped.

Parameter	Description
INDICES	Index to tones in the form [n1, n2, ..., nK, +/-1]. • nj is the index of the j-th HB tone and the .HB statement contains K tones • +/-1 is the index of the HBAC tone Wildcards are not supported if this parameter is used. You can transform HB data into the time domain and output by using the following syntax: .PRINT HBTRAN ov1 [ov2 ... ovN].PROBE HBTRAN ov1 [ov2 ... ovN]. See TYPE above for voltage and current type definitions.

Time-Sampled (TS) HBAC Output Format

A TS-HBAC analysis produces these output data files:

- Output from the .PRINT and .PROBE statements are written to a *.printhbacts# or *.hbacts# file respectively. The TIMEAXIS parameter on the HBAC statement can be used to select the X-axis for plotting.
 - *TIMEAXIS* = 0 each block contains the data format TYPE Node|Element versus frequency at each TIME point. This allows display of frequency-parameterized curves versus time.
 - *TIMEAXIS* = 1 each block contains the data format TYPE Node|Element versus TIME at each frequency point. This allows display of time-parameterized curves versus frequency.

Time-Averaged (TA) HBAC Output Format

A TA-HBAC analysis produces these output data files:

Output from the .PRINT and .PROBE statements are written to a *.printtbac# or a *.hbac# file respectively. The FREQAXIS parameter on the HBAC statement can be used to select the X-axis for plotting.

FREQAXIS = IN each block contains the data format TYPE Node|Element[INDICES] versus INPUT frequencies.

FREQAXIS = OUT each block contains the data format TYPE Node|Element[INDICES] versus OUTPUT frequencies.

For back compatibility for versions before PrimeSim HSPICE 2020.12 style HBAC output to *.hb#, *.printhb# use the option *HBAC_OUTPUT_TO_HB* = 1. Default is 0.

Using the .MEASURE Command with .HBAC

Since .HBAC requires an .HB analysis, the measure statements for this analysis are the same as for .HB analysis. For example:

```
.MEASURE HB result FIND out_var AT=val
```

The .MEASURE command format allows to access Node/Element values at a given TIME value and a given input or output frequency value.

Note:

When a requested time value lies outside the range (0, Tp), Tp is the period associated with the fundamental frequency, the time value will be appropriately folded into the acceptable range. Negative frequency values are acceptable.

Errors and Warnings

Reported performance log statistics are written to a .lis file:

- Number of nodes
- Memory in use
- CPU time
- Maximum Krylov iterations
- Maximum Krylov dimension
- Target GMRES residual
- GMRES residual
- Actual Krylov iterations considered
- Frequency (swept input frequency values)

The following error and warning messages are used when PrimeSim HSPICE encounters a problem with an HBAC analysis.

Error Messages

- HBAC frequency sweep includes negative frequencies. HBAC allows only frequencies that are greater than or equal to zero.
- No HB statement is specified (error at parser). HBAC requires an HB statement to generate the steady-state solution.

Warning Messages

- More than one HBAC statement (warning at parser). PrimeSim HSPICE advanced analog analyses uses only the last HBAC statement in the netlist.
- No HBAC sources are specified (error at parser). HBAC requires at least one HBAC source.
- GMRES Convergence Failure. When GMRES (Generalized Minimum Residual) reaches the maximum number of iterations and the residual is greater than the specified tolerance. The HBAC analysis generates a warning and then continues as if the data were valid. This warning reports the following information:
 - Final GMRES Residual
 - Target GMRES Residual
 - Maximum Krylov Iterations
 - Actual Krylov Iterations taken
- Time-Sampled HBAC is currently not supported for multi-tone Harmonic Balance. Defaults to Time-Averaged HBAC. (Warning at parser)

Shooting Newton AC Analysis (.SNAC)

You use the Shooting Newton AC (.SNAC) statement for analyzing the linear behavior in large-signal periodic systems. The .SNAC statement uses periodic AC (PAC) and the Shooting Newton algorithm to perform linear analysis of circuits where the linear coefficients are modulated by a periodic, steady-state signal.

The following section describes the periodic AC analysis based on a Shooting Newton algorithm. This functionality is similar to the Harmonic Balance for periodic AC (HBAC) analysis.

The following sections discuss these topics:

- [Prerequisites and Limitations](#)
- [SNAC Example](#)
- [Input Syntax](#)
- [Output Syntax](#)
- [SNAC Output Data Files](#)
- [Using the .MEASURE Command with .SNAC](#)
- [Errors and Warnings](#)

Prerequisites and Limitations

The following prerequisites and limitations apply to SNAC:

- Requires one and only one `.SNAC` statement. If you use multiple `.SNAC` statements, PrimeSim HSPICE advanced analog analyses use only the last `.SNAC` statement.
- Requires one and only one `.SN` statement.
- Requires placing the parameter sweep in the `.SN` statement.
- Requires at least one periodic source.
- Limited to simulations that can be reduced to a single tone SN analysis.
- Supports unlimited number of sources.
- The requested maximum harmonic in a `.PROBE` or `.PRINT` statement must be less than or equal to half the number of harmonics specified in the SN statement (that is, $\text{max_harm} \leq \text{nharms} / 2$).

SNAC Example

The following example is shipped with the PrimeSim HSPICE distribution as `mix_snac.sp` and is available in the directory: `$installdir/demo/hspice/rf_examples/`.

```
* Test SNAC: ideal I,Q mixer -rrd
.OPTIONS PROBE
.OPTIONS POST=2
$.OPTIONS snmaxiter=100
vlo lo 0 1.0 cos(1.0 0.5 1g) $ Periodic, Large-Signal SN Input
rlo lo 0 1.0
rrf rf 0 1.0 $ Noise source
rrf1 rf1 rf 1.0 $ Noise source
g1 0 if cur='1.0*v(lo)*v(rf)' $ mixer element
c1 0 if q='1.0e-9*v(lo)*v(rf)' $ mixer element
rout if ifg 1.0
vctrl ifg 0 0.0
h1 out 0 vctrl 1.0
rh1 out 0 1.0
vrf rf1 0 snac .001 24.0 $ Small signal for SNAC with 1-tone SN Input
.sn tones=1.0g nharms=3
.snac lin 1 0.8g 0.8g
.print sn v(rf1) v(lo) v(out)
.print snfd v(rf1) v(lo) v(out)
.print snac v(rf1) v(lo) v(out)
.measure snac vout1 find v(out) [1,-1] at=0.8g
.measure snac vout2 find v(out) [0,1] at=0.8g
.measure snac vout3 find v(out) [1,1] at=0.8g
.measure sn vlo1 find v(lo) at=0.5n
```

```
.measure sn vlo2 find v(lo) at=1n
.measure snfd vlo3 find v(lo) [1] at=1
.end
```

Input Syntax

```
.SNAC frequency_sweep
```

Parameter	Description
frequency_sweep	Frequency sweep range for the input signal (also referred to as the input frequency band or fin). You can specify LIN, DEC, OCT, POI, or SWEEPBLOCK. Specify the nsteps, start, and stop frequencies using the following syntax for each type of sweep: - LIN nsteps start stop - DEC nsteps start stop - OCT nsteps start stop - POI nsteps freq_values - SWEEPBLOCK=swblockname - DATA=dataname

Output Syntax

This section describes the syntax for the SNAC .PRINT and .PROBE statements. These statements are similar to those used for HB analysis.

.PRINT and .PROBE Statements

```
.PRINT SNAC TYPE (NODES | ELEM) [INDICES]
.PROBE SNAC TYPE (NODES | ELEM) [INDICES]
```

Parameter	Description
TYPE	Specifies a harmonic type node or element. TYPE can be one of the following: - Voltage type – - V = voltage magnitude and phase in degrees - VR = real component - VI = imaginary component - VM = magnitude - VP - Phase in degrees - VPD - Phase in degrees - VPR - Phase in radians - VDB - dB units VDBM - dB relative to 1 mV - Current type – - I = current magnitude and phase in degrees - IR = real component - II = imaginary component - IM = magnitude - IP - Phase in degrees - IPD - Phase in degrees - IPR - Phase in radians - IDB - dB units IDBM - dB relative to 1 mV - Power type – P - Frequency type – hertz[index], hertz[index1, index2, ...] You must specify the harmonic index for the hertz variable. The frequency of the specified harmonics is dumped.
NODES ELEM	NODES or ELEM can be one of the following: - Voltage type – a single node name (n1), or a pair of node names, (n1,n2) - Current type – an element name (elemname) - Power type – a resistor (resistorname) or port (portname) element name - Frequency type – the harmonic index for the hertz variable. The frequency of the specified harmonics is dumped.
INDICES	Index to tones in the form [n1, +/-1]. - n1 is the index of the SN tone +/-1 is the index of the SNAC tone Wildcards are not supported if this parameter is used. You can transform SN data into the time domain and output by using the following syntax: .PRINT SNTRAN ov1 [ov2 ... ovN]. PROBE SNTRAN ov1 [ov2 ... ovN]. See TYPE above for voltage and current type definitions.

SNAC Output Data Files

A SNAC analysis produces these output data files:

- Output from the .PRINT statement is written to a .printsnac# file.
- This data is against the input frequency band points.
- The header contains the large-signal fundamental frequency and the range of small-signal frequencies.

- The columns of data are labeled as F(Hz), followed by the output variable names. Each variable name has the associated mixing pair value appended. All N variable names and all M mixing pair values are printed for each swept small-signal frequency value (a total of N*M for each frequency value).
- Output from the `.PROBE` statement is written to a `.snac#` file.

Reported performance log statistics are written to a `.lis` file:

- Number of nodes
- Number of FFT points
- Number of equations
- Memory in use
- CPU time
- Maximum Krylov iterations
- Maximum Krylov dimension
- Target GMRES residual
- GMRES residual
- Actual Krylov iterations taken
- Frequency (swept input frequency values)

Using the .MEASURE Command with .SNAC

Since `.SNAC` requires an `.SN` analysis, the measure statements for this analysis are the same as for `.SN` analysis. For example,

```
.MEASURE SN result FIND out_var AT=val
```

Errors and Warnings

The following error and warning messages are used when PrimeSim HSPICE encounters a problem with a SNAC analysis.

Error Messages

- SNAC frequency sweep includes negative frequencies. SNAC allows only frequencies that are greater than or equal to zero.
- No SN statement is specified (error at parser). SNAC requires an SN statement to generate the steady-state solution.

Warning Messages

- More than one SNAC statement (warning at parser). PrimeSim HSPICE advanced analog analyses uses only the last SNAC statement in the netlist.
- No SNAC sources are specified (error at parser). SNAC requires at least one SNAC source.
- GMRES Convergence Failure. When GMRES (Generalized Minimum Residual) reaches the maximum number of iterations and the residual is greater than the specified tolerance. The SNAC analysis generates a warning and then continue as if the data were valid. This warning reports the following information:
 - Final GMRES Residual
 - Target GMRES Residual
 - Maximum Krylov Iterations
 - Actual Krylov Iterations taken

Steady-State Voltage and Current Sources

The I (current source) and V (voltage source) elements include extensions that allow you to use them as sources of steady-state sinusoidal signals for HB/HBAC and SN/SNAC analyses. When you use a power parameter to specify the available power, you can also use these elements as power sources.

For a general description of the I and V elements, see Power Sources in the *PrimeSim HSPICE® User Guide: Elements*.

- [I and V Element Syntax](#)
- [Examples](#)

I and V Element Syntax

```
Vxxx p n
+ $ **** Voltage or Power Information ****
+ [[dc] mag] [ac [mag [phase]]] [HBAC [mag [phase]]]
+[SNAC [mag [phase]]]
+ [hb [mag [phase] [harm [tone [modharm [modtone]]]]]]
+ [transient waveform] [TRANFORHB=[1|0]]

+ $ **** Power Switch ****
+ [power=[0 | 1 | W | dbm]] [z0=val] [rdc=val] [rac=val]
+ [RHBAC=val] [rhb=val] [rtran=val]
```

```
Ixxx p n
+ $ **** Current or Power Information ****
+ [[dc] mag] [ac [mag [phase]]] [HBAC [mag [phase]]]
+ [SNAC [mag [phase]]]
+ [hb [mag [phase [harm [tone [modharm [modtone]]]]]]]
+ [transient waveform] [TRANFORHB=[1|0]]

+ $ **** Power Switch ****
+ [power=[0 | 1 | W | dbm]] [z0=val] [rdc=val] [rac=val]
+ [RHBAC=val] [rhb=val] [rtran=val]
```

Parameter	Description
[[dc] mag]	DC voltage or power source value. You do not need to specify DC explicitly (default=0).
[ac [mag [phase]]]	AC voltage or power source value.
[HBAC [mag [phase]]]	Advanced analog HBAC voltage or power source value.
[SNAC [mag [phase]]]	Advanced analog SNAC voltage or power source value.
[hb [mag [phase [harm [tone [modharm [modtone]]]]]]]	Advanced analog HB voltage, current, or power source value. Multiple HB specifications with different harm, tone, modharm, and modtone values are allowed. <i>phase</i> is in degrees <i>harm</i> and <i>tone</i> are indices corresponding to the tones specified in the .HB statement. Indexing starts at 1 (corresponding to the first harmonic of a tone). <i>modtone</i> and <i>modharm</i> specify sources for multi-tone simulation. A source specifies a tone and a harmonic, and up to 1 offset tone and harmonic (<i>modtone</i> for tones and <i>modharm</i> for harmonics). The signal is then described as: $V(\text{or } I) = \text{mag} \cdot \cos(2\pi \cdot (\text{harm} \cdot \text{tone} + \text{modharm} \cdot \text{modtone}) \cdot t + \text{phase})$
[transient waveform]	(Transient analysis) Any one of waveforms: AM, EXP, PULSE, PWL, SFFM, or SIN. Multiple transient descriptions are not allowed.
[power=[0 1 W dbm]]	(PrimeSim HSPICE advanced analog analyses) Power Switch - When 0 (default), element treated as a voltage or current source. - When 1 or W, element treated as a power source, realized as a voltage source with a series impedance. In this case, the source value is interpreted as RMS available power in units of Watts. - When dbm, element treated as a power source in series with the port impedance. Values are in dbms. You can use this parameter for transient analysis if the power source is either DC or SIN.
[z0=val]	(LIN analysis) System impedance used when converting to a power source, inserted in series with the voltage source. Currently, this only supports real impedance. - When power=0, z0 defaults to 0. - When power=1, z0 defaults to 50 ohms. You can also enter <i>zo=val</i> .

Parameter	Description
[rdc=val]	(DC analysis) Series resistance (overrides z0).
[rac=val]	(AC analysis) Series resistance (overrides z0).
[RHBAC=val]	(PrimeSim HSPICE advanced analog HBAC analysis) Series resistance (overrides z0).
[rhb=val]	(PrimeSim HSPICE advanced analog HB analysis) Series resistance (overrides z0).
[rtran=val]	(Transient analysis) Series resistance (overrides z0).
[TRANFORHB=[0 1]]	• 0 (default): The transient description is ignored if an HB value is given or a DC value is given. If no DC or HB value is given and TRANFORHB=0, then HB treats the source as a DC source, and the DC source value is the time=0 value. • 1: HB analysis uses the transient description if its value is VMRF, SIN, PULSE, PWL, or LFSR. If the type is a non-repeating PWL source, then the time=infinity value is used as a DC source value. For example, the following statement is treated as a DC source with value=1 for HB: <code>v1 1 0 PWL (0 0 1n 1 1u 1) TRANFORHB=1</code> In contrast, the following statement is a 0V DC source: <code>v1 1 0 PWL (0 0 1n 1 1u 1) TRANFORHB=0</code> The following statement is treated as a periodic source with a 1us period that uses PWL values: <code>v1 1 0 PWL (0 0 1n 1 0.999u 1 1u 0) R TRANFORHB=1</code> To override the global TRANFORHB option, explicitly set TRANFORHB for a V/I source.

Examples

Example 1

This example shows an HB source for a single tone analysis:

```
.hb tones=100MHz harms=7
I1 1 2 dc=1mA hb 3mA 0. 1 1
```

I1 is a current source with a the following time-domain description:

```
I1=1mA + 3mA*cos(2*pi*1.e8*t)
```

Example 2

This example shows HB sources used for a two-tone analysis:

```
.hb tones=1.e9 1.1e9 intmodmax=5
Vin lo 0 dc=0. hb 1.5 90 1 1
Vrf rf 0 dc=0. hb 0.2 0 1 2
```

These sources have the following time-domain descriptions:

```
Vin=1.5*cos(2*pi*1.e9*t - 90*pi/180) V  
Vrf = 0.2*cos(2*pi*1.1e9*t) V
```

Example 3

The following HB source uses a `modtone` and `modharms`:

```
.hb tones=2.e9 1.9e9 harms=5 5  
Vm input gnd dc=0.5 hb 0.2 0. 1 1 -1 2
```

`Vm` has the following time-domain description:

```
Vm = 0.5 + cos(2*pi*1.e8*t)
```

Example 4

This example uses an HB source specified with a `SIN` source and `TRANFORHB`.

```
.hb tone=1.e8 harms=7  
Vt 1 2 SIN(0.1 1.0 2.e8 0. 0. 90) tranforhb=1
```

`Vt` is converted to the following HB source:

```
Vt 1 2 dc=0.1 hb 1.0 0.0 2 1
```

Example 5

This example shows a power source (the units are Watts).

```
.hb tones=1.1e9 harms=9  
Pt Input Gnd power=1 Z0=50. 1m 0. 1 1
```

`Pt` delivers 1 mW of power through a 50-ohm impedance.

Phase Noise and Buffer Chains

Phase noise is specific to oscillators. However, for buffer chains, you can do periodic noise analysis. Phase noise that may be contributed by buffer, amplifier, divider, or multiplier circuits is often referred to as the *residual phase noise*. There are three commands in PrimeSim HSPICE advanced analog analyses that can be used to help you predict such noise: `.HBNOISE`, `.SNNOISE`, and `.PTDNOISE`. These analyses compute the noise spectral density at an output variable taking into consideration the modulation effects.

`.HBNOISE` and `.SNNOISE` compute the average noise over one period, assuming your circuit is driven by a periodic input signal.

`.PTDNOISE` computes the noise at one specific time point within the period, or it can compute the noise as a function of time over one period. `.PTDNOISE` can also convert the

noise to a “jitter” value, which would be an uncertainty in the timing as opposed to the uncertainty in the output voltage.

See the following sections for detailed information on these commands:

- [Multitone Harmonic Balance Noise \(.HBNOISE\)](#)
- [Shooting Newton Noise Analysis \(.SNNOISE\)](#)
- [Periodic Time-Dependent Noise Analysis \(.PTDNOISE\)](#)

Multitone Harmonic Balance Noise (.HBNOISE)

An HBNOISE (Harmonic Balance noise) analysis simulates the noise behavior in periodic systems and is designed for use with driven circuits. It employs a PAC algorithm to perform noise analysis of non-autonomous (driven) circuits under periodic, steady-state tone conditions. This can be extended to quasi-periodic systems having more than one periodic, steady-state tone. One application for a multitone HBNOISE analysis is determining mixer noise figures under the influence of a strong interfering signal.

The PAC method simulates noise assuming that the stationary noise sources and/or the transfer function from the noise source to a specific output are periodically modulated.

- The modulated noise source (thermal, shot, or flicker) is modeled as a cyclostationary noise source.
- A PAC algorithm solves for the modulated transfer function.
- You can also use the HBNOISE PAC method with correlated noise sources including the MOSFET level 9 and level 11 models and the behavioral noise source in the G-element (voltage-dependent current source).

You use the `.HBNOISE` statement to perform a periodic noise analysis.

The following sections discuss these topics:

- [Supported Features](#)
- [HBNOISE Example](#)
- [Input Syntax](#)
- [Output Syntax](#)
- [Output Data Files](#)
- [Measuring HBNOISE Analyses with .MEASURE](#)

- [Optimizing with HBNOISE Measurement](#)
- [Errors and Warnings](#)

Supported Features

HBNOISE supports the following features:

- All existing PrimeSim HSPICE advanced analog noise models.
- Uses more than one single-tone, harmonic balance to generate the steady-state solution.
- Unlimited number of HB sources (using the same tone, possibly multiple harmonics).
- Includes stationary, cyclostationary, frequency-dependent, and correlated noise effects.
- Swept parameter analysis.
- Results are independent of the number of HBAC sources in the netlist.

Prerequisites and Limitations

The following prerequisites and limitations apply to HBNOISE:

- Requires one `.HB` statement (which determines the steady-state solution).
- Requires at least one HB source or one TRANFORHB source.
- Requires placing the parameter sweep in the `.HB` statement.
- The requested maximum harmonic in `.HBNOISE` must be less than or equal to half the number of harmonics used in harmonic balance (that is, $max_harm \leq num_hb_harms/2$).

HBNOISE Example

This example performs an HB analysis, then runs an HBNOISE analysis over a range of frequencies, from 9.0e8 to 9.2e8 Hz. Simulation outputs the output noise at V(out) and the single side-band noise figure versus the input frequency band, from 1e8 to 1.2e8 Hz, to the `*.pn0` file. The following is the netlist for this example:

```
$$*-Ideal mixer + noise source
$ prints total noise at the output (1.57156p V/sqrt-Hz),
$ single-sideband noise figure, (3.01 dB)
$ double-sideband noise figure. (0 dB)
.OPTION PROBE
.OPTION POST=2
vlo lo 0 0.0 hb 1.0 0 1 1 $ Periodic, HB Input
ilo lo 0 0
```

```
rsrc rfin rfl 1.0 $ Noise source
c1 0 if q='1.0e-9*v(lo)*v(rfin)' $ mixer element
g1 0 if cur='1.0*v(lo)*v(rfin)' $ mixer element
rout if 0 1.0
vrf rfl 0 $ hbac 2.0 0.0
.hb tones=1.0g nharms=4 $ sweep mval 1 2 1
.HBNOISE rout rsrc lin 11 0.90g 0.92g
.print HBNOISE onoise ssnf dsnf
.probe HBNOISE onoise ssnf dsnf
.end
```

Input Syntax

```
.HBNOISE outvar [outvar2 ... outvarN] insrc parameter_sweep
+ [n1, n2, ..., nk, +|-1]
+ [listfreq=(frequencies|none|all)] [listcount=val]
+ [listfloor=val] [listsources= on|off]
+ [TOL=value]
+ [KRYLOVDIM=value]
+ [KRYLOVITER=value]
```

Parameter	Description
outvar [outvar2 ... outvarN]	Nodal voltage or branch current output variables. Define the nodes or branches at which the PrimeSim HSPICE simulator sums the noise. You can specify an arbitrary number of output variables. However, you must specify at least one outvar.
insrc	Input source. If this is a resistor, PrimeSim HSPICE uses it as a reference noise source to determine the noise figure. If the resistance value is 0, the result is an infinite noise figure.

Parameter	Description
<code>parameter_sweep</code>	Frequency sweep range for the input signal. Also referred to as the input frequency band (IFB) or <i>fin</i>. You can specify the sweep type as either LIN, DEC, OCT, POI, or, SWEEPBLOCK. Specify the <i>nsteps</i> or <i>npoints</i>, <i>start</i> and <i>stop</i> values using the following syntax for each sweep type: • LIN <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • POI <i>npoints p1, p2, ...pn</i> • SWEEPBLOCK = <i>swblockname</i> • DATA= <i>dataname</i> • OPTIMIZE=OPTxxx • MONTE=<i>val</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave(OCT). • <i>npoints</i> = number of points for linear (LIN) or points ofinterest (POI). • <i>start</i> = starting value. • <i>stop</i> = ending value. • <i>p1, p2, ... pn</i> = point values. • <i>swblockname</i> = sweep block name. • <i>dataname</i> = data block name. • OPTxxx = optimization model name. • <i>val</i> = number of Monte Carlo sweeps.
<code>n1,n2,...,nk, + -1</code>	Index term defining the output frequency band (<i>fout</i>) at which the noise is evaluated. Generally, $f_{out} = \text{ABS}(n_1 \cdot f_1 + n_2 \cdot f_2 + \dots + n_k \cdot f_k + /-f_{in})$ where: • <i>f1,f2,...,fk</i> are the first through <i>kth</i> steady-state tones determined from the harmonic balance solution • <i>fin</i> is the input frequency band defined by <i>parameter_sweep</i>. The default index term is [1,1,...1,-1]. For a single tone analysis, the default mode (index term [1,-1]) is consistent with simulating a high-side, down conversion mixer where the RF signal is specified by the input frequency band and the noise is measured at a down-converted frequency that the output frequency band specifies: $f_{out} = \text{abs}(f_{LO} - f_{in})$. Where <i>fLO</i> corresponds to the large signal specified by the first index 1 and <i>fin</i> is the input RF signal specified by the second index -1. In general, you can use the [n1,n2,...,nk,+/-1] index term to specify an arbitrary offset. The noise figure measurement is also dependent on this index term.
<code>listfreq</code>	Prints the element noise value to the .lis file. You can specify at which frequencies the element noise value is printed. The frequencies must match the sweep_frequency values defined in the <i>parameter_sweep</i>, otherwise they are ignored. In the element noise output, the elements that contribute the largest noise are printed first. The frequency values can be specified with the NONE or ALL keyword, which either prints no frequencies or every frequency defined in <i>parameter_sweep</i>. Frequency values must be enclosed in parentheses. For example: <code>listfreq=(none)</code> <code>listfreq=(all)</code> <code>listfreq=(1.0G)</code> <code>listfreq=(1.0G, 2.0G)</code>. The default value is NONE.

Parameter	Description
<code>listcount</code>	Prints the element noise value to the <code>.lis</code> file, which is sorted from the largest to smallest value. You do not need to print every noise element; instead, you can define <code>listcount</code> to print the number of element noise frequencies. For example, <code>listcount=5</code> means that only the top 5 noise contributors are printed. The default value is 1.
<code>listfloor</code>	Prints the element noise value to the <code>.lis</code> file and defines a minimum meaningful noise value (in $\text{V/Hz}^{1/2}$ units). Only those elements with noise values larger than <code>listfloor</code> are printed. The default value is $1.0\text{e-}14 \text{ V/Hz}^{1/2}$.
<code>listsources</code>	Prints the element noise value to the <code>.lis</code> file when the element has multiple noise sources, such as a FET, which contains the thermal, shot, and 1/f noise sources. You can specify either ON or OFF: ON Prints the contribution from each noise source and OFF does not. The default value is OFF. When <code>listsources</code> is turned on, the element noise source contributions will also be output into the <code>*.pn#</code> file.
<code>TOL</code>	Same as <code>.OPTION PHASENOISETOL</code> .
<code>KRYLOVDIM</code>	Same as <code>.OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM</code> .
<code>KRYLOVITER</code>	Same as <code>.OPTION PHASENOISEKRYLOVITER / PHASENOISE_KRYLOV_ITR</code> .

Output Syntax

The PrimeSim HSPICE advanced analog HB and SN noise analyses can output the output noise (`onoise`), noise figures (`NF`, `SSNF`, and `DSNF`) and, the input referred noise (`inoise`). This section describes the syntax for the HBNOISE `.PRINT` and `.PROBE` statements.

.PRINT and .PROBE Statements

```
.PRINT HBNOISE [ONOISE] [NF] [SSNF] [DSNF] [INOISE]
.PROBE HBNOISE [ONOISE] [NF] [SSNF] [DSNF] [INOISE]
```

Parameter	Description
<code>ONOISE</code>	Outputs the voltage noise at the output frequency band across the output nodes in the <code>.HBNOISE</code> statement. The data is plotted as a function of the input frequency band points. Units are in $\text{V/Hz}^{1/2}$. Simulation ignores <code>ONOISE</code> when applied to autonomous circuits.
<code>NF SSNF</code>	NF and SSNF both output a single-side band noise figure as a function of the input frequency band points: $\text{NF} = \text{SSNF} = 10 \text{ Log}(\text{SSF})$ Single side-band noise factor, $\text{SSF} = \{(\text{Total Noise at output, at the output frequency band, originating from all frequencies}) - (\text{Load Noise originating from the output frequency band})\} / (\text{Input Source Noise originating from the input frequency band})$.

Parameter	Description
DSNF	DSNF outputs a double side-band noise figure as a function of the input frequency band points. $DSNF = 10 \log(DSF)$ Double side-band noise factor, $DSF = \{(\text{Total Noise at output, at the output frequency band, originating from all frequencies}) - (\text{Load Noise originating from the output frequency band})\} / (\text{Input Source Noise originating from the input frequency band and from the image of the input frequency band})$.
INOISE	Outputs input referred noise which can be printed, probed, or measured.

Output Data Files

An HBNOISE analysis produces these output data files:

- Output from the `.PRINT` statement is written to a `.printpn#` file.
- Output from the `.PROBE` statement is written to a `.pn#` file.

Both the `*.printpn#` and `*.pn#` files output data against the input frequency band points.

- When multiple output variables are specified, multiple `*.printpn#` and `*.pn#` files are generated, each with one output variable name in its file name as in `*.printpn#@outvar` and `*.pn#@outvar`.
- Standard output information is written to a `.lis` file:
 - simulation time
 - HBNOISE linear solver method
 - HBNOISE simulation time
 - total simulation time

See also [Using Noise Analysis Results as Input Noise Sources](#).

Measuring HBNOISE Analyses with .MEASURE

Note:

A `.MEASURE HBNOISE` statement cannot contain an expression that uses a `HBNOISE` variable as an argument. Also, you cannot use a `.MEASURE HBNOISE` statement for error measurement and expression evaluation of `HBNOISE`.

The `.MEASURE HBNOISE` syntax supports several types of measurements:

- Find-when

```
.MEASURE HBNOISE result FIND out_var1  
+ AT = Input_Frequency_Band value
```

The previous measurement yields the result of a variable value at a specific input frequency band point.

```
.MEASURE HBNOISE result FIND out_var1  
+ WHEN out_var2 = out_var3
```

The previous measurement yields the result at the input frequency point when `out_var2 == out_var3`.

```
.MEASURE HBNOISE result WHEN out_var2 = out_var3
```

The previous measurement yields the input frequency point when `out_var2 == out_var3`.

- Average, RMS, min, max, and peak-to-peak

```
.MEASURE HBNOISE result [RMS] out_var [FROM = IFB1]  
+ [TO = IFB2]
```

- Integral evaluation

```
.MEASURE HBNOISE result INTEGRAL out_var  
+ [FROM = IFB1] [TO = IFB2]
```

This measurement integrates the `out_var` value from the IFB1 frequency to the IFB2 frequency.

- Derivative evaluation

```
.MEASURE HBNOISE result DERIVATIVE out_var AT = IFB1
```

This measurement finds the derivative of `out_var` at the IFB1 frequency point.

Note:

`.MEASURE HBNOISE` cannot contain an expression that uses an *hbnoise* variable as an argument. You also cannot use `.MEASURE HBNOISE` for error measurement and expression evaluation of `HBNOISE`.

- Input referred noise

```
.MEASURE [HBNOISE|SNNOISE] result FIND inoise  
+ AT = IFB_value
```

This measurement yields the result of the input referred noise at a specific input frequency band point.

```
.MEASURE [HBNOISE|SNNOISE] result FIND inoise  
+ WHEN out_var2 = out_var3
```

This measurement yields the result at the input frequency point when `out_var2 == out_var3`.

```
.MEASURE HBNOISE result func inoise [FROM = IFB1]  
+ [TO = IFB2]
```

Where `func` is one of the following measurement types:

- **AVG (average):** Calculates the area under the `inoise` curve, divided by the periods of interest.
- **MAX (maximum):** Reports the maximum value of `inoise` over the specified interval.
- **MIN (minimum):** Reports the minimum value of `inoise` over the specified interval.
- **PP (peak-to-peak):** Reports the maximum value minus the minimum value of `inoise` over the specified interval.
- **RMS (root mean squared):** Calculates the square root of the area under the `inoise` curve, divided by the period of interest.

```
.MEASURE HBNOISE result INTEGRAL inoise  
+ [FROM =IFB1] [TO = IFB2]
```

This measurement integrates the `inoise` value from the IFB1 frequency to the IFB2 frequency.

```
.MEASURE HBNOISE result DERIVATIVE inoise AT = IFB1
```

This measurement finds the derivative of `inoise` at the IFB1 frequency point.

The PrimeSim HSPICE advanced analog analyses optimization flow can read the measured data from a `.MEASURE HBNOISE` analysis. This flow can be combined in the PrimeSim HSPICE advanced analog analyses optimization routine with a `.MEASURE HBTR` analysis (see [Using .MEASURE with .HB Analyses](#)) and a `.MEASURE PHASENOISE` analysis (see [Measuring Phase Noise with .MEASURE PHASENOISE](#)).

Optimizing with HBNOISE Measurement

The required statements are:

- Analysis statement

```
.HB TONES=f1[f2...fn] [NHARMS=h1 [,h2...hn]]  
+ SWEEP OPTIMIZE=OPTxxx RESULT=measname MODEL=mname
```


For example:

```
.HBOSC tone=1g nharms=5 PROBENODE=out,gnd,0.8  
+ SWEEP OPTIMIZE=opt1 RESULT=y1,y2 MODEL=m1  
.MODEL m1 OPT level=0
```

- Measure statement

```
.MEASURE HBNOISE measname FIND out_var1 AT=val GOAL=val
```

Errors and Warnings

For .HBNOISE errors, see the list of .HBAC [Errors and Warnings](#).

Shooting Newton Noise Analysis (.SNNOISE)

A Shooting Newton noise (SNNOISE) analysis simulates the noise behavior in periodic systems. It uses a PAC algorithm to perform noise analysis of non-autonomous (driven) circuits under periodic, steady-state tone conditions. SNNOISE is similar to the HBNOISE analysis.

The PAC method simulates noise assuming that the stationary noise sources and/or the transfer function from the noise source to a specific output are periodically modulated.

- The modulated noise source (thermal, shot, or flicker) is modeled as a cyclostationary noise source.
- A PAC algorithm solves the modulated transfer function.
- You can also use the SNNOISE PAC method with correlated noise sources including the MOSFET Level 9 and Level 11 models and the behavioral noise source in the G-element (voltage-dependent current source).

You use the .SNNOISE statement to perform a periodic noise analysis.

The following sections discuss these topics:

- [Supported Features](#)
- [SNNOISE Analysis Example](#)
- [Input Syntax](#)
- [Output Syntax](#)
- [Output Data Files](#)
- [Measuring SNNOISE Analyses with .MEASURE](#)

Supported Features

SNNOISE supports the following features:

- All existing PrimeSim HSPICE advanced analog noise models.
- Uses Shooting Newton to generate the steady-state solution.
- Unlimited number of sources.
- Includes stationary, cyclostationary, frequency-dependent, and correlated noise effects.
- Swept parameter analysis.
- Results are independent of the number of SNAC sources in the netlist.

Prerequisites and Limitations

The following prerequisites and limitations apply to SNNOISE:

- Requires one `.SN` statement (which determines the steady-state solution).
- Requires at least one periodic source. Does not recognize HB sources.
- Requires placing the parameter `sweep` in the `.SN` statement.

SNNOISE Analysis Example

This example performs an SN analysis, then runs an SNNOISE analysis over a range of frequencies from 9.0e8 to 9.2e8 Hz. Simulation outputs the output noise at V(out) and the single side-band noise figure versus the input frequency band, from 9.0e8 to 9.2e8 Hz, to the `*.pn0` file. The following is the netlist for this example:

```
*
$*-Ideal mixer + noise source
$ prints total noise PSD at the output (2.47e-20 V^2) when q=0
$ single-sideband noise figure, (3.01 dB)
$ double-sideband noise figure. (0 dB)
.OPTION PROBE
.OPTION POST=2
vlo lo 0 0.0 cos (0 1.0 1.0g 0 0 0)
llo lo 0 0
rsrc rfin rf1 1.0 $ Noise source
gl 0 if cur='1.0*v(lo)*v(rfin)' $ mixer element
cl 0 if q='1.0e-9*v(lo)*v(rfin)' $ mixer element
rout if 0 1.0
vrf rf1 0 $ hbac 2.0 0.0
.option delmax=0.002n
.SN tones=1G nharms=4 trstab=10n
.SNNOISE rout rsrc lin 11 0.90g 0.92g
.probe SNNOISE onoise ssnf dsnf
```

```
.print SNNOISE onoise ssnf dsnf
.end
```

Input Syntax

```
.SNNOISE outvar [outvar2 ... outvarN] insrc frequency_sweep
+[n1, +|-1]
+[listfreq=(frequencies|none|all)] [listcount=val]
+[listfloor=val] [listsources= on|off]
+ [TOL=value]
+ [KRYLOVDIM=value]
+ [KRYLOVITER=value]
```

Parameter	Description
outvar [outvar2 ... outvarN]	Nodal voltage or branch current output variables. Define the nodes or branches at which the PrimeSim HSPICE simulator sums the noise. You can specify an arbitrary number of output variables. However, you must specify at least one outvar.
insrc	Input source.
frequency_sweep	Frequency sweep specification. Specifies the type, nsteps or npoints and fstart and fstop frequency or frequency points for each sweep type, where: • type = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN npoints fstart fstop • OCT nsteps fstart fstop • DEC nsteps fstart fstop • POI npoints p1, p2, ...pn • SWEEPBLOCK = swblockname • Where: • nsteps = number of steps per decade (DEC) or octave (OCT). • npoints = number of points for linear (LIN) or points of interest (POI). • fstart = starting frequency. • fstop = ending frequency. • p1, p2, ... pn = frequency points.
n1, + -1	Index term defining the output frequency band at which the noise is evaluated. The output frequency is computed according to $f_{out}= n1*f1 +/- fin $, where f1 is the fundamental tone (inverse of the fundamental period) and fin is from the frequency sweep. For a single tone analysis, the default mode (index term [1,-1]) is consistent with simulating a high-side, down conversion mixer where the RF signal is specified by the input frequency band and the noise is measured at a down-converted frequency that the output frequency band specifies: $f_{out}=abs(f_{LO}-fin)$. Where fLO corresponds to the large signal specified by the first index 1 and fin is the input RF signal specified by the second index -1. See Specifying Variant Indices and Measuring SNNOISE Analyses with .MEASURE .
listfreq	Prints the element noise value to the .lis file; the default is none.
listcount	Prints the element noise value to the .lis file, sorted from the largest to smallest value.

Parameter	Description
listfloor	Prints the element noise value to the .lis file and defines a minimum meaningful noise value. Only those elements with noise values larger than listfloor are printed. The default value is 1.0e-14 V/sqrt(Hz).
listsources	Prints the element noise value to the .lis file when the element has multiple noise sources. The default is off. When listsources is turned on, the element noise source contributions will also be output into the*.snpn# file.
TOL	Same as .OPTION PHASENOISETOL.
KRYLOVDIM	Same as .OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM.
KRYLOVITER	Same as .OPTION PHASENOISEKRYLOVITER / PHASENOISE_KRYLOV_ITR.

Specifying Variant Indices

.SNNOISE is the appropriate PrimeSim HSPICE advanced analog analysis for the computation of noise at the output of a sample and hold circuit. When using .SNNOISE, you need to specify the indices as [0,1] instead of the default [1,-1]. When you specify the indices as [0,1], you get results that are “what you see is what you get” with respect to the frequency sweep specified in the .SNNOISE command. There are two more important things to consider when using .SNNOISE: you must use enough harmonics to resolve the clock edge and you will want a high density of SN time points. It is recommended that the number of time points be between 2 and 20 times the number of harmonics used. To increase the density of the time points during the SN analysis, you can use the option DELMAX to specify a maximum time step.

For example, you can use the following:

```
.OPTION DELMAX=5p
.SN TONES=5e6 nharms=2000 trinit=800n
.SNNOISE v(out) rrf lin 10 0.6g 1.6g [1,-1]
```

This example uses 100 points per decade for the frequency sweep, instead of 1000 points per decade. This is to speed up the simulation. It does not affect the accuracy of the results.

Output Syntax

This section describes the syntax for the SNNOISE .PRINT and .PROBE statements.

.PRINT and .PROBE Statements

```
.PRINT SNNOISE [ONoise] [NF] [SSNF] [DSNF] [INoise]
.PROBE SNNOISE [ONoise] [NF] [SSNF] [DSNF] [INoise]
```

Parameter	Description
ONoise	Outputs the voltage noise at the output frequency band across the output nodes in the .SNNOISE statement. The data is plotted as a function of the input frequency band points. Units are in $V/Hz^{1/2}$. Simulation ignores ONoise when applied to autonomous circuits.
NF SSNF	NF and SSNF both output a single-side band noise figure as a function of the input frequency band points: $NF = SSNF = 10 \text{ Log}(SSF)$ Single side-band noise factor, $SSF = \{(\text{Total Noise at output, at the output frequency band, originating from all frequencies}) - (\text{Load Noise originating from the output frequency band})\} / (\text{Input Source Noise originating from the input frequency band})$.
DSNF	DSNF outputs a double side-band noise figure as a function of the input frequency band points. $DSNF = 10 \text{ Log}(DSF)$ Double side-band noise factor, $DSF = \{(\text{Total Noise at output, at the output frequency band, originating from all frequencies}) - (\text{Load Noise originating from the output frequency band})\} / (\text{Input Source Noise originating from the input frequency band and from the image of input frequency band})$.
INOISE	Outputs input referred noise which can be printed, probed, or measured.

Output Data Files

An SNNOISE analysis produces these output data files:

- Output from the `.PRINT` statement is written to a `.printsnpn#` file.
- Output from the `.PROBE` statement is written to a `.snpn#` file.

Both the `*.printsnpn#` and `*.pnn#` files output data against the input frequency band points.

- When multiple output variables are specified, multiple `*.printsnpn#` and `*.snpn#` files are generated, each with one output variable name in its file name as in `*.printsnpn#@outvar` and `*.snpn#@outvar`.
- Standard output information is written to a `.lis` file:
 - simulation time
 - SNNOISE linear solver method
 - SNNOISE simulation time
 - total simulation time

See also [Using Noise Analysis Results as Input Noise Sources](#).

Measuring SNNOISE Analyses with .MEASURE

Note:

A `.MEASURE SNNOISE` statement cannot contain an expression that uses a `SNNOISE` variable as an argument. Also, you cannot use a `.MEASURE SNNOISE` statement for error measurement and expression evaluation of `SNNOISE`.

The `.MEASURE SNNOISE` syntax supports four types of measurements:

- Find-when

```
.MEASURE SNNOISE result FIND out_var1  
+ At = Input_Frequency_Band value
```

The previous measurement yields the result of a variable value at a specific input frequency band point.

```
.MEASURE SNNOISE result FIND out_var1  
+ WHEN out_var2 = out_var3
```

The previous measurement yields the result at the input frequency point when `out_var2 == out_var3`.

```
.MEASURE SNNOISE result WHEN out_var2 = out_var3
```

The previous measurement yields the input frequency point when `out_var2 == out_var3`.

- Average, RMS, min, max, and peak-to-peak

```
.MEASURE SNNOISE result [RMS] out_var [FROM = IFB1]  
+ [TO = IFB2]
```

- Integral evaluation

```
.MEASURE SNNOISE result INTEGRAL out_var  
+ [FROM = IFB1] [TO = IFB2]
```

This measurement integrates the `out_var` value from the IFB1 frequency to the IFB2 frequency.

- Derivative evaluation

```
.MEASURE SNNOISE result DERIVATIVE out_var AT = IFB1
```

This measurement finds the derivative of `out_var` at the IFB1 frequency point.

Note:

.MEASURE SNNOISE cannot contain an expression that uses an *hbnoise* variable as an argument. You also cannot use .MEASURE SNNOISE for error measurement and expression evaluation of SNNOISE.

- Input referred noise:

```
.MEASURE SNNOISE result FIND inoise  
+ AT = IFB_value
```

This measurement yields the result of the input referred noise at a specific input frequency band point.

```
.MEASURE SNNOISE result FIND inoise  
+ WHEN out_var2 = out_var3
```

This measurement yields the result at the input frequency point when out_var2 == out_var3.

```
.MEASURE SNNOISE result func inoise [FROM = IFB1]  
+ [TO = IFB2]
```

Where *func* is one of the following measurement types:

- AVG (average): Calculates the area under the *inoise* curve, divided by the periods of interest.
- MAX (maximum): Reports the maximum value of *inoise* over the specified interval.
- MIN (minimum): Reports the minimum value of *inoise* over the specified interval.
- PP (peak-to-peak): Reports the maximum value minus the minimum value of *inoise* over the specified interval.
- RMS (root mean squared): Calculates the square root of the area under the *inoise* curve, divided by the period of interest.

```
.MEASURE SNNOISE result INTEGRAL inoise  
+ [FROM = IFB1] [TO = IFB2]
```

This measurement integrates the *inoise* value from the IFB1 frequency to the IFB2 frequency.

```
.MEASURE SNNOISE result DERIVATIVE inoise AT = IFB1
```

This measurement finds the derivative of *inoise* at the IFB1 frequency point.

Multitone Harmonic Balance Transfer Function Analysis (.HBXF)

The `.HBXF` command calculates the transfer function from a given source in the circuit to a designated output. Frequency conversion is calculated from the input frequencies to a single output frequency that is specified with the command. The relationship between the `.HBXF` command and the input/output is expressed in the following equation:

Equation 36

$$Y_m(j\omega_0) = \sum_{\omega \in W} HBXF_{m,n}(j\omega_0, j(\omega + \Delta\omega)) \cdot X_n(j(\omega + \Delta\omega))$$

where:

- $HBXF_{m,n}(j\omega_0, j(\omega + \Delta\omega))$ is the transfer function from input port n to the output port m
- W is the set of all possible harmonics
- $\omega + \Delta\omega$ is the input frequency
- $\Delta\omega$ is the offset frequency
- m is the output node number
- n is the input node number
- ω_0 is the output frequency
- Y is the output (voltage or current)
- X is the input (voltage or current)

The following sections discuss these topics:

- [Supported Features](#)
- [Example](#)
- [HBXF Test Listing](#)
- [Input Syntax](#)
- [Output Syntax](#)
- [Output Data Files](#)
- [Using the .MEASURE Command with .HBXF](#)

Supported Features

The `.HBXF` command supports the following features:

- All existing PrimeSim HSPICE advanced analog models and elements
- Sweep parameter analysis
- Unlimited number of HB sources

Prerequisites and Limitations

The following prerequisites and limitations apply to the `.HBXF` command:

- Only one `.HBXF` statement is required. If you use multiple `.HBXF` statements, PrimeSim HSPICE advanced analog analyses use only the last `.HBXF` statement.
- At least one `.HB` statement is required, which determines the steady-state solution.
- Parameter sweeps must be placed in the `.HB` statements.

Example

Based on the HB analysis, the following example computes the trans-impedance from `isrc` to `v(1)`.

```
.hb tones=1e9 nharms=4
.hbxf v(1) lin 10 1e8 1.2e8
.print hbxf tfv(isrc) tfi(n3)
```

HBXF Test Listing

```
* Test HBXF: nonlinear order-2 poly equation
.OPTIONS PROBE
.OPTIONS POST=2
vlo lo 0 cos(0 1.0 1g 0 0) tranforhb=1
rlo lo 0 50
vrfl rfl 0 0
rrfl rfl 0 50
El out 0 POLY(2) lo 0 rfl 0 0 1 1 1 10 1
rout out 0 50
.hb tones=1g nharms=5
.hbxf v(out) lin 2 100meg 200meg
.print hb v(out) v(rfl) v(lo)
.print hbxf tfv(vrfl) tfv(vlo)
.end
```

Input Syntax

```
.HBXF out_var freq_sweep  
+[n1, +|-1]
```

Parameter	Description
out_var	Specify $i(2_port_elem)$ or $v(n1[, n2])$
freq_sweep	Frequency sweep range for the input signal (also referred to as the input frequency band (<i>fin</i>)). A sweep of type LIN, DEC, OCT, POI, or SWEEPBLOCK. Specify the <i>nsteps</i>, start, and stop frequencies using the following syntax for each type of sweep: - LIN <i>nsteps start stop</i> - DEC <i>nsteps start stop</i> - OCT <i>nsteps start stop</i> - POI <i>nsteps freq_values</i> - SWEEPBLOCK=<i>swblockname</i> Specify the frequency sweep range for the output signal. PrimeSim HSPICE advanced analog analyses determines the offset frequency in the input sidebands; for example, $f1 = \text{abs}(f_{out} - k \cdot f_0)$ s.t. $f1 \leq f_0/2$. The f_0 is the steady-state fundamental tone, and $f1$ is the input frequency.
n1, + -1	Optional argument. Specifies the index term defining the output frequency band to be evaluated. The output frequency is computed according to $f_{out} = n1 \cdot f1 \pm fin$, where $f1$ is the fundamental tone (inverse of the fundamental period) and fin is from the frequency sweep. Default is 1, +1. If <i>n1, + -1</i> is omitted, the evaluation is performed in the frequency range given by <i>freq_sweep</i>.

Example:

```
.HBXF v(1) DEC 10K 100MEG [1,+1]
```

Output Syntax

This section describes the syntax for the HBXF .PRINT and .PROBE statements.

.PRINT and .PROBE Statements

```
.PRINT HBXF TYPE (NODES | ELEM) [INDICES]  
.PROBE HBXF TYPE (NODES | ELEM) [INDICES]
```

Parameter	Description
TYPE	TYPE can be one of the following: - TFV = existing source - TFI = placeholder value for the current source attached to the given node. The transfer function is computed on the output variables and input current or voltage.
NODES ELEM	NODES or ELEM can be one of the following: - Voltage type – a single node name (n1), or a pair of node names (n1,n2) - Current type – an element name (elemname) - Power type – a resistor (resistorname) or port element name (portname).
[INDICES]	Specify the index notation for the output. Example: <code>.probe HBXF tfv(vdd) [0,1]</code>

Output Data Files

An HBXF calculation produces these output data files:

- Output from the `.PRINT` statement is written to a `.printxf#` file.
 - The output is in ohms, siemens, or undesignated units, and the header in the output file is `Z(..)`, `Y(..)` or `GAIN(..)`.
- Output from the `.PROBE` statement is written to an `.xf#` file.
- Reported performance log statistics are written to a `.lis` file:
 - HBXF CPU time
 - HBXF peak memory usage

Using the .MEASURE Command with .HBXF

Since `.HBXF` requires an `.HB` analysis, the measure statements for this analysis are the same as for `.HB` analysis. For example,

```
.MEASURE HB result FIND out_var AT=val
```

Shooting Newton Transfer Function Analysis (.SNXF)

The `.SNXF` command calculates transfer functions from an arbitrary number of small signal sources to a designated output in a circuit under periodic steady-state conditions.

Frequency conversion is calculated from multiple input frequencies to a single output at a single frequency that is specified on the command line.

The following sections discuss these topics:

- [Prerequisites and Limitations](#)
- [Example](#)
- [SNXF Test Listing](#)
- [Input Syntax](#)
- [Output Syntax](#)
- [Output Data Files](#)
- [Using the .MEASURE Command with .SNXF](#)

Prerequisites and Limitations

The following prerequisites and limitations apply to the .SNXF command:

- Only one .SNXF statement is required. If you use multiple .SNXF statements, PrimeSim HSPICE advanced analog analyses uses only the last one.
- At least one .SN statement is required, which determines the steady-state solution.
- Parameter sweeps must be placed in the .SN statements.

Example

Based on the SN analysis, the following example computes the transimpedance from `isrc` to `v(1)`.

```
.SN tones=1e9 nharms=4
.SNXF v(1) lin 10 1e8 1.2e8
print SNXF TFV(isrc) TFI(n3)
```

SNXF Test Listing

```
* Test SNXF: nonlinear order-2 poly equation
.OPTIONS PROBE
.OPTIONS POST=2
vlo lo 0 cos(0 1.0 1g 0 0)
rlo lo 0 50
vrfl rfl 0 0
rrfl rfl 0 50
El out 0 POLY(2) lo 0 rfl 0 0 1 1 1 10 1
```

```
rout out 0 50
.opt delmax=.01n
.sn tones=lg nharms=5
.snxf v(out) lin 2 100meg 200meg
.print sn v(out) v(rf1) v(lo)
.print snxf tfv(vrf1) tfv(vlo)
.end
```

Input Syntax

```
.SNXF out_var freq_sweep
+[n1, +|-1]
```

Parameter Description

Parameter	Description
out_var	Specify $i(2_port_elem)$ or $v(n1[,n2])$
freq_sweep	Frequency sweep range for the input signal (also referred to as the input frequency band (fin)). A sweep of type LIN, DEC, OCT, POI, or SWEEPBLOCK. Specify the nsteps, start, and stop frequencies using the following syntax for each type of sweep: - LIN <i>nsteps start stop</i> - DEC <i>nsteps start stop</i> - OCT <i>nsteps start stop</i> - POI <i>nsteps freq_values</i> - SWEEPBLOCK=<i>swblockname</i> Specify the frequency sweep range for the output signal. PrimeSim HSPICE advanced analog analyses determines the offset frequency in the input sidebands Fin, where $Fin = \text{abs}(n \cdot F0 \pm Fout)$. F0 is the steady-state fundamental tone, and Fout is the output frequency. SNXF then generates the transfer functions from all of the input sidebands (the Fin values) to the output frequency Fout.
n1, + -1	Optional argument. Specifies the index term defining the output frequency band to be evaluated. The output frequency is computed according to $fout = n1 \cdot f1 \pm fin$, where f1 is the fundamental tone (inverse of the fundamental period) and fin is from the frequency sweep. Default is 1, +1. If n1, + -1 is omitted, the evaluation is performed in the frequency range given by <i>freq_sweep</i>.

Example:

```
.SNXF v(1) DEC 10K 100MEG [1,+1]
```

Output Syntax

This section describes the syntax for the SNXF `.PRINT` and `.PROBE` statements.

.PRINT and .PROBE Statements

```
.PRINT SNXF TYPE(NODES | ELEM) [INDICES]
.PROBE SNXF TYPE(NODES | ELEM) [INDICES]
```

Parameter Description

`TYPE` can be one of the following:

- `TFV` = existing source
- `TFI` = placeholder value for the current source attached to the given node.

The transfer function is computed on the output variables and input current or voltage

`NODES | ELEM`.

`NODES` or `ELEM` can be one of the following:

- Voltage type – a single node name (`n1`), or a pair of node names, (`n1,n2`)
- Current type – an element name (`elemname`)
- Power type – a resistor (`resistorname`) or port (`portname`) element name

`[INDICES]` specify the index notation for the output.

Example:

```
.probe SNXF tfv(vdd) [0,1]
```

Output Data Files

An SNXF calculation produces these output data files:

- Output from the `.PRINT` statement is written to a `.printsnxf#` file. The output is in ohms, siemens, or undesignated units, and the header in the output file is `Z(..)`, `Y(..)` or `GAIN(..)`.
- Output from the `.PROBE` statement is written to an `.snxf#` file.

Reported performance log statistics are written to a `.lis` file:

- SNXF CPU time
- SNXF peak memory usage

Using the .MEASURE Command with .SNXF

Since .SNXF requires an .SN analysis, the measure statements for this analysis are the same as for .SN analysis. For example,

```
.MEASURE SN result FIND out_var AT=val
```

HB- and SN-Based Stability Analysis

Negative feedback techniques are widely used in analog and RF design to improve circuit properties such as variation tolerance, bandwidth, impedance matching, and output waveform distortion. Stability is always a serious concern for feedback circuits. Self-oscillations have been found above and/or beyond the bandwidth in diversified circuits such as optical fiber system receivers, power amplifiers, and distributed microwave amplifiers. It is critical for designers to evaluate stability and stability margin of a feedback circuit. Such information can be used for optimization in the early design stage and for diagnostic purposes in the physical realization stage.

PrimeSim HSPICE .LSTB analysis is used to study the stability margin of analog circuits without large signal presence. It is based on a linearized circuit at a given DC operating point. When the input signal is small, the amplifier is largely linear and .LSTB results are accurate enough. However, when the input signal becomes large, the amplifier becomes more nonlinear. This will change the loop gain of the feedback system. PrimeSim HSPICE now provides .HBSTB and .SNSTB analyses designed for the purpose of analyzing small-signal loop gain for circuits with negative feedback loops, in the presence of a large signal, such as an amplifier.

- [Algorithm Description](#)
- [.HBSTB and .SNSTB](#)

Algorithm Description

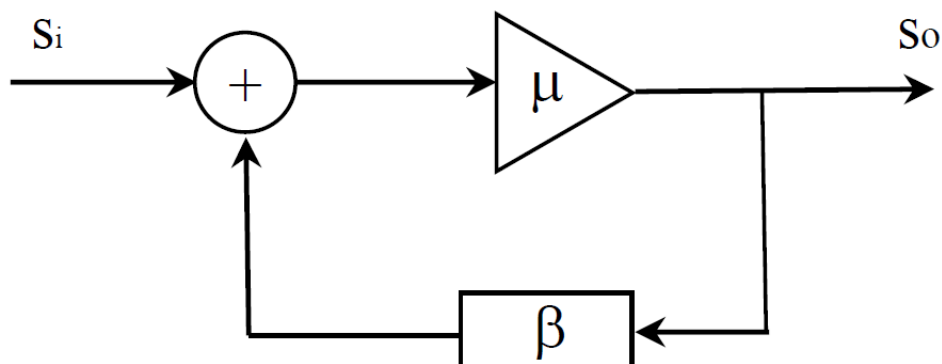
Figure 19 shows an ideal single loop feedback network. The closed-loop transfer function is given by:

$$A = \frac{\mu}{1 - \mu\beta}$$

Where $1 - \mu\beta$ is referred to as the feedback factor.

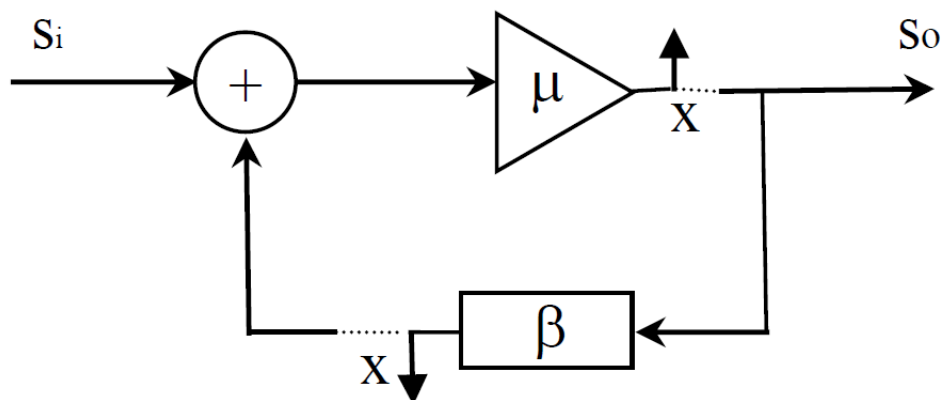
Under the condition $1 - \mu\beta = 0$, a feedback network will function like an oscillator. The frequency of a linearized oscillator is determined by the condition that the phase shift of $\mu\beta$ is zero provided that the magnitude of $\mu\beta$ equals unity. $\mu\beta$ is referred to as loop gain and is also defined as the return ratio.

Figure 19 The ideal single-loop feedback network



To calculate the loop gain of a feedback network, according to [9], we can break the feedback loop at any location on either the active path or the feedback path, as long as the bias (DC or steady state) impedance characteristics are not disturbed (Figure 20).

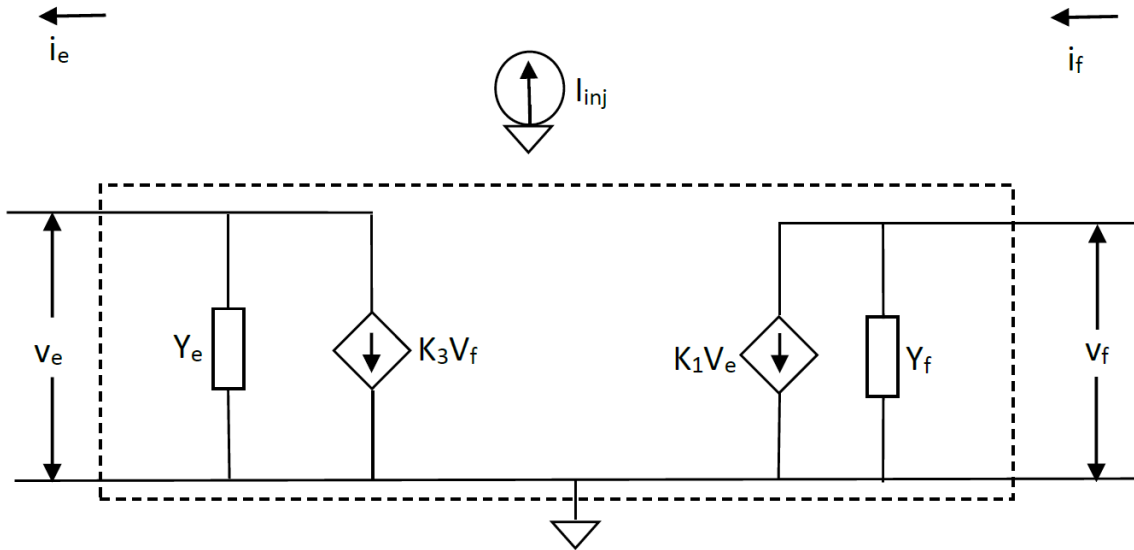
Figure 20 Break the loop at any location



In low frequency applications, signals flow through the feedback loop unilaterally. But for most practical designs, the active path may not be strictly unilateral, and the feedback path is usually bilateral. Thus, both the normal and the reverse loop transmission exist around the feedback loop. To accurately evaluate the stability of a feedback network, both normal and reverse loop transmission should be considered. Therefore, the double injection technique introduced in [9] based on bilateral return loop method, to calculate the loop gain of a big signal driven system with feedback loops, is used.

The calculation is processed by successive injection [9] (Middlebrook's Technique). A voltage source V_{inj} and a current source I_{inj} are inserted as shown in Figure 21.

Figure 21 Double-injection technique based on the bilateral return loop model



The V_{inj} source is placed in series in the loop: one pin of the voltage loop must be connected to the loop input, the other pin to the loop output. The orientation of inserted voltage sources in differential and common-mode testing is significant. It is required that the positive terminal of both voltage sources go to the input of the amplifier or to the output of the amplifier.

The I_{inj} source is inserted with one node connected to the loop, while the other end is connected to GND. In Figure 21, The part in the dashed line box is the Norton and Thevenin equivalents of the feedback loop.

At the normal circuit operation, both I_{inj} and V_{inj} are set to zero. Thus the loop properties are undisturbed. When calculating feedback loop gain, let I_{inj} and V_{inj} be the driving (input) signals and i_f and v_e be the dependent (output) signals. Since the feedback network is linear, the only constraint to be placed upon each dependent signal, by superposition, is the linear sum of the two values it would have due to each input signal alone. Thus,

$$\begin{bmatrix} i_f \\ v_e \end{bmatrix} = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} I_{inj} \\ V_{inj} \end{bmatrix}$$

Where:

$$B = i_f: I_{inj}=0, V_{inj}=1$$

$$D = v_e: I_{inj}=0, V_{inj}=1$$

$$A = i_f: I_{inj}=1, V_{inj}=0$$

$$C = v_e: I_{inj}=1, V_{inj}=0$$

The loop gain is then given as [9]:

Equation 37 Loop gain

$$T = \frac{2(AD-BC) - A + D}{2(BC-AD) + A - D - 1}$$

Stability analysis is a small signal analysis based on steady state analysis. Therefore, it is carried out after HB or SN analysis is completed.

When doing HB or SN analysis, the injection V_{inj} and I_{inj} are zero. Once the HB or SN is successfully done, all the stimulus in the circuits are turned off, and V_{inj} and I_{inj} are injected respectively. As mentioned previously, when $V_{inj}=1V$, $I_{inj}=0$, B and D are calculated; while when $V_{inj}=0$, $I_{inj}=1A$, A and C are calculated. The return ratio (loop gain) is calculated using A, B, C, and D according to Equation 37.

.HBSTB and .SNSTB

PrimeSim HSPICE provides two commands namely .HBSTB for HB-based loop stability analysis and .SNSTB for SN-based loop stability analysis.

See the *PrimeSim HSPICE® Reference Manual: Commands and Control Options* for a detailed description of these commands and their outputs.

Since the probe can only be inserted into one feedback loop, when there are multiple feedback loops in the circuit, you have to insert multiple probes, to calculate the loop gain of each feedback loop. Or, if the feedback loops share one particular path, the probe can be inserted into this particular path, to break all the feedback loops, and calculate the loop gain of the entire feedback system. When multiple probes are inserted, multiple .HBSTB/.SNSTB commands should be specified, each with its own name. PrimeSim HSPICE performs the loop gain calculations one by one.

The outputs are in the .hbcx# and .hbstb# files for .HBSTB, and .sncx# and .snstb# files for .SNSTB. The dB, magnitude, phase, and the real and imaginary parts of the loop gain parameters are printed.

Note:

When the HB/SN tone values are small, the .HBSTB/.SNSTB results are close to .LSTB results.

References

- [1] S. Maas, *Nonlinear Microwave Circuits*, Chapter 3, IEEE Press, 1997.
- [2] R. Gilmore and M.B. Steer, "Nonlinear Circuit Analysis Using the Method of Harmonic Balance - A Review of the Art, Part I, Introductory Concepts." *International Journal of*

- Microwave and Millimeter-wave Computer-Aided Engineering*, Volume 1, No. 1, pages 22-37, 1991.
- [3] R. Gilmore and M.B. Steer, "Nonlinear Circuit Analysis Using the Method of Harmonic Balance - A Review of the Art. Part II. Advanced Concepts." *International Journal of Microwave and Millimeter-wave Computer-Aided Engineering*, Volume 1, No. 2, pages 159-180, 1991.
 - [4] V. Rizzoli, F. Mastri, F. Sgallari, G. Spaletta, "Harmonic-Balance Simulation of Strongly Nonlinear Very Large-Size Microwave Circuits by Inexact Newton Methods," *MTT-S Digest*, pages 1357-1360, 1996.
 - [5] S. Skaggs, *Efficient Harmonic Balance Modeling of Large Microwave Circuits*, Ph.D. thesis, North Carolina State University, 1999.
 - [6] R.S. Carson, *High-Frequency Amplifiers*, 2nd Edition, John Wiley & Sons, 1982
 - [7] S.Y. Liao, *Microwave Circuit Analysis and Amplifier Design*, Prentice-Hall, 1987.
 - [8] Y. Saad, *Iterative Methods for Sparse Linear Systems*, PWS Publishing Company, 1995.
 - [9] Michael Tian V. Visvanathan, Jeffrey Hantgan, and Kenneth Kundert, "Striving for Small-signal Stability: Loop-Based and Device-Based Algorithms for Stability Analysis of Linear Analog Circuits in the Frequency Domain", *Circuits & Devices*, January 2001.

9

Large Signal S-parameter Analyses

Describes how to do frequency translation and large-signal S-parameter extraction, as well as noise parameter calculation.

This chapter discusses various techniques supported in PrimeSim HSPICE advanced analog analyses for extracting circuit scattering parameters. Since advanced analog circuits can operate under large-signal and small-signal conditions, there are several types of scattering parameters that are useful to measure.

Linear small-signal scattering parameters represent the RF frequency-domain transfer characteristics for a circuit that is operating at its DC bias condition, but the stimulus and response signals are sufficiently small that they do not influence the operating point. This type of analysis is performed using the `.LIN` analysis. For information on doing small-signal S-parameter analysis (`.LIN`), see Linear Network Parameter Analysis in the *PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis*.

In the case of RF mixers and receiver front-ends, some of the input and output frequencies of interest involve a frequency translation. This translation is intentional and caused by nonlinear mixing in the circuit due to devices being driven by large-signal periodic waveforms. This type of scattering parameter analysis therefore must begin by solving the large-signal periodic response and then finding the small-signal behavior about this large-signal operating point. This capability is provided by the `.HBLIN` analysis which has setup and analysis control options similar to `.LIN`, but is capable of extracting S-parameters about a large-signal periodic steady-state operating point.

In the case of circuits such as power amplifiers, the extraction of scattering parameters is also important, but the circuit stimulus and response signals may themselves be large-signal periodic waveforms. And, it can be important to analyze how these S-parameters vary as a function of input power levels. This capability is provided by the `.HBLSP` large-signal S-parameter analysis, which uses large-signal stimulus for the S-parameter extractions.

The following topics are presented in this chapter:

- [Frequency Translation S-Parameter \(HBLIN\) Extraction](#)
- [Large-Signal S-parameter \(HBLSP\) Analysis](#)

Frequency Translation S-Parameter (HBLIN) Extraction

Frequency translation scattering parameter (S-parameter) extraction is used to describe N-port circuits that exhibit frequency translation effects, such as mixers. The analysis is similar to the existing LIN analysis, except that the circuit is first linearized about a periodically-varying operating point instead of a simple DC operating point. After the linearization, the S-parameters between circuit ports that convert signals from one frequency band to another are calculated.

You use the `.HBLIN` statement to extract frequency translation S-parameters and noise figures.

Frequency translation S-parameter describes the capability of periodically linear time varying systems to shift signals in frequency. The S-parameters for a frequency translation system are similar to the S-parameters of a linear-time-varying system. They are defined as:

Equation 38

$$b = S \cdot a$$

$$S_{i,j;m,n}(\omega) = \frac{b_{i,m}(\omega)}{a_{j,n}(\omega)}$$

$$a_{k \neq j, p \neq n}(\omega) = 0$$

The incident waves, $a_{i,n}(\omega)$ and the reflected waves, $b_{i,n}(\omega)$, are defined using these equations:

Equation 39

$$a_{i,n}(\omega) = \frac{V_i(\omega+n\omega_0) + Z_{0i}I_i(\omega+n\omega_0)}{2\sqrt{Z_{0i}}}$$

$$b_{i,n}(\omega) = \frac{V_i(\omega+n\omega_0) - Z_{0i}I_i(\omega+n\omega_0)}{2\sqrt{Z_{0i}}}$$

where:

- ω_0 is the fundamental frequency (tone).
- n is a signed integer.
- i is the port number.
- $a_{i,n}(\omega)$ is the input wave at the frequency $\omega + n\omega_0$ on the i^{th} port.
- $b_{i,n}(\omega)$ is the reflected wave at the frequency $\omega + n\omega_0$ on the i^{th} port.
- $V_i(\omega + n\omega_0)$ is the Fourier coefficient at the frequency $\omega + n\omega_0$ of the voltage at port i .

- $I_i(\omega + n\omega_0)$ is the Fourier coefficient at the frequency $\omega + n\omega_0$ of the current at port i.
- Z_{oi} is the reference impedance at port i.
- V and I definitions are Fourier coefficients rather than phasors.

For a multi-tone analysis, they can be expressed as:

Equation 40

$$b = S \cdot a$$

$$S_{i,j;m_1..m_N,n_1,n_2..n_N}(\omega) = \frac{b_{i,m_1,m_2..m_N}(\omega)}{a_{j,n_1,n_2..n_N}(\omega)}$$

$$a_{k,p_1,p_2..p_N} |_{k \neq j, \forall p_q \neq n_q}(\omega) = 0$$

Equation 41

$$a_{i,n_1,n_2..n_N}(\omega) = \frac{V_i(\omega + \sum_{j=1}^N n_j \omega_j) + Z_{oi} I_i(\omega + \sum_{j=1}^N n_j \omega_j)}{2\sqrt{Z_{oi}}}$$

$$b_{i,n_1,n_2..n_N}(\omega) = \frac{V_i(\omega + \sum_{j=1}^N n_j \omega_j) - Z_{oi} I_i(\omega + \sum_{j=1}^N n_j \omega_j)}{2\sqrt{Z_{oi}}}$$

where:

- ω_j is the i^{th} tone.

The frequency translation S-parameters are calculated by applying different n_j ($j = 1 \sim N$) to different ports.

Limitations

The HBLIN analysis has these known limitations:

- Noise parameters are not calculated for a mixed-mode operation.
- Only the S-parameters corresponding to the set of frequencies specified at each port are extracted.
- Multiple small-signal tones are not supported.
- The port element impedance cannot be specified as a complex number.

The following sections discuss these topics:

- [HB Analysis](#)
- [Port Element](#)

-
- [Output Syntax](#)
- [Output Data Files](#)

HB Analysis

An HB analysis is required prior to an HBLIN analysis. To extract the frequency translation S-parameters, a sweep of the small-signal tone is necessary. You can identify the small-signal tone sweep in the `.HBLIN` command or in the `.HB` command together with a `SS_TONE` specification.

For additional information regarding HB analysis, see [Harmonic Balance Analysis on page 42](#).

Port Element

You must use a port (P) element as the termination at each port of the system. To indicate the frequency band that the S-parameters are extracted from, it is necessary to specify a harmonic index for each P-element.

Port Element Syntax

Without `SS_TONE`

```
Pxxx p n n_ref PORT=portnumber  
+ [HBLIN = [H1, H2, ... HN, +/-1]] ...
```

With `SS_TONE`

```
Pxxx p n n_ref [PORT=portnumber]  
+ [HBLIN = [H1, H2, ... +/-1 ... HN]] ...
```

Parameter	Description
n_ref	Reference node used when a mixed-mode port is specified.
PORT	The port number. Numbered sequentially beginning with 1 with no shared port numbers.
HBLIN	Integer vector that specifies the harmonic index corresponding to the tones defined in the <code>.HB</code> command. The <code>+/-1</code> term corresponds to the small-signal tone specified by <code>SS_TONE</code> in the <code>.HB</code> command. If there is no <code>SS_TONE</code> in the <code>.HB</code> command, the <code>+/-1</code> term must be at the last entry of the <code>HBLIN</code> vector.

HBLIN Analysis

You use the `.HBLIN` statement to extract frequency translation S-parameters and noise figures.

- [Input Syntax](#)
- [HBLIN Analysis](#)

Input Syntax

Without `SS_TONE`

```
.HBLIN frequency_sweep
+ [NOISECALC = [1|0|yes|no]] [FILENAME=file_name]
+ [DATAFORMAT = [ri|ma|db]]
+ [MIXEDMODE2PORT = [dd|cc|cd|dc|sd|sc|cs|ds]]
```

With `SS_TONE`

```
.HBLIN [NOISECALC = [1|0|yes|no]] [FILENAME=file_name]
+ [DATAFORMAT = [ri|ma|db]]
+ [MIXEDMODE2PORT = [dd|cc|cd|dc|sd|sc|cs|ds]]
```

Parameter	Description
<i>frequency_sweep</i>	Frequency sweep range for the input signal (also referred to as the input frequency band or fin). You can specify LIN, DEC, OCT, POI, or SWEEPBLOCK. Specify the nsteps, start, and stop frequencies using the following syntax for each type of sweep: • LIN <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • POI <i>nsteps freq_values</i> • SWEEPBLOCK=<i>swblockname</i> • DATA=<i>dataname</i>
NOISECALC	Enables calculating the noise figure. The default is no (0).
FILENAME	Specifies the output file name for the extracted S-parameters or the object name after the -o command-line option. The default is the netlist file name.
DATAFORMAT	Specifies the format of the output data file. • dataformat=RI, real-imaginary. • dataformat=MA, magnitude-phase. This is the default format for Touchstone files. • dataformat=DB, DB(magnitude)-phase.

Parameter	Description
MIXEDMODE2PORT	Describes the mixed-mode data map of output mixed mode S-parameter matrix. The availability and default value for this keyword depends on the first two port (P element) configuration as follows: - case 1: p1=p2=single-ended (standard-mode P element) available: ss default: ss - case 2: p1=p2=balanced (mixed-mode P element) available: dd, cd, dc, cc default: dd - case 3: p1=balanced p2=single-ended available: ds, cs default: ds - case 4: p1=single p2=balanced available: sd, sc default: sd

Examples

Example 1

Single-tone analysis with frequency translation example. In this example, the 2-port S-parameters from RF (1G-del_f) to IF (del_f) are extracted. The LO signal is specified by normal voltage source Vlo. The frequency on port 1 is in the RF band, 1G-del_f, and the frequency on port 2 is in the IF band, del_f. The IF band is swept from 0- to 100-MHz. The results are output to the file ex1.s2p.

```
p1 RFin gnd port=1 HBLIN=(1,-1)
p2 IFout gnd port=2 HBLIN=(0,1)
Vlo LOin gnd DC 0 HB 2.5 0 1 1
.HB tones=1G harms=5
.HBLIN lin 5 0 100meg noisecalc=no filename=ex1
+ dataformat=ma
```

Example 2

Another single-tone analysis with frequency translation example. In this example, the 3-port S-parameters are extracted. Port 3 provides the periodic large signal. The frequency on port 1 is del_f, the frequency on port 2 is 1G*2-del_f, and the frequency on port 3 is 1G*1+del_f. The small-signal frequency is swept from 0 to 100MHz. HBNOISE calculation is required. The results are output to the file ex2.s3p.

```
p1 1 0 port=1 HBLIN=(0, 1)
p2 2 0 port=2 HBLIN=(2, -1)
p3 3 0 port=3 hb 0.5 0 1 1 HBLIN=(1, 1)
.HB tones=1G harms=5
.HBLIN lin 5 0 100meg noisecalc=yes filename=ex2
```

Output Syntax

This section describes the syntax for the HBLIN .PRINT and .PROBE statements.

.PRINT and .PROBE Statements

```
.PRINT HBLIN Smn | Smn(TYPE) | S(m,n) | S(m,n)(TYPE)
.PROBE HBLIN Smn | Smn(TYPE) | S(m,n) | S(m,n)(TYPE)
.PRINT HBLIN SXYmn | SXYmn(TYPE) | SXY(m,n) | SXY(m,n)(TYPE)
.PROBE HBLIN SXYmn | SXYmn(TYPE) | SXY(m,n) | SXY(m,n)(TYPE)
.PRINT HBLIN NF SSNF DSNF
.PROBE HBLIN NF SSNF DSNF
```

Parameter	Description
Smn Smn(TYPE) S(m,n) S(m,n)(TYPE) SXYmn SXYmn(TYPE) SXY(m,n) SXY(m,n)(TYPE)	Complex 2-port parameters. Where: • m = 1 or 2 • n = 1 or 2 • X and Y are used for mixed-mode S-parameter output. • The values for X and Y can be D (differential), C (common), or S (single-end). TYPE = R, I, M, P, PD, D, DB, or DBM • R = real • I = imaginary • M = magnitude • P = PD = phase in degrees • D = DB = decibels • DBM = decibels per 1.0e-3
NF SSNF	NF and SSNF both output a single side-band noise figure as a function of the input frequency band points: $NF = SSNF = 10 \log(SSF)$ Single side-band noise factor, $SSF = \frac{\text{(Total Noise at output, at the output frequency band, originating from all frequencies)} - \text{(Load Noise originating from the output frequency band)}}{\text{(Input Source Noise originating from input frequency band)}}$.
DSNF	DSNF outputs a double side-band noise figure as a function of the input frequency band points. $DSNF = 10 \log(DSF)$ Double side-band noise factor, $DSF = \frac{\text{(Total Noise at output, at the output frequency band, originating from all frequencies)} - \text{(Load Noise originating from the output frequency band)}}{\text{(Input Source Noise originating from the input frequency band and from the image of input frequency band)}}$.

Output Data Files

An HBLIN analysis produces these output data files:

- The S-parameters from the .PRINT statement are written to a .printhl# file.
- The extracted S-parameters from the .PROBE statement are written to a .hl# file.

Large-Signal S-parameter (HBLSP) Analysis

Use the `.HPSLP` command to invoke periodically-driven nonlinear circuit analyses for power-dependent S-parameters. An HBLSP analysis provides three kinds of analyses for periodically-driven nonlinear circuits, such as those that employ power amplifiers and filters:

- Two-port power-dependant (large-signal) S-parameter extraction
- Two-port small-signal S-parameter extraction
- Two-port small-signal noise parameter calculation

Unlike small-signal S-parameters, which are based on linear analysis, power-dependent S-parameters are based on the harmonic balance simulation. Its solution accounts for nonlinear effects such as compression and variation in the power levels.

The definition for power-dependent S-parameters is similar to that for small-signal parameters. Power-dependent S-parameters are defined as the ratio of reflected and incident waves by using this equation:

$$b = S * a ; S[i, j] = b[i, n] / a[j, n] \text{ when } a[k, n] (k \neq j) = 0$$

The incident waves, $a[i, n]$, and reflected waves, $b[i, n]$, are defined by using these equations:

$$a[i, n] = (V[i](n*W_0) + Z_o[i] * I[i](n*W_0)) / (2 * \text{sqrt}(Z_o[i]))$$

$$b[i, n] = (V[i](n*W_0) - Z_o[i] * I[i](n*W_0)) / (2 * \text{sqrt}(Z_o[i]))$$

Where:

- W_0 is the fundamental frequency (tone).
- n is a signed integer.
- i is the port number.
- $a[i, n]$ is the input wave at the frequency $n*W_0$ on the i^{th} port.
- $b[i, n]$ is the reflected wave at the frequency $n*W_0$ on the i^{th} port.
- $V[i](n*W_0)$ is the Fourier coefficient at the frequency $n*W_0$ of the voltage at port i .
- $I[i](n*W_0)$ is the Fourier coefficient at the frequency $n*W_0$ of the current at port i .
- $Z_o[i]$ is the reference impedance at port i .

An HBLSP analysis only extracts the S-parameters on the first harmonic (that is, $n=1$).

- [Input Syntax](#)
- [Examples](#)
- [Output Syntax](#)
- [Output Data Files](#)
- [HBLSP Analysis Limitations](#)

Input Syntax

```
.HBLSP NHARMS=nh [POWERUNIT=[dbm|watt]]  
+ [SSPCALC=[1|0|YES|NO]] [NOISECALC=[1|0|YES|NO]]  
+ [FILENAME=file_name] [DATAFORMAT=[ri|ma|db]]  
+ FREQSWEEP freq_sweep POWERSWEEP power_sweep
```

Parameter	Description
NHARMS	Number of harmonics in the HB analysis triggered by the .HBLSP statement.
POWERUNIT	Power unit. Default is watt.
SSPCALC	Extract small-signal S-parameters. Default is 0 (NO).
NOISECALC	Perform small-signal 2-port noise analysis. Default is 0 (NO).
FILENAME	Output data .p2d filename. Default is the netlist name or the object name after the -o command-line option.
DATAFORMAT	Format of the output data file. Default is ma (magnitude, angle).
FREQSWEEP	Frequency sweep specification. A sweep of type LIN, DEC, OCT, POI, or SWEEPBLOCK. Specify the <i>nsteps</i> , <i>start</i> , and <i>stop</i> times using the following syntax for each type of sweep: • LIN <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • POI <i>nsteps freq_values</i> • SWEEPBLOCK=<i>swblockname</i> This keyword must appear before the POWERSWEEP keyword.
POWERSWEEP	Power sweep specification. A sweep of type LIN, DEC, OCT, POI, or SWEEPBLOCK. Specify the <i>nsteps</i> , <i>start</i> , and <i>stop</i> frequencies using the following syntax for each type of sweep: • LIN <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • POI <i>nsteps power_values</i> • SWEEPBLOCK=<i>swblockname</i> This keyword must follow the FREQSWEEP keyword.

Note:

The `FREQSWEEP` and `POWERSWEEP` keywords must appear at the end of an `.HBLSP` statement.

Examples

The following example does a 2-port single-tone, power-dependent S-parameter extraction, without frequency translation:

- Frequency sweep: The fundamental tone is swept from 0 to 1G
- Power sweep: The power input at port 1 is swept from 6 to 10 Watts.
- Five harmonics are required for the HB analysis. Large-signal S-parameters are extracted on the first harmonic.
- Five harmonics are required in the HBLSP triggered HB analysis.
- The DC value in `p1` statement is used to set the DC bias, which is used to perform small-signal analyses.
- Small-signal S-parameters are required extracted.
- Small-signal two-port noise analysis is required.
- The data will be output to the `ex1.p2d` file.

Example 19 2-Port, Single Tone

```
p1 1 0 port=1 dc=1v
p2 2 0 port=2
.hblsp nharms=5 powerunit = watt
+ sspcalc=1 noisecalc=1 filename=ex1
+ freqsweep lin 5 0 1G powersweep lin 5 6 10
```

The following example generates large scale S-parameters as a function of input for a differential equalizer.

Example 20 4-Port Network

```
* hblsp example
.opt post
p1 n1 0 port=1 ac=1
p2 n2 0 port=2
*** put your DUT
R1 n1 n2 10***
.hblsp nharms=5
+ freqsweep lin 4 1k 10k
+ powersweep lin 2 5 10
.end
```

Output Syntax

This section describes the syntax for the `HBLSP .PRINT` and `.PROBE` statements. These statements only support S and noise parameter outputs. Node voltage, branch current, and all other parameters are not supported in `HBLSP .PRINT` and `.PROBE` statements.

.PRINT and .PROBE Statements

```
.PRINT HBLSP Smn | Smn(TYPE) | S(m, n) | S(m, n) (TYPE)
+ ...small signal 2-port noise params...
.PROBE HBLSP Smn | Smn(TYPE) | S(m, n) | S(m, n) (TYPE)
+ ...small signal 2-port noise params...
```

Parameter	Description
Smn Smn(TYPE) S(m,n) S(m,n)(TYPE)	Complex 2-port parameters. Where: • m = 1 or 2 • n = 1 or 2 • TYPE = R, I, M, P, PD, D, DB, or DBM - R = real I = imaginary M = magnitude - P = PD = phase in degrees - D = DB = decibels - DBM = decibels per 1.0e-3
... small signal 2-port noise parameters ...	G_AS NF RN YOPT GAMMA_OPT NFMIN VN2 ZCOR GN RHON YCOR ZOPT IN2For a description of these parameters, see Linear Network Parameter Analysis in the <i>PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis</i> .

Output Data Files

An HBLSP analysis produces these output data files:

- The large-signal S-parameters from the `.PRINT` statement are written to a `.printls#` file.
- The small-signal S-parameters from the `.PRINT` statement are written to a `.printss#` file.
- The large-signal S-parameters from the `.PROBE` statement are written to a `.ls#` file.
- The small-signal S-parameters from the `.PROBE` statement are written to a `.ss#` file.
- The extracted large- and small-signal S and noise parameters are written to a `.p2d` file.

The large-signal and small-signal S-parameters from the `.PROBE` statement are viewable in WaveView.

HBLSP Analysis Limitations

The HBLSP analysis has these known limitations:

- Power-dependent S-parameter extraction is a 2-port analysis only. Multiport power-dependent S-parameters are not currently supported.
- The intermodulation data block (`IMTDATA`) in the `.p2d` file is not supported.
- The internal impedance of the port element can only be a real value. Complex impedance values are not supported.

10

Envelope Analysis

Describes how to use envelope simulation.

The following topics are presented in this chapter:

- [Envelope Simulation](#)
- [Envelope Analysis Commands](#)
- [Output Syntax](#)
- [Envelope Output Data File Format](#)

Envelope Simulation

Envelope simulation combines the features of time- and frequency-domain analysis. Harmonic Balance (HB) solves for a static set of phasors for all the circuit state variables, as shown in this equation:

Equation 42

$$v(t) = a_0 + \sum_{i=1}^N [a_i \cos \omega_i t + b_i \sin \omega_i t]$$

In contrast, envelope analysis finds a dynamic, time-dependent set of phasors, as this equation shows:

Equation 43

$$v(t) = a_0(\hat{t}) + \sum_{i=1}^N [a_i(\hat{t}) \cos \omega_i t + b_i(\hat{t}) \sin \omega_i t]$$

In envelope simulation, each signal is described by the evolving spectrum. Envelope analysis is generally used on circuits excited by signals with significantly different timescales. An HB simulation is performed at each point in time of the slower-moving ($\hat{t}$) timescale. In this way, for example, a 2-tone HB simulation can be converted into a series of related 1-tone simulations where the transient analysis proceeds on the ($\hat{t}$)

timescale, and 1-tone HB simulations are performed with the higher frequency tone as the fundamental frequency.

In PrimeSim HSPICE advanced analog analyses, any voltage or current source identified as a HB source either in a V or I element statement, or by an `.OPTION TRANFORHB` command, is used for HB simulations at each point in $\hat{t}$ time. All other sources are associated with the transient timescale. Also, the input waveforms can be represented in the frequency domain as RF carriers modulated by an envelope by identifying a VMRF signal source in a V or I element statement. The amplitude and phase values of the sampled envelope are used as the input signal for HB analysis.

Some typical applications for envelope simulation are amplifier spectral regrowth, adjacent channel power ratio (ACPR), and oscillator startup and shutdown analyses.

Envelope Analysis Commands

This section describes those commands specific to envelope analysis. These commands are:

- Standard envelope simulation (`.ENV`)
- Oscillator simulation, both startup and shutdown (`.ENVOSC`)
- Envelope Fast Fourier Transform (`.ENVFFT`)

The following sections discuss these topics:

- [Nonautonomous Form](#)
- [Oscillator Analysis Form](#)
- [Fast Fourier Transform Form](#)

Nonautonomous Form

```
.ENV TONES=f1 [f2...fn] NHARMS=h1 [h2...hn]  
+ ENV_STEP=tstep ENV_STOP=tstop
```

Parameter	Description
TONES	Carrier frequencies, in hertz.
NHARMS	Number of harmonics.
ENV_STEP	Envelope step size, in seconds.
ENV_STOP	Envelope stop time, in seconds.

Description

You use the `.ENV` command to do standard envelope simulation. The simulation proceeds just as it does in a standard transient simulation, starting at `time=0` and continuing until `time=env_stop`. An HB analysis is performed at each step in time. You can use Backward-Euler (BE), trapezoidal (TRAP), or level-2 Gear (GEAR) integration.

Recommended option settings are:

- For BE integration, set `.OPTION SIM_ORDER=1`.
- For TRAP, set `.OPTION SIM_ORDER=2 (default) METHOD=TRAP (default)`.
- For GEAR, set `.OPTION SIM_ORDER=2 (default) METHOD=GEAR`.

Example

```
.env tones=1e9 nharms=6 env_step=10n env_stop=1u
```

Oscillator Analysis Form

```
.ENVOSC TONE=f1 NHARMS=h1 ENV_STEP=tstep ENV_STOP=tstop  
+ PROBENODE=n1,n2,vosc <FSPTS=num, min, max>
```

Parameter	Description
TONE	Carrier frequencies, in hertz.
NHARMS	Number of harmonics.
ENV_STEP	Envelope step size, in seconds.
ENV_STOP	Envelope stop time, in seconds.
PROBENODE	Defines the nodes used for oscillator conditions and the initial probe voltage value.
FSPTS	Specifies the frequency search points used in the initial small-signal frequency search. Usage depends on the oscillator type.

Description

You use the `.ENVOSC` command to do envelope simulation for oscillator startup or shutdown.

Oscillator startup or shutdown analysis with this command must be helped along by converting a bias source from a DC description to a PWL description that either:

- Starts at a low value that supports oscillation and ramps up to a final value (startup simulation)
- Starts at the DC value and ramps down to zero (shutdown simulation).

In addition to solving for the state variables at each envelope time point, the `.ENVOSC` command also solves for the frequency. This command is intended to be applied to high-Q oscillators that take a long time to reach steady-state. For these circuits, the standard transient analysis is too costly. Low-Q oscillators, such as typical ring oscillators, are more efficiently simulated with standard transient analysis.

Example

```
.envosc tone=250Meg nharms=10 env_step=20n env_stop=10u
+ probenode=v5,0,1.25
```

Fast Fourier Transform Form

```
.ENVFFT output_var [NP=val] [FORMAT=keyword]
+ [<WINDOW=keyword] [ALFA=val]
```

Parameter	Description
output_var	Any valid output variable.
NP	The number of points to use in the FFT analysis. NP must be a power of 2. If not a power of 2, then it is automatically adjusted to the closest higher number that is a power of 2. The default is 1024.
FORMAT	Specifies the output format:NORM= normalized magnitude UNORM=unnormalized magnitude (default)
WINDOW	Specifies the window type to use: RECT=simple rectangular truncation window (default) BART=Bartlett (triangular) window HANN=Hanning window HAMM=Hamming window BLACK=Blackman window HARRIS=Blackman-Harris window GAUSS=Gaussian window KAISER=Kaiser-Bessel window
ALFA	Controls the highest side-lobe level and bandwidth for GAUSS and KAISER windows. The default is 3.0.

Description

You use the `.ENVFFT` command to perform Fast Fourier Transform (FFT) on the envelope output. This command is similar to the `.FFT` command. The only difference is that the transformation is performed on real data with the `.FFT` command, and with the `.ENVFFT` command, the data being transformed is complex. You usually want to do this for a specific harmonic of a voltage, current, or power signal.

Example

```
.envfft v(out) [1]
```

Output Syntax

The results from envelope simulation can be made available through the `.PRINT`, `.PROBE`, and `.MEASURE` commands. This section describes the basic syntax you can use for this purpose.

.PRINT or .PROBE

You can print or probe the envelope simulation results by using the following commands:

```
.PRINT ENV ov1 <ov2... >  
.PROBE ENV ov1 <ov2... >
```

Where `ov1...` are the output variables to print or probe.

.MEASURE

In PrimeSim HSPICE advanced analog analyses, the independent variable for envelope simulation is the first tone. Otherwise, except for the analysis type, the `.MEASURE` statement syntax is the same as the syntax for HB; for example,

```
.MEASURE ENV result ...
```

Envelope Output Data File Format

The results of envelope simulations are written to `*.ev#` data files by the `.PROBE` statement. The format of an `*.ev#` data file is equivalent to an `*.hb#` data file with the addition of one fundamental parameter sweep that represents the slowly-varying time-envelope variation $\hat{t}$ of the Fourier coefficients and frequencies. You can recognize this swept parameter in the `*.ev#` file by the keyword `env_time`.

Each row in the tabulated data of an *.ev# file includes values for identifying frequency information, the complex data for the output variables, and information on the envelope time sweep. For example, the header for a data file dump for output variables v(in) and v(out), that follow a 2-tone envelope analysis, have entries for:

```
hertz  v(in)  v(out)  n0  f0  n1  f1  sweep  env_time  $& %#
```

Which result in data blocks with floating point values following:

```
env_time[0]
f[0]  a[0]{v(in)}  b[0] {v(in)}  a[0] {v(out)}  b[0] {v(out)}  n0  f0  n1
f1
f[1]  a[1]{v(in)}  b[1] {v(in)}  a[1] {v(out)}  b[1] {v(out)}  n0  f0  n1
f1
...
f[N]  a[N]{v(in)}  b[N] {v(in)}  a[N] {v(out)}  b[N] {v(out)}  n0  f0  n1
f1

env_time[1]
f[0]  a[0]{v(in)}  b[0] {v(in)}  a[0] {v(out)}  b[0] {v(out)}  n0  f0  n1
f1
f[1]  a[1]{v(in)}  b[1] {v(in)}  a[1] {v(out)}  b[1] {v(out)}  n0  f0  n1
f1
...
f[N]  a[N]{v(in)}  b[N] {v(in)}  a[N] {v(out)}  b[N] {v(out)}  n0  f0  n1
f1

...

env_time[M-1]
f[0]  a[0]{v(in)}  b[0] {v(in)}  a[0] {v(out)}  b[0] {v(out)}  n0  f0  n1
f1
f[1]  a[1]{v(in)}  b[1] {v(in)}  a[1] {v(out)}  b[1] {v(out)}  n0  f0  n1
f1
...
f[N]  a[N]{v(in)}  b[N] {v(in)}  a[N] {v(out)}  b[N] {v(out)}  n0  f0  n1
f1
```

Where there are M data blocks corresponding to M envelope time points, with each block containing N+1 rows for the frequency data. The units for the env_time sweep are seconds.

Part 5: Large Signal Noise Analyses

Describes the usage of simulation of Random noise, transient noise analysis and periodic time-dependent noise analysis.

Note: For .AC-related .NOISE analysis, see AC Small-Signal and Noise Analysis in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*. This Part contains the following chapters/topics:

- [Chapter 11, Simulation of Random Noise](#)
- [Chapter 12, Transient Noise Analysis](#)
- [Chapter 13, Periodic Time-Dependent Noise Analysis \(.PTDNOISE\)](#)

11

Simulation of Random Noise

Describes the characteristics of random signals, types of noise, component noise models, and noise simulation in PrimeSim HSPICE.

PrimeSim HSPICE ships several examples for your use; see Applications of General Interest Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

Note:

For .AC-related .NOISE analysis, see AC Small-Signal and Noise Analysis in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

This chapter presents the following topics:

- [Introduction to Noise Sources](#)
- [Characteristics of Random Signals](#)
- [Noise Types](#)
- [Component Noise Models and PrimeSim HSPICE Noise Simulation](#)

Introduction to Noise Sources

One of the essential techniques in designing an analog circuit is to be able to identify major noise sources in the circuit and know how to minimize their effect to an acceptable level. Noise can be loosely defined as a disturbance signal with random amplitude, which is usually an unwanted phenomenon in a circuit.

It is important to limit the power of noise to a low level relative to that of the signal; otherwise, the signal can be greatly distorted or completely indistinguishable. They rely on simulation tools to help them identify the dominant sources of noise in the circuit, measure the noise levels (in power, current, or voltage), and eventually make changes in the design to reduce the noise levels, if necessary.

Two types of noise are commonly defined: inherent and interference.

- Inherent noise is the noise created by the elements internal to the circuit (resistors, diodes, and transistors), caused by movement of electrons in the circuit.
- Interference noise refers to the unwanted signals from the environment absorbed by the circuit.

Eliminating interfering noise can be done by different methods of signal shielding or code modulation so that the signal can be differentiated from the noise. Inherent noise, however, can only be affected by changes in the circuit topology and the bias currents.

This chapter discusses only inherent noise and the PrimeSim HSPICE simulations related to it. Whenever the term “noise” is used, it refers to “inherent noise”.

Presented first in the following sections are some theoretical definitions for noise sources in an analog circuit. These are followed by a discussion on how PrimeSim HSPICE performs a noise simulation and how to interpret outputs from a noise simulation.

Characteristics of Random Signals

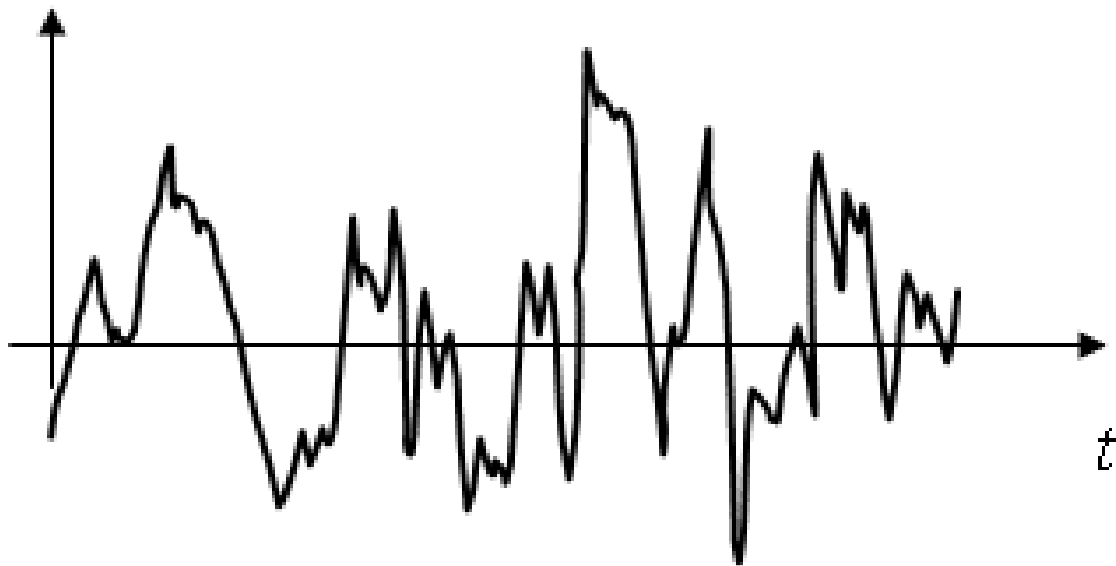
The following sections describe noise sources in the time and frequency domains, and methods to deal with the multitude of noise sources in a circuit:

- [Probability Distribution Function versus Power Spectral Density](#)
- [Multiple Noise Sources in a Circuit](#)
- [Summary](#)

Probability Distribution Function versus Power Spectral Density

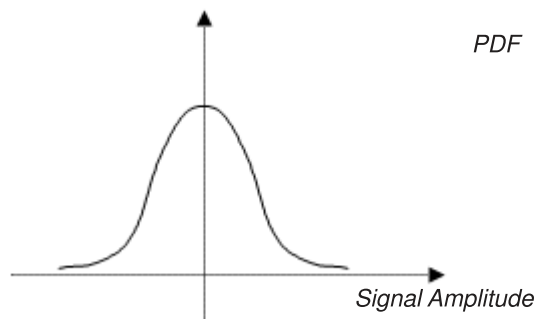
Noise is a signal with random amplitude. [Figure 22](#) illustrates a sample noise signal in the time domain. The vertical axis could be either a branch current or a node voltage.

Figure 22 Typical noise signal (voltage or current) in the time domain



To simulate a noise source in the time domain, you have to know the Probability Distribution Function (PDF) of that signal and then create a voltage or current source that produces a random output signal conforming to the PDF. [Figure 23](#) demonstrates a sample PDF for a random signal with a Gaussian distribution. Extracting PDF parameters from the circuit components is not straightforward and therefore not common in analyzing noise.

Figure 23 PDF for a random signal with Gaussian probability distribution



However, noise can be easily modeled and measured in the frequency domain. The noise models in the frequency domain also relate directly to the component and circuit characteristics. This is the reason noise analyses are usually done in the frequency domain.

While we cannot obtain a deterministic value for the noise amplitude in the time domain because it is a random variable, it is quite possible to determine the power of the noise if the noise is bandwidth limited. In other words, it is true that the amplitude of a noise signal changes randomly in the time domain. However, its power, as defined by [Equation 44](#) is a fixed and finite value if and only if the signal is bandwidth limited.

Definition of Power for a noise voltage (a) and current (b), normalized for a 1-ohm load:

Equation 44

$$P = \lim_{T \rightarrow \infty} \cdot 1 / T \int_{-T/2}^{T/2} |v(t)|^2 dt \quad \text{Eq. a}$$

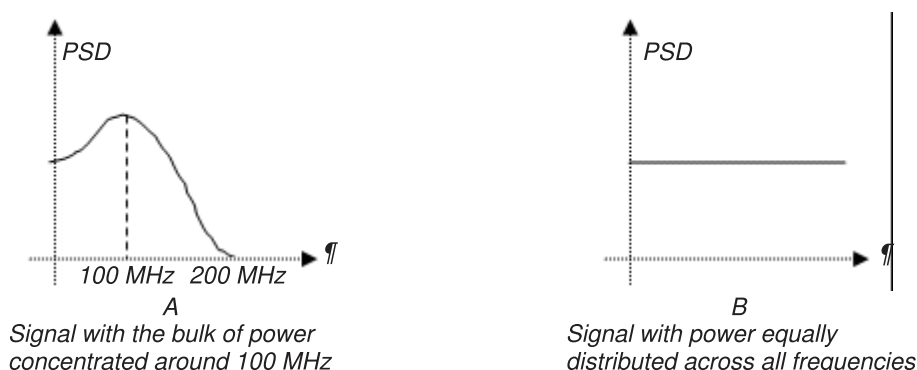
$$P = \lim_{T \rightarrow \infty} \cdot 1 / T \int_{-T/2}^{T/2} |i(t)|^2 dt \quad \text{Eq. b}$$

The power of a signal can also be described in the frequency domain; by means of a Power Spectral Density (PSD). A PSD identifies the portion of signal power present as a function of signal frequency. [Figure 24](#) illustrates two PSD graphs.

Graph A resembles the PSD of a typical data signal in high speed and RF communication; a signal with a limited bandwidth. It can be seen from the graph that the composing frequencies of the signal are between DC (0 Hz) and 200 MHz, with the bulk of power being concentrated around 100 MHz.

Graph B shows a signal with a flat power distribution; meaning that the power is equally distributed across all frequencies. Such a signal is referred to as the “white noise”. Resistor thermal noise possesses such a PSD.

Figure 24 Sample PSD graphs



To obtain the total power of a signal within a certain bandwidth, you must calculate the area under the PSD curve within the given frequency range, as defined in [Equation 45](#).

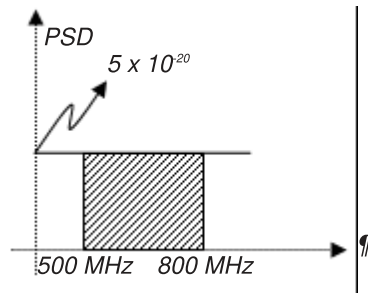
Definition of power based in the frequency domain, normalized for a 1 ohm load:

Equation 45

$$P = \int_{f-1}^{f-2} PSD(f) df$$

An example of power calculation in the frequency domain for a noise signal is shown in [Figure 25](#). In this example, the PSD is a flat curve with a value of $5 \times 10^{-20} \text{ V}^2/\text{Hz}$.

Figure 25 Sample PSD graphs



The power of the noise between the frequencies 500 MHz and 800 MHz is equal to the shaded area under the curve and is given by:

$$P_n = \int_{500M}^{800M} 5 \times 10^{-20} df = (800M - 500M) 5 \times 10^{-20} = 1.5 \times 10^{-11} \text{ V}^2$$

The root mean square (RMS) value of the noise signal is given by [Equation 46](#). The RMS value of a signal is defined as the DC value that would render the same power as the signal in question. For the general sinusoidal signal $A \cos(\omega t)$, the RMS value is $A / \sqrt{2}$.

The RMS value of a signal is the root of the 1 ohm normalized signal power:

Equation 46

$$v_{rms} = \sqrt{P}$$

Which means that the RMS voltage value of the noise signal in the above example is:

$$v_{rms} = \sqrt{1.5 \times 10^{-11}} = 5.48 \mu\text{V}$$

This means that the power of the noise is identical to that of a sinusoidal signal with a RMS value of 5.48 μV .

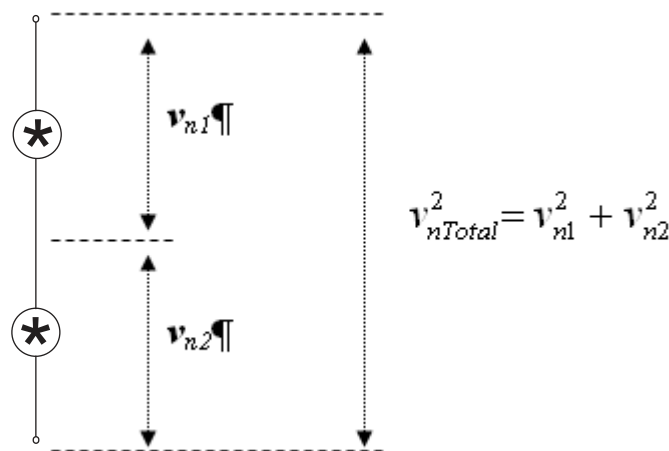
Multiple Noise Sources in a Circuit

In the usual case of dealing with a multitude of noise sources in a circuit, it is important to know how to add up the effects of noise to obtain the accumulated noise value.

If the noise sources are completely independent (that is, un-correlated), such as the noises generated by two separate resistors, their powers add up to their RMS values.

Figure 26 shows how two independent noise sources, in this case both voltage sources, add up.

Figure 26 Noise summation: independent noise signals add in squared values



It might not be evident from the equation for V_{nTotal} in Figure 26 that when there are several noise sources in a circuit, only reducing the biggest noise contributors will reduce the total noise significantly. Reducing the smaller sources, even by large amounts does not have a considerable effect on the overall noise value because the power of two of the signals magnifies the dominance of bigger sources over smaller ones.

To clarify this point, consider the case where two noise sources are present, with RMS values of $v_1 = 2 \mu V$ and $v_2 = 9 \mu V$. The total noise is given by:

$$v_{nTotal} = \sqrt{(2)^2 + (9)^2} = 9.21 \mu V$$

Now if in an attempt to lower the total noise the designer of the circuit cuts the $2 \mu V$ source by 50%, the total noise will be:

$$v_{nTotal} = \sqrt{(1)^2 + (9)^2} = 9.06 \mu V$$

Now, if instead we cut the $9 \mu V$ source by only 10%, the total noise will be:

$$v_{nTotal} = \sqrt{(2)^2 + (8.1)^2} = 8.34 \mu V$$

This emphasizes the fact that to reduce the total noise, we should first concentrate on the strongest source of noise.

Summary

Important points from this section are:

- Inherent noise is best modeled in the frequency domain by its PSD.
- The power of noise depends on the bandwidth of the system and is defined as the area under the PSD curve between two given frequencies.
- When multiple noise sources are present, the square of their voltage (or current) values add up to produce the total noise.
- When multiple sources of noise are present in a circuit, the most effective way of reducing the total noise is to focus on reducing the most dominant noise contributor.

Noise Types

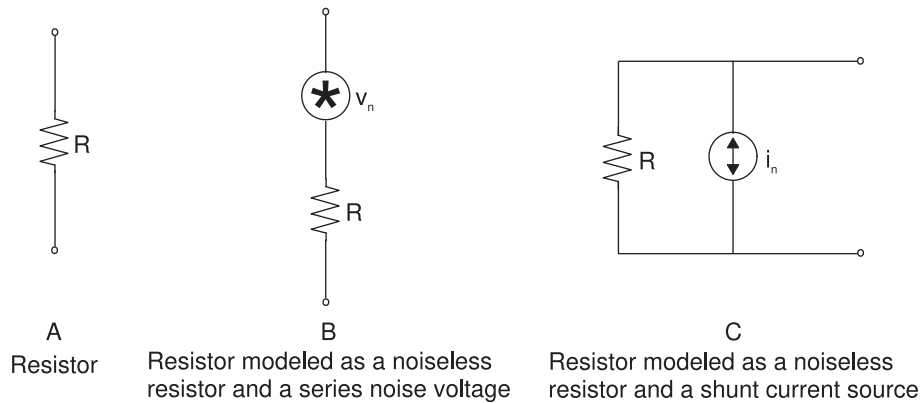
The following sections identify and describe three common types of noise in analog circuits:

- [Thermal Noise](#)
- [Flicker Noise](#)
- [Shot Noise](#)
- [Summary](#)

Thermal Noise

Thermal noise is generated by random movements of electrons in a resistive conductor. Any circuit element with resistive characteristics, whether a resistor or the base junction series resistor in a BJT's small signal model, disseminates thermal noise. The models for thermal noise voltage source and their equivalent current source are shown in [Figure 27](#).

Figure 27 Thermal noise voltage sources and their equivalent circuits



The voltage and current PSD functions for a resistor thermal noise are:

$$V_n^2(f) = 4kTR \quad \text{V}^2/\text{Hz}$$

$$I_n^2(f) = 4kT/R \quad \text{A}^2/\text{Hz}$$

where: k is Boltzmann's constant equal to 1.38×10^{-23} . T is the temperature in Kelvin degrees: $1^\circ\text{K} = ^\circ\text{C} + 273$. For a room temperature of 25°C (88°F), the default temperature setting in PrimeSim HSPICE, T will be 298°K . R is the resistance in ohms.

It should be noted that the two voltage and current PSD models are equivalent, they both produce the same amount of open circuit voltage or short circuit current and can be used interchangeably during noise analysis.

Notice that the thermal noise voltage PSD is proportional to the resistance while the current PSD is inversely proportional to the resistance. By looking at the PSD models, a general conclusion can be made: the presence of bigger resistors in a circuit usually translates to a bigger thermal noise voltage.

The PSD models have been explicitly described as functions of frequency to re-emphasize the fact that the PSD is only meaningful in the frequency domain context.

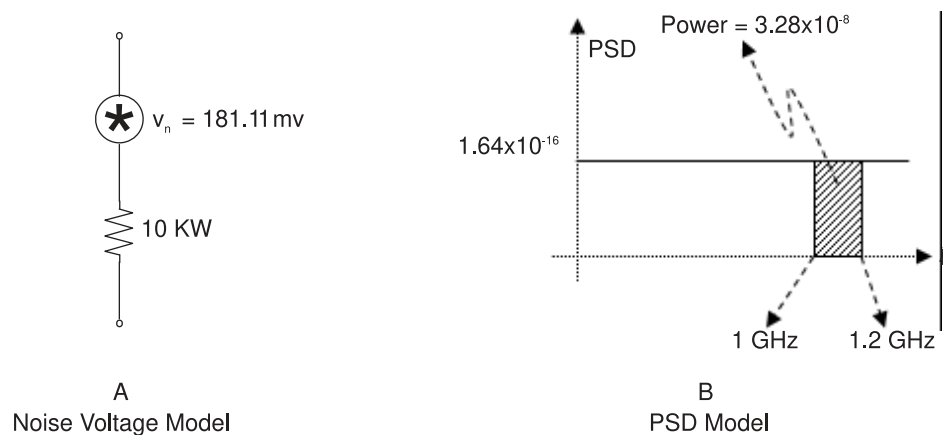
The above models suggest that PSDs are flat across all frequencies. This assumption will no longer be true for frequencies above a few hundred gigahertz.

Example 1

What will be the RMS voltage value of the thermal noise generated by a 10 kohm resistor operating in the 1- to 1.2-GHz frequency range? Assuming a room temperature of 25 °C or 298 °K, the PSD function will be (see also [Figure 28](#)):

$$\begin{aligned}
 V_n^2(f) &= 4kTR = 4(1.38 \times 10^{-23})(298)(10K) = 1.64 \times 10^{-16} & V^2/Hz \\
 v_{nrms}^2 &= \int PSD(f)df = \int (1.64 \times 10^{-16})df = (1.64 \times 10^{-16}) \times (0.2G) = 3.28 \times 10^{-8} & V^2 \\
 v_{nrms} &= \sqrt{3.28 \times 10^{-8}} = 181.11 \mu V
 \end{aligned}$$

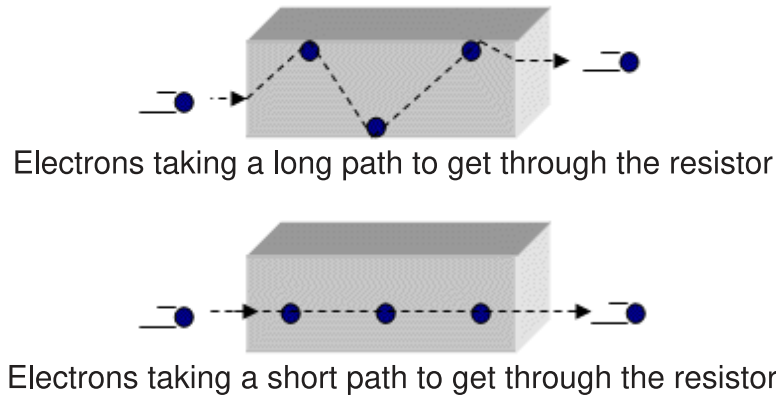
Figure 28 Noise models for 10 kohm resistor



Flicker Noise

Flicker noise is the noise generated by the fluctuations in the average current travel time in a conductor. As electrons take different random paths to get from one end of a resistor to the other, at any given time, the average current will experience different resistance along the way, leading to fluctuations in current, hence the generation of flicker noise. [Figure 29](#) illustrates how electrons can fall into paths with different lengths while flowing through a resistor.

Figure 29 Paths for current through a resistor



By shrinking the cross-section area of the conducting material, there will be smaller differences between different current paths and the flicker noise will shrink. That is why thin film resistors have a much smaller flicker noise than carbon rod resistors, which have a much larger conducting cross-section.

In addition, at high frequencies, the current tends to travel through the outer surface of the conductor, a phenomenon known as “skin effect”, effectively shrinking the conducting cross-section to a very thin layer. This results in smaller differences in the current path lengths and consequently smaller flicker noise. Because of stronger flicker noise effects at higher frequencies, flicker noise is inversely proportional to the frequency. That is why it is also called “1/f noise”. Flicker noise is only significant at lower frequency values, but it can have a major impact on nonlinear circuits at RF frequencies.

The major sources of flicker noise in analog circuits are semiconductor components. Resistors can be manufactured to produce only small amount of flicker noise and are not usually taken into account for the calculation of flicker noise in a circuit.

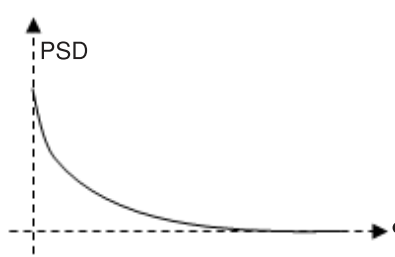
Flicker noise is usually represented as a current PSD for resistors and semiconductors with this equation:

$$I_n^2(f) = K_f I / f \quad A^2 / Hz$$

Where, I is the current through the resistor. f is frequency in hertz. K_f is a constant which depends on the characteristics and geometry of the conductor.

The PSD for flicker noise is depicted in [Figure 30](#).

Figure 30 PSD for flicker noise



PrimeSim HSPICE uses more general models for CMOS flicker noise PSDs. One of the models used for a CMOS transistor is the SPICE2/Berkeley noise model:

$$I_n^2(f) = K_f I^{A_f} / (C_{ox} L^2 f^{E_f}) A^2 / Hz$$

where: K_f is the flicker noise parameter.

A_f is the current exponent.

C_{ox} is the Gate oxide capacitance.

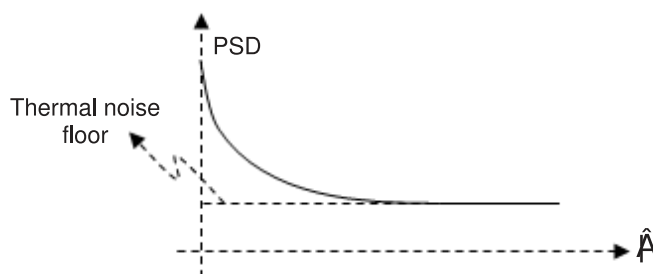
L is the effective length of the transistor.

E_f is the frequency exponent.

Another model used by PrimeSim HSPICE for flicker noise calculation is the BSIM model, which looks very similar to the above equation. It is important to note that you usually must not be concerned with the value settings of these parameters. They must be defined and set to proper values in the transistor models defined in the technology library.

The combined PSD for flicker and thermal noise is shown in [Figure 31 on page 209](#).

Figure 31 PSD of combined thermal and flicker noise



Note that the flicker noise is the dominant source of noise at lower frequencies and as its magnitude shrinks at higher frequencies, the thermal noise will become the dominant source of noise. Thermal noise is also referred to as “the noise floor” because that is the minimum possible amount of noise in a circuit.

Shot Noise

Shot noise is only generated by semiconductor elements and is caused by random passage of electrons and holes across a potential barrier, such as a P-N junction. Shot noise is often represented by a current PSD, known as the “Schottky formula”:

$$I_n^2(f) = 2qI \quad A^2 / Hz$$

Where, $q = 1.6 \times 10^{-19}$ C is the charge of an electron. I is the bias current of the junction and can be I_C , collector bias currents for a BJT transistor.

Shot noise PSD curves are flat across all frequencies.

Summary

The three major types of noise in an analog circuit are:

- Thermal noise, also known as white noise, is generated by the resistors in the circuit. Thermal noise is a function of conductor resistance.
- Flicker noise, also called 1/f noise, is mainly generated by transistors in a circuit. It is a function of the component geometry and its magnitude drops as frequency increases.
- Shot noise is caused by bias currents in the base and collector of BJT transistors.

Component Noise Models and PrimeSim HSPICE Noise Simulation

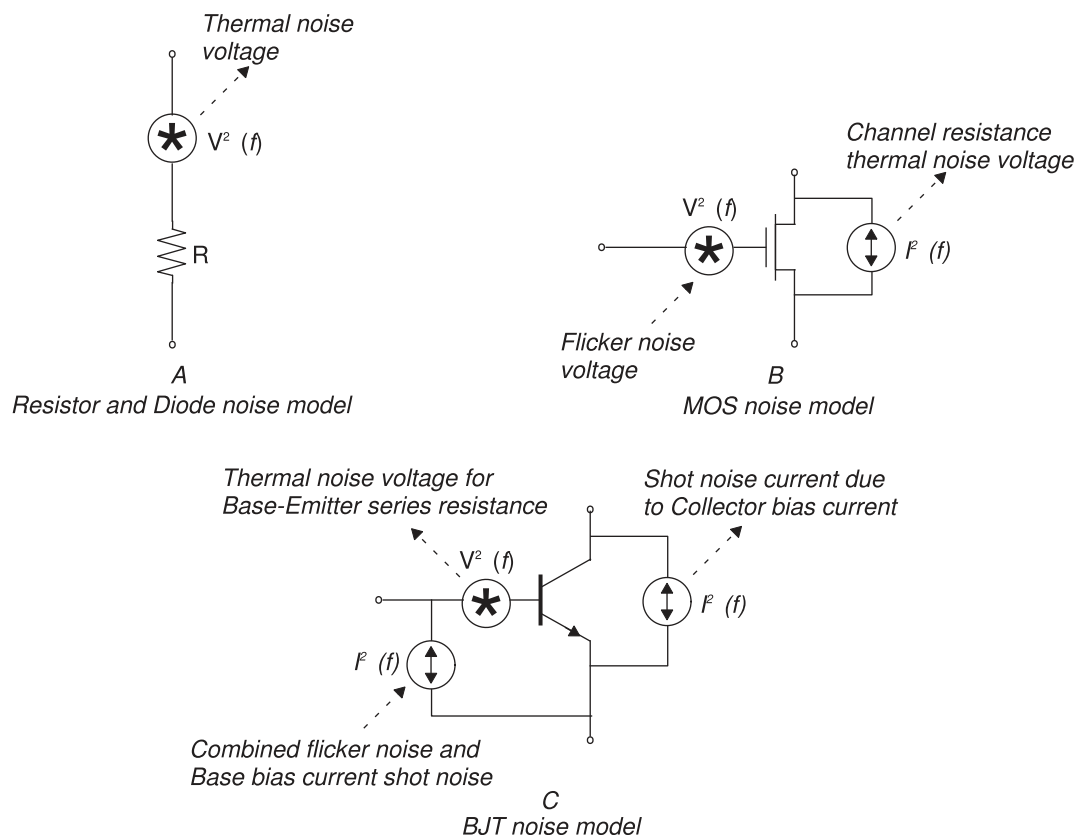
The following sections describes how PrimeSim HSPICE models noisy elements, calculates the total output noise PSD, and obtains the total output noise voltage PSD, and how the RMS output noise voltage can be deduced from the total output noise PSD.

- [Element Noise Models](#)
- [PrimeSim HSPICE Noise Simulation](#)
- [Summary](#)

Element Noise Models

Each resistor, diode, and transistor in a circuit generates at least some type of inherent noise. PrimeSim HSPICE models the noise generating elements in a circuit as a noiseless element combined with a noise current or voltage source with a known PSD. [Figure 32 on page 211](#) includes the listing of the four noise generating elements in analog circuits and their equivalent noise models.

Figure 32 Noise generating circuit elements and their noise models



The noise model for a diode is identical to that of a resistor with a resistance of:

$$R_d = kT / qI_d$$

Where, R_d is the equivalent resistance of the diode. k is Boltzmann's constant. q is the charge of an electron. I_d is the bias current of the diode.

PrimeSim HSPICE Noise Simulation

To perform a noise analysis in PrimeSim HSPICE, a `.NOISE` statement along with an `.AC` statement must be used. The `.NOISE` syntax for the noise analysis is:

```
.NOISE outvar1 [outvar2 outvar3...] [vin] [interval]
+ [portnoise[=1|0]]
+ [listckt [=1|0]]
+ [listfreq=frequencies|none|all]
+ [listcount=num] [listfloor=val]
```

```
+ [listsources=1|0|yes|no]
+ [listtype=1|0]
```

The frequency points at which the noise calculations are performed are the same points defined by the `.AC` statement. The noise calculations for each frequency point will be output to the listing file.

The following sections discuss these topics:

- [RX Transfer Function](#)
- [Example](#)

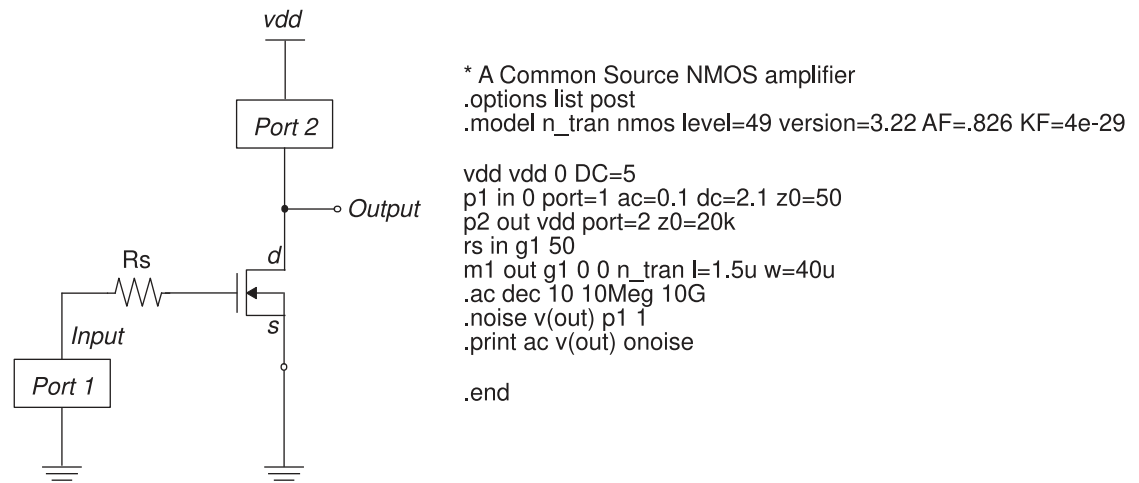
RX Transfer Function

RX is the transfer function from the noise source to the circuit output. Since the noise sources for built-in devices are all current sources with PSDs (power spectral densities) in A^2/Hz , and the output is usually a voltage (specified in the `.NOISE` statements), RX is a transimpedance (V/A). The total output noise voltage is the integrated output noise. The output noise at a given frequency is a PSD in V^2/Hz . This can be integrated over the frequency range (using the `.AC` command) to get a total output noise in V^2 . If you take the square root, you get the total output noise in V.

Example

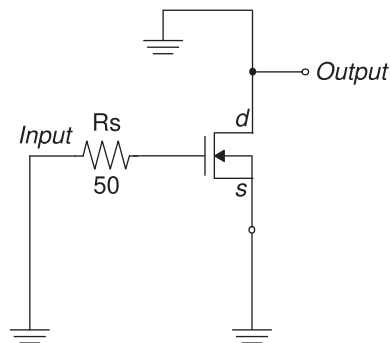
For example (see [Figure 33](#)), take a simple common source NMOS amplifier to show how PrimeSim HSPICE calculates the output noise. In this example, the output port element (P2) is used as a pull-up resistor in the circuit. For the path to the demo file, `noise_app_orig.sp`, see Applications of General Interest Examples in *PrimeSim HSPICE User Guide: Demonstration Netlists*. The full path to this example is `$installdir/demo/hspice/apps/noise_app_orig.sp`.

Figure 33 A common source NMOS amplifier and its netlist



The first step in the noise analysis is to set all the signal voltage and current sources to 0. The equivalent circuit for our example, after setting $v_{dd}=0$, is shown in Figure 34.

Figure 34 Equivalent circuit after independent V and I sources are set 0



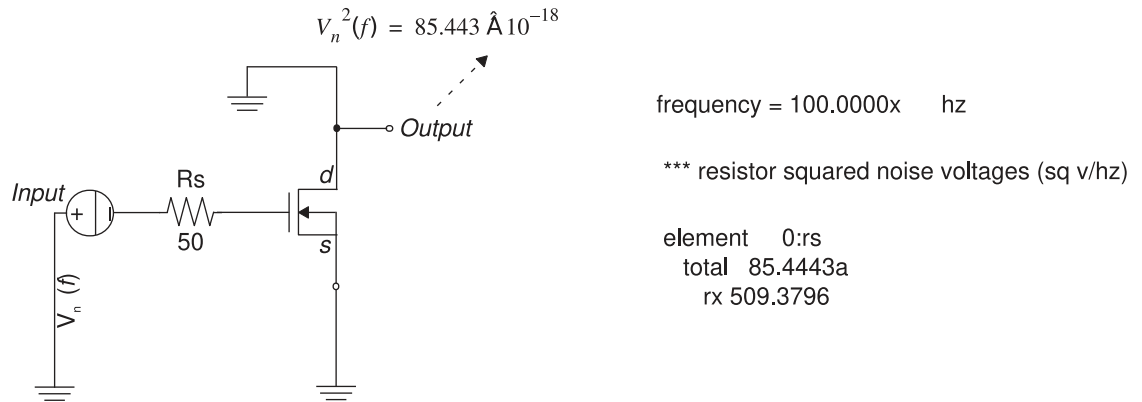
PrimeSim HSPICE models each resistor, diode, and transistor with its noise model, and then calculates the output voltage resulting from the noise signal, one element at a time.

To start with, it replaces R_s with its noise model, as shown in Figure 35, and calculates the PSD of the noise voltage as seen at the output port. PrimeSim HSPICE reports an output voltage PSD of $85.4443 \times 10^{-18} \text{ V}^2/\text{Hz}$, caused by the thermal noise model of R_s . The value rx shown is used to obtain the voltage transfer function from the R_s noise source to the output port:

$$\text{Voltage Transfer Function} = rx = \frac{\text{output_voltage}}{\text{element_noise_current}}$$

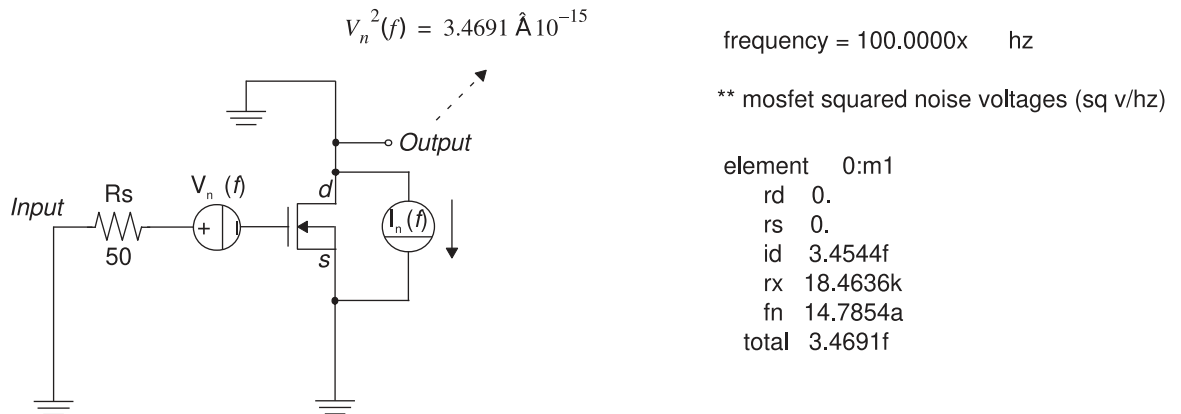
PrimeSim HSPICE uses rx for its internal calculations and you need not pay any particular attention to it.

Figure 35 Circuit noise model for R_s and analysis output at 100 MHz



The circuit for calculating the PSD of the output noise generated by the NMOS is given in [Figure 36](#).

Figure 36 Circuit noise model for NMOS and analysis output



The total PSD at 100 MHz will be the sum of all individual PSDs (the square of noise signals addition):

$$V_{nTotal}^2(f = 100MHz) = 85.4443 \times 10^{-18} + 3.4691 \times 10^{-15} = 3.5546 \times 10^{-15} V^2 / Hz$$

$$V_{nTotal}(f = 100MHz) = \sqrt{3.5546 \times 10^{-15}} = 59.6204 \times 10^{-9} V / \sqrt{Hz}$$

It is common to represent the total noise generated by a circuit as the equivalent input noise (otherwise called as the *input referred noise*). The input referred noise voltage/current is the voltage/current that, if applied at the input of the noise-free circuit, would have generated the same output voltage as the total output noise voltage.

To calculate the equivalent input noise signal, you have to obtain the transfer function from the input to the output. For the example circuit above, the transfer function will be:

$$TF = \frac{\text{Output Voltage}}{\text{Input Voltage}} = \frac{V(out)}{VS}$$

In the case where the input source is a current source instead of a voltage source, the TF would be the ratio of the output voltage over the input current.

The equivalent input noise voltage (or current in case of the input current source) is given by:

$$\text{Input Referred Noise} = \frac{\text{Total Output Noise Voltage}}{TF}$$

The input referred noise is useful to calculate the circuit's "noise figure," which is a measure of how much a circuit is generating inherent noise. A large noise figure indicates a high level of noise generation by the circuit. Noise figure is defined as:

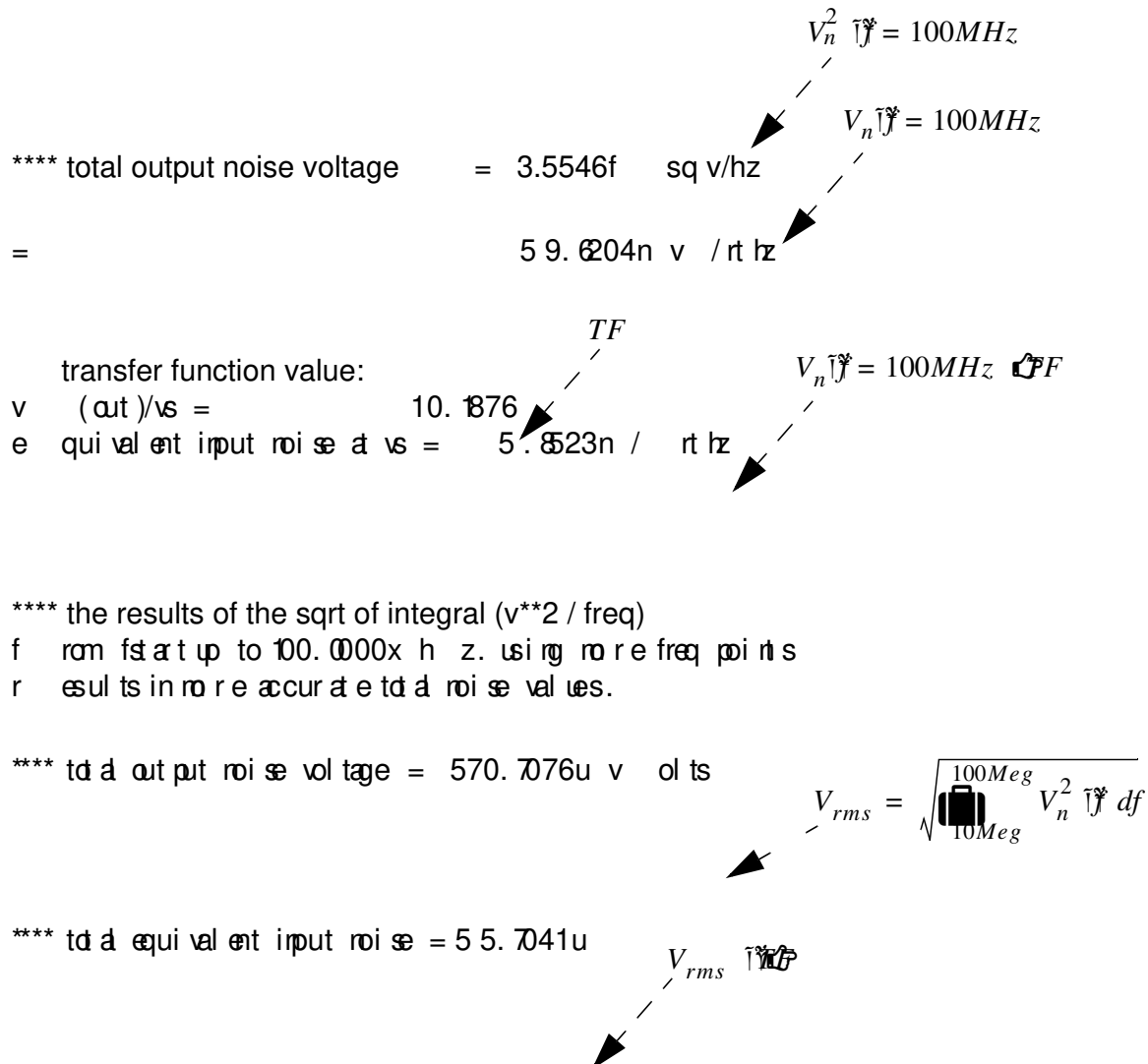
$$\text{Noise Figure} = 10 \cdot \log \cdot (\text{Noise Factor})$$

Where Noise Factor is given by:

$$\text{Noise Factor} = 1 + \frac{\text{Input Referred Inherent Noise Power}}{\text{Power of External Noise at the Input}}$$

[Figure 37](#) shows the PrimeSim HSPICE noise analysis summary output for the above circuit.

Figure 37 Noise summary for noise performed at 100 MHz



Summary

To summarize,

- PrimeSim HSPICE model elements as noiseless elements connected to noise-generating voltage and current sources.
- To calculate the total output noise PSD, set all signal and supply sources to zero. The simulator then replaces, one-by-one, the noise-generating elements with their equivalent noise models and calculates the noise voltage PSD at the output caused

by the element's noise source (similar to an `.AC` analysis). It then repeats the same process for the next noise-generating element.

- Once the output noise PSDs are calculated for all elements, PrimeSim HSPICE adds the PSDs together to obtain the total output noise voltage PSD.
- The RMS output noise voltage and the input referred noise can be deduced from the total output noise PSD.

12

Transient Noise Analysis

Describes the PrimeSim HSPICE solutions to perform transient noise analysis and compute noise statistics and their variation over time for circuits driven with non-periodic waveforms.

Transient noise analysis shows the effect of noise on the signal magnitude. It is also useful to see how noise affects the timing of the signal. Transient noise simulations include statistically accurate noise source contributions from all devices. The noise source frequency response is adjustable. From the transient noise analysis results, you can extract and measure phase noise and jitter. This capability is especially useful in characterizing phase locked loops (PLL). The two jitter measurements are time interval error (TIE) and autocorrelation function. TIE measures the time-shift behavior relative to a reference signal. The autocorrelation function tracks the relative time-shift behavior of the signal.

This chapter describes two approaches:

- Monte Carlo (default), where PrimeSim HSPICE uses random signal sources to predict the statistical characteristics of the circuit performance due to device noise. (See [Monte Carlo Noise Analysis on page 221](#).)
- Stochastic Differential Equation (SDE), for advanced users, which makes a direct prediction of the actual statistics of the output waveforms. (See [Stochastic Differential Equation Analysis on page 236](#).)

PrimeSim HSPICE includes several different algorithms for understanding circuit behavior based on the noise generated internally by electronic devices and thermal noise. PHASENOISE analysis computes the effects that noisy elements have on the output spectrum of oscillators. HBNOISE and SNNOISE analyses compute the small-signal variations that noise can create under large-signal steady-state operating conditions (see [Multitone Harmonic Balance Noise \(.HBNOISE\)](#) or [Shooting Newton Noise Analysis \(.SNNOISE\)](#)). Periodic time-domain noise (PTDNOISE) analysis computes the noise statistics of a periodic signal, and how those statistics vary with time over the period of the steady-state signal (see [Periodic Time-Dependent Noise Analysis \(.PTDNOISE\)](#)). What makes transient noise analysis unique compared to these other approaches is that its noise analysis is a complement to the fully nonlinear dynamic time-domain .TRAN simulation of PrimeSim HSPICE.

For additional information and graphical illustration of the PrimeSim HSPICE transient noise solution, see the webinar link: **Jitter Analysis Using HSPICE Transient Noise Techniques** at <https://www.synopsys.com/verification/ams-verification/hspice.html> and https://www.synopsys.com/news/pubs/snug/sanjose11/ma6_tutorial_HSPICE_trannoise.pdf (requires your Synopsys SolvNetPlus user ID and password).

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

Note:

For .AC-related .NOISE analysis, see AC Small-Signal and Noise Analysis in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

The following sections discuss these topics:

- [Overview of PrimeSim HSPICE Transient Noise Analysis](#)
- [Modeling Frequency-Dependent Noise Sources](#)
- [Monte Carlo Noise Analysis](#)
- [Setting up a Monte Carlo Transient Noise Analysis](#)
- [Controlling Noise](#)
- [Setting Noise Bandwidth](#)
- [TIE and Spectrum Jitter Measurements from Transient Noise Results](#)
- [Phase Noise and Jitter from Transient Noise](#)
- [Power Spectral Density from Transient Noise](#)
- [Stochastic Differential Equation Analysis](#)
- [Error Handling, Error Recovery, Status Reporting](#)
- [References](#)

Overview of PrimeSim HSPICE Transient Noise Analysis

A variety of noise measurements are desirable from circuit simulation. The traditional SPICE .noise analysis provides a measurement of the RMS noise voltage at an output node as a function of frequency. This RMS value is similar to a measurement of the standard deviation of an equivalent Gaussian distribution of noise present at the

output node of interest due to the contributing random noise sources within the circuit. The `.noise` analysis is a small-signal analysis that gives an output noise (onnoise) value over the `.ac` frequency range.

More advanced examples of noise measurements include phase noise and timing jitter. Timing jitter, in particular, is a measurement of a clock or oscillator's random noise over a time interval. Timing jitter represents the standard deviation (or variance) of the timing uncertainty (that is, the random drift of the clock edges) as a function of time. It is therefore a time-domain noise measurement that PrimeSim HSPICE typically evaluates at each cycle or half-cycle. For some clocks, oscillators, and PLLs, you can separate this measurement into time-independent and time-dependent contributions written as:

Equation 47

$$\sigma^2(\tau) = \frac{2}{\omega_0^2} [R_\phi(0) - R_\phi(\tau)]$$

where $\sigma^2(\tau)$ represents the time-dependent variance, τ is the time advance, and $R_\phi(\tau)$ is the autocorrelation function that relates to the power spectrum of phase variations as:

Equation 48

$$R_\phi(\tau) = \frac{1}{2\pi} \int_{-\infty}^{\infty} S_\phi(\omega) e^{j\omega\tau} d\omega$$

These special relationships allow the computation of timing jitter from the results of `.phasenoise` analysis, since we can assume $L(f) \cong S_\phi(f)$ and therefore derive the time-varying noise from the frequency-domain phase noise simulation solution.

Other measurements of time-dependent or time-varying noise are also desirable for circuits other than clocks and oscillators, and for situations other than the steady-state operation. The purpose of such measurements is usually similar: derive useful information on the time-varying statistical behavior of the circuit due to its internal noise sources. Transient noise analysis is a typical `.TRAN` simulation but with all random noise sources within the circuit activated as contributing signal sources. The Monte Carlo noise analysis is a transient noise simulation approach that uses uncorrelated random signal sources for device noise in such a way that you can seed all noise signals uniquely. In addition, you can repeatedly, from run-to-run, predict the statistical characteristics of the circuit performance due to device noise. You can typically examine the resulting outputs by using histogram plots to measure their statistical behavior.

Modeling Frequency-Dependent Noise Sources

Transient noise techniques require that you model noise sources in the time domain. These techniques model device noise sources in terms of the standard white Gaussian noise processes. You write the standard white Gaussian noise process as $\zeta(t)$.

You calculate white noise source models in terms of intensity functions. For example, if Equation 49 gives the (frequency dependent) noise from a time varying conductance:

Equation 49

$$i_n^2 = 4kTG(t)\Delta f$$

Then you can model in terms of intensity as the random time-domain current given by:

Equation 50

$$j_n = \sqrt{2kTG(t)} \zeta(t)$$

In the case of flicker noise, it is necessary to create the 1/f power spectrum of flicker noise sources by filtering a $\zeta(t)$ process. You can accomplish this with the following rational function transfer relationship:

Equation 51

$$H(s) = \prod_{i=1}^N \frac{(s + \omega_{zi})}{(s + \omega_{pi})}$$

Since you can model all flicker sources over the same frequency range (that is, bandwidth), you can use this same transfer function for all sources. Tests show that the above fit is very accurate using one pole/zero per octave. This modeling technique shows reasonable fits using one pole/zero for every two octaves, or even with one pole/zero per decade. This fitting algorithm is therefore frequency range-specific, which is why you use the parameters FMAX and FMIN for specifying frequency ranges for transient noise analysis similar to the modeling of the frequency-dependent S-element.

The TRANNOISE analysis includes a time-domain noise source for all noise contributions within all devices. So, for example, if the transistor model in use includes induced gate noise, so does .TRANNOISE. For example, if you use a BSIM4 model and the model parameter set includes values for Induced Gate Noise, then .TRANNOISE includes this noise in the simulation.

Monte Carlo Noise Analysis

The equations for Monte Carlo transient noise analysis define the Monte Carlo approach.

The time-domain system of equations for transient analysis drives the Modified Nodal Analysis (MNA) system described by the following vector state equation:

Equation 52

$$f(\dot{x}, x, t) = 0$$

Consider this the noiseless system that PrimeSim HSPICE solves for during a normal transient analysis. You can consider the transient noise analysis as a similar analysis where we now inject noise from all device noise models to give the modified equation:

Equation 53

$$f(\dot{x}, x, t) = -j_n(t)$$

This system reflects the added noise sources and PrimeSim HSPICE solves it in the same manner as transient analysis. However, if we consider each noise source related to a white Gaussian noise function, we must create an *ensemble* of waveforms for our unknown $x(t)$ vector to predict the output statistics. This type of simulation involves generating multiple uncorrelated noise source waveforms for all noise sources, and then running multiple simulations in a Monte Carlo fashion. This approach is called the Monte Carlo noise simulation. This method cannot directly measure the statistics of output signals due to input noise (as with `.NOISE`), but instead models noise sources as independent time-domain stimuli. Generating statistical information in this approach requires running a plurality of simulations over a variety of random noise-source sequences (to create an ensemble of output waveforms) and then analyzing the statistics of the ensemble using histograms or other plots. When the system behavior is *ergodic*, it is possible to run a single very long duration simulation in order to capture the statistical variations of the output signal over time (as with an eye-diagram). Monte Carlo modeling of the noise sources uses a sum of sinusoids with random phases [2], or random number generators with the appropriate statistical distributions [3]. An advantage of the Monte Carlo approach is its ability to capture very nonlinear noise behaviors. This is useful, for example, when you know that the responses of circuits with noise have non-Gaussian variations about their noiseless simulations.

Setting up a Monte Carlo Transient Noise Analysis

The transient noise analysis requires an accompanying `.tran` analysis which determines the time-sampling, matrix solutions, and deterministic output waveforms. The `.TRANNOISE` command activates transient noise and computes the additional noise variables. This is consistent with how `.NOISE` computes additional noise outputs when you add it to an `.AC` analysis.

The following sections discuss these topics:

- [Transient Noise and Monte Carlo Noise Analysis Techniques Available in PrimeSim HSPICE](#)
- [Monte Carlo Output Data](#)
- [Monte Carlo Examples](#)

Transient Noise and Monte Carlo Noise Analysis Techniques Available in PrimeSim HSPICE

The four methods of using Monte Carlo analysis and transient noise analysis are:

- [Traditional Monte Carlo with Transient Noise Disabled](#)
- [Transient Noise Analysis without Monte Carlo \(Single Sample\)](#)
- [Transient Noise Analysis with Monte Carlo](#)
- [Both Transient Noise and Traditional Monte Carlo Enabled](#)

Traditional Monte Carlo with Transient Noise Disabled

To use traditional Monte Carlo with transient noise disabled, specify a Monte Carlo run with the `.TRAN` command.

For example:

```
.TRAN 10p 8u SWEEP MONTE=20 FIRSTRUN=2
* gives 20 Monte Carlo samples from 2-21
* no .trannoise
```

This method uses the standard `SWEEP MONTE` keywords to activate Monte Carlo parameter sampling for `GAUSS`, `AGAUSS`, `UNIF`, and `AUNIF` defined parameters. For this analysis, the transient noise is not included. The result of the simulation is an ensemble of output waveforms that reflect no noise, but only the traditional Monte Carlo parameter variations.

Transient Noise Analysis without Monte Carlo (Single Sample)

A single transient noise analysis includes statistically accurate time-varying noise contributions for all noise sources present in the circuit, including SPICE models. The bandwidth is adjustable for all noise-source frequency responses. This analysis approach is the fastest possible based on single `.TRAN` analysis and is the most effective for viewing statistical variations over time (ergodic simulations).

To use transient noise analysis without Monte Carlo, specify a single sample transient noise analysis.

For example:

```
.TRAN 10p 8u          $.tran without SWEEP MONTE
.TRANNOISE v(out<0>)  $ output of interest
+ fmin=1k fmax=1e8    $ sets noise bandwidth
```

When setting the noise bandwidth, `FMIN` is used to set the low-frequency flicker noise limits and `FMAX` is used to set the maximum noise-source waveform bandwidth and ensure Nyquist sampling. The seed values can be reset to re-run simulations with uniquely

different noise waveforms. The output waveforms can be post-processed to measure the resulting noise effects.

Transient Noise Analysis with Monte Carlo

To use transient noise analysis with Monte Carlo, specify a multi-sample transient noise analysis.

For example:

```
.TRAN 10p 8u          $.tran without SWEEP MONTE
.TRANNOISE v(out<0>)  $ output of interest
+ METHOD=MC           $ use random noise waveform
+ SAMPLES=20          $ 20 samples
+ SEED=2              $ begin with seed=2, ends at 21
+ fmin=1k fmax=1e8    $ sets noise bandwidth
```

For this analysis, traditional Monte Carlo parameter sampling is disabled and only the nominal values are used for GAUSS, AGAUSS, UNIF, and AUNIF defined parameters during the simulation.

The transient noise `METHOD=MC` keyword will inject random noise waveforms for all noise sources present in the circuit, including SPICE models. The result of the simulation is an ensemble of output waveforms that reflect the random noise waveform injections. Each noise seed sample will have a unique waveform. This approach is useful for characterizing statistics at specific time events and the waveforms can be post-processed as a histogram.

Both Transient Noise and Traditional Monte Carlo Enabled

To use transient noise analysis with the traditional Monte Carlo analysis enabled, specify both a traditional Monte Carlo run and a transient noise analysis.

For example:

```
.TRAN 10p 8u SWEEP MONTE=20 FIRSTRUN=2
* sets Monte seed values from 2-21
.TRANNOISE v(out<0>)  $ add .trannoise
+ METHOD=MC
+ SAMPLES=1 SEED=2    $ THIS IS IGNORED
* Monte Carlo seeds 2-21 defined in .TRAN are used
+ fmin=1k fmax=1e8    $ sets noise bandwidth
```

The Monte Carlo seeds are set in the transient analysis setting and the same seed values are shared as the noise seeds in the transient noise analysis. The traditional Monte Carlo parameter sampling is used for random values from the GAUSS, AGAUSS, UNIF, and AUNIF defined parameters. The transient noise `METHOD=MC` keyword will inject random noise waveforms for all noise sources present in the circuit, including SPICE models. The simulation result is an ensemble of output waveforms that reflect both the traditional Monte

Carlo parameter variations and the random noise waveform injections from transient noise analysis.

Note:

You will receive a warning message about the `.TRANNOISE` sweep being ignored.

Monte Carlo Output Data

The `.TRANNOISE` analysis outputs raw data to the `*.trpn0` and `*.printtr0` files. Formatting of the data output is consistent with that used for `*.pn0` and `*.snpn0` data files. These output files organize data according to the sample number in the Monte Carlo index. The first sample (index=1) creates a noise-free simulation, that is, disables all noise sources for this simulation. With index=2, all subsequent simulations use unique random number seeding to create unique simulation results due to noise.

Monte Carlo Examples

The following example generates 30 Monte Carlo noise simulations that starts with a noiseless (index=1) simulation.

```
.TRANNOISE v(out) SAMPLES=30
```

The following example generates 20 Monte Carlo noise simulations that starts with the seed value (that is, index) of 31 for the first simulation.

```
.TRANNOISE v(out) SAMPLES=20 SEED=31
```

The following example generates a single noise simulation, with a seed value of 50, and amplifies all noise sources by a factor of 10.

```
.TRANNOISE v(out) SEED=50 SCALE=10.0
```

The following example generates six Monte Carlo transient noise simulations with seed values of 1, 3, 4, 5, 9, and 10. Normalized autocorrelation is computed for each `v(out)` output.

```
.TRANNOISE v(out) SAMPLES=LIST(1,3:5,9:10) AUTOCORRELATION=2
```

Controlling Noise

Substantial performance improvements can be obtained by removing noise from the non-critical circuit blocks. It can also be useful to disable various noise contributors to isolate and identify the dominant circuit noise effects. Transient noise analysis allows you to ignore noise contributions at the subcircuit level.

To ignore the noise contribution, add the `NOISE=0` parameter to any subcircuit instance specified by an X-element. `NOISE=0` is treated as a hierarchical parameter, causing noise to be disabled down through the subcircuit's hierarchy.

By default, all noise sources are activated at `time=0` in transient analysis. Using the `START` keyword in the `.TRANNOISE` command allows the noise sources to be activated at a specified time.

For example:

```
.TRAN 10p 10u
.TRANNOISE v(out)
+ fmin=1k fmax=1e8
+ start=2u $ enable noise sources at 2uS
```

Additional noise control options are also available to turn on or off noise types for subcircuits, instances, and devices. The options `noise_on_list` and `noise_off_list` provide the flexibility to turn on and off noise sources by noise type (thermal, shot, flicker, or all) and element type (D, E, G, J, M, P, Q, R, S, W, or X). The options are supported for all available noise analyses. For more information, see `.OPTION NOISE_ON_LIST` and `.OPTION NOISE_OFF_LIST` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Note:

When there is a conflict between `noise=0|1` and the `noise_on_list/noise_off_list` options, the latter take precedence.

Setting Noise Bandwidth

The `FMAX` and `FMIN` keywords in the `.TRANNOISE` command are used to set the noise bandwidth of the circuit. You can expect some significant differences in transient noise analysis performance with `FMAX` and `FMIN` changes. Noise power increases linearly with `FMAX` and the flicker noise power can scale as $1/FMIN$.

By default, PrimeSim HSPICE estimates the circuit noise bandwidth and uses the estimated value for `FMAX`. The estimate is based on the circuit conductance and capacitance matrices internal to PrimeSim HSPICE.

You can also manually set the `FMAX` value. Use caution when setting the `FMAX` value manually since it has a dramatic effect on transient noise. Since `FMAX` controls the amount of energy each noise source can emit, huge values of `FMAX` (such as 100G) can result in huge instantaneous noise levels. `FMAX` is therefore a critical parameter for controlling the transient noise analysis performance.

Additionally, because `FMAX` sets the noise bandwidth of noise sources, it also controls the maximum time step taken. Therefore, a large `FMAX` can cause very slow simulation

performance. Setting F_{MAX} should be based on the known operating and clock frequencies, or technology limits.

The F_{MIN} keyword establishes the lower frequency bound for flicker noise modeling. You need to use this control only when running very long simulations to capture the effects of low frequency noise.

TIE and Spectrum Jitter Measurements from Transient Noise Results

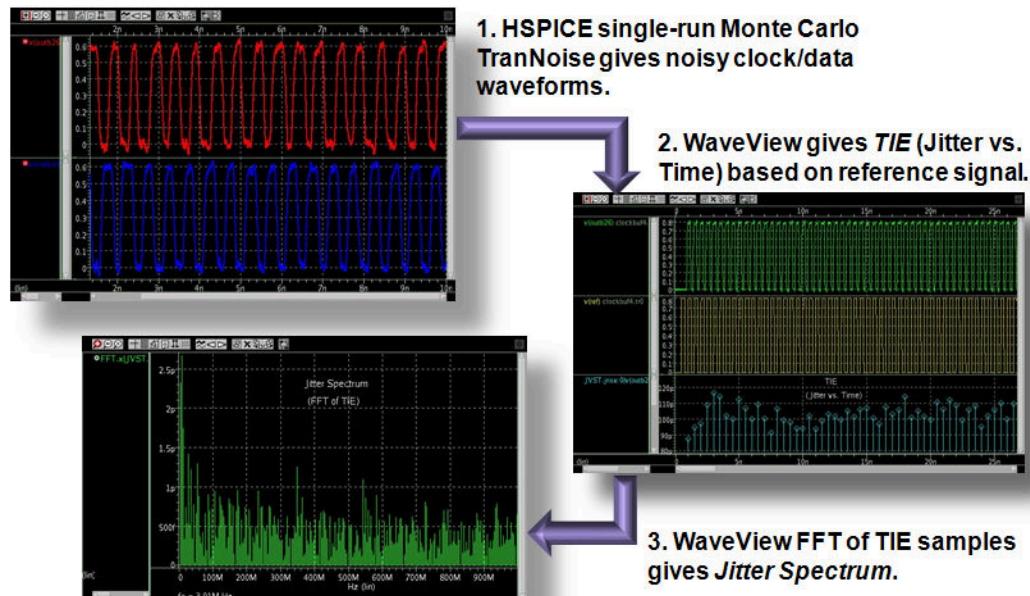
While transient noise analysis shows the effect of noise on the signal magnitude, it is also useful to see how the noise affects the timing of the signal.

From the transient noise analysis results, jitter can be measured. The two jitter measurements are *time interval error* or TIE, and *autocorrelation function*. TIE measures the time-shift behavior relative to a reference signal and is best measured using WaveView. The autocorrelation function tracks the relative time-shift behavior of the signal.

The following sections discuss these topics:

- [Measure TIE Jitter and Jitter Spectrum Example](#)
- [Autocorrelation Function Example](#)

Measure TIE Jitter and Jitter Spectrum Example



The following clock buffer example circuit uses a single run Monte Carlo transient noise analysis. Post-processing is performed using WaveView.

The complete `clockbuf4.sp` input file for this example is located at `$installdir/demo/hspice/rf_examples/`.

The transient noise analysis uses the following:

- A `.TRAN` command and a `.TRANNOISE` command:

```
.tran '0.05*period' '520*period'
.trannoise v(outb26) FMAX=50G SCALE=10
```

The noise bandwidth is set by `FMAX` and the `SCALE` factor is used to scale the noise sources in the circuit.

- A pulse source to define the input clock source. The pulse source will also be used as the ideal reference source for comparison with the noisy buffer output signals.

```
Vsrc ref gnd DC 0 PULSE (0.0 'vdd'
+ '0.975*period' '0.05*period' '0.05*period'
+ '0.45*period' 'period')
```

- A `.PROBE` command for plotting the output signals:

```
.probe tran v(ref) v(outa) v(outb14) v(outb26)
```

Analysis Setup Details

```
* Transient noise example
.option post probe
.option runlvl=5
.param freq=2000MEG period='1/freq'
* Reference Clock
Vsrc ref gnd DC 0 PULSE (0.0 'vdd'
+ '0.975*period' '0.05*period' '0.05*period'
+ '0.45*period' 'period')
* Clock Buffer Circuit
* Analysis setup
.tran '0.05*period' '520*period'
.trannoise v(outb26) FMAX=50G SCALE=10
.probe tran v(ref) v(outa) v(outb14) v(outb26)
.end
```

To run this netlist, type the following command:

```
hspice -i clockbuf4.sp -o
```

This produces an output file `clockbuff4.tr0`, containing the transient noise output.

To measure the TIE jitter, use the following steps:

1. Type `wv` at the prompt to invoke WaveView.
2. Click the **Open Waveform** icon and select the `clockbuf4.tr0` file from the Open: Waveform Files dialog box.
3. Select the `v(outb26)` signal from the `clockbuf4.tr0` file in the signal browser. Double-click the signal name or drag and drop the signal into the waveview. The `v(outb26)` signal is the noisy output of the clock buffer.
4. To plot the jitter versus time, click **Jitter vs. Time** in the Utilities section of the **Waveform** tab. In the Jitter vs. Time Plot window, set the **Reference Clock** to **An Analog Signal**. Set the **External Reference Signal** to be the ideal reference by dragging and dropping the `v(ref)` signal into the box. Select **Display Reference Clock Signal**. Set the Rise (V) and Fall (V) for both the **Analog External Reference Signal Threshold** and the **Analog Target Signal Threshold** to 0.4 volts. The **Jitter Type** should be set to **Trigger Time**. Select **Always Search Forward for Target Edge**. Set the **Jitter Output** to **Bar Chart**. Click **OK**. The reference signal, `v(ref)`, and the jitter versus time waveforms will now be displayed. If you zoom in, the waveforms will look similar to the following figure:

Figure 38 WaveView: Jitter vs. Time measurement to get Timing Interval Error (TIE) Jitter vs. Time. Measurement based on the reference signal



5. To get the jitter spectrum, click **FFT/DFT** in the Utilities section of the **Waveform** tab. In the FFT/DFT Parameters window, set the **# of points** to 1024. Deselect **Sampling Rate**. The box next to **Stop** will now be checked. Enter 260n for **Stop**.

Select **Spectrum Panel** for **Display Type** and click **OK**. The FFT of the jitter will be displayed in a new WaveView window.

6. Change the Y-axis to dB10 by right clicking the Y-axis and selecting **Plot Complex > dB10(mag)** from the menu.
7. Change the spectrum to lines by right clicking inside the waveview window and selecting **Configure Spectrum** from the menu. In the Spectrum Settings window, select **Draw Lines** and click **OK**. The jitter spectrum looks similar to the following figure:

Figure 39 FFT of the TIE Jitter samples show the Jitter Spectrum



Autocorrelation Function Example

The autocorrelation function is given by the following equation:

$$R(\tau) = \int_{-\infty}^{\infty} f(t)f(t+\tau) dt$$

The example netlist used to illustrate the measurement of TIE jitter can also be used to illustrate the use of the autocorrelation function. To activate the autocorrelation calculation for output terminals, the transient noise analysis statement is:

```
.trannoise v(outb26) FMAX=20G SCALE=10 autocorrelation=1
```

The output data from the autocorrelation function calculation is output to an ASCII formatted file with the extension `.trnz#`. The autocorrelation function can be viewed in

WaveView by adding a `.PROBE` statement to probe `ONoise`. In the example netlist, the probe statement is:

```
.PROBE trannoise onoise
```

For the `clockbuf4.sp` example, the autocorrelation file is named `clockbuf4.trnz0` and it contains the output `v(outb26)` and the autocorrelation function of the output, `autocorr(outb26)`. The following figure shows the output signal and the autocorrelation function plotted in WaveView and zoomed in to show more detail of the waveforms.

Figure 40 Autocorrelation Measurement



Phase Noise and Jitter from Transient Noise

Phase noise and jitter characterization of closed loop PLLs can be difficult using the HB or SN simulation algorithms. The PLL must be in steady-state and this requires that the PLL must be simulated well beyond the transient start-up period. Because of multiple time scales due to dividers that are present in the typical PLL, the HB and SN algorithms may have convergence issues. With additional processing, phase noise and jitter measurements can be extracted from the transient noise simulation.

Extracting the phase noise of a PLL from transient noise analysis can reveal noise shaping and spurious responses. Two methods are available in transient noise analysis to characterize the phase noise and jitter. Additionally, the number of window segments, used by the spectral density estimation algorithm in transient noise analysis, can be set using

the option `TRNOISESEGMENTS`. Setting the option results in a smoother frequency domain spectral density response.

The following sections discuss these topics:

- [Delay Line Method](#)
- [Phase Detector Method](#)
- [Phase Noise Output](#)
- [Jitter Output](#)
- [Recommendations](#)

Delay Line Method

Phase noise can be measured by multiplying a signal by the delayed version of itself. This results in a spectral density of frequency fluctuations (FM noise) that is then converted to phase noise.

$$L(f_m) = (S_{\Delta f_m}(f_m)) / (2f_m^2) \approx (\Delta V_{rms}^2(f_m)) / (2K_\phi 2\pi f_m \tau)^2$$

The advantages of this method are less sensitivity to reference frequency errors and the requirement of a smaller total transient time (`TSTOP`). The delay line method is better for measuring the phase noise of VCO circuits.

Using Delay Line Method

The delay line method is selected by setting `phasenoise=1` in the transient noise command.

A reference source definition is required to establish the phase reference for phase noise and jitter calculations. PrimeSim HSPICE now provides the capability to automatically compute the frequency for use as the reference for phase noise and jitter measurements. The automatic frequency detection algorithm is based on the time-frequency analysis of signals based on the Ramanujan Fourier Transform expansion, which has special properties in its ability to estimate frequency content as a function of time (see [7]).

To activate the automatic frequency detection algorithm, set `REF=auto` in the `.TRANNOISE` command. Alternatively, you can also specify a reference source present in the netlist or a frequency value (in Hz), for use as the reference for phase noise and jitter measurements. For detailed information on using `REF` for `.TRANNOISE`, see `.TRANNOISE` in *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Jitter can be measured and probed by setting `jitter=1`.

```
.option RUNLVL=5          $ for phase accuracy
.tran 10p 42u             $ TSTOP=42u
```



```
Vref refn gnd SIN (0.5 0.5 'VCO_ref')      $ reference
Rref refn gnd 1k
.option TRNOISESEGMENTS=4      $ smoothing
.trannoise v(vco_out) FMAX=4G
+ start=2u                      $ noise begins at 2u
+ ref=Vref                      $ specify reference source
+ phasenoise=1                  $ use delay line method
+ jitter=1                      $ extract jitter from phase noise
.probe trannoise phnoise        $ probe phase noise
.probe trannoise jitter         $ probe TIE jitter
```

Phase Detector Method

The phase detector method compares the zero crossings between the noisy signal and an ideal sinusoidal reference signal, making a direct measurement of phase fluctuations and their spectral density.

$$L(f_m) = \frac{1}{2} (S_{\phi}(f_m))$$

This method does not require calibration of the phase detector gain ($K\phi$) and is immune to DC offsets, AM noise, and delay values. The phase detector method is better for measuring the phase noise of PLL circuits.

Using the Phase Detector Method

The delay line method is selected by setting `phasenoise=2` in the transient noise command. The jitter can be extracted from the phase noise by setting `jitter=2`.

```
.option RUNLVL=5      $ for phase accuracy
.tran 10p 42u         $ TSTOP=42u
Vref refn gnd SIN (0.5 0.5 'frq_ref*divideN')  $ reference
Rref refn gnd 1k
.option TRNOISE_SEGMENTS=4      $ smoothing
.trannoise v(xmc_top.mc_clk_vco_out)
+ samples=1 seed=2 FMAX=4G
+ start=2u                      $ noise begins at 2u
+ ref=Vref                      $ specify reference source
+ phasenoise=2                  $ use phase detector method
+ jitter=2                      $ extract jitter from phase noise
.probe trannoise phnoise        $ probe phase noise
.probe trannoise jitter         $ probe TIE jitter
```

Phase Noise Output

The keyword `phnoise` specifies the phase noise for the whole circuit. The phase noise value for a circuit element can be specified using the `phnoise(element_name)` keyword.

```
.PRINT TRANNOISE phnoise phnoise(element_name)
.PROBE TRANNOISE phnoise phnoise(element_name)
```

In this syntax, the standalone `phnoise` is the phase noise parameter.

For example: `.PROBE TRANNOISE phnoise.`

When phase noise is extracted from transient noise analysis, the phase noise data is output to the `*.trpn#` file when `.OPTION POST` is set and `.PROBE` statements are used. The phase noise data is output to the `*.printtrpn#` file when `.PRINT` statements are used. The unit of the phase noise data is decibels, relative to the carrier signal, per hertz, across the output nodes specified in the `.TRANNOISE` statement (Units: dBc/Hz). The phase noise data plot is a function of the offset frequency.

Jitter Output

The timing jitter calculations are derived from the results of phase noise analysis. The phase noise output syntax supports the `JITTER` keyword as an output keyword in addition to the `PHNOISE` keyword.

```
.PRINT TRANNOISE PHNOISE JITTER  
.PROBE TRANNOISE PHNOISE JITTER
```

When jitter is extracted in addition to phase noise from transient noise analysis and the `JITTER` keyword is present in the print and probe statements, the `.TRANNOISE` analysis also outputs the raw jitter data to the `*.trjt0` file when `.OPTION POST` is set and `.PROBE` statements are used. The raw jitter data is output to the `*.printtrjt0` file when `.PRINT` statements are used. The jitter data is the TIE timing jitter (Accumulated Jitter) and is a function of time in units of seconds.

For a discussion on the different jitter measurements available in PrimeSim HSPICE, see [Jitter Definitions on page 115](#).

Recommendations

The following recommendations should be followed to get the best phase noise and jitter results from transient noise analysis.

When simulating a PLL, it is important to know when the PLL has reached the steady-state. It is recommended that the `START` parameter in the transient noise command is used to activate noise calculations, at a time after the transient analysis has reached steady-state and after the PLL loop dynamics have stabilized. This will speed up the simulation and prevent errors in measuring phase noise and jitter caused by a PLL that has not yet stabilized to its steady-state output frequency.

It is important to know the PLL or VCO output frequency since a `SIN` or `PULSE` source that operates at the PLL or VCO output frequency is used as a reference source for the phase noise and jitter calculations.

To get the low offset frequency phase noise response, a large `TSTOP` value in the transient noise command is required. The recommended `TSTOP` value is given as:

$$TSTOP \approx \text{Smoothing} / \text{lowest_frequency}$$

Where the phase noise smoothing is controlled by the option `TRNOISESEGMENTS`.

Use caution when using transient noise analysis for open-loop oscillators because low-Q VCOs may have considerable frequency variations due to unbounded phase drifts. With unstable frequencies, the phase-detector method can give erroneous results. Hence, for open-loop oscillators, the delay line method is a better alternative. For most open-loop VCOs, the best phase noise analysis is `.PHASENOISE` based on `.SNOSC` or `.HBOSC` analysis.

The `.PHASENOISE` analysis is designed to be used with autonomous (non-driven) circuits using `.SNOSC` and `.HBOSC`, since it uses frequency domain methods to achieve faster speed and better dynamic range than time domain methods. In addition, special windowing functions are used during the spectral estimation phase to reduce the numerical noise.

Because insufficient accuracy settings may cause numerical noise to look like phase noise or jitter, consider checking the dynamic range of the noise measurements. To do this, set `SEED=1` in the transient noise command to test noise measurements with noise disabled. The result will be the numerical noise floor. To ensure the accuracy of the noise results, look for 10dB *noise to noise* ratio. One should always expect a numerical noise floor limit in measurements.

Power Spectral Density from Transient Noise

An important application of transient noise analysis is to generate output spectra of circuit waveforms that include electronic noise. In cases where transient noise analysis results are being compared to the PrimeSim HSPICE small signal noise (`.NOISE`) analysis results, the desired output measurements are in terms of the noise power spectral density or PSD. The required outputs are frequency-domain density responses in the units of V^2/Hz and A^2/Hz (or $V/\text{sqrt}(\text{Hz})$ and $A/\text{sqrt}(\text{Hz})$) and the total noise effects are computed by integrating the density values.

To compute PSD values as a function of frequency from time-domain signals, PrimeSim HSPICE uses special spectral estimation techniques. In these techniques, the estimation of the noise output level is considered equally important as the signal level, to extract basic frequency-domain noise power spectral density output measurement from Monte Carlo transient noise analysis.

Using PSD from Transient Noise

Power spectral density can be extracted by adding the keyword `PSD=1` to the transient noise analysis command.

An important PrimeSim HSPICE option that can be used to control the PSD calculations is:

```
.option TRNOISESEGMENTS=int
```

Where the value *int* represents the number of window segments used by PrimeSim HSPICE for the spectral density estimation algorithm. As this value is increased, additional Welch noise averaging is used to provide a smoother frequency domain noise density response. In addition, the `.TRANNOISE START` parameter value is used to identify when the circuit has reached steady state and as the starting time for performing PSD calculations.

```
.option runlvl=5  
.option trnoisesegments=20  
.tran 10p 10u  
.trannoise v(out) fmax=10g psd=1  
.probe trannoise onoise(m)
```

Stochastic Differential Equation Analysis

Transient noise analysis predicts waveform statistics at particular time points. One method to accomplish this is to run the multi-sample Monte Carlo approach and then use the ensemble of simulation results to generate histograms at the time points of interest. Histograms that measure vertical distributions reveal voltage noise. Histograms that measure horizontal time-shift distributions reveal jitter. In those cases where it is desirable to have a highly accurate distribution curve, it may be necessary to simulate with a large number of Monte Carlo samples. Stochastic differential equation (SDE) analysis can provide a more efficient alternative in such cases. Instead of requiring a large ensemble of results, SDE performs special calculations that directly predict the statistics of the output waveforms.

Let the time-domain signal resulting from a regular transient analysis (with noise ignored) for a specific output node be written as $v_{out}^s(t)$.

Let the time-domain signal resulting from an analysis with signal and noise for the same output node be written as $v_{out}^{s+n}(t)$.

We can define the noise voltage component $v_{out}^s(t)$ to be:

Equation 54

$$v_{out}^n(t) = v_{out}^{s+n}(t) - v_{out}^s(t)$$

We can define a variance equal to the expected value of this noise component at a given time to be:

Equation 55

$$\sigma_n^2(t) = v_{out}^n(t) \cdot v_{out}^n(t)$$

If we assume that the noise variations are small, we can create a linear Stochastic Differential Equation (SDE) for the noise contribution vector $x_n(t)$ (one entry in the vector being the output noise $v_{out}^n(t)$). This SDE may be formulated in terms of time-varying coefficient matrices that are evaluated for a normal transient analysis, which are functions of the noise-free solution vector $x_s(t)$ and derived from the noise-free transient analysis computations [1]:

Equation 56

$$A(t)x_n + C(t)\dot{x}_n + B(t)v \cong 0$$

$$x_n(0) = x_{n,0}$$

We can then create and solve a linear ordinary differential equation (ODE) system for the time-varying noise correlation matrix $K(t) = x_n(t)x_n(t)^T$:

Equation 57

$$\dot{K}(t) = E(t)K(t) + F(t)F(t)^T$$

where $E(t)$ and $F(t)$ are derived from $A(t)$, $B(t)$, and $C(t)$, and T indicates the transpose operation.

In general, simultaneously solving for both the deterministic and stochastic differential equations can give us the complete time-dependent output signal waveform vector $x_s(t)$, as well as the complete time-varying noise correlation/covariance matrix $K(t) = x_n(t)x_n(t)^T$. The entries in this matrix represent time-dependent variance values $\sigma_n^2(t)$ for output signals $v_{out}^n(t)$. This output can be interpreted and plotted as a time-varying RMS noise voltage waveforms for $v_{RMS}^n(t) = \sqrt{\sigma_n^2(t)}$. The results of such a transient + noise analysis include the usual deterministic transient analysis waveforms, including the mean voltage output $v_{out}^s(t)$ and also its (stochastic) time-varying RMS noise component $v_{RMS}^n(t)$. In this sense, the SDE analysis method provides typical SPICE output waveforms for circuit unknowns, plus the additional waveform representing the time-varying statistics of the circuit. The `.TRANNOISE` SDE method therefore activates this special analysis and makes this output available to the user.

Stochastic Differential Equation (SDE) dynamic noise statistics are computed in the form of a time-varying covariance matrix. SDE techniques allow the output of a variance

waveform for any output signal. Variance waveforms can be used to construct probability density plots.

The SDE approach gives probability density information and, like the multi-run Monte Carlo approach, is useful for characterizing statistics at specific time events.

The following topics are discussed in these sections:

- [SDE Output Data](#)
- [SDE Examples](#)

SDE Output Data

The `.TRANNOISE` SDE analysis outputs raw data to the `*.tr0` and `*.printtr0` files consistent with transient analysis. The `.PRINT/.PROBE` output syntax supports the following measurements:

```
.print trannoise ONOISE ONOISE(M) VRMS(n1[,n2])
.probe trannoise ONOISE ONOISE(M) VRMS(n1[,n2])
```

where:

- The `ONOISE` and `ONOISE(M)` outputs are the same. They represent noise voltage or current at the node or branch specified by the `output` keyword. The `ONOISE` represents the RMS noise voltage component (square root of the variance), units in Volts, of the noise at the specified output present in addition to the noise-less transient voltage.

Note:

`ONOISE` is only output when you use the SDE method; when using the Monte Carlo method, `ONOISE` is 0.

- `VRMS`: The output of RMS noise voltages at other nodes (that is, the output for general nodal noise voltage values).

The actual instantaneous output voltage is the sum of the signal plus noise components:

Equation 58

$$v_{out}^{s+n}(t) = v_{out}^s(t) + v_{out}^n(t)$$

Where the noise component $v_{out}^n(t)$ has an assumed Gaussian distribution (in x) as:

$$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma^2}}$$

And the output signal $v_{out}^s(t)$ is that resulting from the (deterministic) `.TRAN` analysis. The time-varying RMS noise voltage waveforms (for `onoise`) are related

to the variance at the specified outputs as given by $v_{RMS}^n(t) = \sqrt{\sigma_n^2(t)}$ where:

$$\sigma_n^2(t) = v_{out}^n(t) \cdot v_{out}^n(t) .$$

SDE Examples

Example 1. SDE method with maximum frequency of 5GHz.

```
.TRANNOISE v(7) METHOD=SDE FMAX=5g
```

Example 2. Activates SDE noise analysis and dumps the `ONoise` output to the `*.tr0` file:

```
.TRANNOISE v(out) METHOD=SDE  
.PROBE TRANNOISE ONoise
```

Example 3. Activates SDE noise analysis placing a lower bound on flicker noise to be 10kHz and an upper bound on all noise power at 100MHz:

```
.TRANNOISE v(out) METHOD=SDE FMIN=10k FMAX=100MEG
```

Error Handling, Error Recovery, Status Reporting

The following error checks are made with transient noise analysis:

- Verify that the specified output node exists.
- Verify non-negative integer values for “list” entries.

Note:

Invalid values for `FMIN` and `FMAX` will be ignored and default values will be used.

The SDE solving can be substantially slower than transient analysis. Status reporting includes the progress of the noise analysis.

References

- [1] A. Demir, E.W.Y. Liu, A.L. Sangiovanni-Vincentelli, “Time-domain non-Monte Carlo noise simulation for nonlinear dynamic circuits with arbitrary excitations,” *IEEE Trans. CAD*, vol. 15, no. 5. May 1996.
- [2] P. Bolcato and R. Poujois, “A new approach for noise simulation in transient analysis”, *Proc. IEEE Int. Symp. Circuits Syst.*, pp. 887-890, May 1992.
- [3] J. A. McNeill, “Jitter in ring oscillators,” Ph.D. dissertation, Boston University, 1994.
- [4]

- M. Okumura and H. Tanimoto, "A time-domain method for numerical noise analysis of oscillators", Proc. IEEE Asia Pacific Design Automation Conference, 1997.
- [5] A. van der Ziel, Noise in Solid State Devices and Circuits, John Wiley & Sons, 1986.
- [6] A. Hajimiri, S. Limotyrakis, and T.H. Lee, "Jitter and phase noise in ring oscillators," IEEE J. Solid-State Circuits, vol. 34, no. 6, pp. 790-804, June 1999.
- [7] P.P. Vaidyanathan, "Ramanujan sums in the context of signal processing", IEEE Trans. Signal Processing, vol. 62, no. 16, August 15, 2014.

13

Periodic Time-Dependent Noise Analysis (.PTDNOISE)

Describes periodic time-dependent noise analysis.

This chapter discusses these topics:

- [Introduction to Periodic Time-Dependent Noise Analysis](#)
- [PTDNOISE Input Syntax](#)
- [PTDNOISE Output Syntax and File Format](#)
- [Errors and Warnings](#)
- [Usage Example](#)
- [Troubleshooting .PTDNOISE Simulation](#)
- [References](#)

Introduction to Periodic Time-Dependent Noise Analysis

While HBNOISE and SNNOISE calculate a time-averaged power spectral density, there are applications where a characterization of the time-dependence of the noise is required. These applications include the computation of the jitter associated with a noisy signal crossing a threshold and the computation of the noise associated with the discretization of an analog signal, which computes the noise in a periodically-driven circuit at a point in time. Periodic Time-Dependent noise analysis (PTDNOISE) calculates the noise spectrum and the total noise at a point in time. Jitter in a digital threshold circuit can then be determined from the total noise and the digital signal slew rate.

Circuits driven by large periodic signals produce cyclostationary noise, that is, the noise characteristics are periodic in time. Cyclostationary noise can be characterized in several ways, with the particular application determining which is appropriate.[\[1\]](#) The time-average power spectral density (PSD) ignores frequency correlations in the noise, but is adequate when the fundamental frequency of the cyclostationary noise is much larger than the

bandwidth of interest. The time-average PSD is calculated in the HBNOISE/SNNOISE analyses. [2]

The harmonic power spectral density (HPSD) or equivalently, the auto-correlation function, $R(t_1, t_2)$, contains the correlation information between noise sidebands. This information is necessary to build behavioral cyclostationary noise sources and to separate the amplitude modulation (AM) and the phase modulation (PM) noise components. (See [Amplitude Modulation/Phase Modulation Separation](#) for more information.)

The time-dependent power spectral density (TDPSD) can be integrated over frequency to yield the time-dependent noise (TDN). TDN can then be used to determine the jitter associated with a noisy signal crossing a threshold. PTDNOISE analysis allows the calculation of TDPSD, TDN, and jitter. In addition, you can calculate both the time-domain power spectral density (TDSN) and the integrated noise (time-dependent noise, TDN) at multiple time points.

By measuring the jitter associated with a noisy signal crossing a threshold, the jitter is modeled by displacing the time in a noise free signal $v(t)$ with a stochastic process $j(t)$.

Equation 59

$$V_{jitter}(t) = v(t + j(t))$$

We can also determine the voltage at this node including the time-dependent noise $n(t)$:

Equation 60

$$Vn(t) = v(t) + n(t)$$

By equating these two representations, expanding in a Taylor series, and dropping higher order terms, as follows:

Equation 61

$$V(t) + n(t) = v(t + j(t)) = v(t) + dv(t) / dt \cdot j(t) + \dots$$

Equation 62

$$N(t) = dv(t) / dt \cdot j(t)$$

In terms of variances, jitter is then defined as:

Equation 63

$$Var(j(t)) = n^2(t) / (dv(t) / dt)^2$$

PTDNOISE Input Syntax

```
.PTDNOISE outvar [outvar2 ... outvarN] TIME=[val|meas|sweep]
+ [TDELTA=time_delta]
```

```
+ frequency_sweep
+ [listfreq=(frequencies|none|all)] [listcount=val]
+ [listfloor=val] [listsources= on|off]
+ [TOL=value]
+ [KRYLOVDIM=value]
+ [KRYLOVITER=value]
```

Parameter	Description
<i>outvar</i> [<i>outvar2</i> ... <i>outvarN</i>]	Nodal voltage or branch current output variables. Define the nodes or branches at which the PrimeSim HSPICE simulator sums the noise. You can specify an arbitrary number of output variables. However, you must specify at least one <i>outvar</i> .
TIME	Time point at which time domain noise is evaluated. Specify either a time point explicitly, such as: TIME= <i>value</i> , where <i>value</i> is either numerical or a parameter name or a .MEASURE name associated with a time domain .MEASURE command located in the netlist. .PTDNOISE uses the time point generated from the .MEASURE command to evaluate the noise characteristics. This is useful if you want to evaluate noise or jitter when a signal reaches some threshold value.
TDELTA	A time value used to determine the slew rate of the time-domain output signal. Specified as TDELTA= <i>value</i> . The signal slew rate is then determined by the output signal at TIME +/- TDELTA and dividing this difference by 2 x TDELTA. This slew rate is then used in the calculation of the strobed jitter. If this term is omitted a default value of 0.01 x the .SN period is assumed.
<i>frequency_sweep</i>	Specifies the <i>type</i> , <i>nsteps</i> or <i>npoints</i> and <i>fstart</i> and <i>fstop</i> frequency or frequency points for each sweep type, where: • <i>type</i> = the frequency sweep type which can be OCT, DEC, LIN, POI, or SWEEPBLOCK. • LIN <i>npoints</i> <i>fstart</i> <i>fstop</i> • OCT <i>nsteps</i> <i>fstart</i> <i>fstop</i> • DEC <i>nsteps</i> <i>fstart</i> <i>fstop</i> • POI <i>npoints</i> <i>p1</i>, <i>p2</i>, ...<i>pn</i> • SWEEPBLOCK = <i>swblockname</i> • Where: • <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). • <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). • <i>fstart</i> = starting frequency. • <i>fstop</i> = ending frequency. • <i>p1</i>, <i>p2</i>, ... <i>pn</i> = frequency points.

Parameter	Description
listfreq	Prints the element noise value to the <code>.lis</code> file. This information is only printed if a noise spectrum is requested in a PRINT or PROBE statement. (See PTDNOISE Output Syntax and File Format.) You can specify which frequencies the element noise is printed. The frequencies must match the sweep frequency values defined in the <code>frequency_sweep</code>, otherwise they are ignored. In the element noise output, the elements that contribute the largest noise are printed first. The frequency values can be specified with the NONE or ALL keyword, which either prints no frequencies or every frequency defined in <code>frequency_sweep</code>. Frequency values must be enclosed in parentheses. For example: • <code>listfreq=(none)</code> • <code>listfreq=(all)</code> • <code>listfreq=(1.0)</code> • <code>listfreq=(1.0G, 2.0G)</code> The default value is NONE.
listcount	Prints the element noise value to the <code>.lis</code> file, which is sorted from the largest to smallest value. You do not need to print every noise element; instead, you can define <code>listcount</code> to print the number of element noise frequencies. For example, <code>listcount=5</code> means that only the top 5 noise contributors are printed. The default value is 1.
listfloor	Prints the element noise value to the <code>.lis</code> file and defines a minimum meaningful noise value (in $V/Hz^{1/2}$ units). Only those elements with noise values larger than <code>listfloor</code> are printed. The default value is $1.0e-14 V/Hz^{1/2}$.
listsources	Prints the element noise value to the <code>.lis</code> file when the element has multiple noise sources, such as a MOSFET, which contains the thermal, shot, and 1/f noise sources. You can specify either ON or OFF: ON prints the contribution from each noise source and OFF does not. The default value is OFF. When <code>listsources</code> is turned on, the element noise source contributions will also be output into <code>*.snptn#</code> file.
TOL	Same as .OPTION PHASENOISETOL.
KRYLOVDIM	Same as .OPTION PHASENOISEKRYLOVDIM / PHASENOISE_KRYLOV_DIM.
KRYLOVITER	Same as .OPTION PHASENOISEKRYLOVITR / PHASENOISE_KRYLOV_ITR.

PTDNOISE Output Syntax and File Format

PTDNOISE output syntax allows for the output of a single parameter: `onoise`, where `onoise` is the noise voltage spectral density at each frequency point specified by the `frequency_sweep` keyword for the time points specified by the `TIME` keyword. The units are $\frac{V}{\sqrt{Hz}}$.

```
.PROBE PTDNOISE onoise
.PRINT PTDNOISE onoise
```

Parameter	Units	Description
onoise	$\frac{V}{\sqrt{Hz}}$	Noise voltage spectral density at each frequency point specified by <i>frequency_sweep</i> at the time point specified by <i>time_value</i>

Output File Format

The following PTDNOISE output files are generated depending on the user input:

File	Description
*.printptn#	Writes output from the .PRINT statements when using HB to obtain the steady state solution.
*.ptn#	Outputs waveform data from the .PROBE statements when using HB to obtain the steady state solution.
*.printsnpn#	Writes output from the .PRINT statements when using SN to obtain the steady state solution.
*.snptn#	Outputs waveform data from the .PROBE statements when using SN to obtain the steady state solution.

Caution:

When multiple output variables are specified, multiple *.printptn#, *.ptn#, *.printsnpn#, and *.snptn# files are generated, each with one output variable name in its file name as in *.printptn#@outvar, *.ptn#@outvar, *.printsnpn#@outvar, and *.snptn#@outvar.

*.lis	Standard output file *.lis contains the following information: • Performance Statistics Log • Number of Nodes • Number of FFT Points • Number of Equations • Memory in use • Maximum Krylov iterations • Maximum Krylov Dimension • Target GMRES Residual • Gmres Residual • Actual Krylov Iterations taken • Frequency (swept input frequency values) Noise source contributions are listed sequentially and are controlled by the PTDNOISE command line parameters: listtime, listfreq, listcount, listfloor, and listsources.
-------	--

.MEASURE Syntax and File Format

The .MEASURE PTDNOISE command gives a special combination measurement using the keyword STROBEJITTER, to provide integrated noise, slew-rate, and jitter as a function of the PTDNOISE time sweep.

The syntax for .MEASURE PTDNOISE is:

```
.MEASURE PTDNOISE result STROBEJITTER onoise freq_sweep  
+[PRINT= 0|1]
```

Argument	Description
result	Name associated with the calculated value of strobe jitter which is the integrated value of the <i>onoise</i> noise voltage (integrated over the default or specified <i>freq_sweep</i> range), divided by the slew-rate. This result is computed for each time value specified in the PTDNOISE command. The results should be interpreted as periodic (strobed), with time values relative to one period. Unit: seconds. Note: The STROBEJITTER keyword will cause the parameters listed below to also be included in the *.msnptn# output file: - timepnt : Time value associated with the strobe jitter output. Unit: seconds. - integptdnoise: The <i>onoise</i> noise voltage integrated over the specified frequency sweep. Unit: V. - slewrate: The slew-rate of the steady-state output waveform at the time value associated with the strobe jitter output. Unit: V/seconds.
PRINT	- print=0 prevents printing a measure result into the measure output file. - print=1 (Default) prints the measure result into the output file.

Note:

.MEASURE PTDNOISE is ignored when a TIME=sweep is specified in the netlist and a warning message is issued.

Measure File Format

File	Description
*.msnptn#	Contains output from the .MEASURE statement when using .SN to obtain the steady state solution.

Errors and Warnings

Error messages are generated under the following circumstances:

- PTDNOISE frequency sweep includes negative frequencies. PTDNOISE allows only frequencies that are greater than or equal to zero.
- PTDNOISE time sweep includes negative times. PTDNOISE allows only time points that are greater than or equal to zero.
- No SN statement is specified (error at parser). PTDNOISE requires a SN statement to generate the steady-state solution.
- Incorrect match to .MEASURE statement.

A warning is issued for a PTDNOISE convergence failure. When the GMRES solver reaches the maximum number of iterations and the residual is greater than the specified tolerance, PTDNOISE generates a warning and then continues as if the data were valid. The warning reports the following information:

- Final GMRES residual
- Target GMRES residual
- Maximum Krylov iterations
- Actual Krylov iterations taken

Usage Example

The following test case illustrates the PTDNOISE analysis for a simple inverter.

```
* Simple RC + Inverter - rcInvPTDNoise.sp
* rrd Jan 03, 2007
* Simulates PSD(t,f) of a simple inverter
* sweep time points
.param f0 = 5.0e8
.sn tones=f0 nharms=4 trinit=10n
.PTDNOISE v(out1) TIME=lin 3 0 2n TDELTA=.1n dec 5 1e5 1e10
+ listfreq=(1e6,1e8)
+ listcount=1
+ listsources=ON

.MEASURE PTDNOISE strobejit STROBEJITTER onoise FROM = 1e4 TO = 1e10
$.measure SN t1 trig AT=0 targ v(out1) val=1.5 fall=1

.opt post
.probe ptdnoise onoise
.print ptdnoise onoise
```

```
.probe sn v(out1)

vd      vdd 0    3.0
.global vdd
vgate in0 0    COS(1.5 1.4 'f0' 0 0 0)
rin      in0 in1 50
rout     out1 0    .1g

xol in1 out1 inv

.subckt inv in out
m1 out in 0 0 n l=350e-9 w=4.5e-6
m2 out in vdd vdd p l=350e-9 w=4.5e-6
.ends

.MODEL N NMOS
+Level= 49 Tnom=27.0 version =3.1 TLEV= 1
*
***

.MODEL P PMOS
+Level= 49 Tnom=27.0 version =3.1 TLEV= 1

.end
```

Troubleshooting .PTDNOISE Simulation

When performing jitter measurements on a clock buffer using PTDNOISE analysis, you may get unusual results. If you define noise frequencies that are multiples of your input clock, you get large values in your jitter measurement. When you sweep through frequencies that are multiples of the clock, PTD noise up-converts $\text{frequency}=0$ flicker noise. (These contributions can be large, since flicker noise typically has a freq^n dependence where $n \cong 1$.) For this reason you should avoid sweeping through multiples of the clock frequency. Also, the flicker noise is limited at very small frequencies so you do not see infinite noise.

References

- [1] J. Roychowdhury, D. Long, and P. Feldmann, "Cyclostationary Noise Analysis of Large RF Circuits with Multitone Excitations," *IEEE Journal of Solid-State Circuits*, volume 33, pages 324–336, March 1998.
- [2] A. Demir, A. Sangiovanni-Vincentelli, "Analysis and Simulation of Noise in Nonlinear Electronic Circuits and Systems", Kluwer Academic, 1998.

Part 6: Behavioral Modeling Applications

Describes the usage of behavioral modeling of nonlinear circuits and laplace transform modeling of filters and networks.

This Part contains the following chapters/topics.

- [Chapter 14, Behavioral Modeling of Nonlinear Circuits](#)
- [Chapter 15, Laplace Transform Modeling of Filters and Networks](#)

14

Behavioral Modeling of Nonlinear Circuits

Describes how to create and use behavioral models.

Behavioral modeling substitutes more abstract, less computationally intensive, circuit models for lower-level descriptions of analog functions. These simpler models emulate the transfer characteristics of the circuit elements that they replace, but with increased efficiency. Behavioral modeling substantially reduces the actual simulation time per circuit. At the level of an entire design and simulation cycle, design efficiency greatly increases, and you can complete a design (from conception to a marketable product) in substantially less time.

PrimeSim HSPICE ships numerous examples for your use; see Behavioral Application Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

This chapter presents the following topics:

- [Behavioral Design Process](#)
- [Using Behavioral Elements](#)
- [Voltage and Current Controlled Elements](#)
- [Modeling with Digital Behavioral Components](#)
- [Calibrating Digital Behavioral Components](#)
- [Analog Behavioral Elements](#)
- [Op-Amps, Comparators, and Oscillators](#)
- [Phase-Locked Loops \(PLL\)](#)
- [References](#)

Behavioral Design Process

PrimeSim HSPICE provides specific modeling elements that promote the use of behavioral and mixed signal techniques. These models include controllable sources that

you can configure, to emulate op-amps, single-input or multi-input logic gates, or any system with a continuous algebraic transfer function.

- You can create these functions in algebraic form, or in the form of coordinate pairs.
- You can use digital stimulus files to enter logic waveforms into the simulation deck, rather than using piecewise linear sources to enter digital waveforms.
- You can define the clock rise times, fall times, periods, and voltage levels.

With PrimeSim HSPICE behavioral models, the typical design cycle for a circuit or system is:

1. Fully simulate a subcircuit, with pertinent inputs, characterizing its transfer functions.
2. Determine which PrimeSim HSPICE elements, singularly or in combination, accurately describe the transfer function.
3. Reconfigure the subcircuit appropriately.
4. After you verify the behavioral model, substitute the model into the larger system, in place of the lower-level subcircuit.

Using Behavioral Elements

Behavioral elements offer a higher level of abstraction and faster processing, compared to a lower-level description of an analog function.

- System-level designers can use function libraries of subcircuits containing these elements, to describe parts such as:
 - Op-amps
 - Vendor-specific output buffer drivers
 - TTL drivers
 - Logic-to-analog converters
 - Analog-to-logic simulator converters
- Integrated Circuit designers can use these elements to reduce the design time, especially when designing filters and signal processors.

Behavioral elements use an arbitrary algebraic equation as a transfer function for either a voltage (E) or a current (G) source. This function can include:

- Nodal voltages
- Element currents

- Time
- Other parameters which you define

A good example of this is a VCO, where `control` is the input voltage node, and `osc` is the oscillator output:

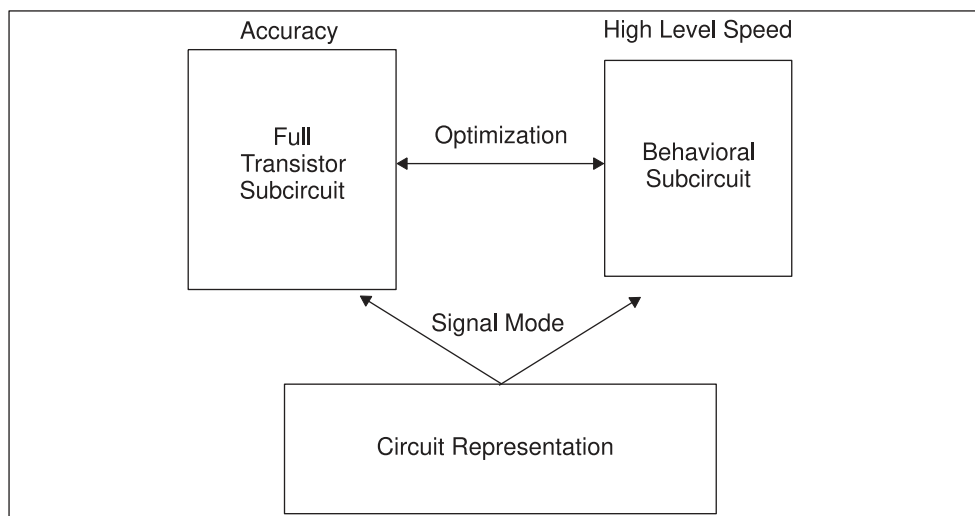
```
Evco osc 0 VOL='voff+gain*\\  
SIN(6.28*freq*(1+V(control))*TIME) '
```

You can use subcircuits to encapsulate a function.

- If you split the function definition from the use, you create a hierarchy.
- If you pass parameters into the subcircuit, you create a parameterized cell.
- If you create a full transistor cell library and a behavioral representation library, you can include mixed-signal functions within PrimeSim HSPICE.

You can use the built-in `OPTIMIZE` function to calibrate the behavioral elements from a full transistor circuit.

Figure 41 Netlisting by Signal Mode



The following sections discuss these topics:

- [Controlled Sources](#)
- [Libraries](#)

Controlled Sources

Controlled sources model both analog and digital circuits, at the behavioral level.

This reduces simulation times for mixed signals and models system-level operations.

Controlled sources also model gate-switching action for the behavioral modeling of digital circuits. For analog behavioral modeling, you can program the controlled sources as mathematical functions. These functions can be either linear or non-linear, depending on other nodal voltages and branch currents.

Libraries

The Discrete Device Library contains standard industry IC components. You can use this library to model board-level designs that contain any of the following:

- Transistors
- Diodes
- Opamps
- Comparators
- Converters
- IC pins
- Printed circuit board traces
- Coaxial cables

You can also model drivers and receivers, to analyze transmission line effects, power line noise, and signal line noise.

Voltage and Current Controlled Elements

PrimeSim HSPICE provides two voltage-controlled and two current-controlled elements, known as E, F, G, and H-elements. For a description of these elements, see Independent Sources and Stimuli, in *PrimeSim HSPICE® User Guide: Elements*.

Modeling with Digital Behavioral Components

The following sections show how to model using digital behavioral components:

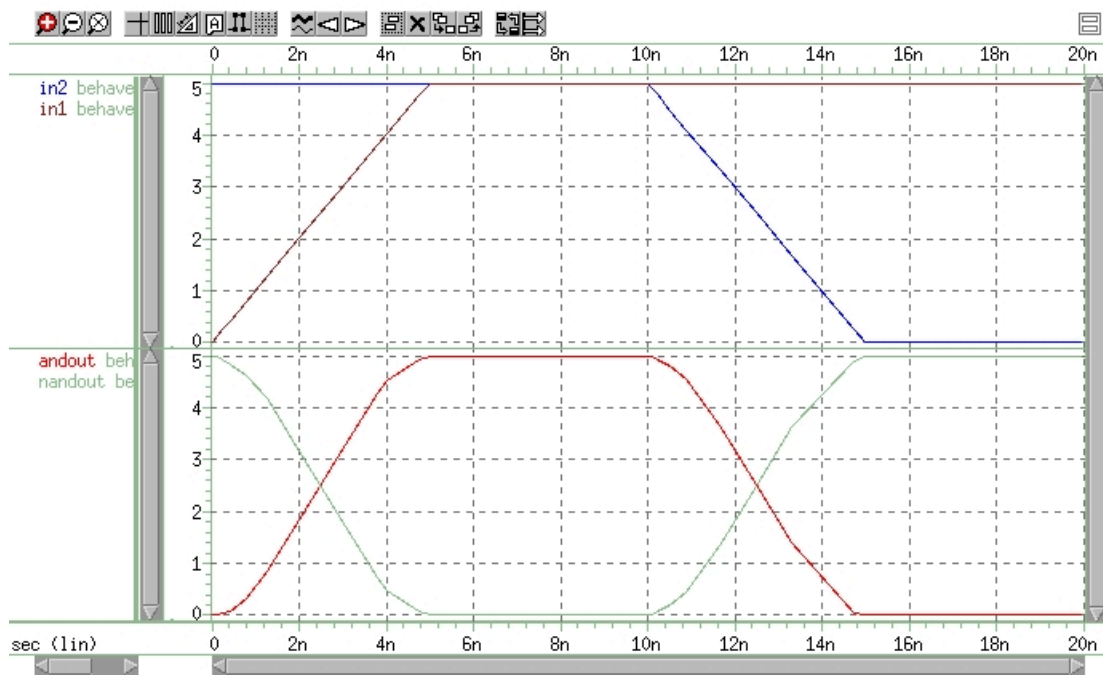
- [Behavioral AND and NAND Gates](#)
- [Behavioral D-Latch](#)
- [Behavioral Double-Edge Triggered Flip-Flop](#)

Behavioral AND and NAND Gates

The following example uses a G Element to model a 2-input AND gate. An E-element models a two-input NAND gate. [Figure 42](#) shows the resulting waveforms. This example is located in the following directory:

```
$installdir/demo/hspice/behave/behave.sp
```

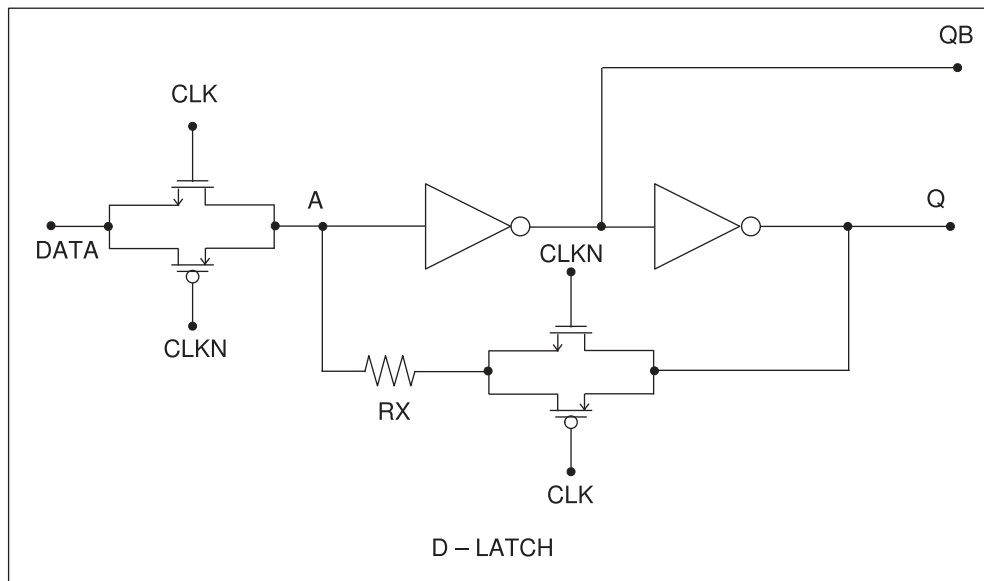
Figure 42 NAND/AND Gates



Behavioral D-Latch

This example uses one input NAND gate and NPWL/PPWL functions, to model a D flip-flop.

Figure 43 D-Latch



Example

This example is located in the following directory:

```
$installdir/demo/hspice/behave/dlatch.sp
```

The file contains the following examples:

- Waveforms
- Subcircuit definitions for Behavioral N-Channel MOSFET
- Behavioral P-Channel MOSFET

Figure 44 D-Latch Response



Behavioral Double-Edge Triggered Flip-Flop

This example uses the D_LATCH subcircuit from the previous example and several NAND gates, to model a double-edged, triggered flip-flop.

Example

This example is located in the following directory:

```
$installdir/demo/hspice/behave/det_dff.sp
```

- Main Circuit
- Subcircuit Definitions

Figure 45 Double-Edge Triggered Flip-Flop

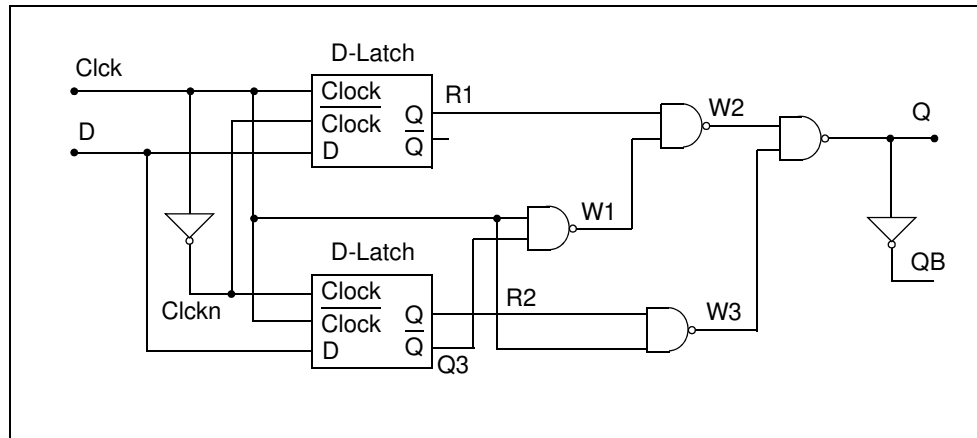


Figure 46 Double Edge Triggered Flip-Flop Response



Calibrating Digital Behavioral Components

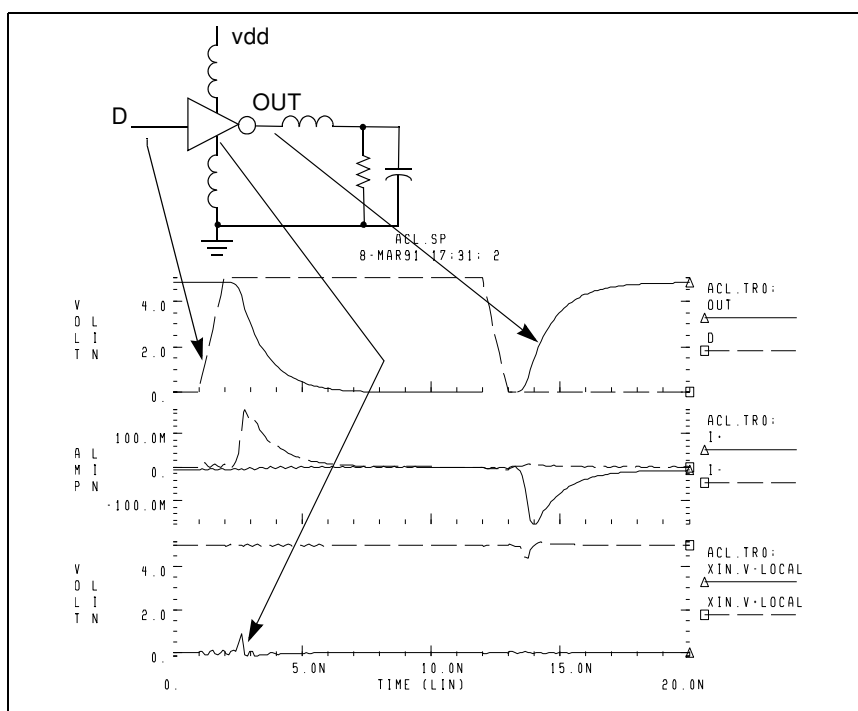
The following sections discuss these topics:

- [Building Behavioral Lookup Tables](#)
- [Optimizing Behavioral CMOS Inverters](#)
- [Optimizing Behavioral Ring Oscillators](#)

Building Behavioral Lookup Tables

The following simulation demonstrates an ACL family output buffer, with 2ns delay, and 1.8ns rise and fall time. It also shows ground and VDD supply currents, and internal ground bounce due to the package.

Figure 47 ACL Family Output Buffer



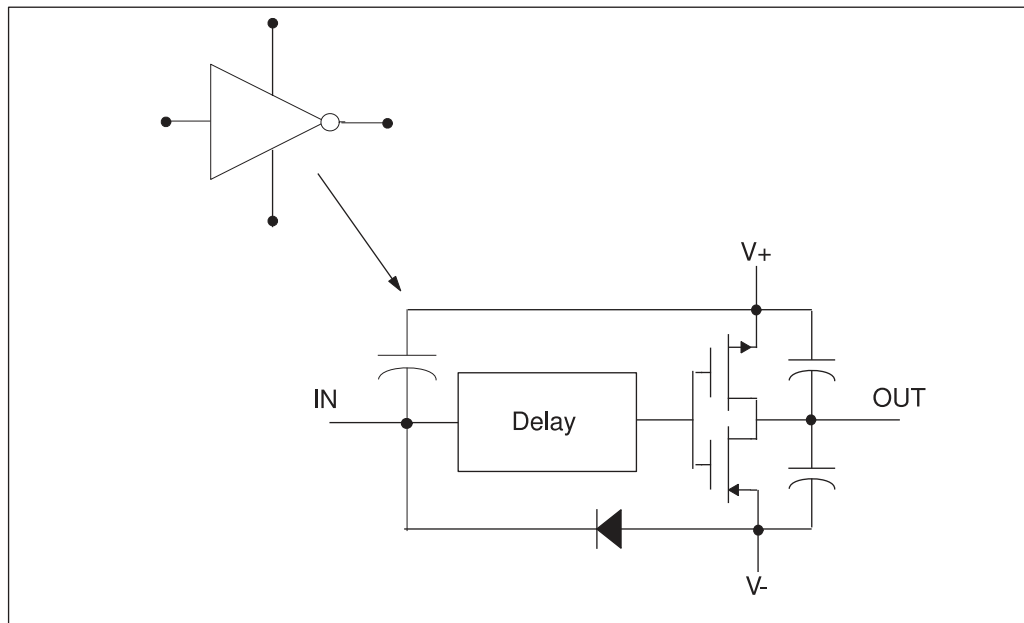
The following commands automatically measure the datasheet quantities, such as TPHL, risetime, maximum power dissipation, and ground bounce.

```
.MEAS tphl trig v(D) val='.5*vdd' rise=1
+ targ v(out) val='.5*vdd' fall=1
.MEAS risetime trig v(out) val='.1*vdd' rise=1
```

```
+ targ v(out) val='.9*vdd' rise=1
.MEAS max_power max power
.MEAS bounce max v(xin.v_local)
```

The inverter consists of capacitors, diodes, one-dimensional lookup table MOSFETs, and a special low-pass delay element. A property of the low-pass delay element is to attenuate pulses that are narrower than the delay value.

Figure 48 Inverter



Subcircuit Definition

```
.subckt inv in out v+ v-
cout+ out_l v+ 2p
cout- out_l v- 2p
xmp out_l inx v+ pmos
xmn out_l inx v- nmos
e inx v- delay in v- td=1n
din v- in dx
.model dx d cjo=2pf
chi in v+ .5pf
.ends inv
```

One-dimensional lookup tables represent the behavioral MOSFETs.

Behavioral N-Channel MOSFET

The following example is a Drain Gate source.

```
.subckt nmos 1 2 3
gn 3 1 VCR npwl(1) 2 3 scale=0.008
* VOLTAGE RESISTANCE
+ 0. 495.8840g
+ 200.00000m 456.0938g
+ 400.00000m 141.6902g
+ 600.00000m 7.0624g
+ 800.00000m 258.9313meg
+ 1.00000 6.4866meg
+ 1.20000 842.9467k
+ 1.40000 21.6882k
+ 1.60000 170.8367k
+ 1.80000 106.4944k
+ 2.00000 72.7598k
+ 2.20000 52.4632k
+ 2.40000 38.5634k
+ 2.60000 8.8056k
+ 2.80000 5.2543k
+ 3.00000 4.3553k
+ 3.40000 3.4950k
+ 3.80000 2.0534k
+ 4.20000 2.7852k
+ 4.60000 2.5k
+ 5.0 2.3k
.ends nmos
```

The preceding example is a voltage-versus-resistance table. It shows, for example, that the resistance at 5 V is 2.3 kohms.

Creating a Behavioral Inverter Lookup Table

You can create an inverter lookup table in three simple steps:

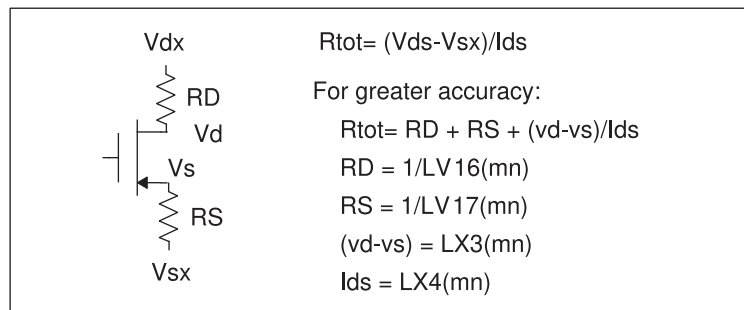
1. Simulate an actual transistor level inverter, using a DC sweep of the input.
2. Print the V/I output, for the output pullup and pulldown transistors.
3. Copy the printed output into the volt lookup table element, for the controlled resistor.

The following test file, `inv_vin_vout.sp`, calculates R_N (the effective pulldown resistor transfer function) and R_P (the pullup transfer function).

- R_N is calculated as $V_{out}/I(mn)$, where mn is the pulldown transistor.
- R_P is calculated as $(VCC-V_{out})/I(mp)$, where mp is the pullup transfer function.

The actual calculation uses a more accurate method to obtain the series resistance of the transistor, as in [Figure 49](#).

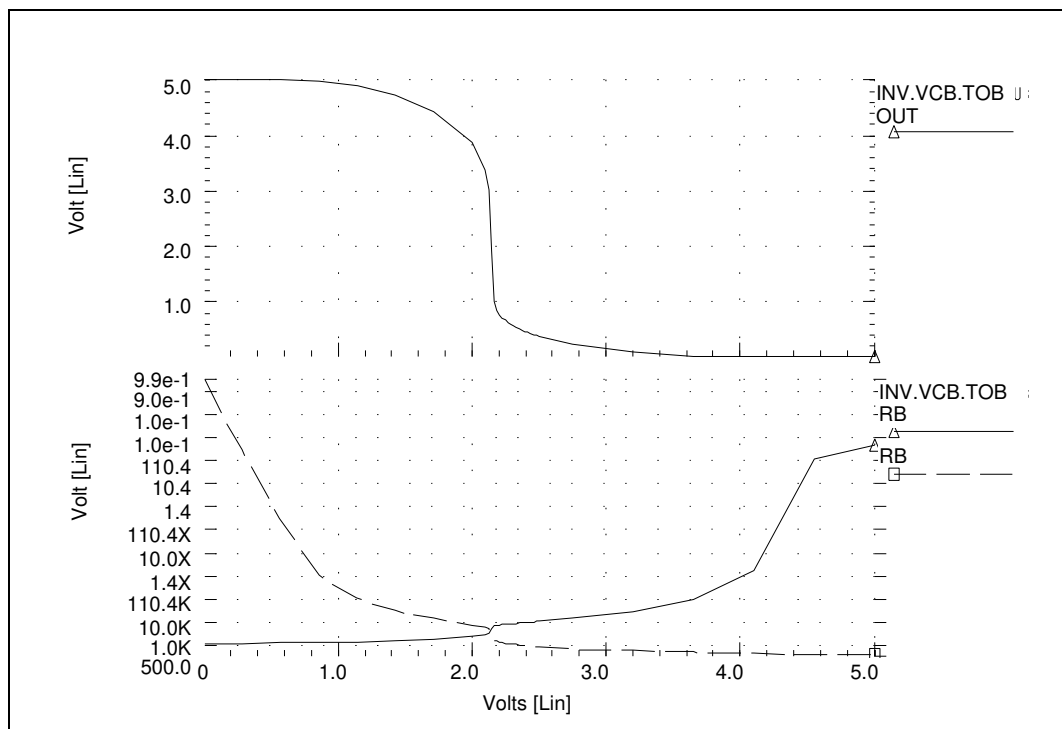
Figure 49 VIN versus VOUT



The first graph in Figure 50 shows VIN versus VOUT.

The second graph shows the computed transfer resistances (R_P and R_N), as a function of VIN.

Figure 50 RP and RN as a Function of VIN



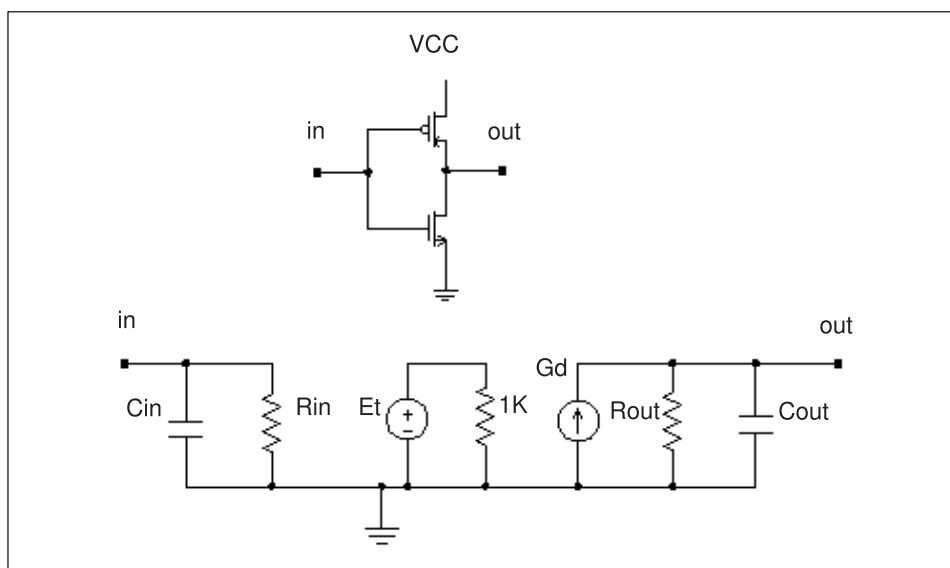
The PrimeSim HSPICE file used to calculate R_P and R_N is located in the following directory:

`$installdir/demo/hspice/behave/inv_vin_vout.sp`

Optimizing Behavioral CMOS Inverters

To calibrate behavioral models, run PrimeSim HSPICE on the full transistor version of a cell. Then optimize the behavioral model to this data.

Figure 51 CMOS Inverter and its Equivalent Circuit



In this example, PrimeSim HSPICE uses the LEVEL 3 MOSFET model to simulate the CMOS inverter.

1. To obtain the input and output resistances, PrimeSim HSPICE performs a `.TF` transfer function analysis (`.TF V(out) Vin`).
2. To obtain the transfer function table of the inverter, PrimeSim HSPICE performs the DC analysis and sweeps the input voltage (`.DC Vin 0 5 .1`).
3. PrimeSim HSPICE uses this table, in the PWL element, to represent the transfer function of the inverter.
4. A voltage-controlled PWL capacitance adjusts the rise and fall time of the inverter, in the equivalent circuit, across the output resistance.
5. The delay element obtains the propagation delay across the output RC circuit.
6. PrimeSim HSPICE uses the inverter in a ring oscillator, to adjust the input capacitance.
7. PrimeSim HSPICE uses optimization analysis for all adjustments in this example. The data file and the results are shown.

Example

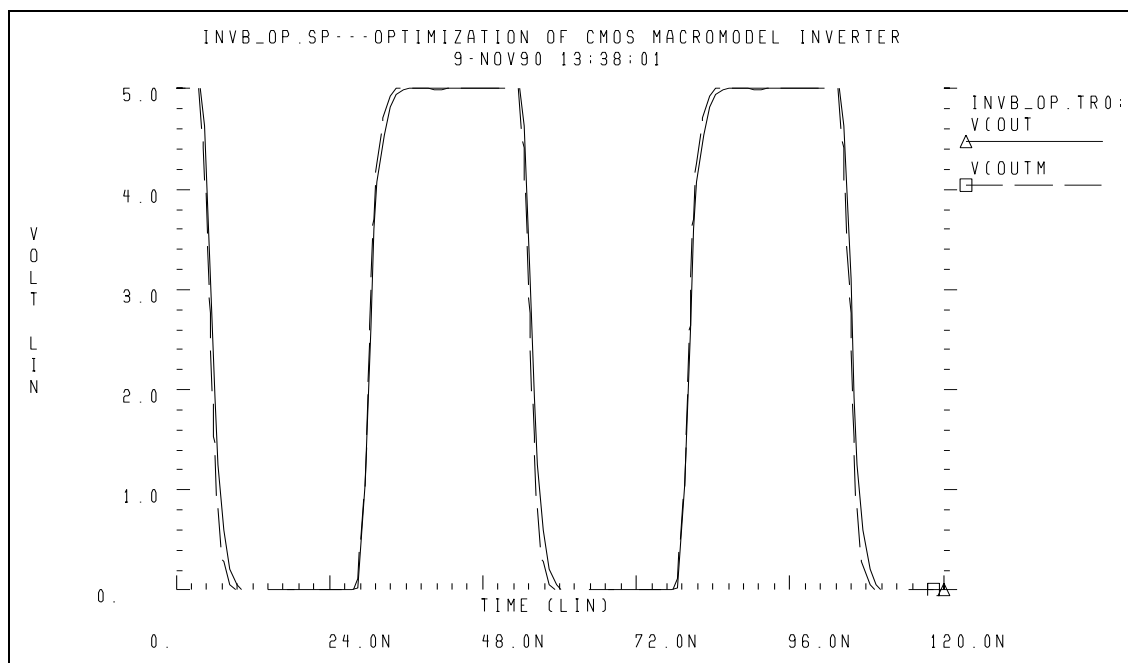
This example is located in the following directory:

```
$installdir/demo/hspice/behave/invb_op.sp
```

The `invb_op.sp` file contains the following sections:

- Subcircuit Definition
- Inverter Using Model
- Optimization Results
- Optimization Completed
- Optimized Parameters OPTINV
- Optimize Results Measure Names and Values

Figure 52 CMOS Inverter Response



Optimizing Behavioral Ring Oscillators

To optimize the behavioral ring oscillator performance, review the examples in this section.

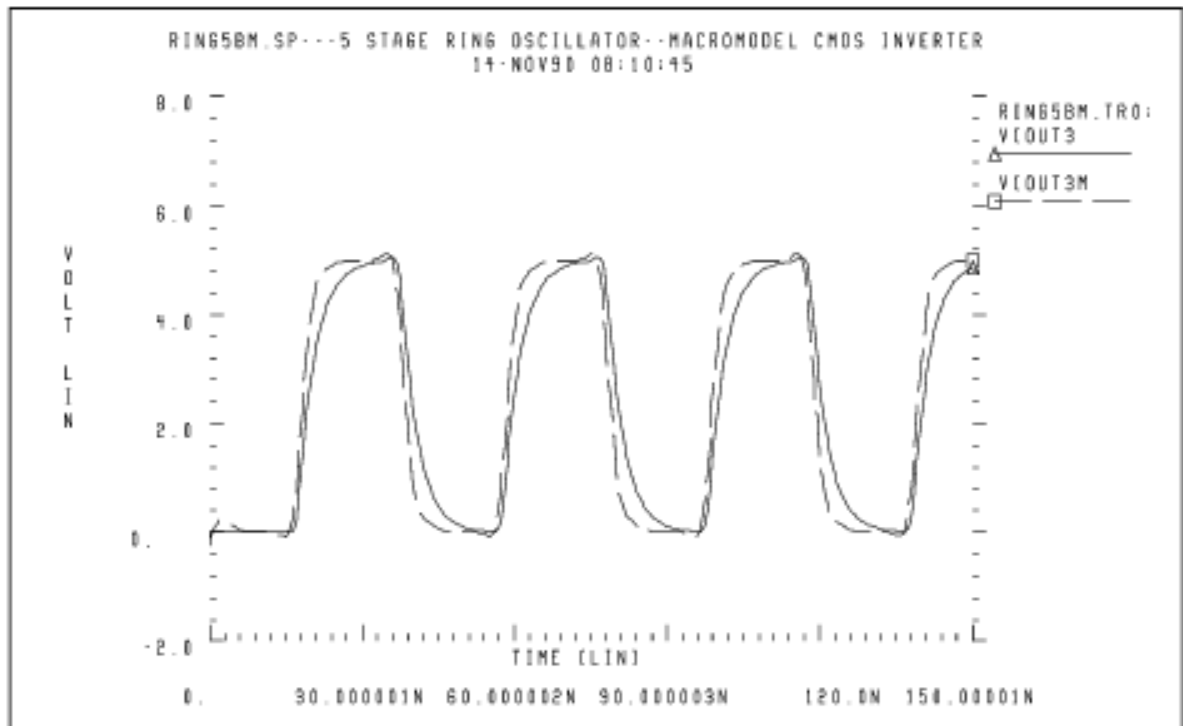
Example: Five-Stage Ring Oscillator

This example is located in the following directory:

```
$installdir/demo/hspice/behave/ring5bm.sp
```

The ring5bm.sp file also contains the results of the five-stage ring oscillator example.

Figure 53 Ring Oscillator Response



Analog Behavioral Elements

The following components are examples of analog behavioral building blocks. Each component demonstrates a basic PrimeSim HSPICE feature:

- integrator: ideal op-amp E-element source
- differentiator: ideal op-amp E-element source
- ideal transformer: ideal transformer E-element source
- AM modulator: algebraic G-element source
- data sampler: algebraic E-element source

The following sections discuss these topics:

- [Behavioral Integrator](#)
- [Behavioral Differentiator](#)
- [Behavioral Amplitude Modulator](#)
- [Behavioral Data Sampler](#)

Behavioral Integrator

PrimeSim HSPICE uses an ideal op-amp to model the integrator circuit and a VCVS to adjust output voltage. The following equation calculates output of the integrator:

$$V_{out} = -\frac{gain}{R_i \cdot C_i} \cdot \int_0^t V_{in} \cdot dt + V_{out}(0)$$

Figure 54 Integrator

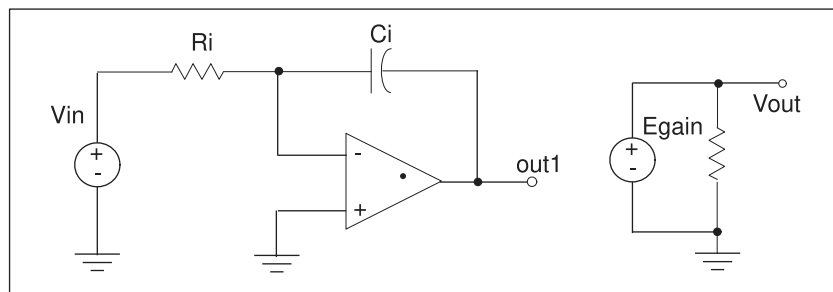


Figure 55 Response of Integrator to a Triangle Waveform



Example

This example is located in the following directory:

```
$installdir/demo/hspice/behave/integ.sp
```

The `integ.sp` file also contains the following sections:

- Control and options
- Subcircuit definition
- Circuit

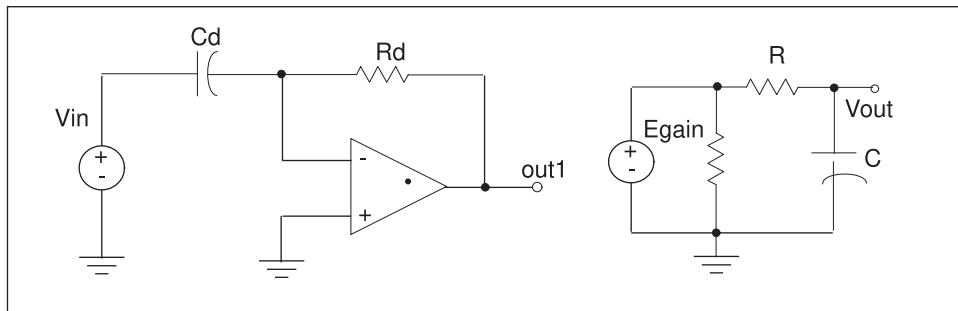
Behavioral Differentiator

PrimeSim HSPICE uses an ideal op-amp to model a differentiator and a VCVS to adjust the magnitude and polarity of the output. The following equation calculates the differentiator response:

$$V_{out} = -gain \cdot R_d \cdot C_d \cdot \frac{d}{dt} V_{in}$$

For a high-frequency signal, the output of a differentiator can overshoot the edges. To smooth this out, you can use a simple RC filter.

Figure 56 Differentiator



Example

This example is located in the following directory:

```
$installdir/demo/hspice/behave/diff.sp
```

The `diff.sp` file also contains the following sections:

- Control and Options
- Subcircuit Definition
- Circuit

Figure 57 Response of a Differentiator to a Triangle Waveform



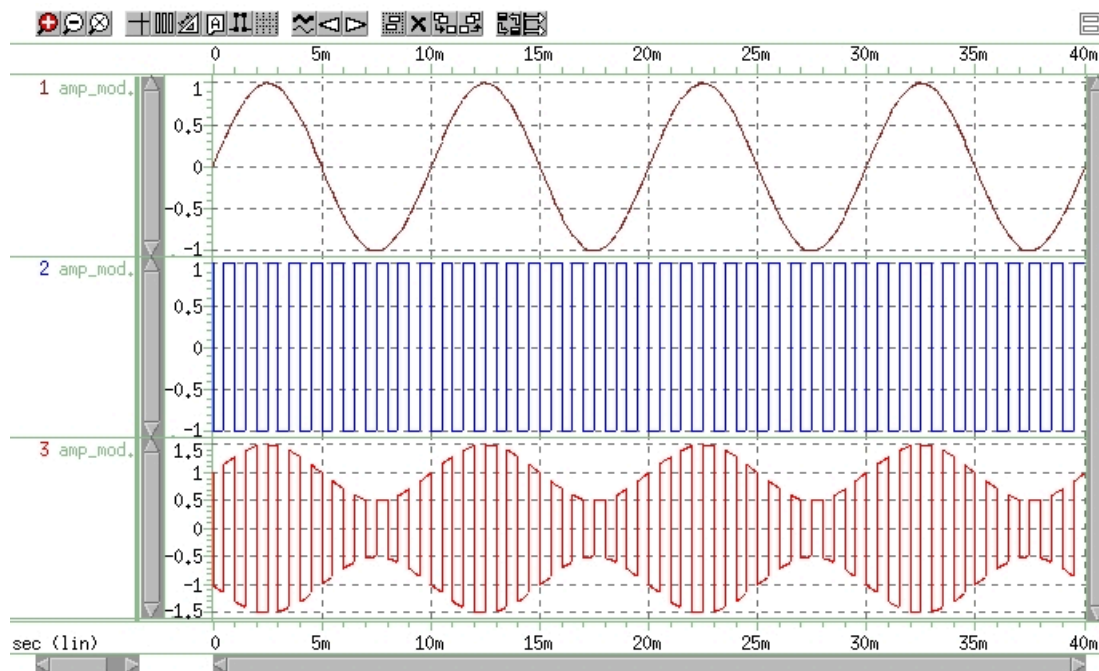
Behavioral Amplitude Modulator

This example, which uses a G-element as an amplitude modulator with a pulse waveform carrier, is located in the following directory:

```
$installdir/demo/hspice/behave/amp_mod.sp
```

See also AM Modulation in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Figure 58 Amplitude Modulator Waveforms

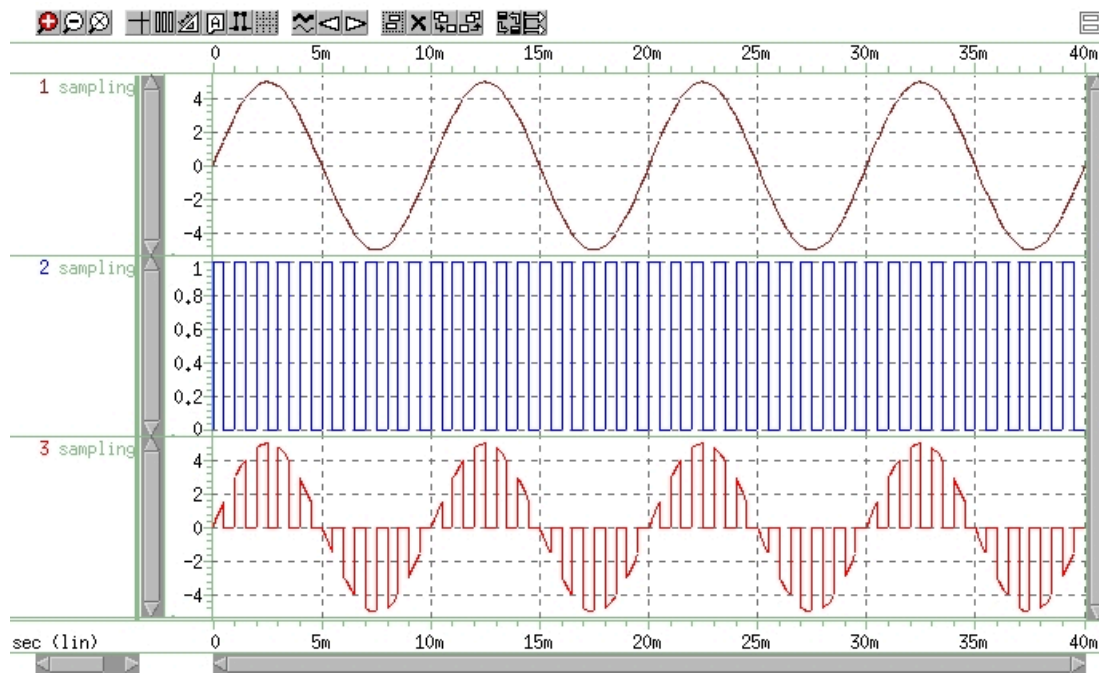


Behavioral Data Sampler

An example of sampling the behavioral data is located in the following directory:

```
$installdir/demo/hspice/behave/sampling.sp
```

Figure 59 Sampled Data



Op-Amps, Comparators, and Oscillators

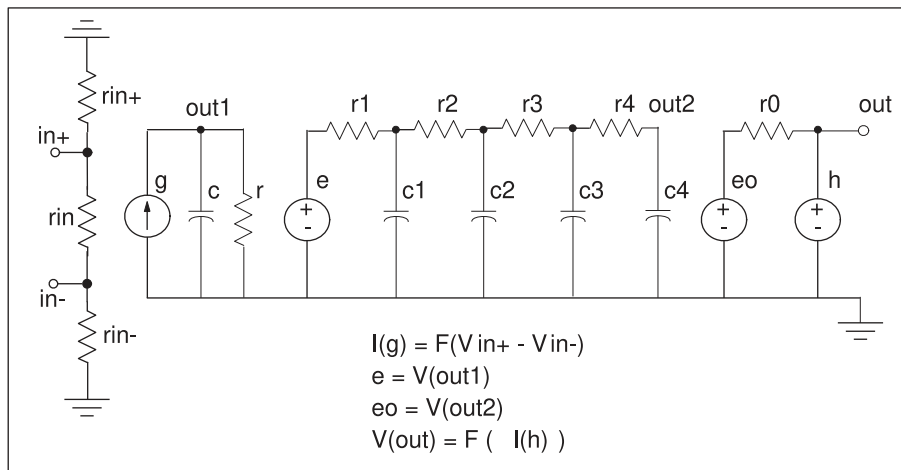
The following sections discuss these topics:

- [741 Op-Amp from Controlled Sources](#)
- [Inverting Comparator with Hysteresis](#)
- [Voltage-Controlled Oscillator \(VCO\)](#)
- [LC Oscillator](#)

741 Op-Amp from Controlled Sources

To model the μ A741 operational amplifier, use PWL controlled sources. A piecewise linear CCVS (source “h”) limits the output to ± 15 V.

Figure 60 Operational Amplifier



Example

This example is located in the following directory:

`$installdir/demo/hspice/behave/op_amp.sp`

The `op_amp.sp` file also contains the following sections:

- Main Circuit
- RC Circuit With Pole At 9 MHz
- Output Limiter to 15 V

Figure 61 AC Analysis Response

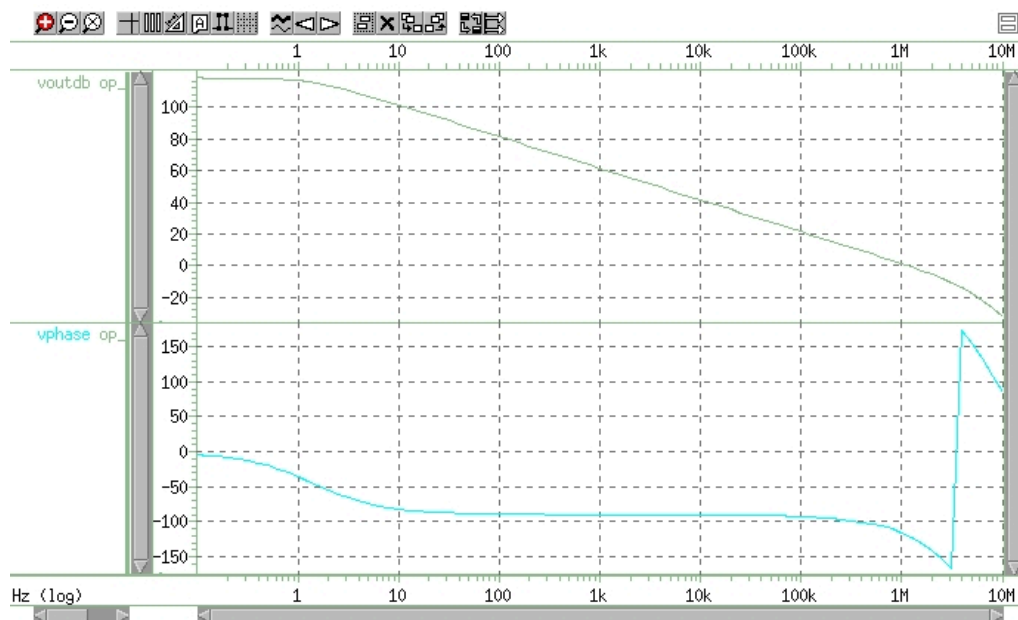


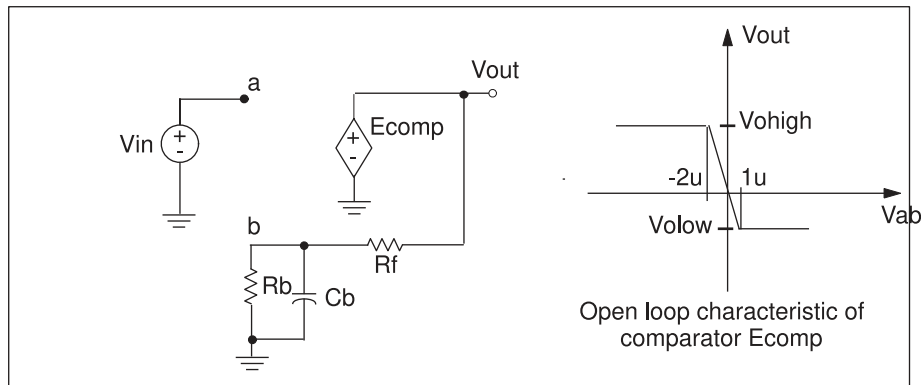
Figure 62 Transient Analysis Response



Inverting Comparator with Hysteresis

A piecewise-linear VCVS models an inverting comparator.

Figure 63 Inverting Comparator with Hysteresis



Two reference voltages correspond to the v_{olow} and v_{ohigh} voltages of E_{comp} :

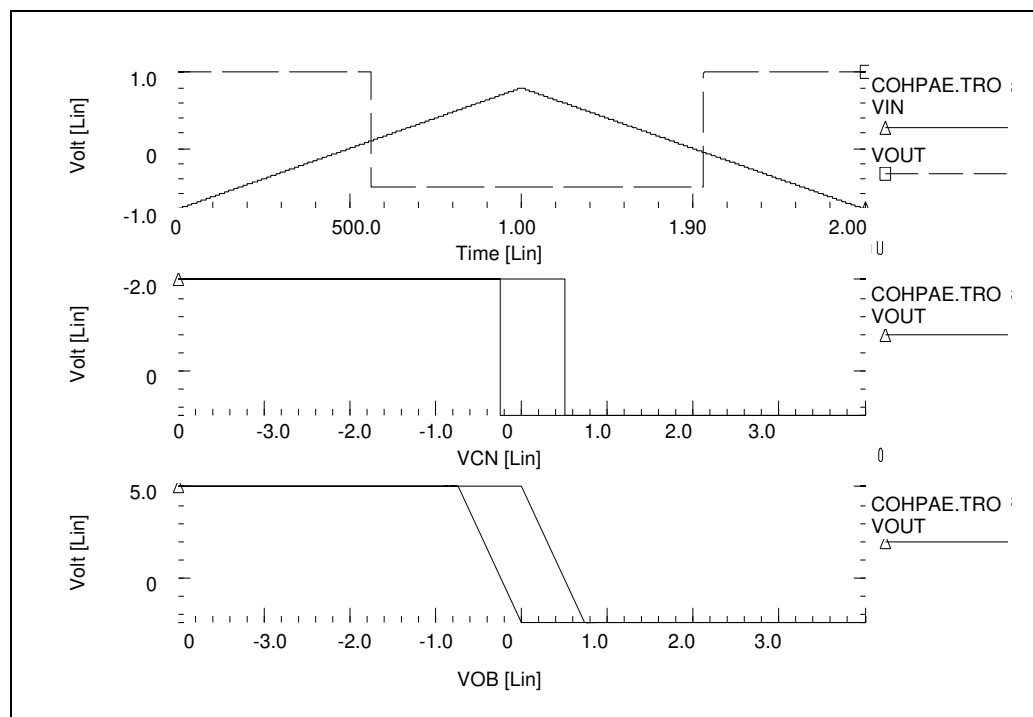
$$V_{reflow} = \frac{V_{olow} \cdot R_b}{R_b + R_f} \quad V_{refhigh} = \frac{V_{ohigh} \cdot R_b}{R_b + R_f}$$

When V_{in} exceeds $V_{refhigh}$, the V_{out} output changes to V_{olow} . For V_{in} values less than V_{reflow} , the output changes to V_{ohigh} .

An example is located in the following directory:

`$installdir/demo/hspice/behave/compar.sp`

Figure 64 Response of Comparator



Voltage-Controlled Oscillator (VCO)

In this example, a one-input NAND (functioning as an inverter) models a five-stage ring oscillator. PWL capacitance switches the load capacitance of this inverter from 1pF to 3pF. As the simulation results indicate, the oscillation frequency decreases as the load capacitance increases.

Example

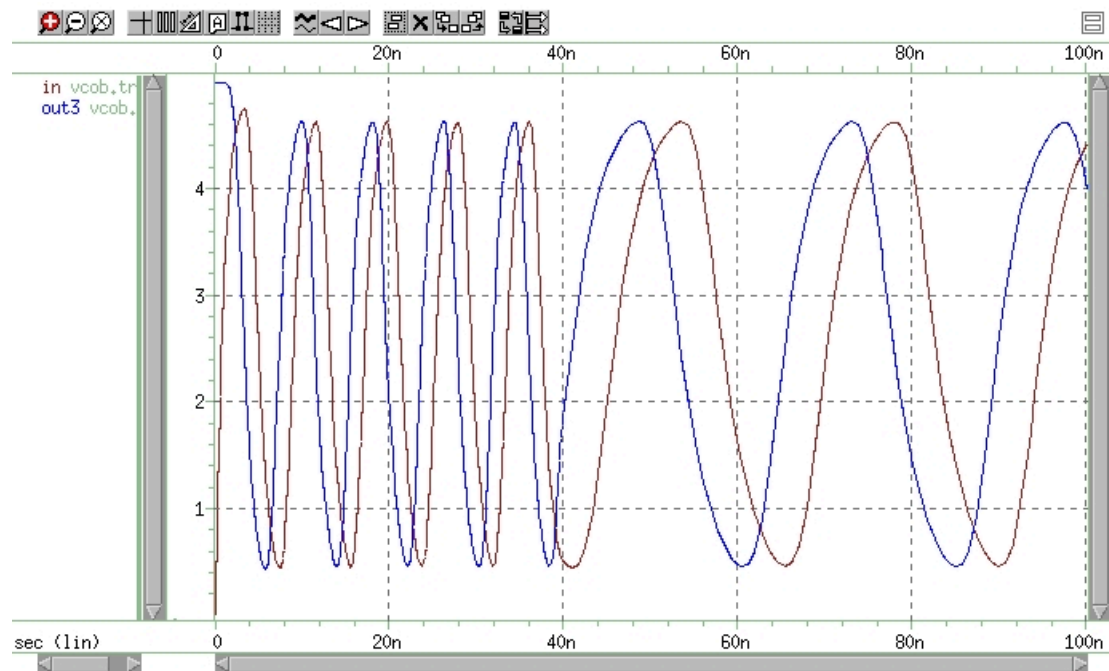
This example is based on the demonstration netlist `vcob.sp`, which is available in the `$installdir/demo/hspice/behave` directory. This file also contains a sample subcircuit definition.

```
vcob.sp voltage controlled oscillator using pwl functions
.option post
.global ctrl
.tran 1n 100n
.ic v(in)=0 v(out1)=5
.probe tran v(in)
x1 in out1 inv
x2 out1 out2 inv
x3 out2 out3 inv
x4 out3 out4 inv
```

```
x5 out4 in inv
vctrl ctrl 0 pwl(0,0 35n,0 40n,5)
*
* macro definitions
*
.subckt inv in out rout=1k
gcout out 0 pwl(1) ctrl 0 level=2 delta=.01
+ 4.5 1p
+ 4.6 3p
rout out 0 rout
gn 0 out nand(1) in 0 scale='1.0k/rout'
+ 0. 5.00ma
+ 0.25 4.95ma
+ 0.5 4.85ma
+ 1.0 4.75ma
+ 1.5 4.42ma
+ 3.5 1.00ma
+ 4.000 0.50ma
+ 4.5 0.20ma
+ 5.0 0.05ma
.ends inv
*

.end
```

Figure 65 Voltage Controlled Oscillator Response



LC Oscillator

The initial capacitor charge is 5 V. The value of the capacitance is the function of voltage at node 10. The capacitance value becomes four times higher at the t2 time. The following equation calculates the frequency of this LC circuit:

$$freq = \frac{1}{6.28 \cdot \sqrt{L \cdot C}}$$

At the t2 time, the frequency must be halved. The amplitude of oscillation depends on the condition of the circuit when the capacitance value changes.

The stored energy is:

$$E = (0.5 \cdot C \cdot V^2) + (0.5 \cdot L \cdot I^2)$$

$$E = 0.5 \cdot C \cdot Vm^2, I = 0 \qquad E = 0.5 \cdot L \cdot Im^2, V = 0$$

At the t2 time, when V=0, if C changes to A · C, then:

$$0.5 \cdot L \cdot Im^2 = 0.5 \cdot Vm^2 = 0.5 \cdot (A \cdot C) \cdot Vm'^2$$

and from the above equation:

$$Vm' = \frac{Vm}{\sqrt{A}}$$

$$Qm' = \sqrt{A} \cdot Vm$$

The second condition that PrimeSim HSPICE considers is when V=Vin, if C changes to A · C, then:

$$Qm = Qm'$$

$$C \cdot Vm = A \cdot C \cdot Vm'$$

$$Vm' = \frac{Vm}{A}$$

Therefore, PrimeSim HSPICE modifies the voltage amplitude between Vm/sqrt(A) and Vm/A, depending on the circuit condition when the circuit switches. This example tests the CTYPE=0 and 1 results. The result for CTYPE=1 must be correct because capacitance is a function of voltage at node 10, not a function of the voltage across the capacitor itself.

Example

This example is based on demonstration netlist `calg2.sp` which is available in the `$installdir/demo/hspice/behave` directory.

```
* in this example the ctype 0 and 1 is tested. the result for
* ctype=1 must be correct because capacitance is function of
* voltage at node 10, not voltage across itself.
*
.option post
.ic v(1)=5 v(2)=5
c1 1 0 c='1e-9*v(10)' ctype=1
l1 1 0 1m
*
c2 2 0 c='1e-9*v(10)' ctype=0
l2 2 0 1m
*
v10 10 0 pwl(0sec,1v t1,1v t2,4v)
r10 10 0 1
```

Figure 66 Correct Result Corresponding to CTYPE=1

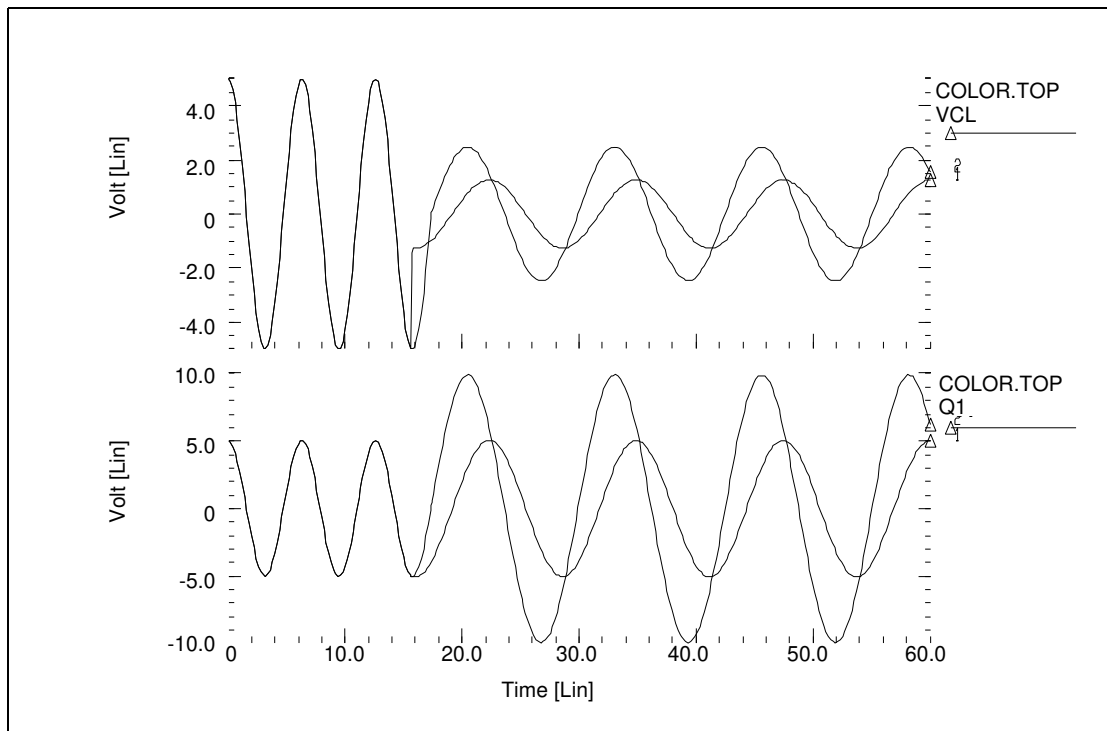
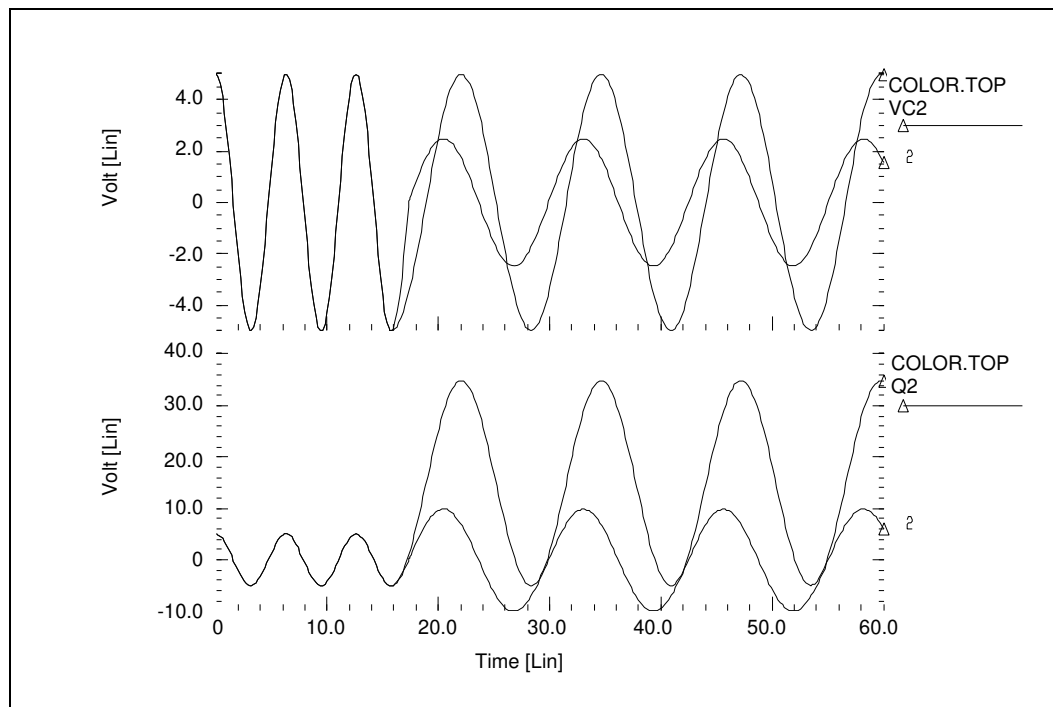


Figure 67 Incorrect Result Corresponding to CTYPE=0



Phase-Locked Loops (PLL)

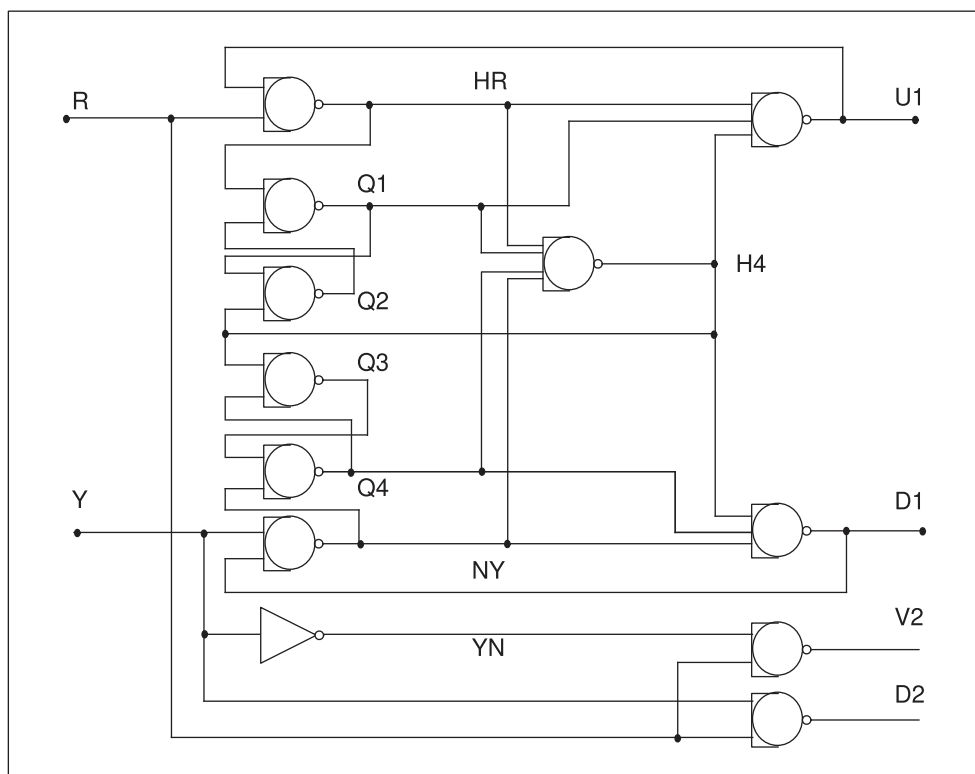
The following sections discuss phase-locked loops:

- [Phase Detector, with Multi-Input NAND Gates](#)
- [Phase Locked Loop Modeling](#)

Phase Detector, with Multi-Input NAND Gates

This circuit uses behavioral elements to implement the inverters, with 2, 3, and 4 input NAND gates.

Figure 68 Phase Detector



Example

An example is located in the following directory:

```
$installdir/demo/hspice/behave/pdb.sp
```

This file also contains sample subcircuit definitions.

Figure 69 Phase Detector Response

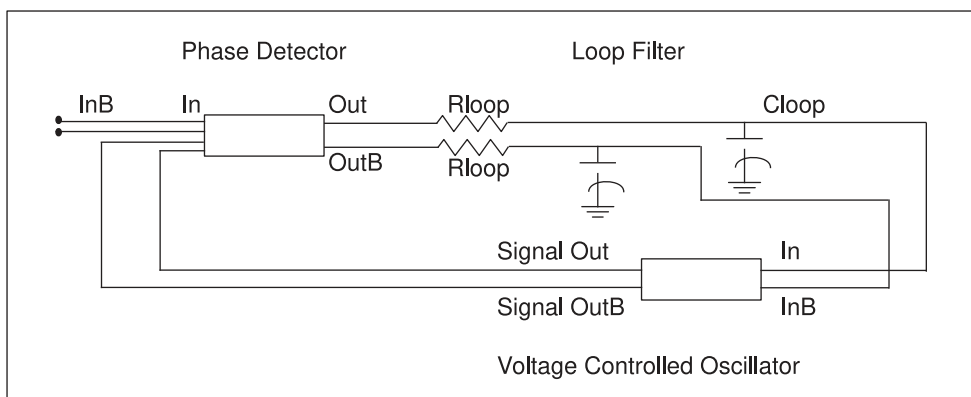


Phase Locked Loop Modeling

A phase-locked loop (PLL) circuit synchronizes to an input waveform, within a selected frequency range. This returns an output voltage that is proportional to the variations in the input frequency. It has three basic components:

- A voltage-controlled oscillator (VCO) which returns an output waveform that is proportional to its input voltage.
- A phase detector which compares the VCO output to the input waveform and returns an output voltage depending on their phase difference.
- A loop filter which filters the phase detector voltage. Returns an output voltage which forms the VCO input (and external voltage output) of the PLL.

Figure 70 Behavioral Phase-Locked Loop



The PLL can be implemented using behavioral elements (Figure 70) or using bipolar transistors (Figure 71 on page 281 and Figure 72 on page 282).

The netlist for the behavioral PLL is in the `pll_bvp.sp` file and the netlist for the full bipolar PLL is in the `pll.sp` file. The netlist for the full bipolar PLL contains the loop filter and the output circuit. Both these netlist files are available in the directory: `$installdir/demo/hspice/behave`.

The PLL transfer function shows a linear region of voltage vs. (periodic) time which is defined as the “lock” range.

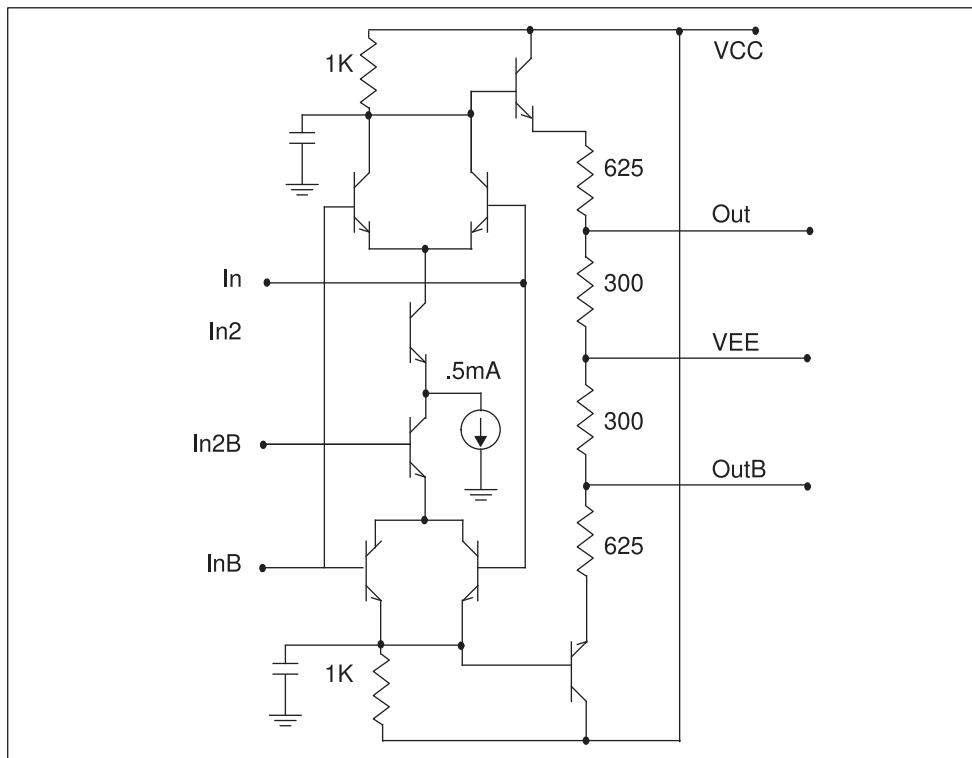
The results of transient simulations (Figure 71) show minimal difference between implementations. However, run time statistics show that the behavioral model reduces the simulation time, to one-third that of the full circuit.

If you use this PLL in a larger system simulation (for example, an AM tracking system), include the behavioral model. This model substantially reduces the simulation run time and still accurately represents the subcircuit.

Figure 71 Behavioral (PLL_BVP Curve) vs. Bipolar (PLL_Curve) Simulation



Figure 72 Bipolar Phase Detector



References

- [1] Chua & Lin. *Computer Aided Analysis of Electronic Circuits*. Englewood Cliffs: Prentice-Hall, 1975, page 117. See also "SPICE2 Application Notes for Dependent Sources," by Bert Epler, *IEEE Circuits & Devices Magazine*, September 1987.

15

Laplace Transform Modeling of Filters and Networks

Describes modeling filters and networks, including Laplace transforms.

When you apply Kirchhoff's laws to circuits that contain energy storage elements, the result is simultaneous differential equations in the time domain. A simulator must solve these equations, to analyze the circuit's behavior. Solving any equation that is higher than the first order can be difficult and classical methods cannot easily solve some driving functions.

In both cases, to simplify the solution, you can use Laplace transforms. These transforms convert time domain equations, containing integral and differential terms, into algebraic equations in the frequency domain.

PrimeSim HSPICE ships numerous examples for your use; see Filters Examples in *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

The following sections discuss these topics:

- [Transient Modeling](#)
- [Using G- and E-elements](#)
- [Laplace and Pole-Zero Modeling](#)
- [Modeling Switched Capacitor Filters](#)
- [References](#)

Transient Modeling

The Laplace transform method provides an easy way to relate a circuit's behavior in time and frequency-domains. This facilitates simultaneous work in those domains.

The algorithm that the PrimeSim HSPICE simulator uses for Laplace and pole/zero transient modeling, offers better performance than the Fast Fourier Transform (FFT) algorithm. To invoke Laplace and pole/zero transient modeling, use a `LAPLACE` or `POLE` function call in a source element statement.

Laplace transfer functions are especially useful in top-down system design, when you use ideal transfer functions instead of detailed circuit designs. In PrimeSim HSPICE, you can also mix Laplace transfer functions with transistors and passive components. Using this capability, you can model a system as the sum of the contributing ideal transfer functions. You can then progressively replace these functions with detailed circuit models, as they become available. Conventional uses of Laplace transfer functions include control systems and behavioral models that contain non-linear elements.

Laplace transforms reduce the time needed to design and simulate large interconnect systems such as clock distribution networks. You can use the asymptotic waveform evaluation (AWE) and other methods, to create a Laplace transfer function model. The AWE model can use only a few poles to represent the large circuit. You can input these poles through a Laplace transform model, to closely approximate the delay and overshoot characteristics of many networks, within a fraction of the original simulation time.

You can use pole/zero analysis to help determine the stability of the design. You can use the `POLE` function in PrimeSim HSPICE when the poles and zeros of the circuit are specified, or you can use the `.PZ` statement (see `.PZ` in *PrimeSim HSPICE Reference Manual: Commands and Control Options*) to derive the poles and zeros from the transfer function.

Frequency response is an important analog circuit property. It is normally the ratio of two complex polynomials (functions of complex frequencies), with positive real coefficients. The form of frequency response can be either the locations of poles and zeros, or a frequency table.

The usual way to design complex circuits is to interconnect smaller functional blocks of known frequency responses, either in the pole/zero or the frequency table form. For example, to design a band-reject filter, you can interconnect a low-pass filter, a high-pass filter, and an adder. Study the function of the complex circuit in terms of its component blocks, before you design the actual circuit. After you test the functionality of the component blocks, you can use these blocks as a reference in optimization techniques, to determine the value of the complex element.

Using G- and E-elements

The following sections discuss these topics:

- [Laplace Transform Function Call](#)
- [G- and E-element Notes](#)
- [Laplace Band-Reject Filter](#)
- [Laplace Low-Pass Filter](#)
- [Circular Convolution Example](#)

Laplace Transform Function Call

Use the G- and E-elements as linear functional blocks or as elements with specific frequency responses.

In the following equations, $H(s)$ denotes the frequency response (also called the impulse response), where s is a complex frequency variable ($s = j2\pi f$). To obtain the frequency response, perform an AC analysis and set $AC=1$ in the input source (the Laplace transform of an impulse is 1). The following expression relates the input and output of the G- and E-elements, with the specified frequency response:

$$Y(j2\pi f) = H(j2\pi f) \cdot X(j2\pi f)$$

where X is the input, Y is the output, and H is the transfer function, at the f frequency.

AC analysis uses the above relation, at any frequency, to determine the frequency response. For operating point and DC sweep analysis, the relation is the same but the frequency is zero.

The transient analysis is more complicated than the frequency response. The output is a convolution of the input waveform with the impulse response $h(t)$:

$$y(t) = \int_{-\infty}^t x(\tau) \cdot h(t-\tau) \cdot d\tau$$

In discrete form, the output is:

$$y(k\Delta) = \Delta \sum_{m=0}^k x(m\Delta) \cdot h[(k-m) \cdot \Delta], k = 0, 1, 2, \dots$$

where you can obtain $h(t)$ from $H(f)$, using the inverse Fourier integral:

$$h(t) = \int_{-\infty}^{\infty} H(f) \cdot e^{j2\pi ft} \cdot df$$

The following equation calculates the inverse discrete Fourier transform:

$$h(m\Delta) = \frac{1}{N \cdot \Delta} \sum_{n=0}^{N-1} H(f_n) \cdot e^{-\frac{j2\pi nm}{N}}, m = 0, 1, 2, \dots, N-1$$

where N is the number of equally-spaced time points and Δ is the time interval or time resolution.

For the frequency response table form (FREQ) of the `LAPLACE` function, PrimeSim HSPICE uses a performance-enhanced algorithm, to convert $H(f)$ to $h(t)$. This algorithm requires N to be a power of 2. The following equation determines the f_n frequency point:

$$f_n = \frac{n}{N \cdot \Delta}, n = 0, 1, 2, \dots, N-1$$

where $n > N/2$ represents the negative frequencies. The following equation determines the Nyquist critical frequency:

$$f_c = f_{N/2} = \frac{1}{2 \cdot \Delta}$$

Because the negative frequency responses are the image of the positive responses, you need to specify only $N/2$ frequency points, to evaluate N time points of $h(t)$. The larger the value of f_c , the more accurate the transient analysis results. However, for large f_c values, the Δ becomes smaller and the computation time increases.

The maximum frequency of interest depends on the functionality of the linear network. For example, in a low-pass filter, you can set f_c to the frequency at which the response drops by 60 dB (a factor of 1000).

$$|H(f_c)| = \frac{|H_{max}|}{1000}$$

After you select or calculate f_c , the following equation can determine Δ :

$$\Delta = \frac{1}{2 \cdot f_c}$$

The following equation calculates the frequency resolution:

$$\Delta f = f_1 = \frac{1}{N \cdot \Delta}$$

which is inversely proportional to the maximum time ($N \cdot \Delta$), over which PrimeSim HSPICE evaluates $h(t)$. Therefore, the transient analysis accuracy also depends on the frequency resolution, or the number of points (N).

You can specify the frequency resolution (`DELF`) and the maximum frequency (`MAXF`) in the G- or E-element statement. To calculate N , PrimeSim HSPICE uses $2 \cdot \text{MAXF}/\text{DELF}$. Next, PrimeSim HSPICE modifies N as a power of 2. The effective `DELF` is $2 \cdot \text{MAXF}/N$, to reflect the changes in N .

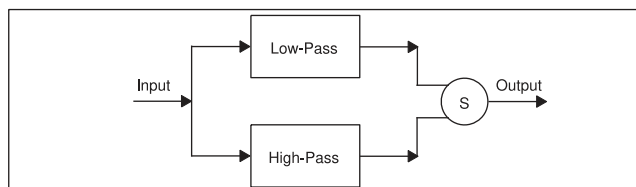
G- and E-element Notes

- If elements in the data file specify frequency responses, do not use pole/zero analysis. If you specify `MAXF=<par>` in a G- or E-element Statement, PrimeSim HSPICE warns that it is ignoring `MAXF`. This is normal.
- PrimeSim HSPICE performs circular convolution, when the G-element `ACCURACY = 0` or 1 (see [Figure 79 on page 291](#)).

Laplace Band-Reject Filter

This example models an active band-reject filter with 3-dB points at 100 and 400 Hz and <35 dB of attenuation between 175 and 225 Hz. The band-reject filter contains low-pass and high-pass filters, and an adder. The low-pass and high-pass filters are fifth-order Chebyshev, with 0.5-dB ripple.

Figure 73 Band-Reject Filter



Example

This example is located in the following directory:

```
$installdir/demo/hspice/filters/BandstopL.sp
```

The `BandstopL.sp` file also contains a sample band-reject filter circuit.

Figure 74 Frequency Response of the Band-Reject Filter

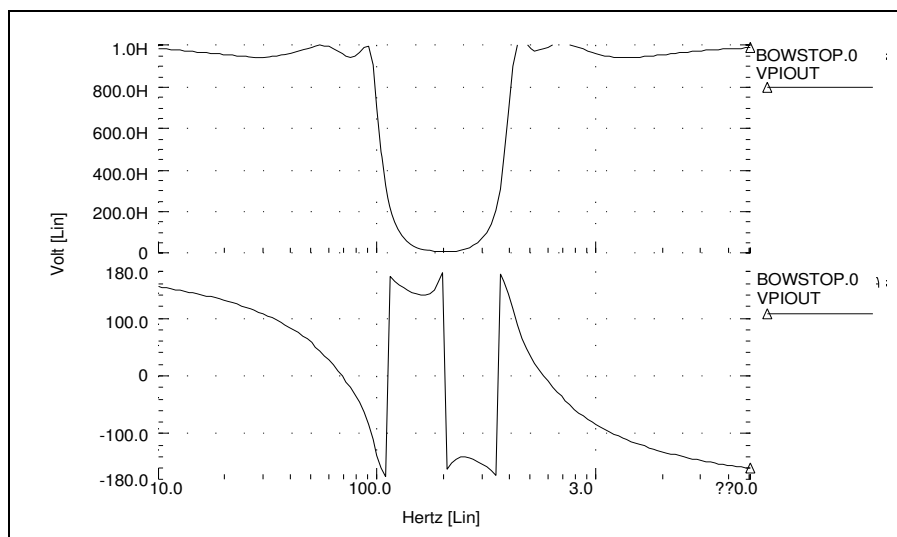
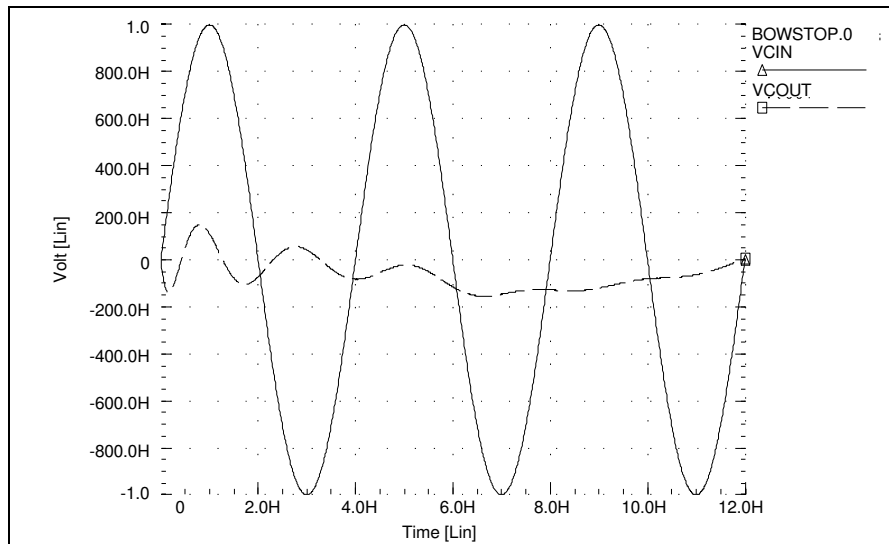


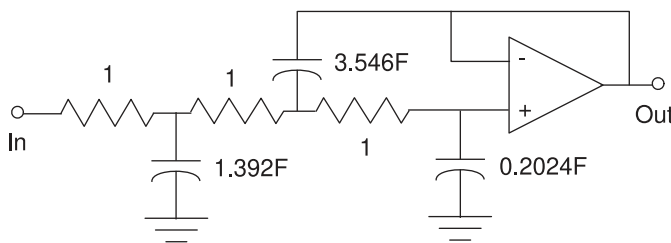
Figure 75 Transient Response of the Band-Reject Filter to a 250 Hz Sine Wave



Laplace Low-Pass Filter

This example simulates a third-order low-pass filter with a Butterworth transfer function. It also compares the results of both the actual circuit and the functional G Element, with the third-order Butterworth transfer function, for AC and transient analysis.

Figure 76 Third-Order Active Low-Pass Filter



The third-order Butterworth transfer function that describes the above circuit is:

$$H(s) = \frac{1.0}{1.0 \cdot (s+1)(s+0.5+j2\pi \cdot 0.1379)(s+0.5-(j2\pi \cdot 0.1379))}$$

The following example is the input listing for the above filter. Parameters set the pole locations for the G-element. Also, this listing specifies only one of the complex poles. The program derives the conjugate pole. The output of the circuit is the `out` node and the output of the functional element is `outg`.

Example

An example of a third-order low-pass Butterworth filter is located in the following directory:

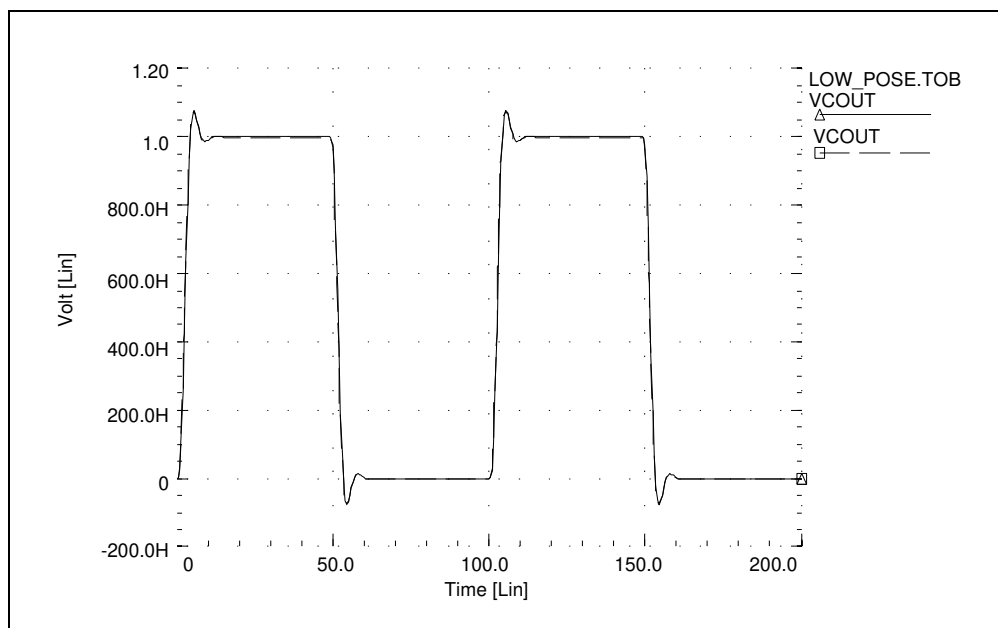
```
$installdir/demo/hspice/filters/Low_Pass.sp
```

The `Low_Pass.sp` file also contains a sample circuit description.

Figure 77 Frequency Response of Circuit and Functional Element



Figure 78 Transient Response of Circuit and Functional Element to a Pulse



Circular Convolution Example

This 30-degree phase-shift filter uses circular convolution. If `DELF=10 MHz`, PrimeSim HSPICE uses inverse fast Fourier transform (IFFT) to obtain the period of time domain response for the G-element. This value is based on the input frequency table and is 100 ns. The `FREQ` G-element performs the convolution integral from $t - T$ to t , assuming that all control voltages at $t < 0$ are zero. t is the target time point and T is the period of the time domain response, for the G-element.

In this example, during time points from 0 to 100 ns, PrimeSim HSPICE uses harmonic components higher than 10 MHz due to the input transition at $t=0$. So the circuit does not behave as a phase shift filter. After one period ($t > 100$ ns), PrimeSim HSPICE performs

circular convolution based on a period of 100 ns. The transient result represents a 30-degree phase shift, for continuous periodic control voltage.

Notes

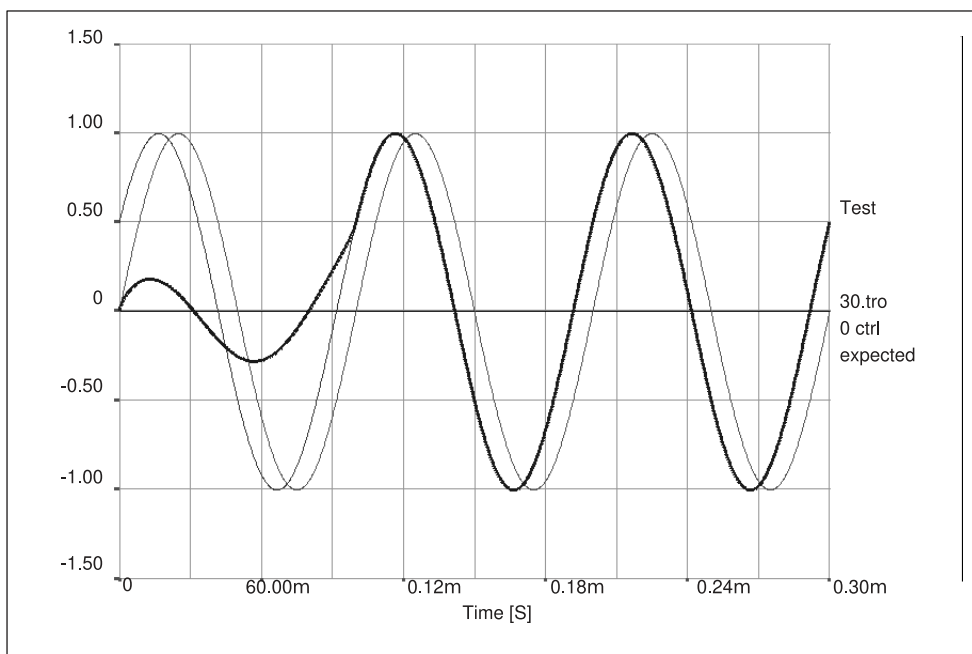
- V(ctrl): control voltage input.
- V(expected): node. Represents an ideal 30-degree shifted wave for the input.
- V(test): output of the G Element.

30-Degree Phase Shift Circuit File

This example illustrates a 30-degree phase-shift filter. It is based on the demonstration netlist `phaseshift.sp` which is available in the directory: `$installdir/demo/hspice/filters`.

```
****
.tran 0.1n 300n
.OPTION post ingold=2 accurate
Vctrl ctrl gnd sin (0 1 10e6)
Gtest gnd test freq ctrl gnd
+ 1.0e00 0 30
+ 1.0e01 0 30
+ 1.0e02 0 30
+ 1.0e03 0 30
+ 1.0e04 0 30
+ 1.0e05 0 30
+ 1.0e06 0 30
+ 1.0e07 0 30
+ 1.0e08 0 30
+ 1.0e09 0 30
+ 1.0e10 0 30
+ MAXF=1.0e9 DELF=10e6
Rtest test gnd 1
Iexpected gnd 3 sin (0 1 10e6 0 0 30)
Vmes 3 expected 0v
Rexpected expected gnd 1
.end
```

Figure 79 Transient Response of the 30 Degree Phase Shift Filter



Laplace and Pole-Zero Modeling

The following sections discuss these topics:

- [Laplace Transform \(LAPLACE\) Function](#)
- [Laplace Transform POLE \(Pole/Zero\) Function](#)
- [AWE Transfer Function Modeling](#)
- [Y-parameter Line Modeling](#)
- [Comparison of Circuit and Pole/Zero Models](#)

Laplace Transform (LAPLACE) Function

PrimeSim HSPICE provides two types of `LAPLACE` function calls: one for transconductance and one for voltage gain transfer functions. See [Using G- and E-elements on page 284](#) for the general forms. For descriptions of the parameters, see G-element Parameters and E-element Parameters in the *PrimeSim HSPICE® User Guide: Elements*.

The following sections discuss these topics:

- [General Form of the Transfer Function](#)
- [Finding the Transfer Function](#)
- [Determining the Laplace Coefficients](#)

General Form of the Transfer Function

To use the `LAPLACE` modeling function, you must find the $k_0, \dots, k_n$ and $d_0, \dots, d_m$ coefficients of the transfer function. The transfer function is the s-domain (frequency domain) ratio of the output for a single-source circuit to the input, with initial conditions set to zero. The following equation represents the Laplace transfer function:

$$H(s) = \frac{Y(s)}{X(s)}$$

where:

- s is the complex frequency, $j2\pi f$.
- $Y(s)$ is the Laplace transform of the output signal.
- $X(s)$ is the Laplace transform of the input signal.

Note:

To obtain the impulse response $H(s)$, PrimeSim HSPICE performs an AC analysis, where `AC=1` represents the input source. The Laplace transform of an impulse is 1. For an element with an infinite response at DC (such as a unit step function $H(s)=1/s$), PrimeSim HSPICE calculations use the value of the `EPSMIN` option (the smallest number possible on the platform) as the transfer function.

The general form of the transfer function $H(s)$ in the frequency domain is:

$$H(s) = \frac{k_0 + k_1s + \dots + k_ns^n}{d_0 + d_1s + \dots + d_ms^m}$$

The order of the numerator for the transfer function cannot be greater than the order of the denominator. The exception is differentiators, for which the transfer function $H(s) = ks$. You can use parameter values for all k and d coefficients of the transfer function, in the circuit descriptions.

Finding the Transfer Function

The first step in determining the transfer function of a circuit is to convert the circuit to the s-domain. To do this, transform the value for each element into its s-domain equivalent form.

Table 3 on page 293 and Table 4 on page 293 show transforms that convert some common functions to the s-domain. The next section provides examples of using transforms to determine transfer functions.

Table 3 Laplace Transforms for Common Source Functions

$f(t), t \geq 0$	Source Type	$L\{f(t)\} = F(s)$
$\sigma(t)$	impulse	1
$u(t)$	step	$\frac{1}{s}$
t	ramp	$\frac{1}{s^2}$
e^{-at}	exponential	$\frac{1}{s+a}$
$\sin \omega t$	sine	$\frac{\omega}{s^2 + \omega^2}$
$\cos \omega t$	cosine	$\frac{s}{s^2 + \omega^2}$
$\sin(\omega t + \phi)$	sine	$\frac{s \sin(\phi) + \omega \cos(\phi)}{s^2 + \omega^2}$
$\cos(\omega t + \phi)$	cosine	$\frac{s \cos(\phi) - \omega \sin(\phi)}{s^2 + \omega^2}$
$\sinh \omega t$	hyperbolic sine	$\frac{\omega}{s^2 - \omega^2}$
$\cosh \omega t$	hyperbolic cosine	$\frac{s}{s^2 - \omega^2}$
te^{-at}	damped ramp	$\frac{1}{(s+a)^2}$
$e^{-at} \sin \omega t$	damped sine	$\frac{\omega}{(s+a)^2 + \omega^2}$
$e^{-at} \cos \omega t$	damped cosine	$\frac{s+a}{(s+a)^2 + \omega^2}$

Table 4 Laplace Transforms for Common Operations

$f(t)$	$L\{f(t)\} = F(s)$
$Kf(t)$	$KF(s)$
$f_1(t) + f_2(t) - f_3(t) + \dots$	$F_1(s) + F_2(s) - F_3(s) + \dots$

Table 4 Laplace Transforms for Common Operations (Continued)

$f(t)$	$L\{f(t)\} = F(s)$
$\frac{d}{dt}f(t)$	$sF(s) - f(0^-)$
$\frac{d^2}{dt^2}f(t)$	$s^2F(s) - sf(0^-) - \frac{d}{dt}f(0^-)$
$\frac{d^n}{dt^n}f(t)$	$s^nF(s) - s^{n-1}f(0^-) - s^{n-2}\frac{d}{dt}f(0^-)$
$\int_{-\infty}^t f(t) dt$	$\frac{F(s)}{s} + \frac{f^{-1}(0)}{s}$
$f(t-a)u(t-a), a > 0$ (u is the step function)	$e^{-as}F(s)$
$e^{at}f(t)$	$F(s+a)$
$f(at), a > 0$	$\frac{1}{a}F\left(\frac{s}{a}\right)$
$tf(t)$	$-\frac{d}{ds}(F(s))$
$t^n f(t)$	$-(1)^n \frac{d^n}{ds^n}F(s)$
$\frac{f(t)}{t}$	$\int_s^\infty F(u) du$ (u is the step function)
$f(t-t_1)$	$e^{-t_1 s}F(s)$

Determining the Laplace Coefficients

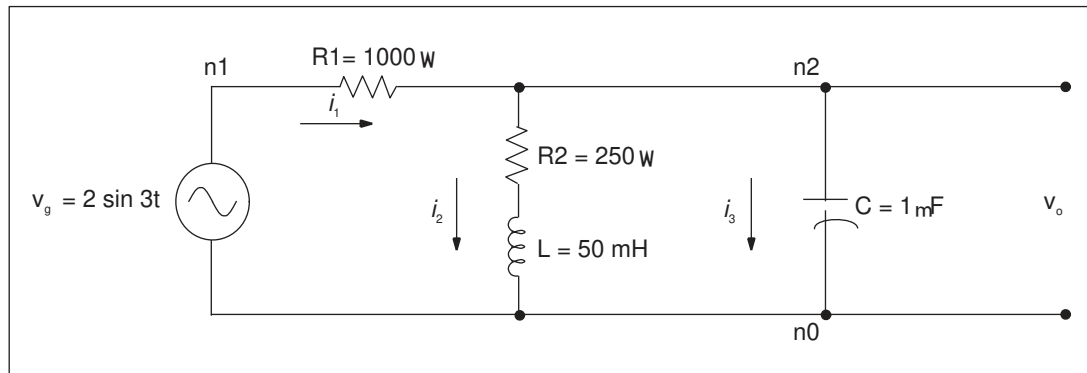
The following examples describe how to determine the appropriate coefficients for the Laplace modeling function call:

- [Laplace Example 1 – Voltage Gain Transfer Function](#)
- [LAPLACE Example 2 – Differentiator](#)
- [LAPLACE Example 3 – Integrator](#)

Laplace Example 1 – Voltage Gain Transfer Function

To find the voltage-gain transfer function for the circuit in [Figure 80](#), convert the circuit to its equivalent s-domain circuit and solve for v_o / v_g .

Figure 80 LAPLACE Example 1 Circuit



Use transforms from [Table 4 on page 293](#) to convert the inductor, capacitor, and resistors. $L\{f(t)\}$ represents the Laplace transform of $f(t)$:

$$L\left\{L\frac{d}{dt}f(t)\right\} = L \cdot (sF(s) - f(0)) = 50 \times 10^{-3} \cdot (s - 0) = 0.05s$$

$$L\left\{\frac{1}{C}\int_0^t f(t) dt\right\} = \frac{1}{C} \cdot \left(\frac{F(s)}{s} + \frac{f^1(0)}{s}\right) = \frac{1}{10^{-6}} \cdot \left(\frac{1}{s} + 0\right) = \frac{10^6}{s}$$

$$L\{R1 \cdot f(t)\} = R1 \cdot F(s) = R1 = 1000W$$

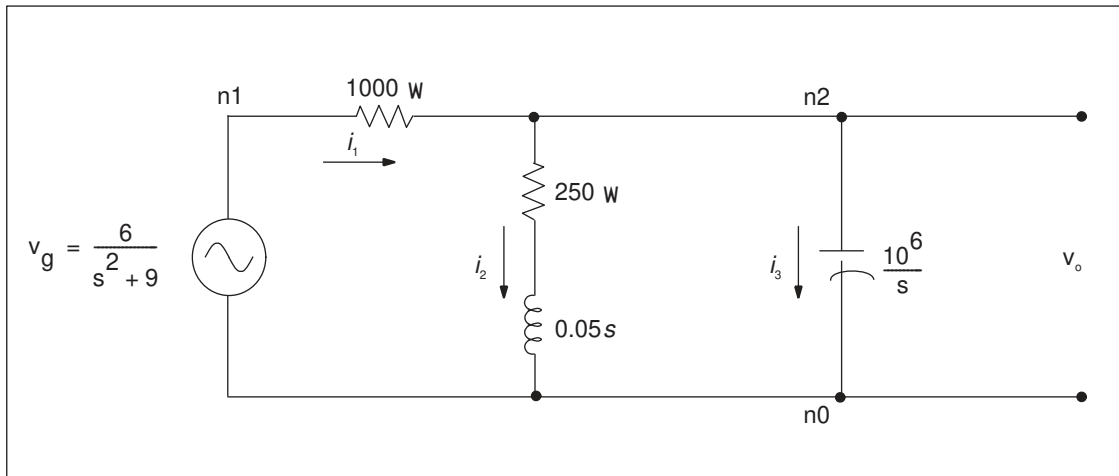
$$L\{R2 \cdot f(t)\} = R2 \cdot F(s) = R2 = 250W$$

To convert the voltage source to the s-domain, use the $\sin \omega t$ transform from [Table 3 on page 293](#):

$$L\{2 \sin 3t\} = 2 \cdot \frac{3}{s^2 + 3^2} = \frac{6}{s^2 + 9}$$

[Figure 81](#) displays the s-domain equivalent circuit.

Figure 81 S-Domain Equivalent of the LAPLACE Example 1 Circuit



Summing the output currents from the n2 node:

$$\frac{v_o - v_g}{1000} + \frac{v_o}{250 + 0.05s} + \frac{v_o s}{10^6} = 0$$

Solve for v_o :

$$v_o = \frac{1000(s + 5000)v_g}{s^2 + 6000s + 25 \times 10^6}$$

The voltage-gain transfer function is:

$$H(s) = \frac{v_o}{v_g} = \frac{1000(s + 5000)}{s^2 + 6000s + 25 \times 10^6} = \frac{5 \times 10^6 + 1000s}{25 \times 10^6 + 6000s + s^2}$$

For the `Laplace` function call, use the k_n and d_m coefficients for the transfer function, in the form:

$$H(s) = \frac{k_0 + k_1s + \dots + k_ns^n}{d_0 + d_1s + \dots + d_ms^m}$$

The coefficients from the above voltage-gain transfer function are:

$$k_0 = 5 \times 10^6 \quad k_1 = 1000$$

$$d_0 = 25 \times 10^6 \quad d_1 = 6000 \quad d_2 = 1$$

Using these coefficients, the following is a Laplace modeling function call for the voltage-gain transfer function of the circuit in [Figure 80 on page 295](#):

LAPLACE Example 2 – Differentiator

To model a differentiator, use either G or E elements, as shown in the following example.

In the frequency domain:

E-element: $V_{out} = ksV_{in}$

G-element: $I_{out} = ksV_{in}$

In the time domain:

E-element:

$$v_{out} = k \frac{dV_{in}}{dt}$$

G-element:

$$i_{out} = k \frac{dV_{in}}{dt}$$

For a differentiator, the voltage gain transfer function is:

$$H(s) = \frac{V_{out}}{V_{in}} = ks$$

In the general form of the transfer function:

$$H(s) = \frac{k_0 + k_1s + \dots + k_ns^n}{d_0 + d_1s + \dots + d_ms^m}$$

If you set $k_1 = k$ and $d_0 = 1$ and the remaining coefficients are zero, then the equation becomes:

$$H(s) = \frac{ks}{1} = ks$$

Using the $k_1 = k$ and $d_0 = 1$ coefficients in the Laplace modeling, the circuit descriptions for the differentiator are:

```
Edif out GND LAPLACE in GND 0 k / 1
Gdif out GND LAPLACE in GND 0 k / 1
```

LAPLACE Example 3 – Integrator

You can use G or E elements to model an integrator, as follows:

In the frequency domain:

E Element: $V_{out} = \frac{k}{s} V_{in}$

G Element: $I_{out} = \frac{k}{s} V_{in}$

In the time domain:

E Element: $v_{out} = k \int V_{in} dt$

G Element: $i_{out} = k \int V_{in} dt$

For an integrator, the voltage gain transfer function is:

$$H(s) = \frac{V_{out}}{V_{in}} = \frac{k}{s}$$

In the general form of the transfer function:

$$H(s) = \frac{k_0 + k_1 s + \dots + k_n s^n}{d_0 + d_1 s + \dots + d_m s^m}$$

As in the previous example, if you set $k_0 = k$ and $d_1 = 1$, then the equation becomes:

$$H(s) = \frac{k + 0 + \dots + 0}{0 + s + \dots + 0} = \frac{k}{s}$$

Laplace Transform POLE (Pole/Zero) Function

The following sections describe the general form of the pole/zero transfer function. It also provides examples of converting specific transfer functions into pole/zero circuit descriptions.

The following sections discuss these topics:

- [POLE Function Call](#)
- [General Form of the Transfer Function](#)

- [Reduced Form of the Transfer Function](#)
- [RC Line Modeling](#)

POLE Function Call

You can use the `POLE` function if the poles and zeros of the circuit are available. You can derive the poles and zeros from the transfer function as described in this chapter, or you can use the `.PZ` statement to find them, as described in *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

PrimeSim HSPICE provides two forms of the `LAPLACE` function call: one for transconductance and one for voltage gain transfer functions. See [Using G- and E-elements on page 284](#) for the general forms and for optional parameters.

To use the `POLE` pole/zero modeling function, find the a , b , f , and α coefficients of the transfer function. The transfer function is the s -domain (frequency domain) ratio of the output, for a single-source circuit, to the input, with initial conditions set to zero.

General Form of the Transfer Function

The general expanded form of the pole/zero transfer function $H(s)$ is:

$$H(s) = \frac{a(s + \alpha_{z1} + j2\pi f_{z1})(s + \alpha_{z1} - j2\pi f_{z1}) \dots (s + \alpha_{zn} + j2\pi f_{zn})(s + \alpha_{zn} - j2\pi f_{zn})}{b(s + \alpha_{p1} + j2\pi f_{p1})(s + \alpha_{p1} - j2\pi f_{p1}) \dots (s + \alpha_{pm} + j2\pi f_{pm})(s + \alpha_{pm} - j2\pi f_{pm})}$$

You can use parameters to set the a , b , α , and f values.

The following is an example:

```
Ghigh_pass 0 out    POLE in 0 1.0    0.0,0.0 / 1.0    0.001,0.0
Elow_pass out 0     POLE in 0    1.0 / 1.0, 1.0,0.0    0.5,0.1379
```

The `Ghigh_pass` statement describes a high-pass filter with the transfer function:

$$H(s) = \frac{1.0 \cdot (s + 0.0 + j \cdot 0.0)}{1.0 \cdot (s + 0.001 + j \cdot 0.0)}$$

The `Elow_pass` statement describes a low-pass filter with the transfer function:

$$H(s) = \frac{1.0}{1.0 \cdot (s + 1)(s + 0.5 + j2\pi \cdot 0.1379)(s + 0.5 - (j2\pi \cdot 0.1379))}$$

To write a pole/zero circuit description for an element, you need to know $H(s)$ transfer function of the element in terms of the a , b , f , and α coefficients.

Before you use the values of these coefficients in `POLE` function calls (in the circuit description), you must simplify the transfer function as described in the next section.

Reduced Form of the Transfer Function

Complex poles and zeros occur in conjugate pairs (a set of complex numbers differ only in the signs of their imaginary parts):

$$(s + \alpha_{pm} + j2\pi f_{pm})(s + \alpha_{pm} - j2\pi f_{pm}), \text{ for poles.}$$

$$(s + \alpha_{zn} + j2\pi f_{zn})(s + \alpha_{zn} - j2\pi f_{zn}), \text{ for zeros.}$$

To write the transfer function in pole/zero format, supply coefficients for one term of each conjugate pair. PrimeSim HSPICE provides the coefficients for the other term. If you omit the negative complex roots, the result is the reduced form of the transfer function, $\text{Reduced}\{H(s)\}$.

To find the reduced form, collect all general-form terms that have negative complex roots:

$$H(s) = \frac{a(s + \alpha_{z1} + j2\pi f_{z1}) \dots (s + \alpha_{zn} + j2\pi f_{zn})}{b(s + \alpha_{p1} + j2\pi f_{p1}) \dots (s + \alpha_{pm} + j2\pi f_{pm})} \cdot \frac{a(s + \alpha_{z1} - j2\pi f_{z1}) \dots (s + \alpha_{zn} - j2\pi f_{zn})}{b(s + \alpha_{p1} - j2\pi f_{p1}) \dots (s + \alpha_{pm} - j2\pi f_{pm})}$$

Then discard the right-hand term, which contains all terms with negative roots. What remains is the reduced form:

$$\text{Reduced}\{H(s)\} = \frac{a(s + \alpha_{z1} + j2\pi f_{z1}) \dots (s + \alpha_{zn} + j2\pi f_{zn})}{b(s + \alpha_{p1} + j2\pi f_{p1}) \dots (s + \alpha_{pm} + j2\pi f_{pm})}$$

For this function, find the a , b , f , and α coefficients to use in a `POLE` function, for a voltage-gain transfer function. The following examples show how to determine the coefficients and write `POLE` function calls for a high-pass filter and a low-pass filter.

See the following examples:

- [POLE Example 1 – High-Pass Filter](#)
- [POLE Example 2 – Low-Pass Filter](#)

POLE Example 1 – High-Pass Filter

For a high-pass filter with a transconductance transfer function, such as:

$$H(s) = \frac{s}{(s + 0.001)}$$

Find the a , b , α , and f coefficients needed to write the transfer function in the general form shown previously. You can then see the conjugate pairs of complex roots. You need to supply only one of each conjugate pair of roots in the `Laplace` function call. PrimeSim HSPICE automatically inserts the other root.

To transform the function into a form that is more similar to the general form of the transfer function, rewrite the transconductance transfer function as:

$$H(s) = \frac{1.0(s+0.0)}{1.0(s+0.001)}$$

Because this function has no negative imaginary parts, it is already in the PrimeSim HSPICE reduced form (reference number 2) shown previously.

You can now identify the a, b, f, and α coefficients, so that the H(s) transfer function matches the reduced form. This matching process obtains the values:

$$n = 1, m = 1$$

$$a = 1.0 \quad \alpha_{z1} = 0.0 \quad f_{z1} = 0.0$$

$$b = 1.0 \quad \alpha_{p1} = 0.001 \quad f_{p1} = 0.0$$

Using these coefficients in the reduced form, provides the transfer function:

$$\frac{s}{(s+0.001)}$$

So the general transconductance transfer function POLE function call:

```
Gxxx n+ n- POLE in+ in- a  $\alpha$  z1,fz1...  $\alpha$  zn,fzn /
b  $\alpha$  p1,fp1...  $\alpha$  pm,fpm
```

for an element named Ghigh_pass, becomes:

```
Ghigh_pass gnd out POLE in gnd 1.0 0.0,0.0 /
+ 1.0 0.001,0.0
```

POLE Example 2 – Low-Pass Filter

For a low-pass filter with the following voltage-gain transfer function:

$$H(s) = \frac{1.0}{1.0(s+1.0+j2\pi \cdot 0.0)(s+0.5+j2\pi \cdot 0.15)(s+0.5-j2\pi \cdot 0.15)}$$

you need to find the a, b, α , and f coefficients, to write the transfer function in the general form, so that you can identify the complex roots with negative imaginary parts.

To separate the reduced form, Reduced{H(s)}, from the terms with negative imaginary parts, rewrite the voltage-gain transfer function as:

$$\begin{aligned} H(s) &= \frac{1.0}{1.0(s+1.0+j2\pi \cdot 0.0)(s+0.5+j2\pi \cdot 0.15)} \cdot \frac{1.0}{(s+0.5-j2\pi \cdot 0.15)} \\ &= \text{Reduced}\{H(s)\} \cdot \frac{1.0}{(s+0.5-j2\pi \cdot 0.15)} \end{aligned}$$

So:

$$\text{Reduced}\{H(s)\} = \frac{1.0}{1.0(s+1.0)(s+0.5+j2\pi \cdot 0.15)}$$

or:

$$\frac{a(s+\alpha_{z1}+j2\pi f_{z1})\dots(s+\alpha_{zn}+j2\pi f_{zn})}{b(s+\alpha_{p1}+j2\pi f_{p1})\dots(s+\alpha_{pm}+j2\pi f_{pm})} = \frac{1.0}{1.0(s+1.0+j2\pi \cdot 0.0)(s+0.5+j2\pi \cdot 0.15)}$$

Now assign coefficients in the reduced form, to match the specified voltage transfer function. The following coefficient values produce the transfer function:

$$\begin{aligned} n &= 0, \quad m = 2, \\ a &= 1.0 \quad b = 1.0 \quad \alpha_{p1} = 1.0 \quad f_{p1} = 0 \quad \alpha_{p2} = 0.5 \quad f_{p2} = 0.15 \end{aligned}$$

You can substitute these coefficients in the POLE function-call, for a voltage-gain transfer function:

```
Exxx n+ n- POLE in+ in- a αz1, fz1...αzn, fzn /  
b αp1, fp1...αpm, fpm
```

for an element named Elow_pass, to obtain the following statement:

```
Elow_pass out GND POLE in 1.0 / 1.0 1.0,0.0 0.5,0.15
```

RC Line Modeling

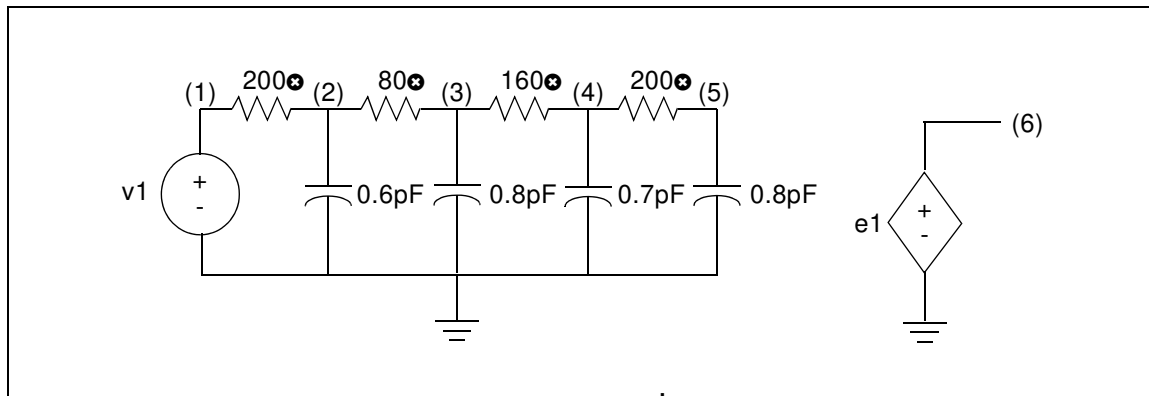
Most RC lines can use very simple models, with only a single dominant pole. You can use AWE methods to find the dominant pole, computed based on the total series resistance and capacitance or determined using the Elmore delay.

The Elmore delay uses the (d1-k1) value as the time constant, for a single-pole approximation to the complete H(s), where H(s) is the transfer function of the RC network for a specified output. The inverse Laplace transform of h(t) is H(s):

$$\tau_{DE} = \int_0^{\infty} t \cdot h(t) \, dt$$

Actually, the Elmore delay is the first moment of the impulse response and so corresponds to a first-order AWE result.

Figure 82 Circuits for a RC Line



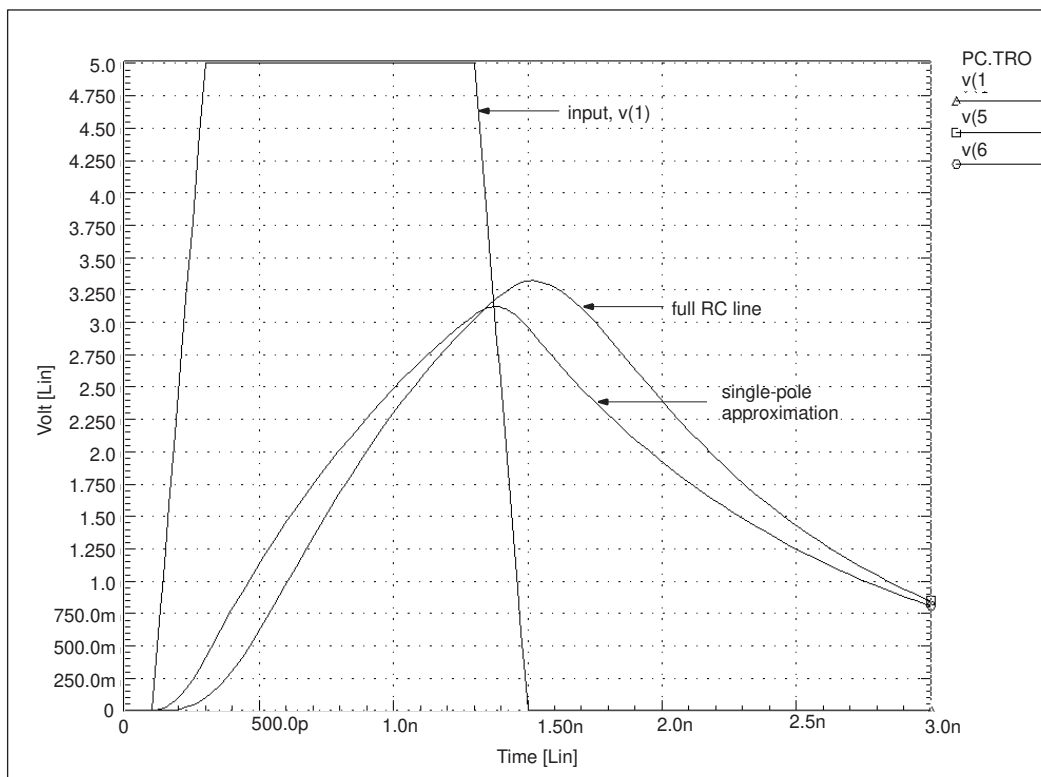
Example

This example is based on demonstration netlist `rcline.sp` which is available in the directory: `$installdir/demo/hspice/filters`.

```
****
.Tran 0.02ns 3ns
.OPTION Post Accurate List Probe
v1 1 0 PWL 0ns 0 0.1ns 0 0.3ns 5 1.3ns 5 1.5ns 0
r1 1 2 200
c1 2 0 0.6pF
r2 2 3 80
c2 3 0 0.8pF
r3 3 4 160
c3 4 0 0.7pF
r4 4 5 200
c4 5 0 0.8pF
e1 6 0 LAPLACE 1 0 1 / 1 1.16n
.Probe v(1) v(5) v(6)
.Print v(1) v(5) v(6)
.End
```

To closely approximate the output of the RC circuit (shown in [Figure 82 on page 303](#)), you can use a single-pole response, as shown in [Figure 83 on page 304](#).

Figure 83 Transient Response of the RC Line and Single-Pole Approximation

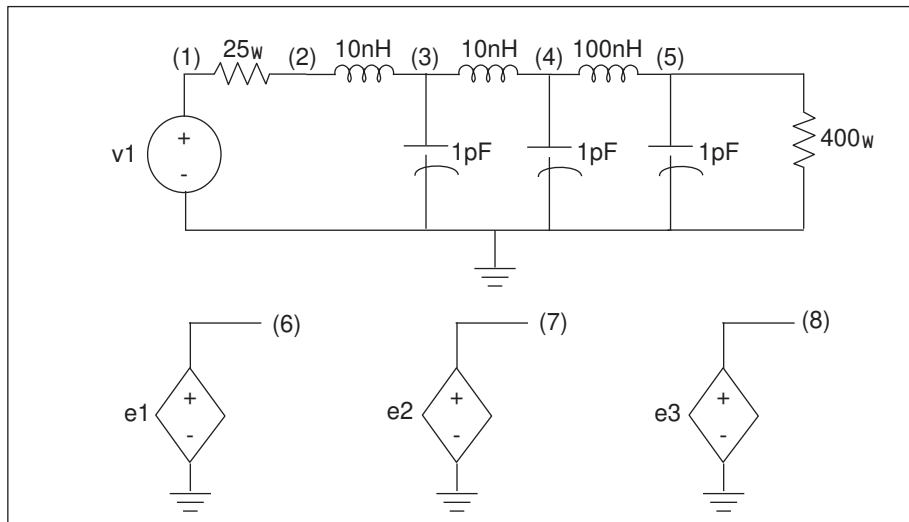


In [Figure 83](#), the single-pole approximation has less delay: 1 ns, compared to 1.1 ns for the full RC line model, at 2.5 V. The single-pole approximation also has a lower peak value than the RC line model. All other things being equal, a circuit with a shorter time constant results in less filtering and allows a higher maximum voltage value. The single-pole approximation produces a lower amplitude (and less delay) than the RC line because the single pole neglects the other three poles in the actual circuit. However, a single-pole approximation still provides very good results for many problems.

AWE Transfer Function Modeling

Approximations using single-pole transfer functions can cause larger errors for low-loss lines than for RC lines because lower resistance allows ringing. Circuit ringing creates complex pole pairs in the transfer function approximation. You need at least one complex pole pair to represent low-loss line response. [Figure 84 on page 305](#) is a typical low-loss line and the transfer function sources used to test various approximations. To obtain the transfer functions, PrimeSim HSPICE uses asymptotic waveform evaluation.

Figure 84 Circuits for a Low-Loss Line

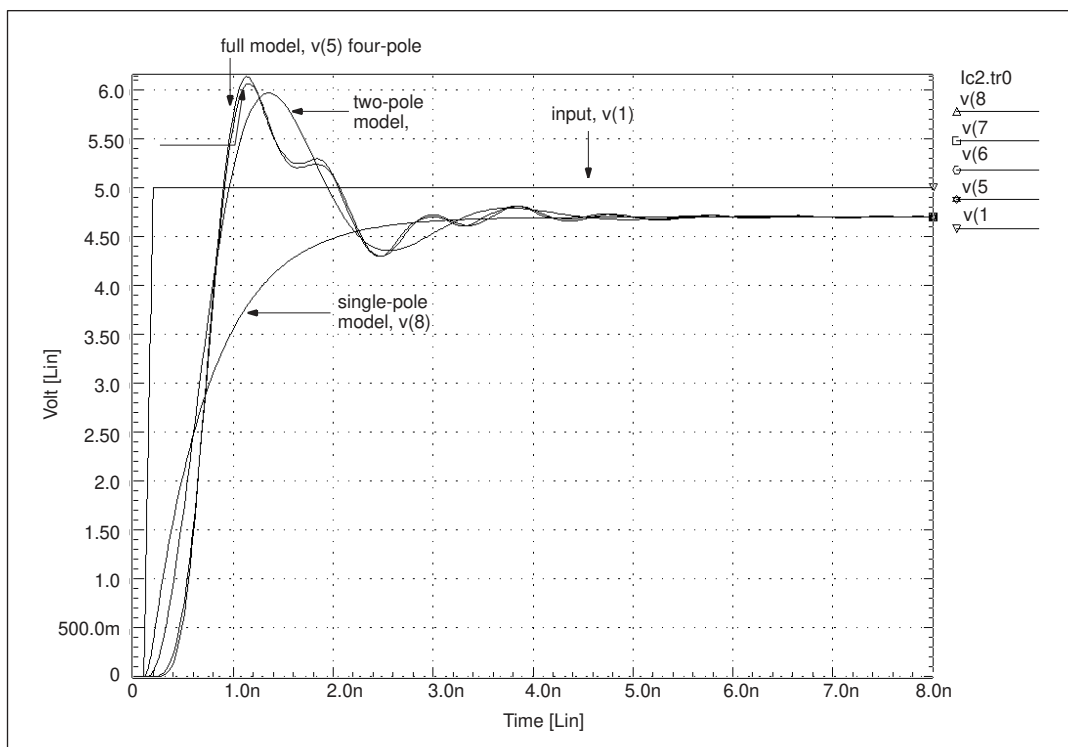


The sample file located in the following directory is a low-loss line circuit file:

```
$installdir/demo/hspice/filters/lowloss.sp
```

Figure 85 on page 306 shows the transient response of a low-loss line. It also shows E-element Laplace models using one, two, and four poles. The single-pole model shows none of the ringing of the higher-order models. Also, all E models must adjust the gain of their response for the finite load resistance, so the models are not independent of the load impedance. The 0.94-gain multiplier in the models take care of the 25-ohm source and the 400-ohm load-voltage divider. These approximations are good delay estimations.

Figure 85 Transient Response of the Low-Loss Line



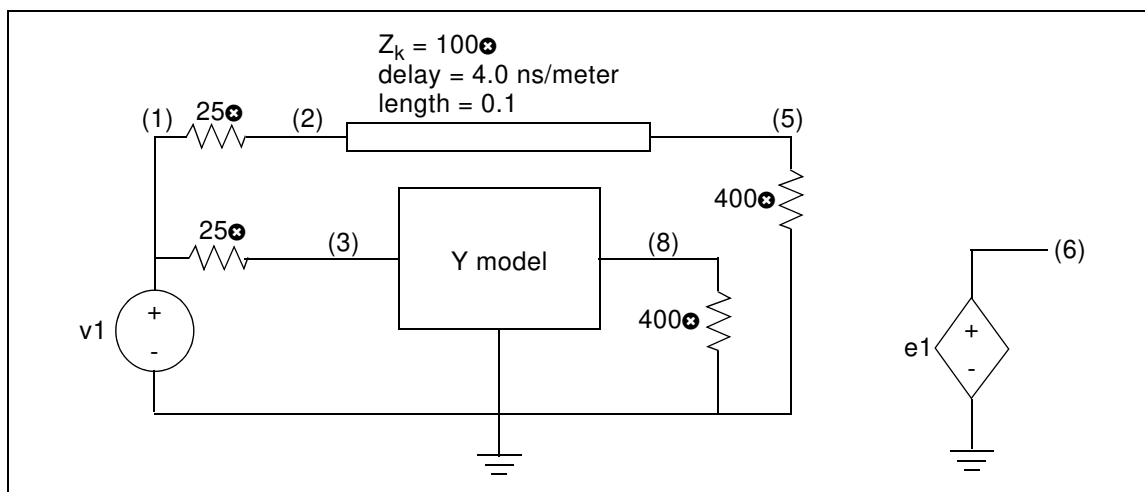
Although the two-pole approximation provides a reasonable agreement with the transient overshoot, the four-pole model offers almost perfect agreement. The actual circuit has six poles. You can use scaling to bring some of the very small numbers in the Laplace model above the $1\text{e-}28$ limit of PrimeSim HSPICE. The `SCALE` parameter multiplies every Y-parameter in the LAPLACE specification by the same value (in this case $1.0\text{e-}20$).

A low-loss line allows reflections between the load and source, compared to the loss of an RC line, which usually isolates the source from the load. So you can either incorporate the load into the AWE transfer function approximation, or create a PrimeSim HSPICE device model that allows source/load interaction. If you allow source/load interaction, you do not need to perform the AWE expansions each time you change load impedances. This allows PrimeSim HSPICE to handle non-linear loads and removes the need for a gain multiplier, as in the circuit file shown. You can use four voltage-controlled current sources or G Elements, to create a Y-parameter model for a transmission line. The Y-parameter network provides the needed source/load interaction. The next example shows such a Y-parameter model for a low-loss line.

Y-parameter Line Modeling

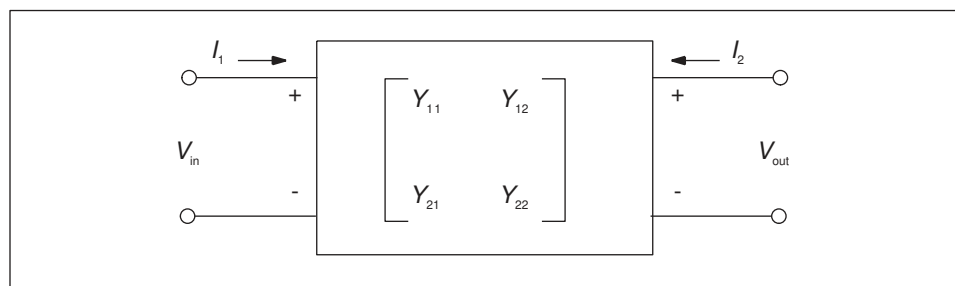
A model that is independent of load impedance is more complicated. You can still use AWE techniques, but you need a way for the load voltage and current to interact with the source impedance. For a transmission line of 100 ohms and 0.4 ns total delay (as shown in [Figure 86](#)), to compare the response of the line, use a Y-parameter model and a single-pole model.

Figure 86 Line and Y-parameter Modeling



[Figure 87](#) shows the voltage and current definitions for a Y-parameter model.

Figure 87 Y Matrix for the Two-Port Network



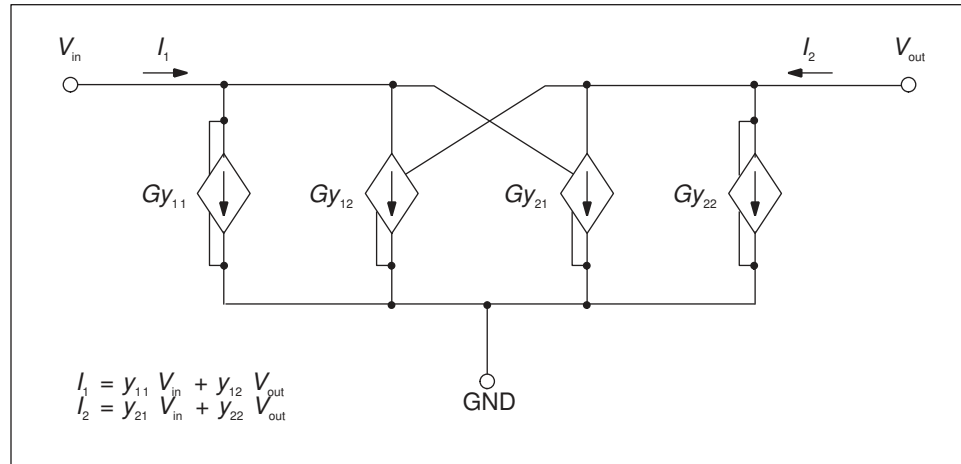
The following equations describe the general network in [Figure 87](#), which you can translate into G-elements:

$$I_1 = Y_{11}V_{in} + Y_{12}V_{out}$$

$$I_2 = Y_{21}V_{in} + Y_{22}V_{out}$$

Figure 88 shows a schematic for a set of two-port Y-parameters. The circuit consists mostly of G-elements.

Figure 88 Schematic for the Y-parameter Network



A Pade expansion of the Y-parameters for a transmission line determines the Laplace parameters for the Y-parameter model, as shown in matrix form in the following equation:

$$Y = \frac{1}{Z_o} \cdot \begin{bmatrix} \coth(p) & -\operatorname{csch}(p) \\ -\operatorname{csch}(p) & \coth(p) \end{bmatrix}$$

where p is the product of the propagation constant times the line length.

A Pade approximation contains polynomials in both the numerator and the denominator. A Pade approximation can model both poles and zeros, and $\coth$ and csch functions also contain both poles and zeros. So, a Pade approximation provides a better low-order model than a series approximation does.

The following equation calculates the Pade expansion of $\coth(p)$ and $\operatorname{csch}(p)$, with a second-order numerator and a third-order denominator:

$$\coth(p) \rightarrow \frac{(1 + \frac{2}{5} \cdot p^2)}{(p + \frac{1}{15} \cdot p^3)} \quad \operatorname{csch}(p) \rightarrow \frac{(1 - \frac{1}{20} \cdot p^2)}{(p + \frac{7}{60} \cdot p^3)}$$

When you substitute $(s \cdot \text{length} \cdot \sqrt{LC})$ for p , PrimeSim HSPICE generates polynomial expressions for each G-element. When you substitute 400 nH for L , 40 pF for C , 0.1 meter for length, and 100 for Z_o , ($Z_o = \sqrt{L/C}$) in the matrix equation above, you can use the resulting values in a circuit file.

The circuit file shown in the following sections uses all of the equation substitutions. The Pade approximations have different denominators for csch and $\coth$ but the circuit file

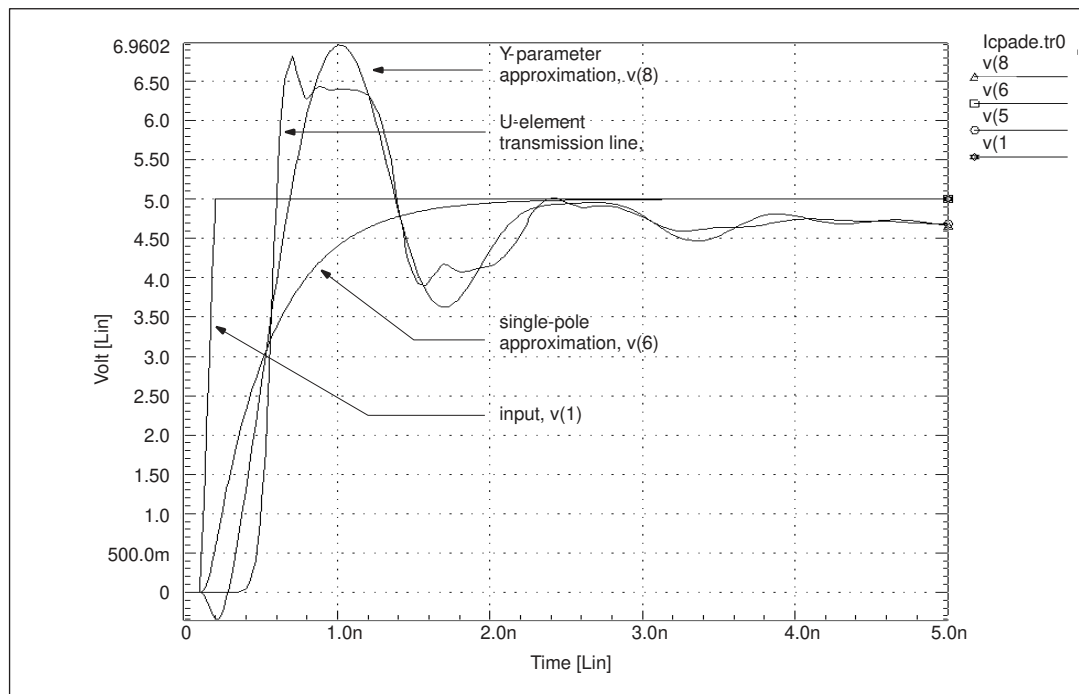
contains identical denominators. Although the actual denominators for $\cosh$ and $\coth$ are only slightly different, using them can cause oscillations in the PrimeSim HSPICE response. To avoid this problem, use the same denominator in the $\coth$ and $\cosh$ functions in the example. The simulation results might vary, depending on which denominator you use as the common denominator because the coefficient of the third-order term changes (but by less than a factor of 2).

This sample LC Line circuit file is located in the following directory:

```
$installldir/demo/hspice/filters/lcline.sp
```

Figure 89 on page 309 compares the output of the Y-parameter model with that of a full transmission line simulation, and with that obtained for a single-pole transfer function. In the latter case, the gain for the load impedance is incorrect, so the function produces an incorrect final voltage level. As expected, the Y parameter model provides the correct final voltage level. Although the Y parameter model provides a good approximation of the circuit delay, it contains too few poles to model all of the transient details. However, the Y-parameter model provides excellent agreement with the overshoot and settling times.

Figure 89 Transient Response of the Y-parameter Line Model



Comparison of Circuit and Pole/Zero Models

This example simulates a ninth-order low-pass filter circuit and compares the results with its equivalent pole/zero description using an E-element. The results are identical, but the pole/zero model runs about 40% faster. The example shown in [Simulation Time Summary on page 310](#) shows the total CPU times for the two methods. For larger circuits, the computation time saving can be much higher.

[Figure 90 on page 311](#) and [Figure 91 on page 311](#) display the transient and frequency response comparisons resulting from the two modeling methods.

Simulation Time Summary

Circuit model simulation times:

```
analysis   time    # points    # iter    conv.iter
op point   0.23     1      3
ac analysis 0.47     151     151
transient   0.75     201     226     113    rev=0
readin      0.22
errchk      0.13
setup       0.10
output      0.00
total cpu time 1.98 seconds
```

Pole/zero model simulation times:

```
analysis   time    # points    #
iter      conv.iter
op point   0.12     1      3
ac analysis 0.22     151     151
transient   0.40     201     222     111    rev=0
readin      0.23
errchk      0.13
setup       0.02
output      0.00
total cpu time 1.23 seconds
```

Figure 90 Transient Responses of the Circuit and Pole/Zero Models

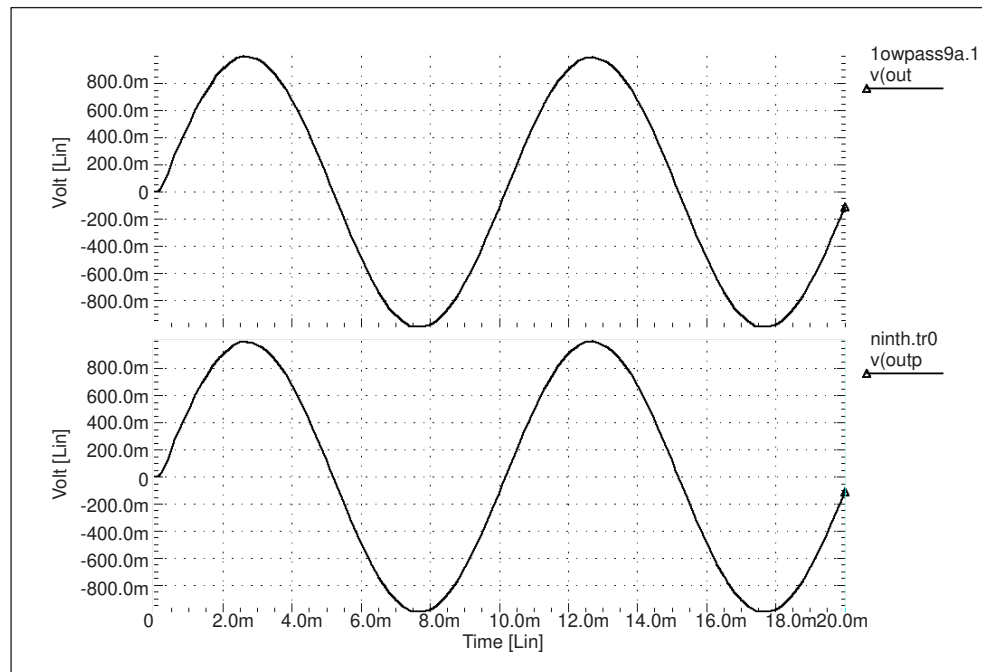
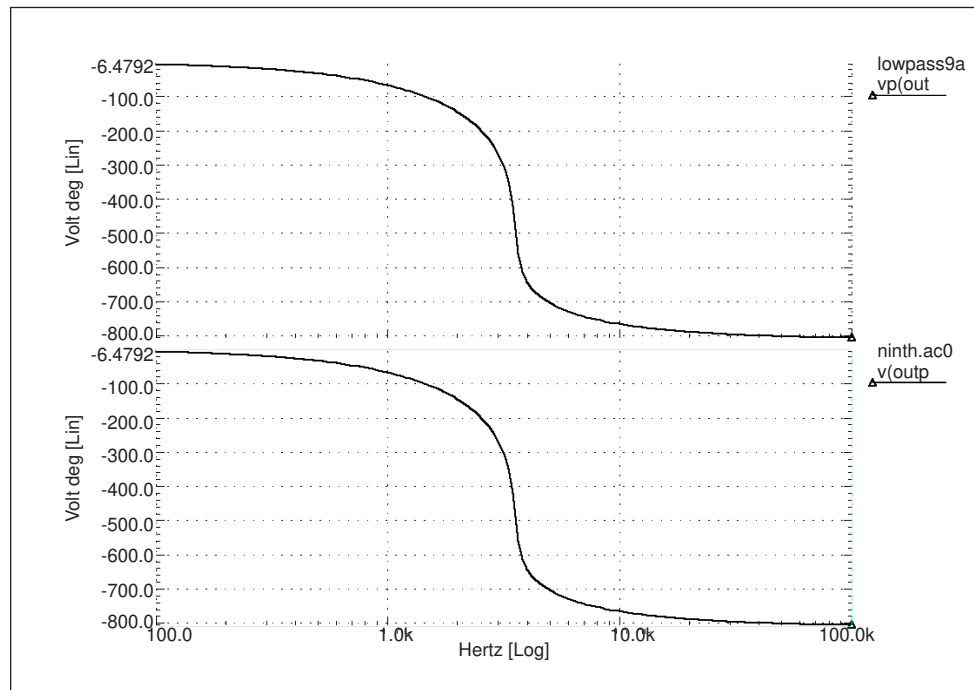


Figure 91 AC Analysis Responses of the Circuit and Pole/Zero Models



Modeling Switched Capacitor Filters

The following sections discuss these topics:

- [Switched Capacitor Network](#)
- [Switched Capacitor Filter Example](#)
- [Input File for Switched Capacitor Filter](#)

Switched Capacitor Network

You can model a resistor as a capacitor and switch combination. The value of the equivalent is proportional to the frequency of the switch, divided by capacitance.

Construct a filter from MOSFETs and capacitors, where the filter characteristics are a function of the switching frequency of the MOSFETs.

To quickly determine the filter characteristics, use ideal switches (voltage-controlled resistors), instead of MOSFETs. The resulting simulation speed-up can be as great as 7 to 10 times faster than a circuit using MOSFETs.

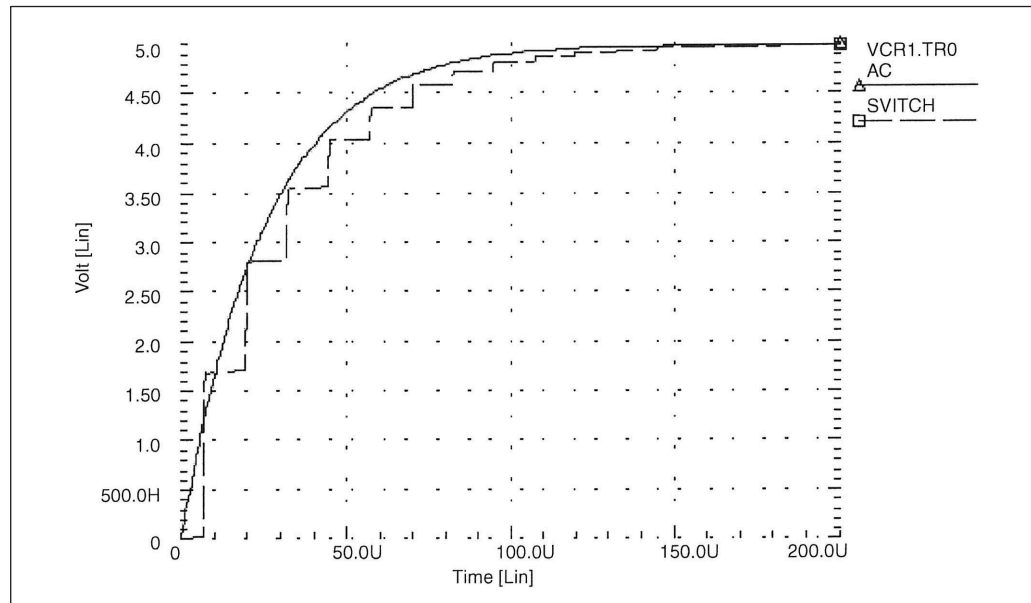
To construct an RC network, the model uses a resistor and a capacitor, along with a switched-capacitor equivalent network. The `RCOUT` node is the resistor/capacitor output and `VCROUT` is the switched-capacitor output.

The `GVCR1` and `GVCR2` switches and the `C3` capacitance model the resistor. The following equation calculates the resistor value:

$$Res = \frac{T_{switch}}{C3}$$

where T_{switch} is the period of the PHI1 and PHI2 pulses.

Figure 92 VCR1.SP Switched Capacitor RC Circuit



Switched Capacitor Filter Example

This example is a fifth-order elliptic, switched-capacitor filter. The passband is 0-1 kHz, with loss less than 0.05 dB. This results from the cascading models of the switches. The resistance is 1 ohm when the switch is closed and 100 Megohm when it is open. The E-element models op-amps as an ideal op-amp. This example provides the transient response of the filter for 1 kHz and 2 kHz sinusoidal input signals.

Figure 93 Linear Section

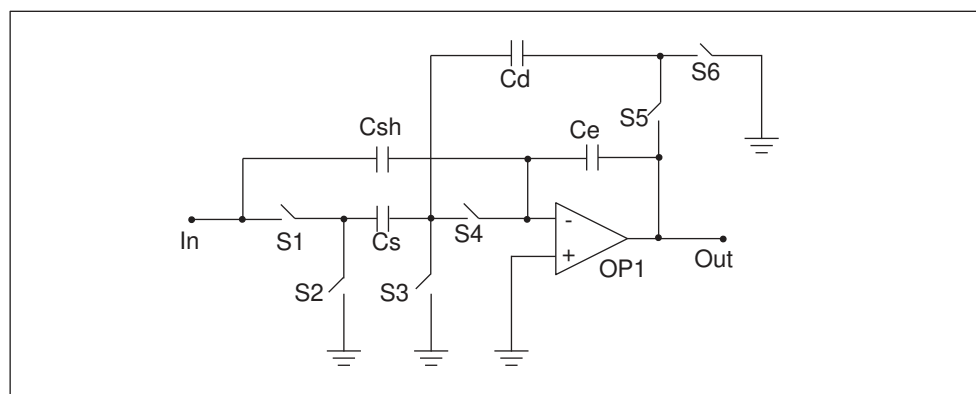


Figure 94 High_Q Biquad Section

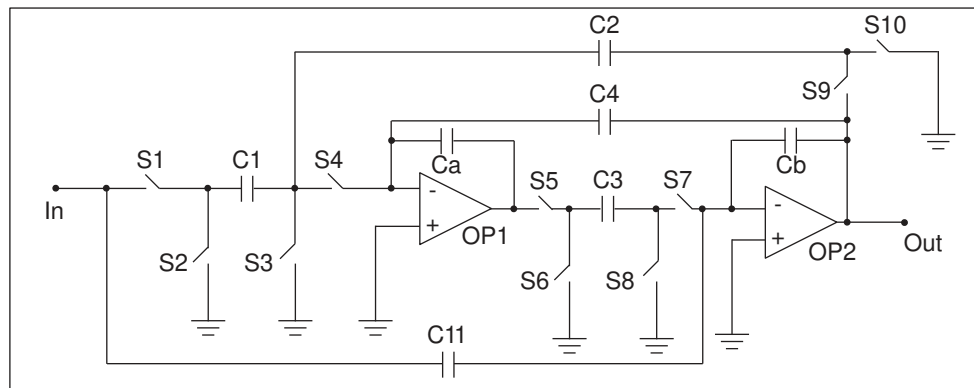
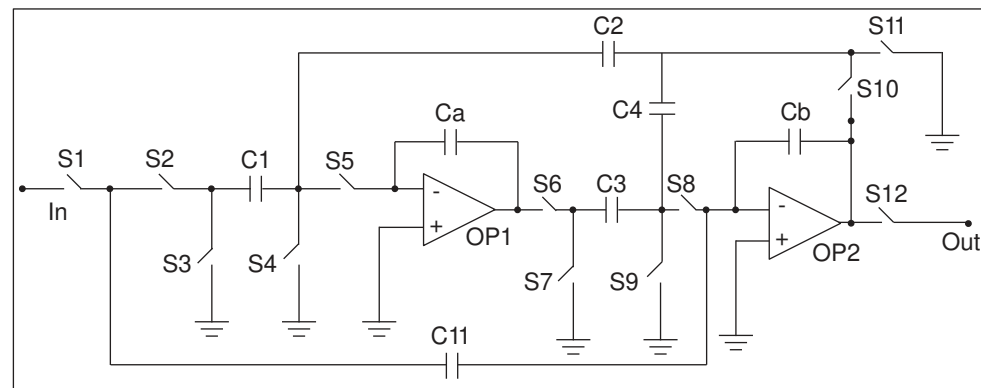


Figure 95 Low_Q Biquad Section



Input File for Switched Capacitor Filter

This sample input file for a switched capacitor filter is located in the following directory:

```
$installdir/demo/hspice/behave/swcap5.sp
```

This file also contains the following examples:

- Sample and Hold
- Linear Section
- High_Q Biquad Section
- Low_Q Biquad Section

Figure 96 Response to 1-kHz Sinusoidal Input

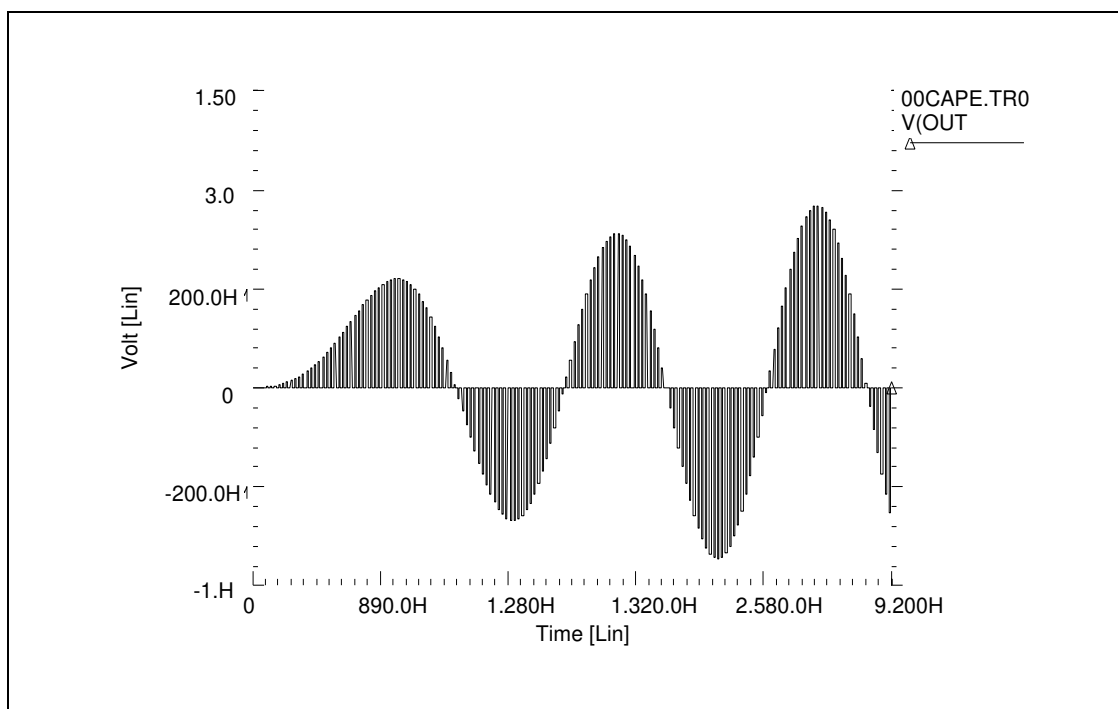
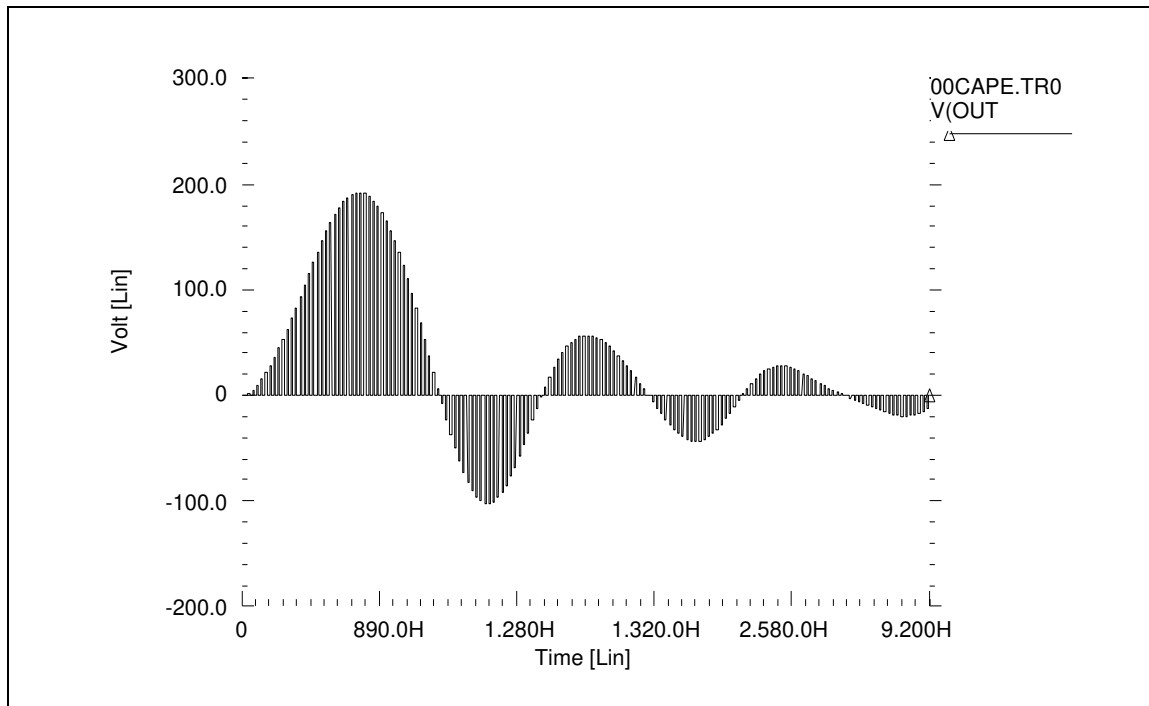


Figure 97 Response to 2-kHz Sinusoidal Input



References

- [1] Williams, Arthur B., and Taylor, Fred J. *Electronic Filter Design Handbook*. New York: McGraw-Hill, 1988, pp. 6-20 to 6-23.
- [2] Nillson, James W. *Electric Circuits*, 4th Edition. Reading, Massachusetts: Addison-Wesley, 1993.
- [3] Edminister, Joseph A. *Electric Circuits*. New York: McGraw-Hill, 1965.
- [4] Ghausi, Kelly, and M.S. "On the Effective Dominant Pole of the Distributed RC Networks", *Jour. Franklin Inst.*, June 1965, pp. 417- 429.
- [5] Elmore, W.C. and Sands, M. *Electronics, National Nuclear Energy Series*, New York: McGraw-Hill, 1949.
- [6] Pillage, L.T. and Rohrer, R.A. "Asymptotic Waveform Evaluation for Timing Analysis", *IEEE Trans. CAD*, Apr. 1990, pp. 352 - 366.
- [7] Kuo, F. F. *Network Analysis and Synthesis*. John Wiley and Sons, 1966.
- [8] Gregorian, Roubik & Temes, Gabor C. *Analog MOS Integrated Circuits*. J. Wiley, 1986, page 354.